Claude Certified Architect, Professional. Curriculum

Key fact Decision rule Exam tip Trap

Read the units in order. Each unit has the same ten parts. Colored text marks a key fact, a decision rule, an exam tip, or a trap.

Topic index

1. **F Foundations** Weight Not weighted
    1. F1 Architecture and business foundations Token cost model, Spend caps and usage tiers, Acceptance thresholds for model output
    2. F2 LLM foundations Context window, Model tier choice, Tool call loop
    3. F3 Software and distributed systems Error classes and retry, Timeouts and long requests, Streaming events and errors
    4. F4 Security and identity API keys and workspace scope, Workload identity federation, Data retention and encryption
2. **D1 Solution design and architecture** Weight 17%
    1. D1.1 Business problem to Claude solution Use-case fit and lane choice, Success criteria and evals
    2. D1.2 End-to-end architecture Request loop with state in the application, Verification and output gates
    3. D1.3 Architectural pattern selection Tool-use loop in application code, Agent loop with model-planned steps
    4. D1.4 Multi-agent orchestration Subagent delegation, Dynamic workflow runs
    5. D1.5 Task decomposition Fan-out with a merge step, Chain and context control
    6. D1.6 Business-value alignment Unit cost from published prices, Value guards for quality and spend
3. **D2 Claude models, prompting, and context engineering** Weight 13%
    1. D2.1 Model selection and tier trade-offs Model tier selection, Effort and adaptive thinking
    2. D2.2 System prompts, templates, and guardrails System prompt with role, scope, and rules, Structured outputs as the output contract
    3. D2.3 Prompt techniques Few-shot examples in multishot prompts, Adaptive thinking and manual chain of thought, Prefill replacement
    4. D2.4 Context and token management Context window and token counting, Compaction of older turns, Context editing by rule
    5. D2.5 Reuse and prompt caching Prompt caching, Automatic caching versus explicit breakpoints, Prompt reuse with templates
4. **D3 Integration** Weight 19%
    1. D3.1 Tool capability scope Tool surface scope, Tool search with deferred tools, Tool definition quality
    2. D3.2 Authentication and authorization in integrations Per-user delegation to MCP servers, Workspace and API key boundary, Cloud platform identity
    3. D3.3 Accuracy and latency trade-offs Model and effort choice for latency, Streaming for perceived latency, Batch processing and service tiers for throughput
    4. D3.4 Observability at scale Request tracing with OpenTelemetry, Usage, cost, and rate limit reports, Claude Code and agent session telemetry
    5. D3.5 The RAG pipeline Search results with citations, Embeddings and reranking models, Chunk granularity for citations
    6. D3.6 Retrieval strategies Hybrid keyword and semantic retrieval, Reranking candidates, Live state through a tool call
    7. D3.7 Integration mechanisms MCP connector in the Messages API, Claude Code MCP servers
    8. D3.8 Progressive discovery and monolithic context Tool search tool in the Messages API, Agent SDK tool search, Agent Skills progressive disclosure
5. **D4 Evaluation, testing, and optimization** Weight 16%

1. D4.1 Evaluation metrics Success criteria and primary metric, Cost and latency metrics
2. D4.2 Evaluation datasets and frameworks Evaluation dataset design, Graders for evaluations
3. D4.3 A/B tests and iteration A/B test design, Iteration by model and effort
4. D4.4 Diagnosis at the correct layer Cache miss diagnosis, Grounding and consistency checks
5. D4.5 Optimization under quality floors Batch processing for work that can wait, Model tier and effort choice
6. D4.6 Production monitoring Usage and cost monitoring, Traces and telemetry, Model version lock and drift

6. **D5 Governance, safety, and risk management** Weight 14%
   1. D5.1 Guardrails and safety controls Input validation and untrusted-content handling, Deterministic tool authorization gate, Sandbox boundary for shell commands
   2. D5.2 Risks and failure modes Hallucination control with grounded quotes, Cost exhaustion limits, Fixed model IDs and retirement
   3. D5.3 Human-in-the-loop validation Pre-action approval for irreversible actions, Post-action review and monitoring, Hook-based approval point in Claude Code
   4. D5.4 Regulatory compliance and data controls Retention limits and zero data retention, Data residency with inference geo and workspace geo, Customer-managed keys and audit evidence
   5. D5.5 Responsible AI Content moderation with Claude, Safety classifiers and refusal handling, Model documentation, transparency, and per-group evaluation

7. **D6 Stakeholder communication and lifecycle management** Weight 14%
   1. D6.1 Structured discovery Success criteria and evaluation sets, Discovery question grid
   2. D6.2 Trade-off communication Model pricing and cost per task, Cost per task measurement, Five-field trade-off record
   3. D6.3 Feedback and expectation alignment Usage and Cost API reads, Workspace spend and rate limits, Feedback contract with review triggers
   4. D6.4 Architecture documentation Architecture decision record, Model ID in decision records
   5. D6.5 Lifecycle phases Lifecycle phase gates, Model lifecycle states and retirement, Workspaces per environment

8. **D7 Developer productivity and operational enablement** Weight 7%
   1. D7.1 Team configuration for Claude Code Shared settings and scopes, Permission rules and hooks, CLAUDE.md and subagent guidance
   2. D7.2 AI-assisted workflows with Claude Code Headless runs, GitHub Actions workflows, Subagent roles for review and exploration
   3. D7.3 Diagnosis and operational resolution Diagnostic commands, Usage, cost, and telemetry, Rewind and file snapshots

F1 Architecture and business foundations

Domain F, Foundations. Task statement: Turn cost, capacity, and quality requirements into numbers that a design can meet. (Inference: task wording from repo lesson 7-4A, not from the exam guide)

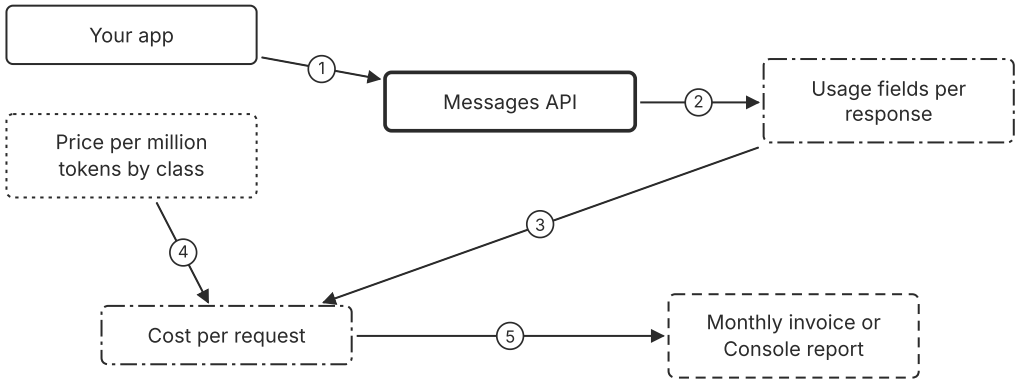
Token cost model

1. Introduction and first principle

Each token class has its own price, so the cost of a request is the sum of its classes.

- Claude Sonnet 5.5 costs 2 dollars per million input tokens and 10 dollars per million output tokens.[1]
- Claude Opus 5.5 costs 4 dollars per million input tokens and 20 dollars per million output tokens.[1]
- Claude Haiku 5.5 starts at 0.10 dollars per million input tokens and 0.50 dollars per million output tokens.[2]
- Batch API requests cost 50 percent less than the base price.[1]
- Regional and multi-region endpoints include a 10 percent premium over global endpoints.[3]
- Setting inference_geo to us on Claude 4.6 and later models applies a 1.1 times pricing multiplier.[3]
- Prompts over 100,000 tokens cost more on Claude Haiku 5.5.[4]

2. Architecture



Schematic drawn from docs. Arrows: 1. send request. 2. return token counts. 3. count per class. 4. rate per class. 5. times daily volume. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. The API reports token counts for each class in the usage fields.
2. Cache writes and cache reads have their own multiplier on the base input price.
3. Thinking tokens bill as output tokens.
4. Batch requests cost 50 percent less than standard requests.
5. Multiply each count by its rate and divide the result by one million.
6. Multiply the cost per request by the daily request volume and the number of days.
7. Compare the monthly figure with the spend cap before launch.

4. Pros and cons

Pros

- The arithmetic gives an exact figure once the token counts are known.
- The cost of a request is visible before any load test runs.
- Usage fields show the split between cached and uncached tokens.

Cons

- Token counts change with the tokenizer, so a sample of real requests gives the best estimate.
- Prices differ by model, endpoint, and platform.

- Prompts over 100,000 tokens cost more on Claude Haiku 5.5.

5. Use when, do not use when

Use when

- A design choice changes the token count per request or the request volume.
- A budget owner must approve a monthly spend figure.

Do not use when

- The work has no token cost, such as a local function with no model call.
- The price for the chosen platform is unknown. Look up the platform price first.

6. Constraint families

Cost

Price each token class separately and add the totals. Combination when one choice is not enough. Add batch pricing for work that can wait up to 24 hours.

Quota

Compare the monthly forecast with the monthly spend cap of the tier. Combination when one choice is not enough. Set a workspace spend limit below the tier cap.

Data location

Use a regional endpoint only when residency requires it. Combination when one choice is not enough. Expect a 10 percent premium on regional endpoints.

Scale

Forecast cost at peak request volume, not at the monthly average.

Operations

Sum the usage fields for each request each day. Combination when one choice is not enough. Alert when the daily total passes the forecast. (Inference)

7. Native cloud services

Amazon Bedrock

Global endpoints carry no pricing premium. Regional endpoints carry a 10 percent premium.[5]

Claude Platform on AWS

Usage bills in Claude Consumption Units. Each unit costs 0.01 dollars, and token rates convert to units.[6]

Claude Models on Google Cloud (Google managed)

Google Cloud invoices usage. Regional and multi-region endpoints carry a 10 percent premium over global endpoints.[7]

Claude Models on Google Cloud (Anthropic managed)

Google Cloud Marketplace bills the usage. One Claude Consumption Unit costs 0.01 dollars.[8]

Microsoft Foundry

Azure Marketplace bills Claude Consumption Units. A US Data Zone deployment applies a 1.1 times multiplier.[9]

8. Exam domain and task statement

F1. Expect a scenario with a price, a volume, and a request for a monthly figure. The credited answer uses token arithmetic per class. (Inference)

If a figure has no unit, convert it to tokens per request and requests per day first.

Multiply each token class by its own rate, add the classes, then multiply by the request volume.

9. Trap

A flat estimate that counts input tokens only looks simple, but output tokens cost more, so the estimate understates the bill.

10. Not in source

- Per-model Google Cloud list prices for Claude models. The docs link to Google Cloud pricing pages. Not in source.
- Monthly spend for any sample workload. Not in source.

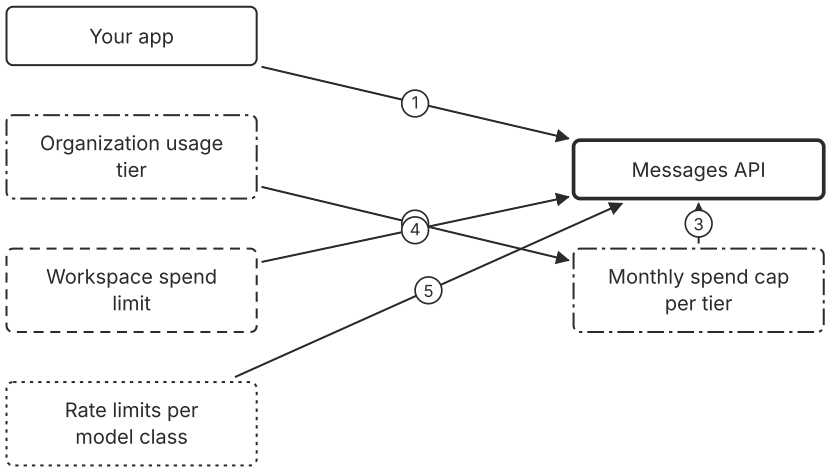
Spend caps and usage tiers

1. Introduction and first principle

A spend cap stops usage at a set monthly amount, so it limits the worst-case bill.

- The Start tier monthly spend cap is 500 US dollars.[10]
- The Build tier monthly spend cap is 1,000 US dollars.[10]
- The Scale tier monthly spend cap is 200,000 US dollars.[10]
- A tier spend cap returns HTTP 429 with no retry-after header.[10]
- A spend limit that you set returns HTTP 400 with an invalid_request_error type.[10]
- An organization has up to 100 workspaces by default.[11]
- The Default Workspace cannot have a spend limit.[11]
- On Claude Platform on AWS, spend can take about 2 hours to reflect recent usage, and the overshoot is billed.[6]

2. Architecture



Schematic drawn from docs. Arrows: 1. send request. 2. sets cap. 3. cap reached: 429 without retry-after. 4. limit reached: 400. 5. requests and tokens per minute. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. The organization sits in a usage tier with a monthly spend cap.
2. Each workspace can set a lower spend limit in the Console.
3. Rate limits apply per model class in requests and tokens per minute.
4. When usage reaches a spend limit you set, requests return HTTP 400.
5. When usage reaches the tier cap, requests return HTTP 429 without a retry-after header.
6. Raise or remove the limit to restore access sooner.

4. Pros and cons

Pros

- A cap bounds the monthly loss from a runaway loop.
- Workspace limits separate budgets for teams and environments.
- A tier move raises the rate limits and the spend cap together.

Cons

- A tier cap returns 429, which looks like a rate limit, but retries fail until access resumes.
- Spend on Claude Platform on AWS can take about 2 hours to show, so usage can pass a cap before requests stop.
- The Default Workspace cannot have a spend limit.

5. Use when, do not use when

Use when

- A team or environment needs its own monthly budget.
- A new workload must not exceed a fixed monthly amount.

Do not use when

- The goal is faster throughput. Rate limits control request speed, not the budget.
- A usage report is the only goal. A report alone does not stop usage.

6. Constraint families

Cost

Set the spend limit below the tier cap. Combination when one choice is not enough. Add an alert threshold for an early warning. (Inference)

Quota

Read the rate limits for each tier before the launch date. Combination when one choice is not enough. Ask the account representative for a tier change on Claude Platform on AWS.

Isolation

Use a separate workspace for each team that needs a separate budget.

Governance

Set a workspace spend limit for each team or environment. Combination when one choice is not enough. Add billing recipients who receive the cap notice.

Operations

Track spend per workspace in the Console.

7. Native cloud services

Amazon Bedrock

The default quota is 2 million input tokens per minute. AWS enforces requests per minute.[5]

Claude Platform on AWS

The tier cap and the rate limits apply. Tier moves go through the account representative or support.[6]

Claude Models on Google Cloud (Google managed)

Google Cloud manages rate limits and quotas for this platform.[7]

Claude Models on Google Cloud (Anthropic managed)

Anthropic manages rate limits. The account representative raises them on request.[8]

Microsoft Foundry

Foundry does not send Anthropic rate limit headers. Limit increases go through Azure.[9]

8. Exam domain and task statement

F1. Expect a scenario with a shared key and a budget owner. The credited answer scopes the budget at the workspace level. (Inference)

A 429 with no retry-after header points to a spend cap, not to a rate limit.

Use a workspace spend limit for budget control and a rate limit change for throughput.

9. Trap

A retry loop for every 429 looks right. A spend cap 429 has no retry-after header, so retries fail until access resumes.

10. Not in source

- Alert threshold percentages for workspace spend. The docs say alerts exist, but they name no values. Not in source.
- Delay between a cap notice and the email delivery. Not in source.

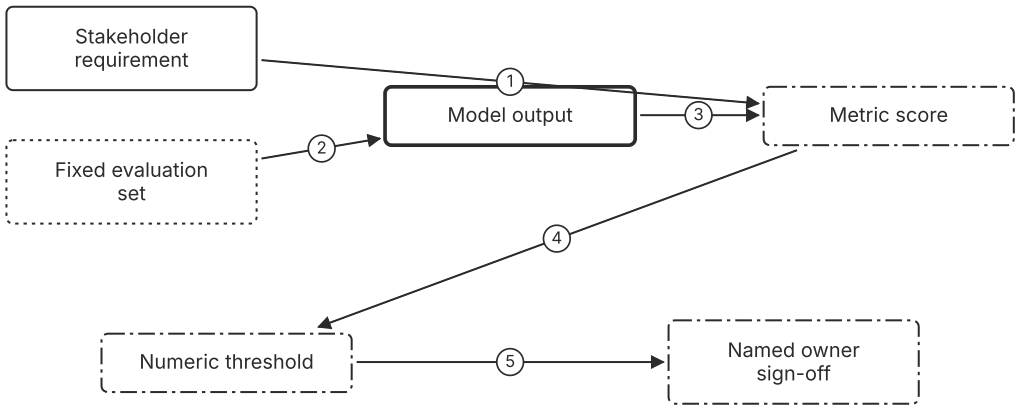
Acceptance thresholds for model output

1. Introduction and first principle

Each quality adjective needs a metric, a threshold, and an owner before the build starts. (Inference)

- Even with temperature set to 0, identical inputs can produce different outputs across API calls.[1][2]
- As token count grows, accuracy and recall degrade, which the docs call context rot.[4]
- The token-count endpoint estimates token usage before a request is sent.[4]

2. Architecture



Schematic drawn from docs. Arrows: 1. turn adjective into metric. 2. run on fixed inputs. 3. score output. 4. compare with line. 5. miss goes to owner. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write the requirement as one sentence that names the quality.
2. Name a metric that measures that quality on a fixed evaluation set.
3. Set a numeric threshold that the metric must meet.
4. Name one owner who signs off the threshold.
5. Run the same evaluation before each release.
6. Review the threshold when the traffic or the task changes.

4. Pros and cons

Pros

- A number gives all builders the same target.

- A threshold makes a release decision testable.
- An owner answers for the number.

Cons

- A threshold chosen without data can be wrong.
- A green metric can hide a user problem when traffic changes.
- A fixed evaluation set can drift from live traffic.

5. Use when, do not use when

Use when

- An output error has a cost, such as a wrong date in a legal form.
- Two or more teams hand off work that depends on the quality claim.

Do not use when

- A one-person demo has no users and no money at stake.
- The output has no quality claim that a test can check.

6. Constraint families

Cost

Measure the cost of a wrong output before setting the threshold. Combination when one choice is not enough. Lower the threshold only when an error costs little. (Inference)

Governance

Name one owner for each threshold. Combination when one choice is not enough. Review the owner sign-off on a fixed date.

Skill

Write each metric in the terms that the task owner uses.

Operations

Run the same evaluation before each release. Combination when one choice is not enough. Add a sample of live traffic when the fixed set no longer matches users. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

F1. Expect an option with a vague quality word and an option with a metric and a threshold. The credited answer names both. (Inference)

If the stem says fast or accurate with no number, the answer adds a metric and a threshold.

Attach a metric, a threshold, and an owner to each quality word before the build starts.

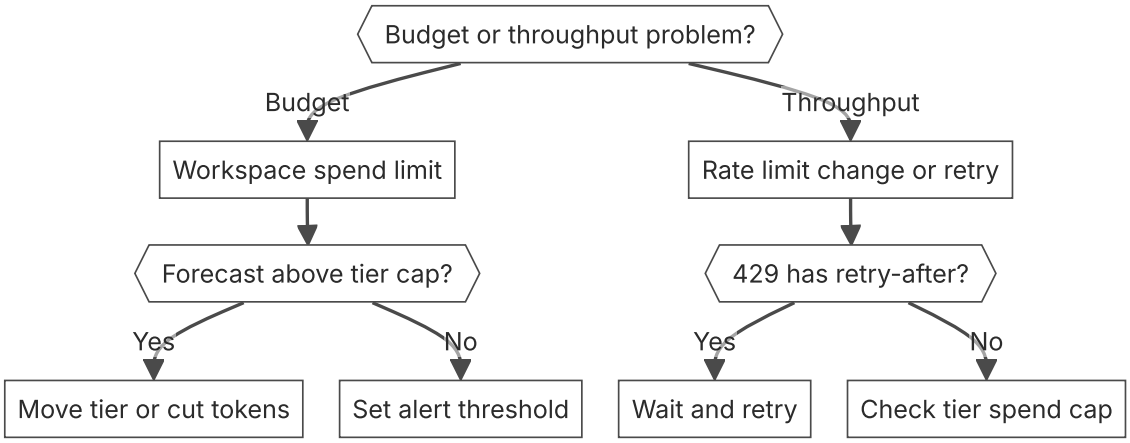
9. Trap

Launch first and tune the threshold later sounds agile, but users meet the errors before any bar exists.

10. Not in source

- A default accuracy target for model output. Not in source.
- A default size for an evaluation set. Not in source.

Decision flow



Unit items not in source

- Acceptance threshold values for any product. The docs name no thresholds. Not in source.
- Exact cost of a delayed refund or other business loss. Not in source.

F2 LLM foundations

Domain F, Foundations. Task statement: Explain how a model produces output, what limits its input and output, and how a tool call returns control to the app. (Inference: task wording from repo lesson 7-3A, not from the exam guide)

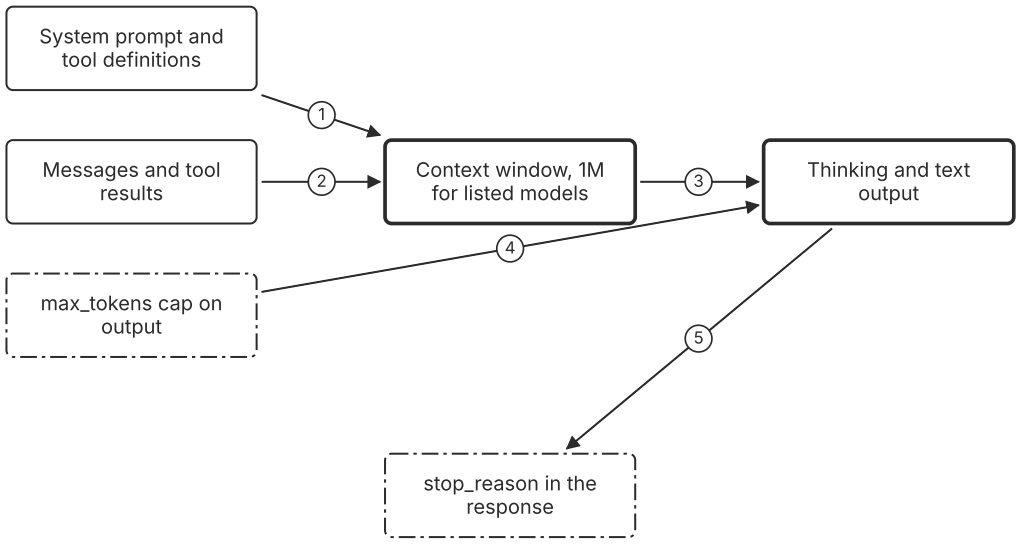
Context window

1. Introduction and first principle

The context window holds every input token and every output token of a turn, so all of them share one budget.

- Claude Opus 5.5 has a 1M-token context window.[1]
- Models with a 1M-token context window generate up to 128k output tokens in one request.[4]
- Other models, such as Claude Sonnet 4.5, have a 200k-token context window.[4]
- An input that alone exceeds the window returns a 400 invalid_request_error.[4]
- Thinking tokens count toward the context window.[4]
- 1M tokens is roughly 555k words on the current tokenizer.[1]
- One request can include up to 600 images or PDF pages on models with a 1M-token window.[4]

2. Architecture



Schematic drawn from docs. Arrows: 1. counts toward window. 2. counts toward window. 3. generate turn. 4. caps output. 5. ends with reason. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Everything in a request counts toward the context window.
2. The system prompt, the messages, the tool results, and the tool definitions all count.
3. Thinking tokens and text output count too.
4. The max_tokens value caps the output for each response.
5. An input longer than the window returns a 400 invalid_request_error.
6. On Claude 4.5 models and newer, output stops with model_context_window_exceeded when generation reaches the limit.
7. Use the token-count endpoint before a large request.

4. Pros and cons

Pros

- A 1M-token window fits long documents in one request for listed models.
- Earlier tool results stay available to the model in the same turn.
- Each response reports its token usage.

Cons

- Accuracy and recall degrade as token count grows, which the docs call context rot.
- Cached prompt prefixes still occupy the window.
- Long inputs raise the token count of every turn.

5. Use when, do not use when

Use when

- The task needs many documents or a long conversation in one request.
- The expected output fits inside the max_tokens cap.

Do not use when

- The input alone exceeds the model window. Shorten or split the input first.
- A short prompt answers the task. Extra context adds cost and can reduce focus.

6. Constraint families

Latency

Set max_tokens to the output size that the task needs. Combination when one choice is not enough. Stream the output when the first tokens matter.

Cost

Estimate tokens before a large request. Combination when one choice is not enough. Cache stable prefixes to cut the cost of repeated input.

Scale

Use the 1M-token window for listed models and the 200k window for older models. Combination when one choice is not enough. Use server-side compaction when a conversation nears the window.

Operations

Read stop_reason before you accept a response. Combination when one choice is not enough. Treat model_context_window_exceeded as a truncated answer.

7. Native cloud services

Claude Platform on AWS

Context-window sizes match the first-party Claude API.[6]

Microsoft Foundry

Listed models have a 1M-token context window on Foundry.[9]

Claude Models on Google Cloud (Google managed)

Listed models have a 1M-token context window on this platform.[7]

8. Exam domain and task statement

F2. Expect a long document and an output limit in one scenario. The credited answer checks the window and max_tokens before the design. (Inference)

If the input alone exceeds the window, the API returns a 400 error and no output.

Fit the request inside the window with room for max_tokens, then select what enters the window.

9. Trap

A bigger window looks like the fix for missed details, but context rot means more tokens can lower recall.

10. Not in source

- Accuracy loss percentage at a given token count. Not in source.
- Token count for text in a language other than English. Not in source.

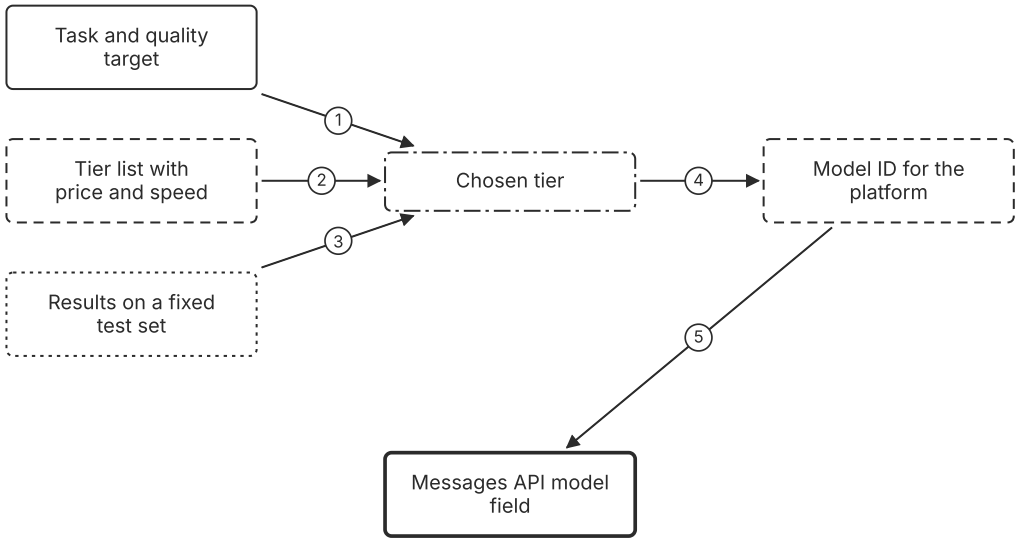
Model tier choice

1. Introduction and first principle

Each tier trades speed and price against capability, so the choice follows the quality target and the budget. (Inference)

- Claude Haiku 5.5 is built for high-volume, latency-sensitive tasks such as classification and routing.[2]
- Claude Haiku 5.5 has a 1M-token context window and up to 128K output tokens.[2]
- Claude Haiku 5.5 uses a tokenizer that counts about 30 percent more tokens than Claude Haiku 4.5 for the same text.[2]
- Claude Sonnet 5.5 costs 2 dollars per million input tokens and 10 dollars per million output tokens.[1]
- Claude Sonnet 5.5 retires not sooner than September 28, 2027.[1]

2. Architecture



Schematic drawn from docs. Arrows: 1. set quality target and budget. 2. filter by price and speed. 3. keep tier that passes. 4. look up platform ID. 5. set model field. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Name the task and the quality target.
2. List the tiers with their price, speed, and context window.
3. Run the same evaluation set on two tiers.
4. Keep the least costly tier that meets the quality target.
5. Set the model field to the ID for the platform in use.
6. Test the choice again after each model migration.

4. Pros and cons

Pros

- Tiers give a clear trade-off between price, speed, and capability.
- A tier change keeps the same request shape.
- A test set replaces a guess about which model is large enough.

Cons

- Model IDs differ by platform, so a copied ID can fail.
- Prices and retirement dates change over time.
- A smaller tier can miss a hard task, and the miss may show no error.

5. Use when, do not use when

Use when

- A high-volume task with a clear output format needs low latency.
- A hard reasoning task fails the quality target on a smaller tier.

Do not use when

- The largest tier is chosen with no quality test. (Inference)
- The design uses a model that is near retirement. Plan a migration first.

6. Constraint families

Latency

Use the fastest tier for high-volume, latency-sensitive tasks. Combination when one choice is not enough. Raise the effort setting when the quality test fails. (Inference)

Cost

Pick the lowest price tier that passes the test set. Combination when one choice is not enough. Price the expected volume before the choice is final.

Governance

Record each model retirement date in the design record. Combination when one choice is not enough. Plan the migration before the retirement date.

Skill

Match the tier to the complexity of the task.

7. Native cloud services

Amazon Bedrock

The model ID is anthropic.claude-haiku-5-5 on Bedrock.[2]

Claude Platform on AWS

The model ID is claude-haiku-5-5, the same as the Claude API.[2]

Claude Models on Google Cloud (Google managed)

The model ID is claude-haiku-5-5 on this platform.[2]

Claude Models on Google Cloud (Anthropic managed)

The model ID is claude-haiku-5-5 on this platform.[2]

Microsoft Foundry

The deployment name becomes the model parameter, and it can differ from the model ID.[9]

8. Exam domain and task statement

F2. Expect a scenario with a volume, a latency limit, and a quality floor. The credited answer picks the tier that meets all three. (Inference)

If the stem names a platform, check the model ID for that platform before you accept an option.

Choose the least costly tier that passes the evaluation for the task.

9. Trap

The largest model looks safest, but it adds cost and latency that the scenario does not require.

10. Not in source

- A quality score that separates the tiers for one task. Not in source.

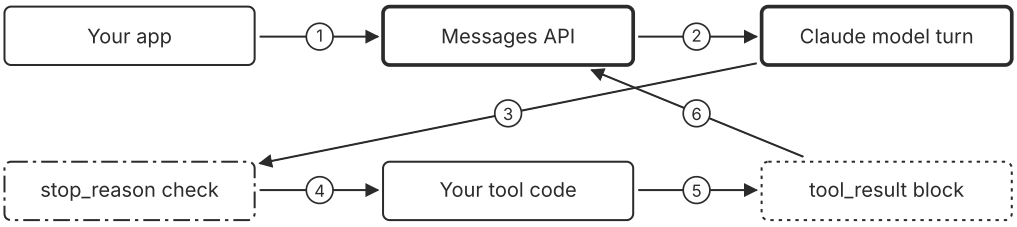
Tool call loop

1. Introduction and first principle

The model requests a tool call, and your code runs the operation and returns the result.

- The model never runs anything on its own.[13]
- A tool_use block includes the tool name and a JSON object of arguments.[13]
- The loop continues while stop_reason is tool_use.[13]
- The loop exits on end_turn, max_tokens, stop_sequence, or refusal.[13]
- A paused server-side loop returns stop_reason pause_turn.[13]
- Anthropic runs server-executed tools such as web_search, web_fetch, code_execution, and tool_search.[13]

2. Architecture



Schematic drawn from docs. Arrows: 1. send tools and messages. 2. generate turn. 3. return stop_reason. 4. tool_use: run each tool. 5. format output. 6. send next request. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Send the tools array and the user message in one request.
2. Claude returns a tool_use block with a tool name and a JSON object of arguments.
3. Your code runs the tool.
4. Claude never receives your implementation of the tool.
5. Your code sends the output back in a tool_result block.
6. Repeat while stop_reason is tool_use.
7. Stop the loop on end_turn, max_tokens, stop_sequence, or refusal.

4. Pros and cons

Pros

- Your app keeps control of execution and permissions.
- The tool schema gives typed inputs to your code.
- Server tools run on Anthropic infrastructure with no code on your side.

Cons

- Each client tool call adds one round trip.
- Your code must enforce a stop rule for the loop. (Inference)
- Tool results count toward the context window.

5. Use when, do not use when

Use when

- The task needs an action with side effects, such as an update to a record.
- The task needs fresh or external data.

Do not use when

- The model can answer from its training. No tool call is needed.
- A single prompt answers the question and no action runs.

6. Constraint families

Latency

Run parallel tool_use blocks concurrently, and return all results in one user message. Combination when one choice is not enough. Use server tools to avoid a client round trip.

Cost

Cache the stable tool definitions with prompt caching.

Governance

Run side-effect tools in your code behind your permission checks. Combination when one choice is not enough. Require human approval for writes. (Inference)

Scale

Defer large tool sets with the tool search tool.

Operations

Log each tool_use block and its result. Combination when one choice is not enough. Record the request ID of each turn.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

F2. Expect a scenario where the model must take an action. The credited answer runs the tool in app code and returns a tool_result. (Inference)

If stop_reason is tool_use, run the tools before you read the text as the final answer.

Run every client tool in your code and return each result in a tool_result block.

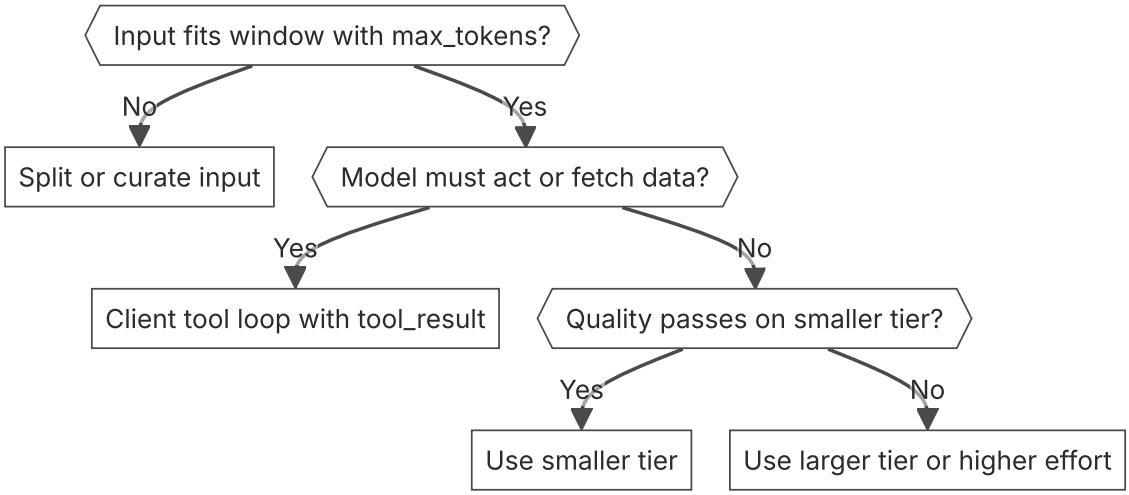
9. Trap

A design where the model calls the external API directly looks simpler. The model never runs anything on its own, so your app must run the client tool.

10. Not in source

- Maximum number of tool calls per turn. Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

F3 Software and distributed systems

Domain F, Foundations. Task statement: Design model calls as remote calls with timeouts, retries, error handling, and streams. (Inference: task wording from repo lesson 7-1A, not from the exam guide)

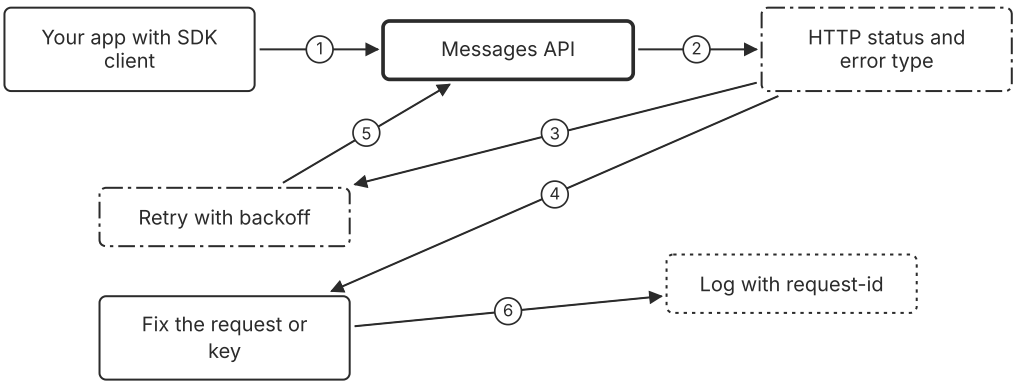
Error classes and retry

1. Introduction and first principle

Each error type tells the caller whether a retry can help.

- The official SDK retries connection errors, rate limits, and 5xx errors twice by default.[15]
- When a rate limit response has a retry-after header, wait that many seconds before retrying.[10]
- A 500 api_error means an unexpected internal error, and the docs say to retry with exponential backoff.[15]
- A 529 overloaded_error means the API is temporarily overloaded.[15]
- Every API response includes a request-id header.[15]
- The Messages API accepts a request up to 32 MB.[15]
- A 413 request_too_large error means the request exceeds the size limit.[15]

2. Architecture



Schematic drawn from docs. Arrows: 1. send request. 2. return status and type. 3. 429, 500, or 529. 4. 400 or 401. 5. resend after delay. 6. record request-id. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. The API returns an HTTP status code and an error type for each failure.
2. A 400 invalid_request_error means the request needs a fix, so a retry does not help.
3. A 401 authentication_error means the key needs a fix, so a retry does not help.
4. A 429 rate_limit_error can succeed after a wait.
5. A 500 api_error or a 529 overloaded_error is a transient 5xx failure, so a retry with backoff can help.
6. Read the retry-after header and wait that many seconds before the next attempt.
7. Record the request-id from each failed response.

4. Pros and cons

Pros

- Typed errors separate faults that need a fix from faults that clear with time.
- The official SDK retries transient failures with backoff by default.
- The request ID links an error to the server-side record.

Cons

- A spend-cap 429 does not clear with a retry.
- Retries add load when the service is already under stress.

- An error can arrive inside a 200 stream after the status line.

5. Use when, do not use when

Use when

- A call fails with 429, 500, or 529 and the request is valid.
- A service calls the API and needs a bounded retry policy.

Do not use when

- The error is a 400 invalid_request_error. Fix the request first.
- The 429 has no retry-after header and comes from a spend cap.

6. Constraint families

Latency

Set a fixed number of retries for each request. Combination when one choice is not enough. Stop retries when the user time budget ends. (Inference)

Quota

Honor the retry-after header on 429 responses. Combination when one choice is not enough. Request a tier increase when 429 retries stay high.

Scale

Increase traffic in steps to avoid acceleration limits.

Operations

Log the request ID and error type for each failure. Combination when one choice is not enough. Alert when 529 errors rise above a set rate. (Inference)

7. Native cloud services

Amazon Bedrock

The default quota is in input tokens per minute. AWS enforces requests per minute.[5]

Claude Platform on AWS

Each response carries an AWS request ID and an Anthropic request ID.[6]

Claude Models on Google Cloud (Google managed)

Google Cloud manages rate limits and quotas for this platform.[7]

Microsoft Foundry

Foundry does not send Anthropic rate limit headers, so back off on the 429 status.[9]

8. Exam domain and task statement

F3. Expect a stem that shows a call that fails. The credited answer matches the error class to a fix in the caller. (Inference)

A 429 with retry-after means wait. A 429 with no retry-after header points to a spend cap.

Retry 429 and 5xx errors with a limit and backoff. Fix 400 and 401 errors first.

9. Trap

A prompt that says try again looks like a retry policy, but the model cannot resend the request or read a retry-after header.

10. Not in source

- Default backoff delay in seconds for the SDK. Not in source.
- Whether a 529 error uses a different retry limit than a 500 error. Not in source.

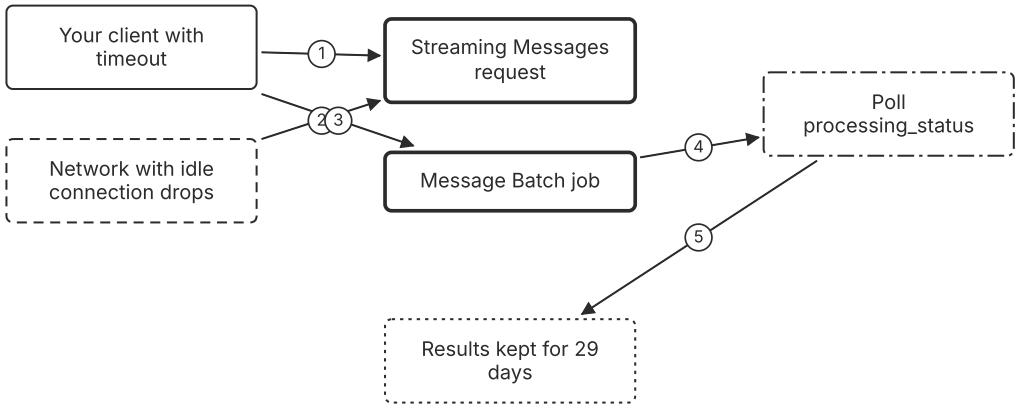
Timeouts and long requests

1. Introduction and first principle

A request that runs too long can lose its connection, so long work uses a stream or a batch.

- The SDK validates that Messages requests that do not stream are not expected to exceed a 10-minute timeout.[15]
- Some networks drop idle connections after a variable period.[15]
- Most batches complete within 1 hour.[16]
- Batches expire if processing does not complete within 24 hours.[16]
- Batch results are available for 29 days after creation.[16]
- The Messages API accepts a request up to 32 MB.[15]
- The Batch API accepts a request up to 256 MB.[15]

2. Architecture



Schematic drawn from docs. Arrows: 1. read events. 2. idle drops risk. 3. create batch. 4. check status. 5. download when ended. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Use streaming for a long response, so your app reads events as they arrive.
2. The SDK checks that a Messages request that does not stream is not expected to pass a 10-minute timeout.
3. Set a TCP keep-alive on direct integrations to reduce idle drops.
4. Create a Message Batch for work that the user does not wait for.
5. Poll the batch until processing_status is ended.
6. Download the results within 29 days of creation.

4. Pros and cons

Pros

- Streaming lets your app process events as they arrive.
- Batches let the caller poll for results instead of one open connection.
- The SDK can return the complete message from a stream.

Cons

- A stream can fail mid-response, so the app must handle partial output.
- Batches expire if processing does not finish within 24 hours.
- Batch wait times vary, and most batches finish within 1 hour.

5. Use when, do not use when

Use when

- The response can run for many minutes and the user reads it as it arrives.
- A job has many requests that can wait up to a day.

Do not use when

- The user waits for a short answer. A standard request works.
- The result must arrive in seconds. A batch is the wrong choice.

6. Constraint families

Latency

Stream long responses that users read. Combination when one choice is not enough. Use batches when the user does not wait.

Cost

Use batches for work that can wait, because batch requests cost 50 percent less.

Quota

Keep Messages API requests under 32 MB. Combination when one choice is not enough. Use the Batch API for payloads up to 256 MB.

Operations

Track processing_status and the expiry time of each batch. Combination when one choice is not enough. Store the results before the 29-day window ends.

7. Native cloud services

Amazon Bedrock

Bedrock streams over standard server-sent events on its Messages API endpoint.[5]

Claude Platform on AWS

Batch operations are CloudTrail data events, so they need explicit logging setup.[6]

Microsoft Foundry

Entra ID tokens typically expire after 1 hour, so long jobs need a token refresh.[9]

Claude Models on Google Cloud (Google managed)

Timeout values and batch wait times on this platform: Not in source.[7]

8. Exam domain and task statement

F3. Expect a long job and a user who either waits or does not wait. The credited answer picks streaming or a batch by who waits. (Inference)

A Messages request that does not stream and may run over 10 minutes needs streaming or a batch.

Stream when the user waits, and batch when the caller can poll later.

9. Trap

A longer client timeout looks like the fix for long calls, but idle connections can drop on some networks no matter the timeout.

10. Not in source

- Idle connection timeout value for each network. Not in source.
- Exact wait time for one batch request. Not in source.

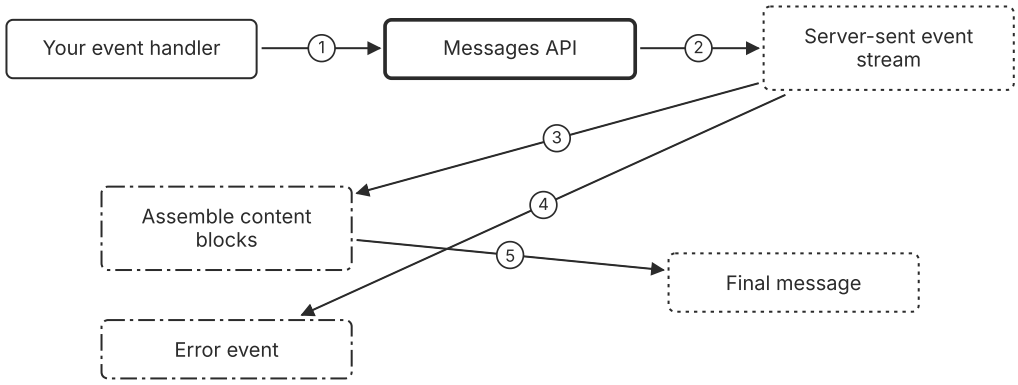
Streaming events and errors

1. Introduction and first principle

A stream sends typed events after a 200 status, so an error can arrive inside the stream.

- An error can occur after the API returns a 200 response during a server-sent event stream.[15]
- An overloaded_error in a stream would normally correspond to HTTP 529.[17]
- Event streams can include any number of ping events.[17]
- Resume of an interrupted stream is documented for Claude 4.5 models and earlier.[17]
- The Bedrock Messages API endpoint uses standard SSE streaming.[5]

2. Architecture



Schematic drawn from docs. Arrows: 1. request stream. 2. send events. 3. apply delta events. 4. overloaded_error event. 5. assemble message. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. The API sends server-sent events after a 200 status.
2. Ping events can appear at any point in the stream.
3. Delta events add partial content to a content block.
4. An error event can arrive in the stream, for example an overloaded_error.
5. Handle error events in the same loop that handles content events.
6. The SDK can assemble the complete message from the stream.
7. For Claude 4.5 models and earlier, an interrupted stream can resume from the point where it stopped.

4. Pros and cons

Pros

- Partial output reaches the user early.
- The SDK returns a complete message when the app needs no events.
- An overloaded_error in a stream uses the same type name as HTTP 529.

Cons

- Error handling must cover failures that arrive after the status line.
- Partial text can show before the stream ends.
- Resume is documented only for Claude 4.5 models and earlier.

5. Use when, do not use when

Use when

- The user reads output while the model writes it.
- The app acts on partial tool input before the response ends.

Do not use when

- A background job needs only the final message.
- The client cannot parse server-sent events.

6. Constraint families

Latency

Stream long responses to show the first tokens early. Combination when one choice is not enough. Use the SDK helper when the app needs no events.

Governance

Treat streamed text as unverified until the final message arrives. (Inference)

Skill

Handle each event type in code.

Operations

Log each error event type and the request ID. Combination when one choice is not enough. Send a new request on newer models, because resume is documented only for Claude 4.5 and earlier. (Inference)

7. Native cloud services

Amazon Bedrock

The Messages API endpoint uses standard server-sent events, unlike the InvokeModel integration.[5]

8. Exam domain and task statement

F3. Expect an option that treats a stream as one full response. The credited answer handles error events and partial text. (Inference)

An overloaded_error event inside a stream corresponds to HTTP 529 in a call that does not stream.

Handle error events inside the stream loop. Do not rely on the HTTP status check alone.

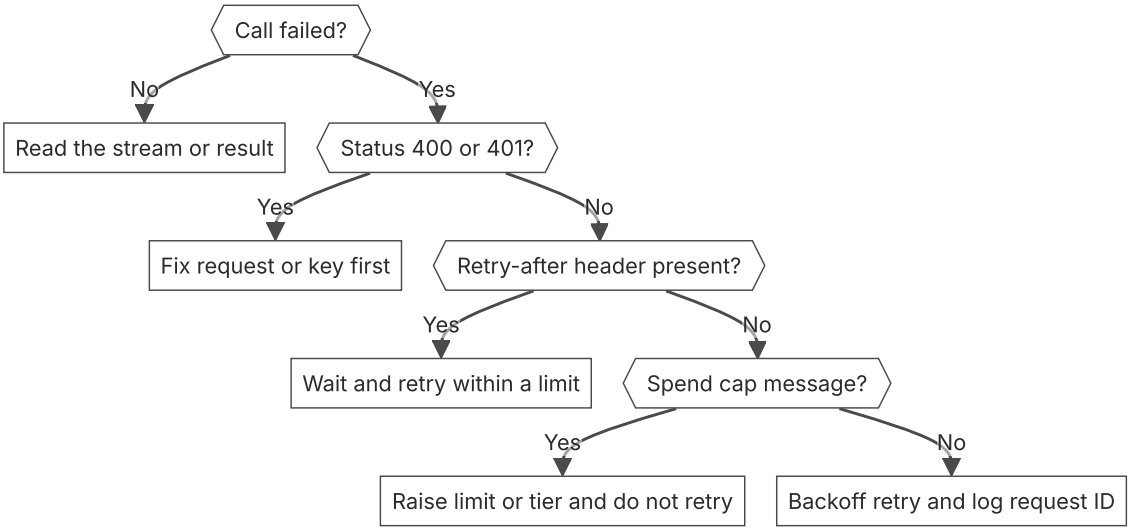
9. Trap

Only the HTTP status code looks sufficient, but a stream can fail after the 200 status.

10. Not in source

- Maximum number of ping events in a stream. Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

F4 Security and identity

Domain F, Foundations. Task statement: Choose the credential, the workspace scope, and the data controls for each caller. (Inference: task wording from repo lesson 7-2A, not from the exam guide)

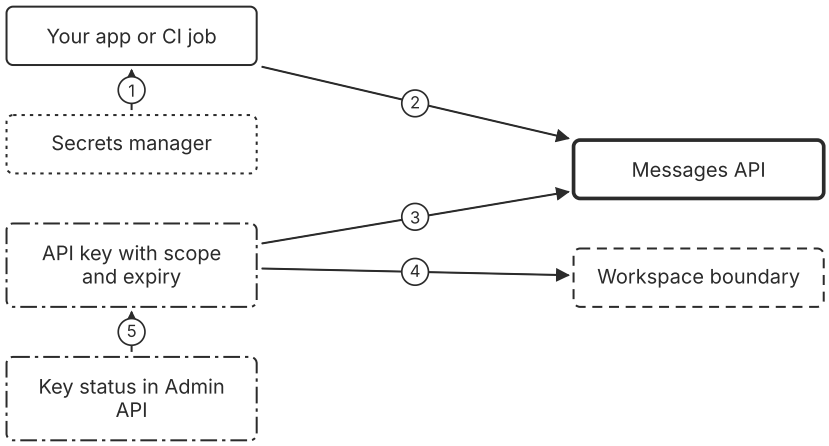
API keys and workspace scope

1. Introduction and first principle

An API key is a bearer secret, so its scope and expiration limit the damage from a leak.

- API keys start with sk-ant- and the Console shows the full key only once.[18]
- Key expiration is a preset of 3 hours, 1 day, 7 days, or 30 days, a custom duration, or Never.[19]
- An expired key returns 401 authentication_error and cannot be reactivated.[19]
- Anthropic emails the key creator 7 days before expiration for keys with a lifetime of at least 14 days.[19]
- Admin API keys start with sk-ant-admin01-.[20]
- Short-term bearer tokens on Amazon Bedrock have a 12-hour maximum.[5]

2. Architecture



Schematic drawn from docs. Arrows: 1. load secret at runtime. 2. Authorization Bearer header. 3. authenticates request. 4. scoped to one workspace. 5. reports status and expires_at. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Create the key in the Console and select its expiration.
2. Choose a service account key for shared or automated workloads.
3. Scope the key to one workspace when the workload needs only that workspace.
4. Store the key in a secrets manager, not in source code.
5. Send the key as a bearer token in the Authorization header.
6. Rotate the key before its expiration date.
7. Disable a suspected leaked key at once.

4. Pros and cons

Pros

- A scoped key limits access to one workspace.
- Expiration limits how long a leaked key works.
- Identity-backed keys become invalid when their identity leaves the organization.

Cons

- A static key stays valid until it expires or is disabled.

- The Console shows the full key once, so a lost key needs a new key.
- Workspace keys are legacy keys and remain usable after their creator leaves.

5. Use when, do not use when

Use when

- A developer runs scripts with a personal key for their own work.
- A shared service needs its own identity. Create a service account key.

Do not use when

- A cloud workload has a platform identity. Use federation instead.
- A team shares one key. The key acts as one person and breaks when that person leaves.

6. Constraint families

Data location

Choose a workspace region on Claude Platform on AWS when residency matters. Combination when one choice is not enough. Send inference_geo us for US inference on Claude 4.6 and later models.

Isolation

Scope keys to one workspace for each environment. Combination when one choice is not enough. Use separate workspaces for tenants that must not share usage.

Governance

Use a service account key for shared workloads. Combination when one choice is not enough. Rotate keys before their expiration dates.

Operations

Review key status and expires_at with the Admin API. Combination when one choice is not enough. Use an Admin API key only for the admin endpoints.

7. Native cloud services

Amazon Bedrock

Short-term bearer tokens have a 12-hour maximum and are the least preferred option.[5]

Claude Platform on AWS

SigV4 signed requests with IAM are the primary method, and API keys are also supported.[6]

Claude Models on Google Cloud (Google managed)

Requests use Google Cloud credentials, and the caller identity needs the aiplatform.endpoints.predict permission.[7]

Claude Models on Google Cloud (Anthropic managed)

Each request carries a Google OAuth 2.0 access token, and three IAM roles control access.[8]

Microsoft Foundry

Foundry supports API keys and Microsoft Entra ID tokens, with Azure RBAC roles.[9]

8. Exam domain and task statement

F4. Expect a leaked key or a shared key in a scenario. The credited answer uses a scoped, identity-backed key or federation. (Inference)

A 401 authentication_error can mean a revoked, malformed, or expired key.

Use a scoped, time-limited, identity-backed key, and store it in a secrets manager.

9. Trap

A key in a prompt or a config file looks convenient, but the model reads the prompt and the file exposes the secret.

10. Not in source

- Key rotation interval that Anthropic recommends. Not in source.
- Maximum number of keys for each workspace. Not in source.

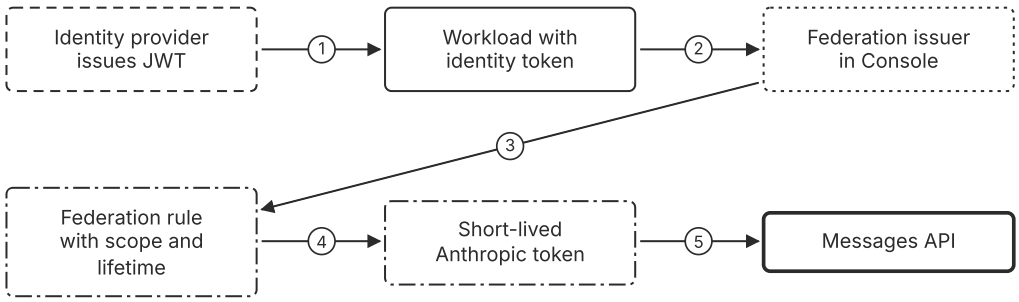
Workload identity federation

1. Introduction and first principle

A workload presents a signed token from its own identity provider and receives a short-lived Anthropic token in return.

- The default token lifetime in a federation rule is 3,600 seconds.[21]
- The minted token lifetime is never less than 60 seconds.[21]
- The SDK starts an advisory refresh at expiry minus 120 seconds.[21]
- The SDK starts a mandatory refresh at expiry minus 30 seconds.[21]
- The default authorization scope is workspace:developer.[21]
- The federation setup wizard listens for a token exchange for 15 minutes.[21]
- Outbound web identity federation on AWS is disabled by default.[6]
- A federation rule accepts token_lifetime_seconds from 60 to 86400, with a default of 3600.[21]

2. Architecture



Schematic drawn from docs. Arrows: 1. issue JWT. 2. present JWT. 3. match claims. 4. mint token. 5. bearer token on requests. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Register the identity provider as a federation issuer in the Console.
2. Create a federation rule with match conditions for the workload.
3. Set the authorization scope and the token lifetime in the rule.
4. The workload presents its signed JWT for a token exchange.
5. Anthropic checks the JWT signature and the rule match.
6. Anthropic returns a short-lived token bound to a service account.
7. The SDK refreshes the token before it expires.

4. Pros and cons

Pros

- No static secret needs storage or rotation.
- Tokens expire in minutes instead of never.
- The SDK re-reads a rotated token file on each exchange.

Cons

- Federation is only as strong as the upstream identity provider.
- Setup needs the admin, owner, or primary owner role.
- Each environment usually needs its own issuer.

5. Use when, do not use when

Use when

- A CI job or Kubernetes workload already has a platform identity.
- A production workload must avoid long-lived API keys.

Do not use when

- A developer needs access for a local script. A personal key suits own development.
- The identity provider publishes no key set. Anthropic cannot verify the JWT.

6. Constraint families

Latency

The SDK starts a refresh 120 seconds before expiry. Combination when one choice is not enough. Expect one token exchange at each workload start. (Inference)

Isolation

Register one issuer for each environment. Combination when one choice is not enough. Create a separate rule for each service account.

Governance

Set the scope in each rule to the least access the workload needs. Combination when one choice is not enough. The default scope is workspace developer, the same as a workspace API key.

Operations

Mint a fresh JWT for each exchange, because JWTs with a jti claim are single-use by default. Combination when one choice is not enough. Check the authentication history page when an exchange fails.

7. Native cloud services

Claude Platform on AWS

AWS STS outbound web identity federation is disabled by default and needs one enable command per account.[6]

Amazon Bedrock

Bedrock uses IAM assumed roles, and AWS STS issues temporary credentials.[5]

Claude Models on Google Cloud (Google managed)

Google Cloud credentials for the caller identity authenticate requests.[7]

Claude Models on Google Cloud (Anthropic managed)

Each request carries a Google OAuth 2.0 access token with the cloud-platform scope.[8]

Microsoft Foundry

Microsoft Entra ID tokens authenticate requests, and they typically expire after 1 hour.[9]

8. Exam domain and task statement

F4. Expect a CI or Kubernetes scenario that asks for no static secret. The credited answer uses federation with a short-lived token. (Inference)

A jti_reused failure means the JWT was presented before, so mint a new JWT for the exchange.

Use federation when the workload has a trusted identity, and use a personal key only for development.

9. Trap

A long-lived key in CI secrets looks simpler, but it is a static secret that stays valid until someone rotates or disables it.

10. Not in source

- Time that the Console keeps authentication history. Not in source.

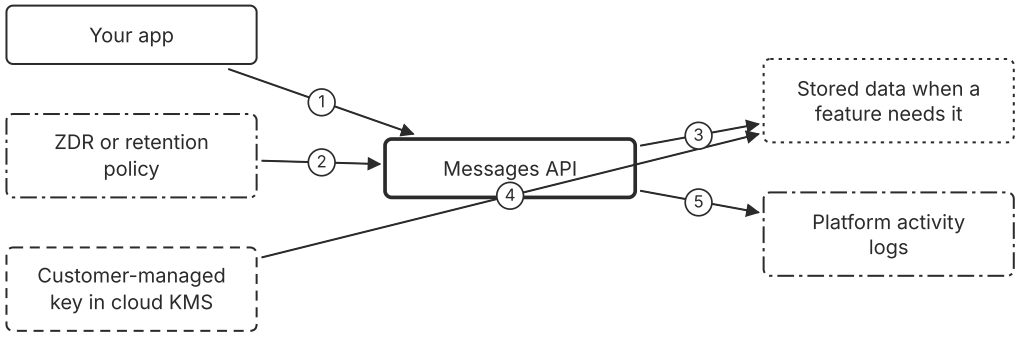
Data retention and encryption

1. Introduction and first principle

A data control names who stores data, for how long, and which key protects it.

- Covered Models require 30-day data retention.[22]
- Each new organization needs ZDR enabled separately by the account team.[22]
- Under ZDR, Anthropic does not store customer prompts or responses at rest after the API response is returned.[22]
- Retained data is never used for model training without express permission.[22]
- CMEK enablement is permanent and can cause irreversible data loss.[23]
- Anthropic keeps no copy of the customer key.[23]
- Anthropic recommends logging activity on at least a 30-day rolling basis.[5]

2. Architecture



Schematic drawn from docs. Arrows: 1. send prompts. 2. no storage at rest under ZDR. 3. store only when required. 4. encrypts stored data. 5. record activity. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Check whether each feature is eligible for zero data retention.
2. Ask the account team to enable ZDR for the organization.
3. Expect 30-day retention for Covered Models, which sit outside ZDR.
4. Choose a customer-managed key in AWS KMS, Google Cloud KMS, or Azure Key Vault when the contract requires one.
5. CMEK enablement is permanent, so confirm the key plan first.
6. Keep activity logs for at least 30 days on a rolling basis.

4. Pros and cons

Pros

- Under ZDR, Anthropic does not store prompts or responses at rest after the response returns.
- A customer-managed key gives the customer control of the encryption key.
- Platform logs record activity for investigations.

Cons

- The Console and some features sit outside ZDR.
- A CMEK misconfiguration can cause permanent data loss.
- CORS is not supported for ZDR organizations, so browser apps need a backend proxy.

5. Use when, do not use when

Use when

- Regulated data passes through the API and the contract forbids storage at rest.

- A customer requires control of the encryption key.

Do not use when

- Sensitive data flows through the Console playground. ZDR does not cover the Console.
- The team has no key recovery plan for CMEK.

6. Constraint families

Data location

Pick the inference geography and the workspace region that residency needs. Combination when one choice is not enough. US inference applies a 1.1 times pricing multiplier on Claude 4.6 and later models.

Isolation

Use CMEK only where the contract requires it.

Governance

Name one owner for each retention and key decision. Combination when one choice is not enough. Record the ZDR enablement date for audit. (Inference)

Operations

Keep activity logs for at least 30 days. Combination when one choice is not enough. Turn on Data Access logs on Google Cloud, which are off by default.

7. Native cloud services

Amazon Bedrock

Data handling follows Amazon Bedrock data protection, and CloudWatch and CloudTrail log activity.[5]

Claude Platform on AWS

The retention policy matches the first-party API, and ZDR is available on request. A separate CMEK setup page covers this platform.[6]

Claude Models on Google Cloud (Google managed)

Agent Platform governs data handling, and request-response logging can record prompts and completions.[7]

Claude Models on Google Cloud (Anthropic managed)

The retention policy matches the API, and ZDR is available on request. Data Access logs are off by default.[8]

Microsoft Foundry

Azure logging is configured in your Azure subscription, for example with Azure Log Analytics.[9]

8. Exam domain and task statement

F4. Expect a regulated data scenario with a retention or key requirement. The credited answer names ZDR or CMEK and the account team step. (Inference)

CMEK enablement is permanent, so the answer confirms the key plan before the change.

Enable ZDR or CMEK only through the account team, after you confirm the eligible features.

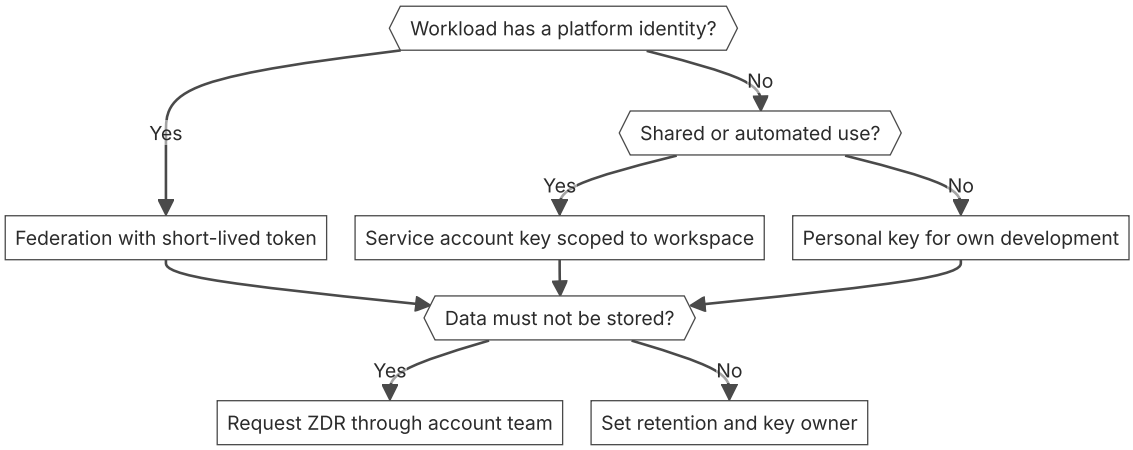
9. Trap

Platform logs look like a data control, but logs do not enable zero data retention.

10. Not in source

- Retention period for Console usage data. The docs say the Console is outside ZDR and name no period. Not in source.

Decision flow



Unit items not in source

- Acceptance rules for the federation rule match conditions beyond the documented fields. Not in source.

D1.1 Business problem to Claude solution

Domain D1, Solution design and architecture. Task statement: Turn a business request into a Claude solution with a stated outcome, constraints, and success criteria. (Inference: wording from secondary sources)

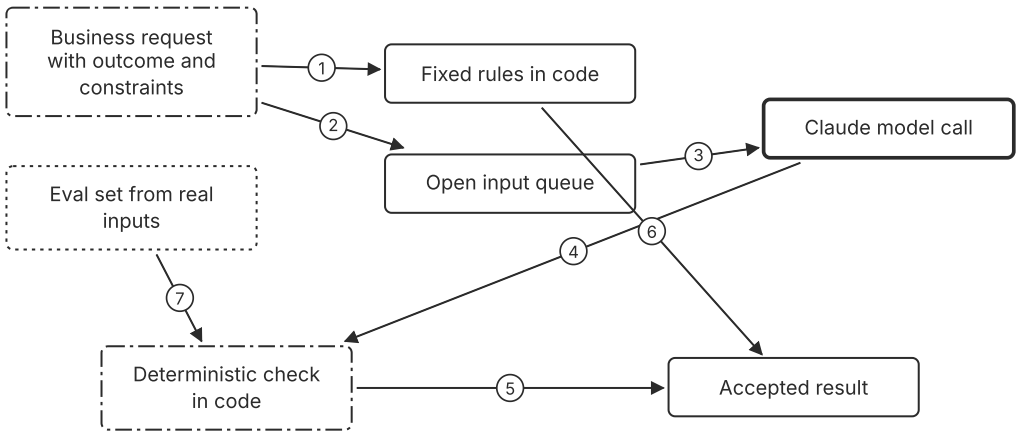
Use-case fit and lane choice

1. Introduction and first principle

Send fixed mapping to code and send open input to Claude only when a check in code can verify the output. (Inference)

- Claude Haiku 5.5 base input price is 0.10 dollars per million tokens for prompts up to 100,000 tokens.[3]
- Claude Haiku 5.5 output price is 0.50 dollars per million tokens for prompts up to 100,000 tokens.[3]
- Claude Haiku 5.5 prompts over 100,000 tokens pay higher prices.[3]
- Claude Opus 5.5 base input price is 4 dollars per million tokens.[3]
- The Message Batches API discount is 50 percent on input and output tokens.[3]
- A batch holds up to 100,000 Message requests or 256 MB, whichever is reached first.[16]

2. Architecture



Schematic drawn from docs. Arrows: 1. fixed input types. 2. open input types. 3. send input. 4. draft result. 5. pass. 6. rule result. 7. pass threshold. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write the outcome, the hard constraints, and the owner before you choose a model.
2. Sort each input type into fixed mapping or open input.
3. Send fixed mapping to code rules with no model call.
4. Send open input to Claude only when a code check can verify the output.
5. Set one metric and one threshold for each success criterion.
6. Name an owner who signs the result when the threshold fails.

4. Pros and cons

Pros

- Fixed rules return the same output for the same input. (Inference)
- Claude handles input whose wording and layout vary.
- A code check limits the damage from a wrong model output. (Inference)

Cons

- Two code paths need maintenance. (Inference)
- The fixed path misses input types that no rule covers. (Inference)

- Each path needs its own success criteria and eval set. (Inference)

5. Use when, do not use when

Use when

- The inputs vary, and a check in code can verify the output.
- The task needs language analysis that rules cannot cover.
- Several input types arrive in one queue.

Do not use when

- The input follows a fixed format, and a rule covers every case.
- No check in code can verify the output, and the error cost is high.
- The team has no success criteria or eval set yet.

6. Constraint families

Latency

Choose Claude Haiku 5.5 when the application needs low latency. Combination when one choice is not enough. Keep fixed steps in code when they need the lowest latency. (Inference)

Cost

Start with Claude Haiku 5.5, which the docs list for the lowest latency and price. Combination when one choice is not enough. Move to a larger model only when evals show a capability gap.

Governance

Name one owner for each success criterion. (Inference) Combination when one choice is not enough. Run the fit test again when the input mix or the error cost changes. (Inference)

Scale

Use the Message Batches API for large volumes that do not need an immediate answer. Combination when one choice is not enough. Expect most batches to finish within 1 hour. A batch that does not finish within 24 hours expires.

Skill

Write the lane rules with the team that owns the process. (Inference)

7. Native cloud services

Amazon Bedrock

Access criteria are set per model. The page lists Claude Haiku 4.5 as open to all customers. Haiku 5.5 access is Not in source.[5]

Claude Platform on AWS

Billing runs through AWS Marketplace in Claude Consumption Units. Anthropic manages rate limits.[6]

Claude Models on Google Cloud (Google managed)

Google Cloud bills the usage as Agent Platform usage. Google Cloud manages rate limits.[24]

Claude Models on Google Cloud (Anthropic managed)

Google Cloud Marketplace bills the usage. Anthropic manages rate limits.[8]

Microsoft Foundry

Billing runs through the Azure Marketplace in Claude Consumption Units.[9]

8. Exam domain and task statement

D1.1. Expect a business request with a cost or accuracy limit. The credited answer names the outcome, the lane, and a measurable success line. (Inference)

If the stem gives no success metric, add the metric before you pick the architecture. (Inference)

Use code for fixed mapping, and use Claude with a code check for open input.

9. Trap

A model call for a fixed format that a rule covers. It adds cost and a failure mode, and it gives no gain. (Inference)

10. Not in source

- Haiku 5.5 access on Amazon Bedrock: Not in source.
- Timeouts and quotas for each native service beyond the rate limit notes above: Not in source.
- Repo tier names Fast, Balanced, and Capable with input prices of 1, 2, and 4 dollars per million tokens: Not in source.
- Console steps: Not in source.

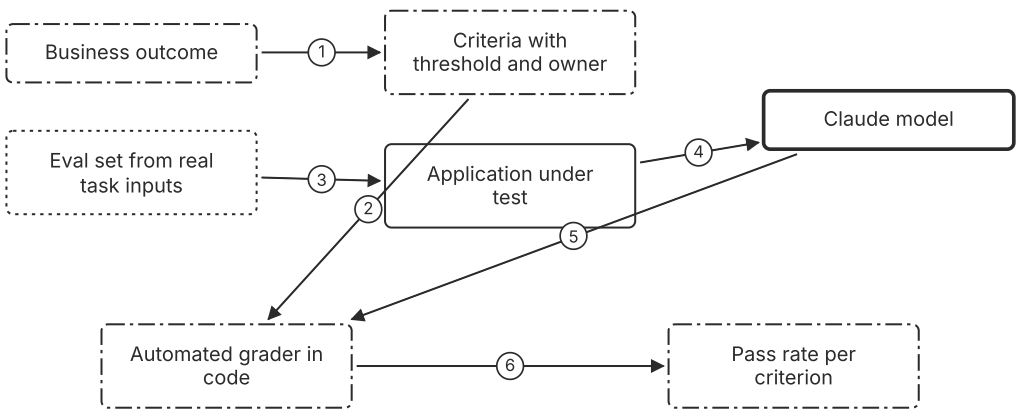
Success criteria and evals

1. Introduction and first principle

Write each success criterion as a measurable target with a threshold, and test it on an eval set before the build scales.

- The docs give an example criterion of an F1 score of at least 0.85 on a held-out test set.[25]
- The docs list specific, measurable, achievable, and relevant as the properties of a success criterion.[25]
- Most batches finish in less than 1 hour.[16]
- Batch results are available for 29 days after creation.[16]

2. Architecture



Schematic drawn from docs. Arrows: 1. translate outcome. 2. define metric. 3. test inputs. 4. model calls. 5. outputs. 6. scores. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write each criterion as a specific, measurable target.
2. Attach a threshold and a named owner to each criterion.
3. Collect test inputs that match the real task distribution.
4. Add edge cases, such as empty input and overly long input.
5. Grade outputs in code where possible, and use an LLM grader where code cannot check the output.
6. Compare each new prompt or model version against the same eval set.

4. Pros and cons

Pros

- Automated scoring supports repeated runs on each change.
- A measurable target shows whether a change helped.
- Quality and operational metrics can share one report.

Cons

- Eval sets take time to build and maintain. (Inference)
- Ambiguous cases split human graders, so the target needs a clear rule.
- LLM-based scoring needs checks of its own. (Inference)

5. Use when, do not use when

Use when

- A team must decide whether to ship a prompt or model change.
- A team compares two prompts or models on the same task.
- An output that customers read needs a quality floor. (Inference)

Do not use when

- The output has no measurable form, and no owner reads the result. (Inference)
- The team needs a result for one demo with no users. (Inference)

6. Constraint families

Cost

Use the Message Batches API for large eval runs that do not need an immediate answer. Combination when one choice is not enough. A batch that does not finish within 24 hours expires.

Governance

Name the owner who signs each threshold. (Inference)

Scale

Prefer more automated test cases over fewer hand-graded cases. Combination when one choice is not enough. Keep a small set of human-graded cases for calibration. (Inference)

Operations

Run the eval set after each prompt or model change. Combination when one choice is not enough. Keep the eval set versioned with the prompt. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D1.1. Expect a stem that asks how to prove success. The credited answer names a measurable criterion with a threshold and an eval set. (Inference)

If an option uses an adjective as the success test, prefer the option that adds a number and a test set. (Inference)

Write each criterion as a metric, a threshold, and an owner before you compare models.

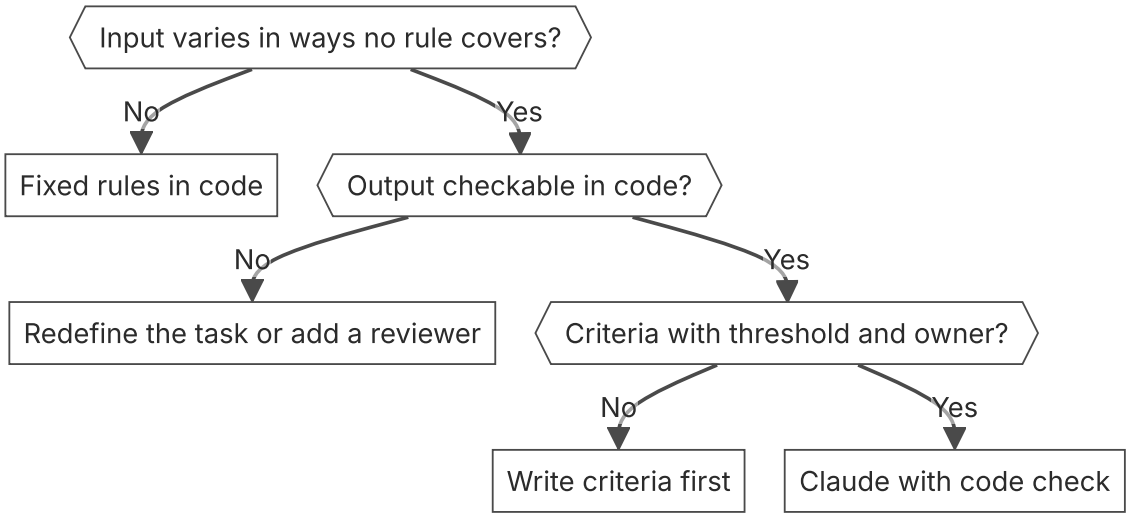
9. Trap

A general quality goal, such as good performance. It cannot pass or fail, so no owner can sign it.

10. Not in source

- Pass rate threshold for any product: Not in source.
- Repo toy holdout size of 500 invoices: Not in source.
- Console steps: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Official name for a fit test as a method: Not in source.

D1.2 End-to-end architecture

Domain D1, Solution design and architecture. Task statement: Design a Claude application with named stages, trust boundaries, state in the application, and a trace for each stage. (Inference: wording from secondary sources)

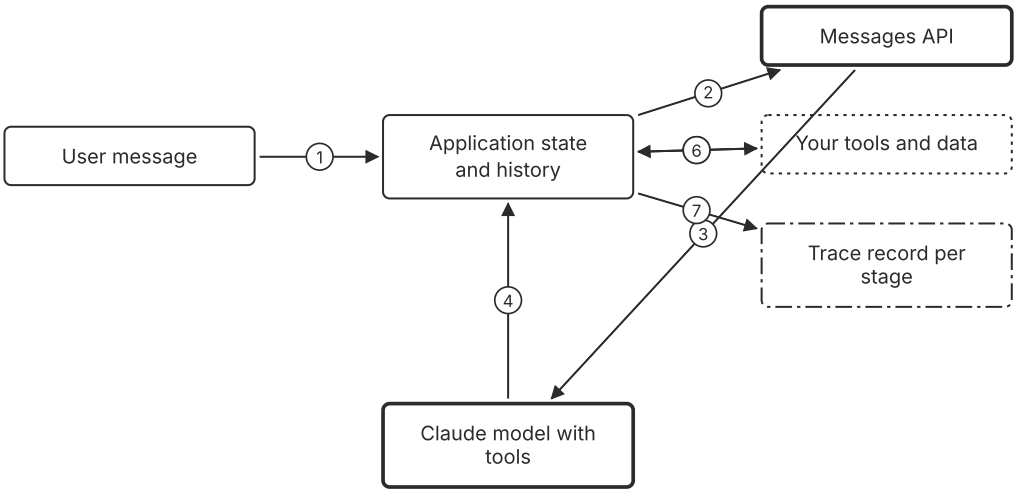
Request loop with state in the application

1. Introduction and first principle

No state persists between Messages API calls. The application resends the conversation on each call and checks each stop reason.

- The Messages API is stateless, so you always send the full conversational history.[27]
- The tool-use loop exits on end_turn, max_tokens, stop_sequence, or refusal.[13]
- A pause_turn stop reason means the work is not complete, and you send the conversation again.[13]
- The Agent SDK exports traces, metrics, and events with OpenTelemetry.[28]
- The docs state that model behavior is not a security control.[29]

2. Architecture



Schematic drawn from docs. Arrows: 1. new message. 2. full history. 3. run turn. 4. stop_reason and blocks. 5. run tool_use. 6. tool_result. 7. log stage. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Send the full conversation history with each Messages API request.
2. Read the stop_reason field before you accept the response.
3. Run each requested tool in your code and return a tool_result block.
4. Repeat the request while stop_reason is tool_use.
5. Handle max_tokens, refusal, and pause_turn as separate outcomes.
6. Log the stage name and the token counts for each call.

4. Pros and cons

Pros

- The request carries its full state, so a run can be replayed from stored inputs. (Inference)
- The stop reasons give a documented exit condition for the loop.
- The application controls each retry and each fallback. (Inference)

Cons

- The application sends more tokens as the history grows. (Inference)

- The team owns the state store and its failure modes. (Inference)
- Each stop reason needs a handler, or the loop fails without a clear cause. (Inference)

5. Use when, do not use when

Use when

- A multi-turn conversation needs context from earlier turns.
- A tool call must return to your code before the next model turn.
- The team needs to replay a run from stored inputs. (Inference)

Do not use when

- One prompt and one answer need no history and no tools. (Inference)
- A managed session already stores the state, and the team accepts its controls. (Inference)

6. Constraint families

Latency

Stream the response when the user waits for the first tokens. Combination when one choice is not enough. Use fine-grained tool streaming when the tool input must appear early.

Isolation

Keep each conversation state scoped to one user or tenant. (Inference)

Scale

Use the Message Batches API for bulk work that does not need an immediate answer.

Operations

Log each stop reason with a count per request. (Inference) Combination when one choice is not enough. Alert when the counts for max_tokens or refusal rise. (Inference)

7. Native cloud services

Amazon Bedrock

Claude in Amazon Bedrock serves the Messages API at /anthropic/v1/messages on AWS-managed infrastructure.[5]

Claude Platform on AWS

Anthropic is the data processor for inference inputs and outputs. Workspace content includes prompts, outputs, files, and batches.[6]

Claude Models on Google Cloud (Google managed)

Google operates Claude models as part of Google Cloud's Agent Platform.[24]

Claude Models on Google Cloud (Anthropic managed)

Anthropic operates the Claude API, reached through your Google Cloud project.[8]

Microsoft Foundry

Claude is available with the hosting options Hosted on Azure and Hosted on Anthropic.[9]

8. Exam domain and task statement

D1.2. Expect a stem with a failure that no stage can explain. The credited answer adds a trace for each stage and a named gate. (Inference)

If a stem shows a failure with no known stage, pick the option that adds a trace per stage. (Inference)

Keep the history, the retries, and the stop reason handling in application code.

9. Trap

A larger model or a longer prompt for a failure that no trace locates. The change treats a symptom and hides the stage that breaks. (Inference)

10. Not in source

- Default timeout for Messages API calls: Not in source.
- Latency budget for each stage: Not in source.
- Console steps: Not in source.

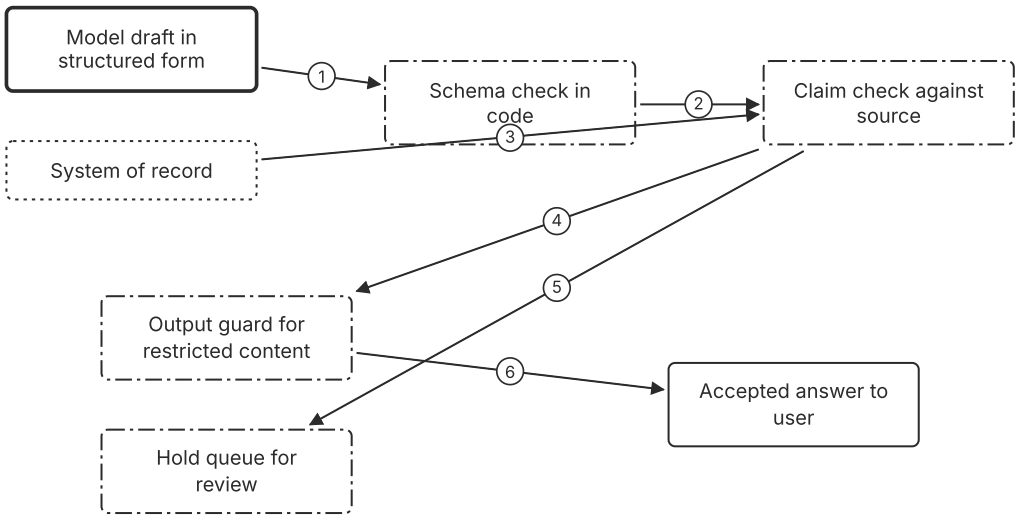
Verification and output gates

1. Introduction and first principle

A check in code compares the draft with the source and the schema before the answer leaves the system. (Inference)

- Structured outputs guarantee schema-compliant responses, because the model output is limited to the schema.[30]
- Strict tool use guarantees schema validation on tool names and inputs.[31]
- The docs state that Claude is inherently resilient to prompt injection, but model behavior is not a security control.[29]

2. Architecture



Schematic drawn from docs. Arrows: 1. structured output. 2. valid object. 3. source records. 4. pass. 5. fail. 6. clean. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Request structured outputs when the answer must match a schema.
2. Check the parsed output against the schema in code.
3. Compare each factual claim with the source record in code.
4. Run an output guard that blocks restricted content.
5. Send each failed draft to a hold queue, not to the user.
6. Log the check result for each draft.

4. Pros and cons

Pros

- Structured outputs guarantee schema-compliant responses.
- A hold queue stops a wrong draft before it reaches the user. (Inference)
- Code checks add no model tokens. (Inference)

Cons

- Schema compliance does not show that the values are correct. (Inference)

- Strict checks reject correct answers that fall outside the rule set. (Inference)
- Each gate adds latency and one more failure point. (Inference)

5. Use when, do not use when

Use when

- A wrong value triggers a payment, a data change, or an action that a patient sees. (Inference)
- A downstream system needs an object with a fixed schema.
- Each claim in the output must trace to a source document. (Inference)

Do not use when

- The output is free text with no downstream system and low risk. (Inference)
- No source record exists to compare against. (Inference)

6. Constraint families

Data location

Use the inference_geo parameter where data residency rules apply. Combination when one choice is not enough. On Claude Platform on AWS, requests with inference_geo on Claude Haiku 4.5 return a 400 error.

Governance

Route each failed draft to a named reviewer. (Inference)

Scale

Run checks in code, so that only the model call adds token cost. (Inference)

Operations

Track the rate of failed checks for each stage. (Inference) Combination when one choice is not enough. Alert when the rate rises after a prompt change. (Inference)

7. Native cloud services

All

Not in source for platform-specific schema support.

8. Exam domain and task statement

D1.2. Expect a stem with a financial or safety-critical output. The credited answer places a deterministic check between the draft and the user. (Inference)

If a stem names a schema, pick structured outputs, and add a code check for the values. (Inference)

Use a check in code for each output whose error costs money, safety, or trust.

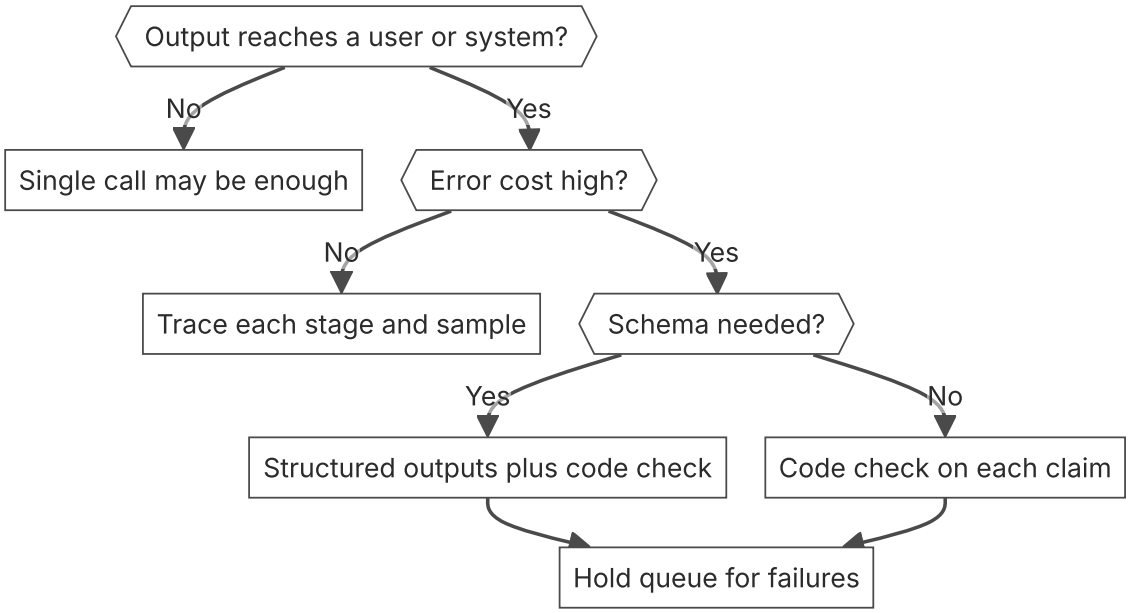
9. Trap

A prompt that asks the model to check its own draft. The check shares the failure mode of the draft, so it gives no independent control. (Inference)

10. Not in source

- Share of schema-compliant outputs that pass a value check in production: Not in source.
- Latency added by each gate: Not in source.
- Console steps: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Seven named gates as an official pattern: Not in source.

D1.3 Architectural pattern selection

Domain D1, Solution design and architecture. Task statement: Select the least autonomous pattern that meets the task constraints. (Inference: wording from secondary sources)

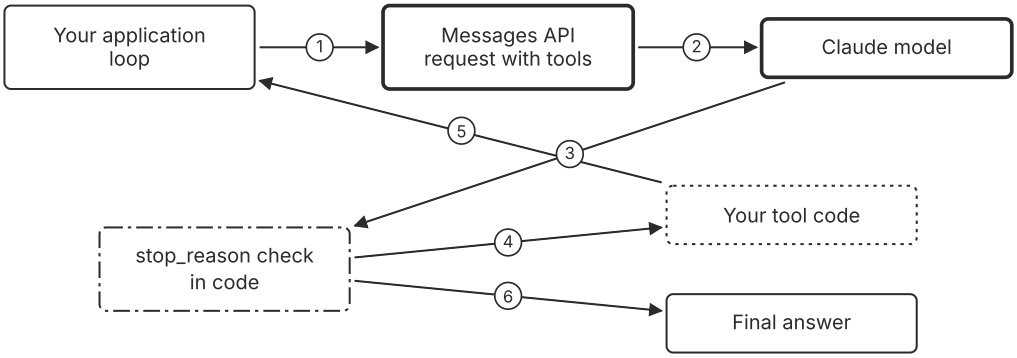
Tool-use loop in application code

1. Introduction and first principle

The model asks for a tool call, and the application runs the tool and returns the result.

- Tool use does not fit when the model can answer from training alone.[13]
- Every tool call is at least one extra round trip.[13]
- Independent, read-only tool operations are usually safe to run in parallel for lower latency.[14]

2. Architecture



Schematic drawn from docs. Arrows: 1. send history and tools. 2. run turn. 3. response. 4. tool_use block. 5. tool_result block. 6. end_turn. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Define each tool with a name, a description, and an input schema.
2. Send the request with the tools array and the user message.
3. Read stop_reason, and run each requested tool in code when it is tool_use.
4. Return each result in a tool_result block on the next request.
5. Exit the loop on end_turn, and handle any other stop reason in code.

4. Pros and cons

Pros

- Your code runs every tool, so your code controls each side effect.
- A tool schema enforces the shape of each input.
- Fixed steps stay in code, not in a model plan. (Inference)

Cons

- Every tool call is at least one extra round trip.
- The loop needs stop reason handling in code. (Inference)
- The model chooses tools, so each tool description must be precise. (Inference)

5. Use when, do not use when

Use when

- The model needs fresh or external data from a tool.
- The task needs one judgment and one side effect in your system. (Inference)
- A tool input must follow a schema.

Do not use when

- The model can answer from training alone.
- A tool call would add more delay than the answer needs. (Inference)

6. Constraint families

Latency

Keep the loop short, with one round trip for each tool call. Combination when one choice is not enough. Run independent, read-only tools in parallel.

Cost

Count one model call for each loop turn. (Inference)

Isolation

Scope each tool to the data the task needs. (Inference)

Operations

Log each tool name, its input, and the size of its result. (Inference)

7. Native cloud services

All

Not in source for platform-specific tool limits.

8. Exam domain and task statement

D1.3. Expect a task with one judgment step and a side effect. The credited answer keeps the steps in application code and calls the model at the judgment step. (Inference)

If the stem names one tool call and a fixed sequence, pick application code over an agent. (Inference)

Use a tool-use loop when the steps are known, and the model only chooses tools.

9. Trap

A full agent for one judgment step. The agent adds planning overhead that the task does not need. (Inference)

10. Not in source

- Tool count limit for a single request: Not in source.
- Console steps: Not in source.

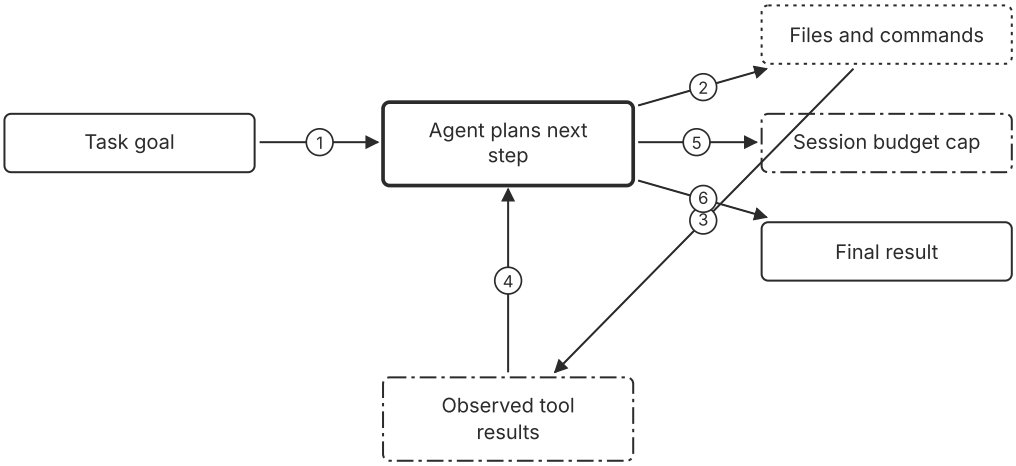
Agent loop with model-planned steps

1. Introduction and first principle

An agent plans its own steps and calls tools until the task is complete or a limit stops the run.

- An agent is an application that plans its own steps and calls tools to complete a task.[32]
- Every agent in a run uses tokens, so a session budget caps the session spend, runs included.[33]
- A session budget stops new model requests when the list cost reaches the budget.[34]
- On Claude Platform on AWS, a Managed Agents session runs without user events for up to 6 hours before it needs reauthentication.[6]

2. Architecture



Schematic drawn from docs. Arrows: 1. task. 2. tool call. 3. output. 4. next plan. 5. spend check. 6. task complete. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Give the agent a goal and the tools it can use.
2. The agent chooses the next tool call from the goal and the results so far.
3. The agent runtime or your application runs each tool and returns the result.
4. The loop continues until the agent stops or a limit applies.
5. Set a session budget so that the session pauses when its list cost reaches the cap.

4. Pros and cons

Pros

- The agent handles steps that are unknown before the run starts.
- The agent can change its plan when a tool result changes the picture.
- The Agent SDK gives an agent built-in tools, an agent loop, and context management.

Cons

- Each run can take a different path. (Inference)
- Every agent in a run uses tokens, so cost rises with the run length.
- Traces show a plan that the team did not write. (Inference)

5. Use when, do not use when

Use when

- The steps depend on results that cannot be listed in advance.
- The task is open research with unknown sources and steps. (Inference)
- A person checks the result before any side effect happens. (Inference)

Do not use when

- The steps are fixed, and a rule covers every case. (Inference)
- The error cost is high, and no check can verify each step. (Inference)

6. Constraint families

Cost

Set a session budget that caps the spend of the run. Combination when one choice is not enough. Set a lower budget for runs with fixed steps. (Inference)

Governance

Limit the agent to the tools and agents that you list. Combination when one choice is not enough. Use Managed Agents permission policies to make server-executed tools wait for approval.

Skill

Write the system prompt with clear rules for when to stop. (Inference)

Operations

Review the run traces and the session events after each run. (Inference)

7. Native cloud services

Claude Platform on AWS

Managed Agents is available. A session can run without user events for up to 6 hours, then needs reauthentication.[6]

Claude Models on Google Cloud (Anthropic managed)

Managed Agents is available, and the roles/claude.user role covers Managed Agents resources.[8]

All

Managed Agents is in beta with the beta header managed-agents-2026-04-01. Availability on Amazon Bedrock, Google managed, and Microsoft Foundry: Not in source.[35]

8. Exam domain and task statement

D1.3. Expect a task whose steps are unknown until the run starts. The credited answer uses an agent loop with a spend cap and a tool list. (Inference)

If the steps are known in advance, pick code with a model call at the judgment step, not an agent. (Inference)

Choose the least autonomous pattern that covers every step the task needs.

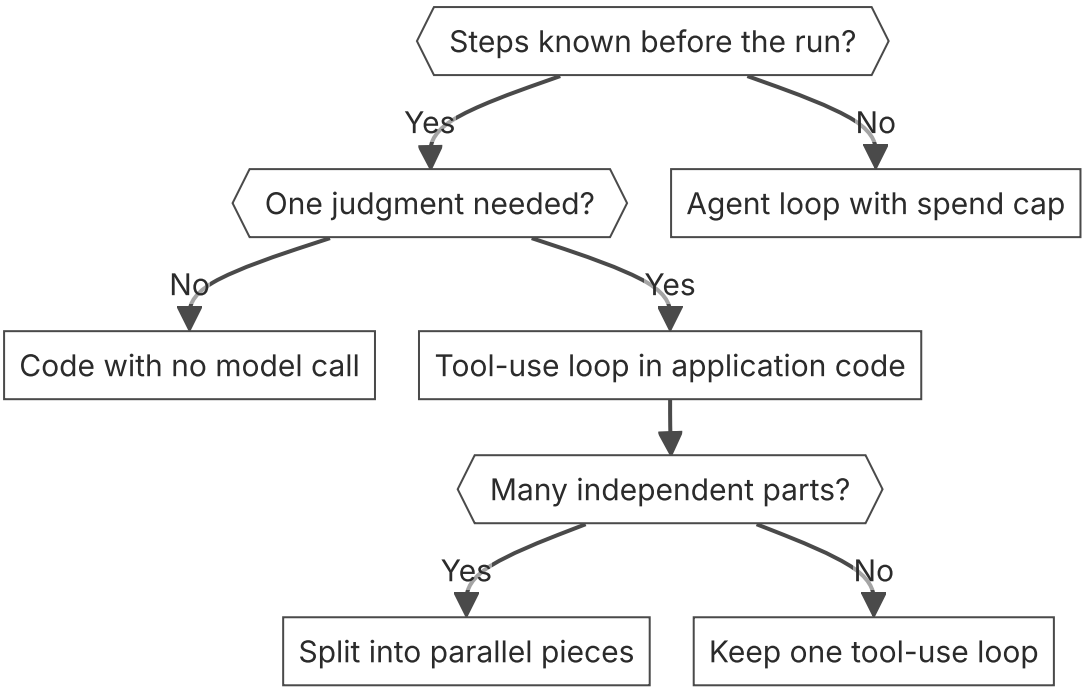
9. Trap

An agent loop for a task with fixed steps. The agent makes more model calls and varies from run to run with no gain. (Inference)

10. Not in source

- Official names for workflow, orchestrator-workers, or routing patterns: Not in source.
- Seven-axis scoring for pattern choice: Not in source. The repo lesson uses it as general practice. (Inference)
- Cost ratio of an agent run to a workflow run: Not in source.
- Console steps: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Official pattern taxonomy in the mirror: Not in source.

D1.4 Multi-agent orchestration

Domain D1, Solution design and architecture. Task statement: Design multi-agent work with clear handoffs, isolated context, and a merge step owned by one agent. (Inference: wording from secondary sources)

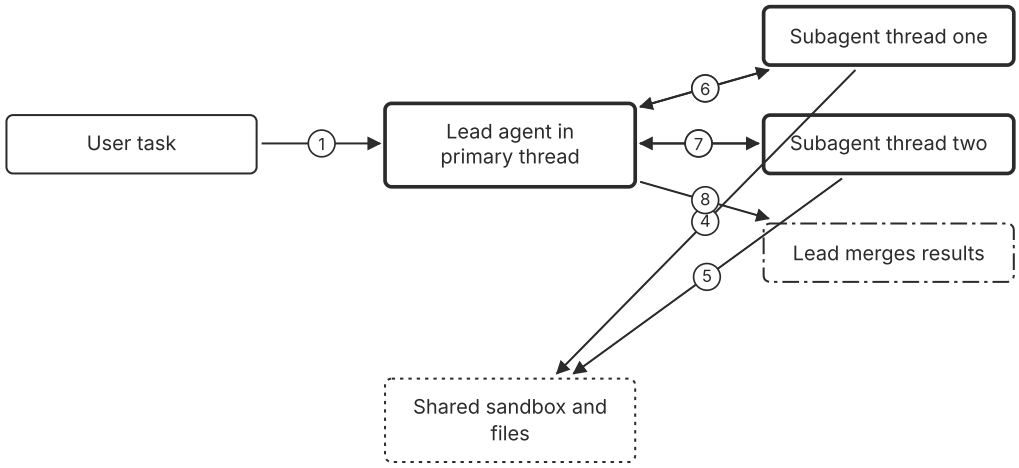
Subagent delegation

1. Introduction and first principle

Each subagent keeps its own conversation history, and the lead agent receives only the subagent's reported results.

- Delegation is one level deep, and a session can have at most 25 child threads at a time.[33]
- All subagents share the sandbox, the filesystem, and the vault credentials, and each has its own conversation history.[33]
- In the Agent SDK, only the final message of a subagent returns to the parent.[38]
- Multiagent orchestration has beta status with the beta header managed-agents-2026-04-01.[33]
- Inline agents are on by default. To allow only listed agents, set inline_agents to disabled.[33]

2. Architecture



Schematic drawn from docs. Arrows: 1. task. 2. delegate scoped task. 3. delegate scoped task. 4. file access. 5. file access. 6. final report. 7. final report. 8. combine. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. List each subagent that the lead agent can call in the multiagent block.
2. Give each subagent a system prompt and tools for one scope.
3. The lead agent delegates a bounded task to each subagent.
4. Each subagent works in its own session thread.
5. The lead agent reads each subagent report and merges the results.
6. Send a follow-up message to a subagent when its report needs more work.

4. Pros and cons

Pros

- Subagents work with isolated context, so their histories do not mix.
- Specialist prompts and tool sets keep each subagent focused.
- A subagent thread stays available for follow-up messages.

Cons

- Delegation is one level deep.

- A session can have at most 25 child threads at a time, idle ones included.
- Every agent in a run uses tokens, so cost rises with each subagent.

5. Use when, do not use when

Use when

- The work splits into well-scoped tasks that feed one goal.
- Specialists need different system prompts or tools.
- The lead agent must check in with a subagent after its report.

Do not use when

- Each subtask needs the result of the previous subtask. (Inference)
- One agent with all tools completes the task inside the latency limit. (Inference)

6. Constraint families

Latency

Parallelize independent subtasks. Combination when one choice is not enough. Compare the total latency with one agent run before you add subagents. (Inference)

Cost

Set a session budget that caps the spend of all child threads.

Governance

List the allowed subagents in the predefined_agents list of subagents. Combination when one choice is not enough. Set inline_agents to disabled, because inline agents are on by default.

Operations

Read each thread through the session event stream.

7. Native cloud services

Claude Platform on AWS

The page lists multiagent orchestration among the Managed Agents features available on this platform.[6]

All

Multiagent orchestration is in beta. Availability on the other platforms, and quotas, are Not in source.[33]

8. Exam domain and task statement

D1.4. Expect independent specialist work with a deadline. The credited answer uses a lead agent, scoped subagents, and a merge step. (Inference)

If the stem needs one coherent result, pick the option with a named lead and a merge step. (Inference)

Delegate only independent, well-scoped work, and keep one lead agent that owns the merge.

9. Trap

Three agents with no lead and no merge. Each agent returns output, and no one owns one coherent result. (Inference)

10. Not in source

- Timeout for a subagent: Not in source.
- Concurrency limit for Agent SDK subagents: Not in source.
- Rule for subagent outputs that differ: Not in source. (Inference: general practice)
- Console steps: Not in source.

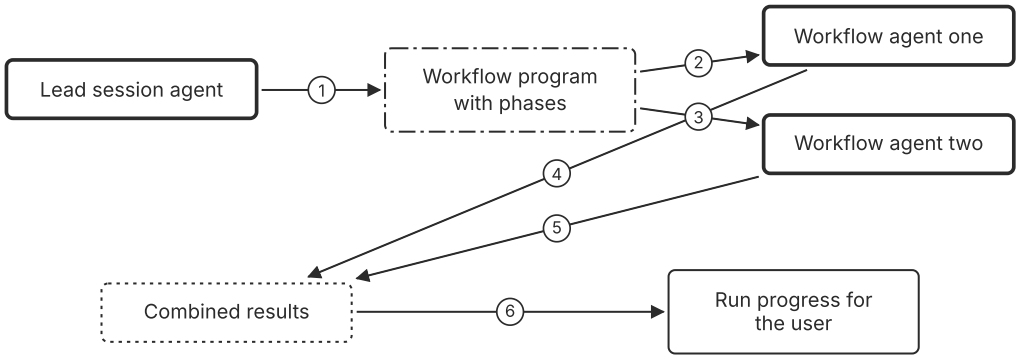
Dynamic workflow runs

1. Introduction and first principle

Claude writes a program that calls many agents in phases and combines their results without the lead agent in each step.

- With dynamic workflows, Claude writes a program that orchestrates agents without Claude's direct involvement.[33]
- The agent cannot send follow-up messages to a run's threads.[33]
- The server archives each workflow run thread by the end of its run.[33]
- Audits, migrations, deep research, and cross-checks are example workflow run uses.[33]

2. Architecture



Schematic drawn from docs. Arrows: 1. writes program. 2. phase one. 3. phase one. 4. output. 5. output. 6. run status. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Tell the agent in its system prompt when to use a workflow run.
2. The agent writes a workflow with phases and the agents for each phase.
3. The server runs the workflow in the background as one workflow run.
4. Context and results pass from one agent to the next in code.
5. Check the run by its phases, and read each thread while the run is active.

4. Pros and cons

Pros

- The main session stays free to talk with the user.
- Runs suit work that has many pieces that can run in parallel.
- Phases give named progress points for each run.

Cons

- You cannot send follow-up messages to a run's threads.
- The server archives each run thread by the end of its run.
- Each agent in a run uses tokens, so each run needs a budget.

5. Use when, do not use when

Use when

- The work has many pieces that run in parallel.
- A long task should finish sooner through parallel phases.
- Audits, migrations, deep research, and cross-checks fit a run.

Do not use when

- The lead agent must send follow-up instructions to each worker during the work.

- The task has one judgment step and no parallel pieces. (Inference)

6. Constraint families

Latency

Use a run for long work that should complete sooner. Combination when one choice is not enough. Split the run into phases with parallel agents in each phase.

Cost

Set a session budget before each run. Combination when one choice is not enough. Set the budget for the whole session, because runs count toward it. (Inference)

Governance

List the allowed agents in the predefined_agents list of workflows, and disable inline_agents. Combination when one choice is not enough. Pin each listed agent to a version number.

Operations

Read thread output while the run is active. Combination when one choice is not enough. The server archives each thread at the end of its run.

7. Native cloud services

Claude Platform on AWS

The page lists multiagent orchestration among the Managed Agents features available on this platform.[6]

All

Multiagent orchestration is in beta. Availability on the other platforms, and quotas, are Not in source.[33]

8. Exam domain and task statement

D1.4. Expect a large parallel job with a progress need. The credited answer uses a workflow run with phases and a session budget. (Inference)

If the stem asks for a follow-up to each worker, pick subagents, not a workflow run. (Inference)

Use a workflow run for many-piece parallel work, and cap its spend with a session budget.

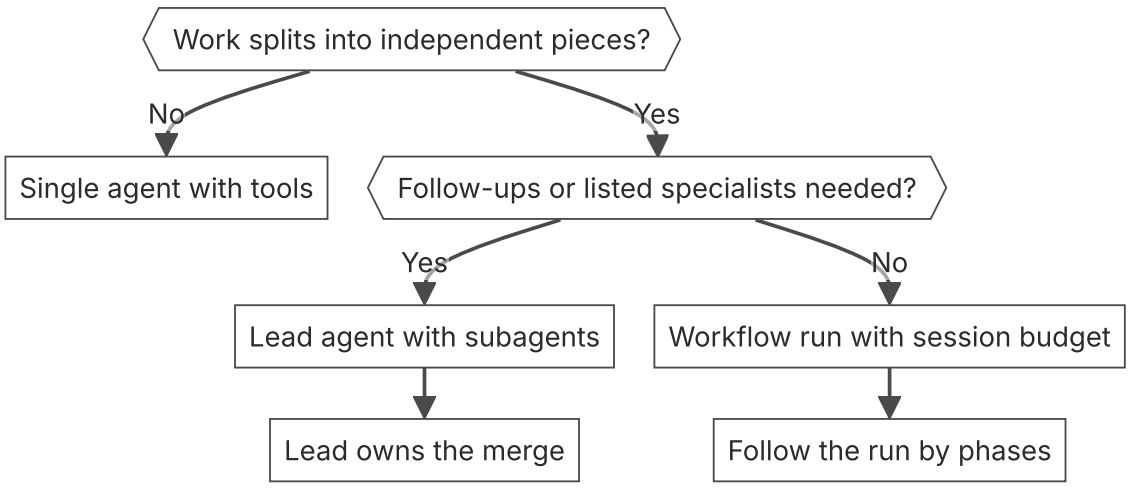
9. Trap

A chat with each agent in turn. The sequence removes the parallel work that the run could split. (Inference)

10. Not in source

- Maximum number of phases in a workflow run: Not in source.
- Maximum number of agents in a workflow run: Not in source.
- Console steps: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Team roles such as reviewer or coordinator as official terms: Not in source.

D1.5 Task decomposition

Domain D1, Solution design and architecture. Task statement: Split a large task into bounded pieces, merge the results, and check the merged result in code. (Inference: wording from secondary sources)

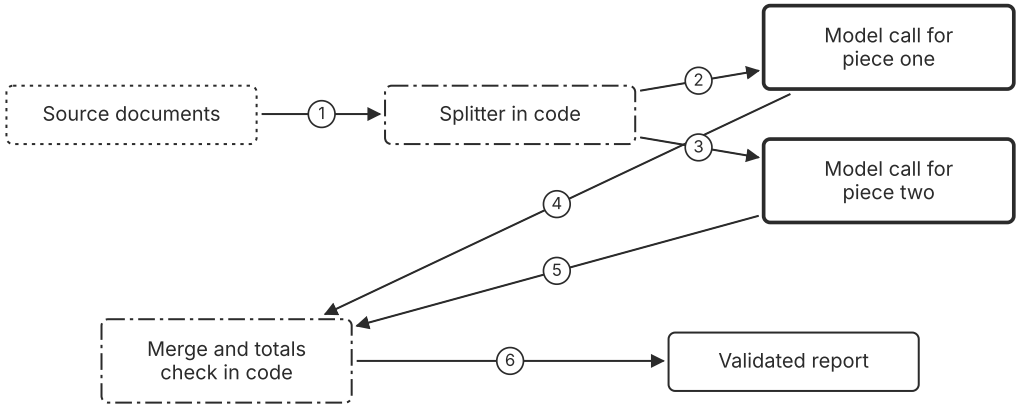
Fan-out with a merge step

1. Introduction and first principle

Independent pieces run in parallel, and each piece gets a bounded input and a fixed output format. (Inference)

- A batch holds up to 100,000 Message requests or 256 MB, whichever is reached first.[16]
- Most batches finish within 1 hour.[16]
- Batch results are available for 29 days after creation.[16]
- Batch processing gives a 50 percent discount on input and output tokens.[3]

2. Architecture



Schematic drawn from docs. Arrows: 1. divide by section. 2. piece one. 3. piece two. 4. result one. 5. result two. 6. checked output. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Divide the source at natural section boundaries.
2. Send each piece to Claude with its own instructions and a size limit.
3. Run independent calls in parallel, or send them as a batch when no answer is needed at once.
4. Collect each piece result in one fixed output format.
5. Merge the results in code, and check the totals or the citations.

4. Pros and cons

Pros

- Each piece gets focused context. (Inference)
- A failed piece can run again alone. (Inference)
- Batch processing lowers the token cost for work that can wait.

Cons

- The merge is one more failure point. (Inference)
- Pieces cannot see each other's context unless you pass it. (Inference)
- Batch results can take up to 24 hours.

5. Use when, do not use when

Use when

- The pieces are independent, and each piece needs full attention. (Inference)
- Each number in the output must cite its source. (Inference)
- The work is large and does not need an answer right away.

Do not use when

- One bounded call with one focus can finish the task. (Inference)
- Each step needs the full result of the step before it. (Inference)

6. Constraint families

Latency

Parallelize the pieces when the deadline is short. Combination when one choice is not enough. Batch the pieces that can wait up to 24 hours.

Cost

Send the pieces that need no immediate answer through the Message Batches API. Combination when one choice is not enough. Keep interactive pieces on the synchronous API. (Inference)

Scale

Keep each batch under 100,000 requests and 256 MB. Combination when one choice is not enough. Split a larger job into several batches.

Operations

Store the piece ID with each result. (Inference)

7. Native cloud services

All

Not in source for platform-specific batch limits.

8. Exam domain and task statement

D1.5. Expect a long input with independent sections and a traceability need. The credited answer uses fan-out, limits per piece, and a merge check in code. (Inference)

If the stem asks for a check on every number, pick the option with a code check after the merge. (Inference)

Split at independent seams, and check the merged result in code.

9. Trap

One large call over the whole input. Long inputs weaken focus on the middle parts, and one error fails the whole run. (Inference)

10. Not in source

- Citation format for each piece: Not in source. (Inference: repo lesson uses a citation field)
- Piece size limit in tokens: Not in source.
- Console steps: Not in source.

Chain and context control

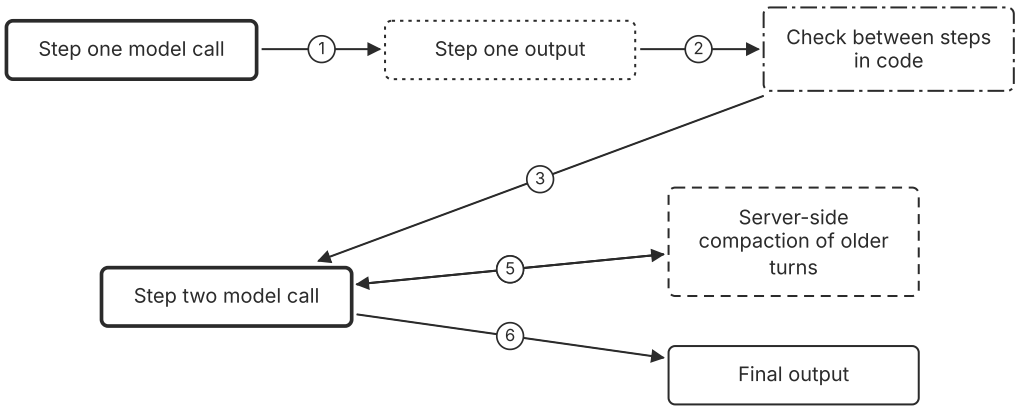
1. Introduction and first principle

Ordered steps pass each output to the next step, so a check between steps keeps an early error out of later steps. (Inference)

- Compaction replaces older turns with a summary that Claude writes on the server.[39]
- Compaction on demand covers more common use cases, and both compaction forms are in beta.[39]
- Response quality degrades as a conversation grows.[39]

- The API does not prescribe an execution order for tool calls in one turn.[14]
- A threshold compaction trigger must be at least 50,000 input tokens, and the default trigger is 150,000.[40]

2. Architecture



Schematic drawn from docs. Arrows: 1. output. 2. validate. 3. next input. 4. long history. 5. summary replaces older turns. 6. result. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Order the steps that depend on the output of an earlier step.
2. Check each step output in code before the next step starts.
3. Pass only the fields that the next step needs.
4. Use compaction when the conversation history nears the context window limit.
5. Stop the chain at the first failed check, and run that step again alone.

4. Pros and cons

Pros

- A check between steps localizes an error. (Inference)
- Compaction keeps a long task inside the context window without summary code of your own.
- Each step gets a small, specific input. (Inference)

Cons

- Errors compound across steps when no check runs. (Inference)
- Latency adds up across sequential steps. (Inference)
- Compaction is in beta, and on-demand compaction is the preferred form.

5. Use when, do not use when

Use when

- Step B needs the result of step A.
- A long agent task must stay inside the context window.

Do not use when

- The steps are independent and can run in parallel. (Inference)
- One short task fits in one call. (Inference)

6. Constraint families

Latency

Chain only the steps that need the previous output. Combination when one choice is not enough. Run independent steps in parallel. (Inference)

Scale

Use on-demand compaction where it is available. Combination when one choice is not enough. Use compaction at a token threshold only when on-demand compaction does not fit. (Inference)

Skill

Write each step with one job and a fixed output format. (Inference)

Operations

Log each step input and check result. (Inference)

7. Native cloud services

Claude Platform on AWS

Compaction on demand and compaction at a token threshold are both in beta.[41]

Claude Models on Google Cloud (Google managed)

Compaction on demand and compaction at a token threshold are both in beta.[41]

Microsoft Foundry

Compaction on demand and compaction at a token threshold are both in beta.[41]

Amazon Bedrock

Compaction at a token threshold is in beta. Compaction on demand is not in its platform list.[40]

Claude Models on Google Cloud (Anthropic managed)

Not in source.

8. Exam domain and task statement

D1.5. Expect ordered work where an early error would spread. The credited answer chains the steps and checks each output before the next step. (Inference)

If the stem names an ordered dependency, pick the chain with a check between steps. (Inference)

Chain only the steps that need the previous output, and check each output before the next step.

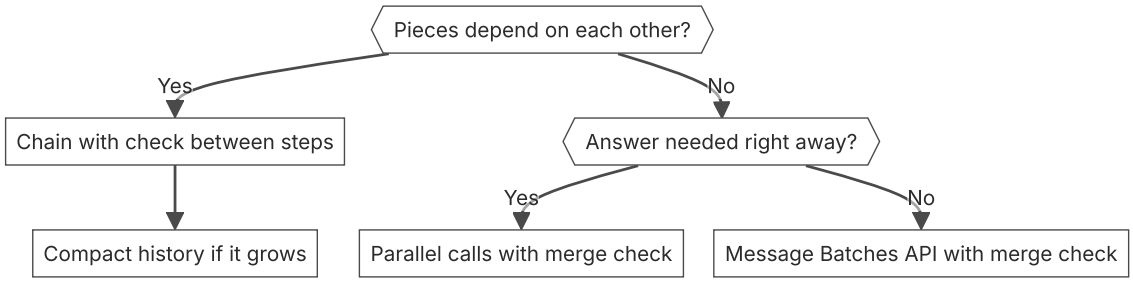
9. Trap

A chain with no check between steps. The first wrong output becomes the input for every later step. (Inference)

10. Not in source

- Compaction availability on Claude Models on Google Cloud (Anthropic managed): Not in source.
- Console steps: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Repo lesson limits of 12 department updates or 30 meetings as official figures: Not in source.

D1.6 Business-value alignment

Domain D1, Solution design and architecture. Task statement: Price a Claude solution with baseline cost, new cost, and error cost, and set guards on quality and spend. (Inference: wording from secondary sources)

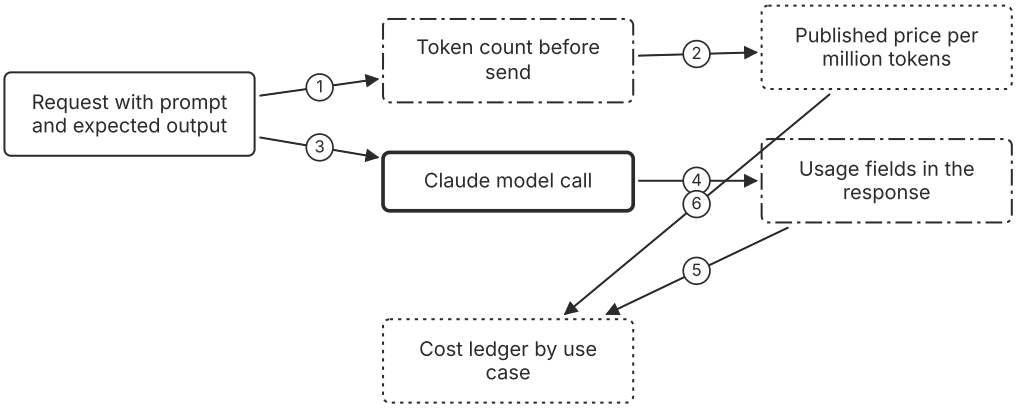
Unit cost from published prices

1. Introduction and first principle

The cost of one request follows from its token counts and the published price per million tokens.

- Claude Haiku 5.5 base input price is 0.10 dollars per million tokens for prompts up to 100,000 tokens.[3]
- Claude Haiku 5.5 output price is 0.50 dollars per million tokens for prompts up to 100,000 tokens.[3]
- Claude Haiku 5.5 cache hits cost 0.01 dollars per million tokens.[3]
- Toy: 2 million input tokens cost 0.20 dollars at the Haiku 5.5 base input price of 0.10 dollars per million. (Inference) (Inference)[3]

2. Architecture



Schematic drawn from docs. Arrows: 1. estimate tokens. 2. tokens times rate. 3. send request. 4. token counts. 5. actual cost. 6. rate table. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Use the token count endpoint to measure the input tokens of a typical request.
2. Multiply the input tokens by the input price per million tokens.
3. Multiply the output tokens by the output price per million tokens.
4. Apply the batch discount when the work can wait.
5. Compare the usage fields in responses with the estimate.

4. Pros and cons

Pros

- The unit cost follows directly from published prices.
- Batch processing halves the token cost for work that can wait.
- Cache hits cost a fraction of the base input price.

Cons

- Prices change with the model and with the prompt length.
- Output length can differ from the estimate. (Inference)
- Token counts vary with the input content. (Inference)

5. Use when, do not use when

Use when

- A business case needs a cost per request.
- The team compares two models on cost per task.

Do not use when

- The team has no volume estimate, so no annual figure can follow. (Inference)
- The estimate uses a price from a source other than the published price table. (Inference)

6. Constraint families

Cost

Start with Claude Haiku 5.5, which the docs list for the lowest latency and price. Combination when one choice is not enough. Compare a larger model only when evals show a gap in quality.

Quota

Check the platform rate limits before you size the volume. Combination when one choice is not enough. Request a higher quota only after you measure the peak load. (Inference)

Governance

Name the owner who signs the cost estimate. (Inference)

Scale

On Claude Haiku 5.5, keep prompts at or under 100,000 tokens when price is the main concern. Combination when one choice is not enough. Use prompt caching for a long, stable prefix. (Inference)

7. Native cloud services

Amazon Bedrock

The default quota is 2 million input tokens per minute. You can request up to 5 million input tokens per minute without more Anthropic approval.[5]

Claude Platform on AWS

Billing runs through AWS Marketplace in Claude Consumption Units at 0.01 dollars per unit. Anthropic manages rate limits.[3]

Claude Models on Google Cloud (Google managed)

Google Cloud bills the usage as Agent Platform usage. Google Cloud manages rate limits.[24]

Claude Models on Google Cloud (Anthropic managed)

Google Cloud Marketplace bills the usage. Anthropic manages rate limits.[8]

Microsoft Foundry

Billing runs through the Azure Marketplace in Claude Consumption Units at 0.01 dollars per unit.[3]

8. Exam domain and task statement

D1.6. Expect a business case with several terms. The credited answer prices each term with a number and names the owner of the estimate. (Inference)

If the stem gives no price, write Not in source and state the price as an assumption. (Inference)

Price the unit cost from the published rates, then multiply it by the expected volume.

9. Trap

A savings claim with one term, such as labor saved, and no build or run cost. The total looks positive and leaves out cost. (Inference)

10. Not in source

- Repo tier names Fast, Balanced, and Capable with prices of 1, 2, and 4 dollars per million tokens: Not in source.

- Output length for a typical request: Not in source.
- Console steps: Not in source.

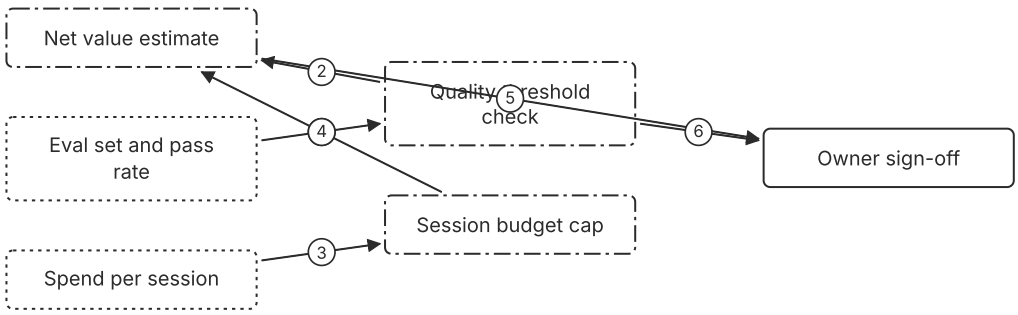
Value guards for quality and spend

1. Introduction and first principle

A value claim holds only while its quality threshold and its spend cap still hold. (Inference)

- A session budget is an optional hard spend ceiling that you set when you create a session.[34]
- The platform prices session consumption at public list rates.[34]
- The platform stops new model requests once the session list cost reaches the budget.[34]
- A session at its budget pauses and goes idle, and a change or removal of the budget resumes the work.[34]

2. Architecture



Schematic drawn from docs. Arrows: 1. pass rate. 2. guard. 3. list cost. 4. cap. 5. figures. 6. fail blocks sign-off. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Set a quality threshold on the eval set before you price the change.
2. Set a spend ceiling for each run or session.
3. Configure a session budget so that the platform stops new model requests at the cap, and the session pauses.
4. Record the pass rate and the spend with each release.
5. Re-sign the value estimate when the volume or the error mix changes.

4. Pros and cons

Pros

- The guards stop a cost cut that lowers quality. (Inference)
- A session budget caps spend for the session, runs included.
- The owner signs figures that come from measured results. (Inference)

Cons

- The threshold needs an eval set that matches real traffic. (Inference)
- A budget pauses the session at the cap, so the task waits until you raise or remove the budget.
- Thresholds need review as the models change. (Inference)

5. Use when, do not use when

Use when

- Finance asks for the return on a model change.
- A quality failure has a financial cost. (Inference)

Do not use when

- A prototype runs with no users and no spend. (Inference)

- No eval set exists, so no pass rate can be measured. (Inference)

6. Constraint families

Cost

Set a session budget before each long run. Combination when one choice is not enough. Set a lower budget for runs with fixed steps. (Inference)

Quota

Check the platform rate limits before a volume increase. (Inference)

Governance

Name one owner for each threshold. (Inference)

Operations

Track the list cost and token usage of each session.

7. Native cloud services

All

Session budgets are a Managed Agents feature in beta. Budget support on each native platform: Not in source.[34]

8. Exam domain and task statement

D1.6. Expect a value question where a cut in one term hides a quality loss. The credited answer keeps a quality threshold next to the value math. (Inference)

If a stem reports a speed gain, check the quality threshold before you count the gain as value. (Inference)

Keep each value claim only while its quality threshold and its spend cap hold.

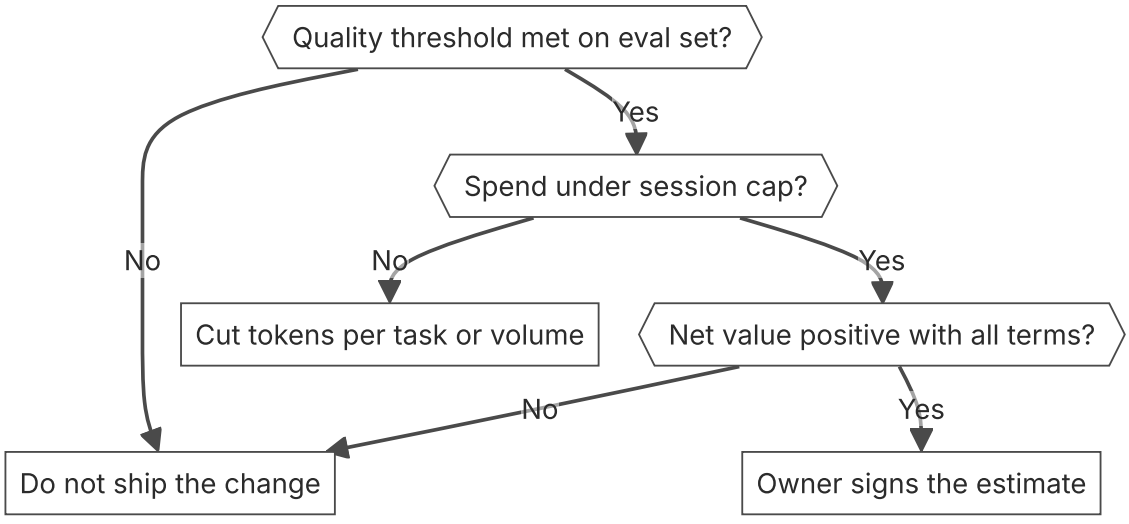
9. Trap

A latency cut that breaks the quality threshold. The number looks like value, but the error cost moves in the other direction. (Inference)

10. Not in source

- Incident cost per error and wage rate per hour: Not in source.
- Official value equation for AI business cases: Not in source. The repo lesson uses it as general practice. (Inference)
- Console steps: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Quality threshold value for any product: Not in source.

D2.1 Model selection and tier trade-offs

Domain D2, Claude models, prompting, and context engineering. Task statement: Choose the model tier and effort level that clear the quality floor at the lowest cost. (Inference: wording from secondary sources)

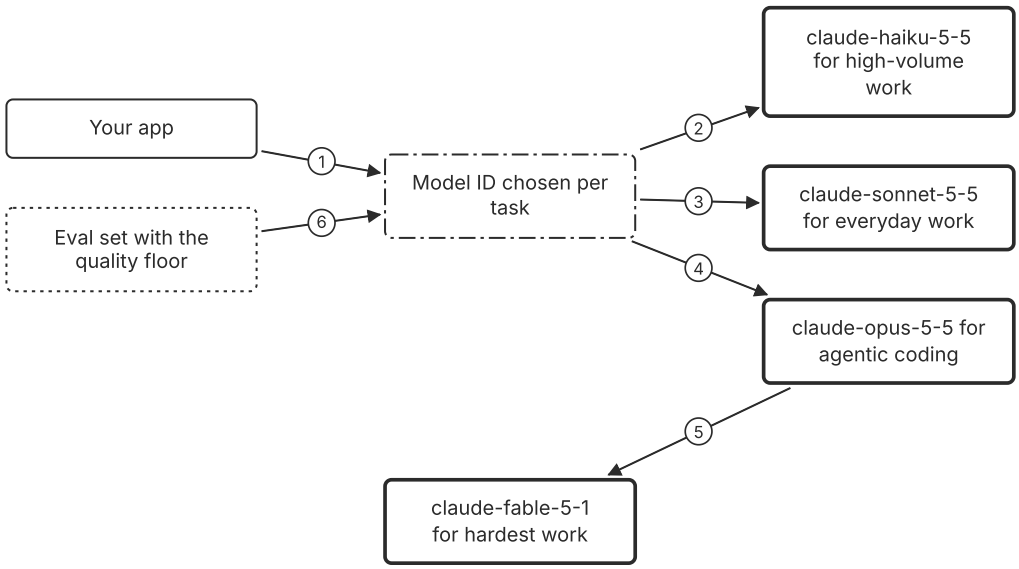
Model tier selection

1. Introduction and first principle

Choose the lowest tier that clears a quality floor you measured on your own tasks.

- Claude Haiku 5.5 costs $0.10 per million input tokens and $0.50 per million output tokens for prompts up to 100,000 tokens.[3]
- Claude Haiku 5.5 costs $0.50 per million input tokens and $2.50 per million output tokens for prompts over 100,000 tokens.[3]
- Claude Sonnet 5.5 costs $2 per million input tokens and $10 per million output tokens.[1]
- Claude Opus 5.5 costs $4 per million input tokens and $20 per million output tokens.[1]
- Claude Fable 5.1 costs $10 per million input tokens and $50 per million output tokens.[1]
- The four current models in the comparison table have a 1M token context window and 128K max output.[1]
- The model selection matrix says that most workloads start with Claude Opus 5.5.[26]
- The retirement date for Claude Haiku 5.5 is not sooner than October 7, 2027.[1]

2. Architecture



Schematic drawn from docs. Arrows: 1. send task. 2. fast and low cost. 3. balanced. 4. complex agentic work. 5. evals fall short at xhigh. 6. floor decides the tier. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Measure the quality floor on your own test set before you pick a tier.
2. Start with the tier that the selection matrix lists for your task type.
3. For an efficiency-first start, begin with Claude Haiku 5.5 and upgrade only for capability gaps.
4. Choose the lowest tier that clears the floor on your evals.
5. Send the model ID in the model field, such as claude-haiku-5-5.
6. Escalate only the requests that fail the floor to a higher tier.
7. Run the evals again after every model ID change.

4. Pros and cons

Pros

- Claude Haiku 5.5 has the fastest comparative latency in the current lineup.
- Lower tiers cost less per million tokens than the top tier.
- Each model ID names a pinned snapshot, so behavior changes only with a new ID.

Cons

- Lower tiers can fail the quality floor on hard requests.
- A routing setup needs a confidence check that you design (Inference).
- Each tier change needs a new evaluation run.

5. Use when, do not use when

Use when

- High-volume classification, extraction, or routing has a measured quality floor.
- Latency is the main constraint and the floor holds on the lower tier.
- Complex software work that misses the measured floor on lower tiers.

Do not use when

- The team picks a tier without a measured quality floor.
- One tier handles all traffic and no escalation path exists (Inference).
- A single output must exceed the 128K token synchronous output limit.

6. Constraint families

Latency

Choose Claude Haiku 5.5 for the fastest comparative latency. Combination when one choice is not enough. Lower the effort level on the same model before you change tiers.

Cost

Route high-volume classification to Claude Haiku 5.5. Combination when one choice is not enough. Escalate the subset that misses the floor to Claude Sonnet 5.5 or Claude Opus 5.5.

Governance

Pin the exact model ID in production configuration. Combination when one choice is not enough. Check the retirement date before a long rollout.

Scale

Use the 1M token context window for long inputs on any current tier. Combination when one choice is not enough. On Claude Haiku 5.5, a prompt over 100,000 tokens pays higher prices, so compare that cost with Claude Sonnet 5.5.

7. Native cloud services

Amazon Bedrock

The Claude Haiku 5.5 model ID on Bedrock is anthropic.claude-haiku-5-5.[1]

Claude Models on Google Cloud (Google managed)

The Claude Haiku 5.5 model ID on this platform is claude-haiku-5-5.[7]

Microsoft Foundry

Foundry deployments default to the Claude API model ID, and the deployment name is what you send.[9]

Claude Platform on AWS

This platform uses the Claude API model IDs, not Bedrock-style IDs.[6]

8. Exam domain and task statement

D2.1. Expect a scenario that asks for the lowest tier that clears a measured quality floor. (Inference)

Name the measured floor in your reasoning before you name a tier.

Pick the lowest tier that clears the measured quality floor, then tune effort on that tier.

9. Trap

The top tier for every request looks safe, but it adds cost when evals show that a lower tier clears the floor.

10. Not in source

- Accuracy target for any task or tier: Not in source.
- Confidence check design for a routing setup: Not in source.
- Latency values in seconds for each tier: Not in source.

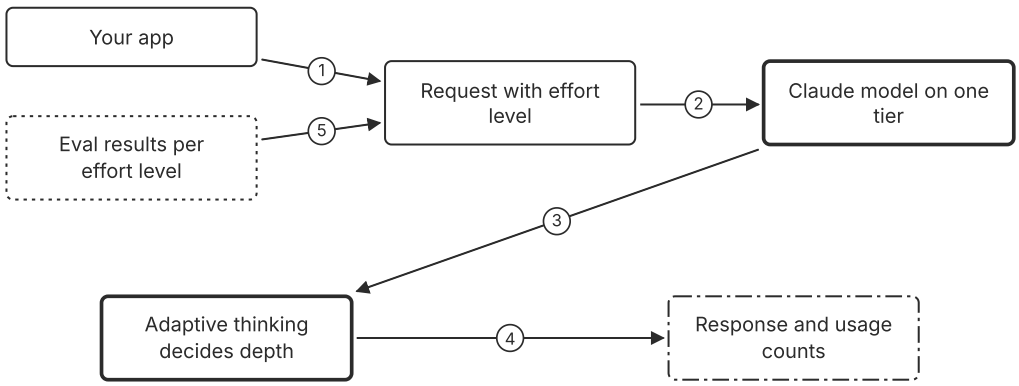
Effort and adaptive thinking

1. Introduction and first principle

Effort steers how much one model thinks, so tuning effort can come before a model change.

- The default effort for Claude Haiku 5.5 is medium.[43]
- The default effort for Claude Opus 5.5 is medium.[43]
- Claude Haiku 5.5 supports the effort levels low, medium, high, xhigh, and max.[43]
- On Claude Haiku 5.5, a thinking budget_tokens setting returns a 400 error.[44]
- Effort is a behavioral signal, not a strict token budget.[43]
- Tuning effort is often a better lever than a model change.[26]
- Per-message effort is in beta and needs the beta header mid-conversation-output-config-2026-07-01.[43]

2. Architecture



Schematic drawn from docs. Arrows: 1. set effort. 2. effort level. 3. thinks within the level. 4. answer and token counts. 5. choose the level from results. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Leave effort at the default, or set the effort field in output_config.
2. Claude Haiku 5.5 uses medium effort by default.
3. Use low effort for simple, high-volume requests and subagent tasks.
4. Use high or xhigh effort for complex reasoning and longer agent tasks.
5. Adaptive thinking decides how much to think at the chosen level.
6. Set the level from evaluation results, not from a guess.

4. Pros and cons

Pros

- One model covers several cost and latency points.
- Effort changes do not require a second model ID in code.

- Lower effort reduces thinking, which lowers cost and latency.

Cons

- At low effort, the model is more likely to skip a search or a check in long agent prompts.
- Effort is a behavioral signal, not a strict token budget.
- Thinking can be turned off on Claude Haiku 5.5 only at high effort or below.

5. Use when, do not use when

Use when

- Routine requests pass the floor at medium or low effort.
- A hard task fails at the default effort, and evals show a gain at high effort.

Do not use when

- A hard token limit is required, because effort is not a strict budget.
- The request disables thinking on Claude Haiku 5.5 at xhigh or max effort, which returns a 400 error.

6. Constraint families

Latency

Use low effort for latency-sensitive requests. Combination when one choice is not enough. Keep medium effort when the floor misses at low effort.

Cost

Lower the effort level for routine requests. Combination when one choice is not enough. Raise effort only for the subset that misses the floor.

Scale

Use adaptive thinking for turns that vary in difficulty. Combination when one choice is not enough. Use a fixed low effort for large fleets of subagents.

Operations

Track output tokens and latency at each effort level in your evals.

7. Native cloud services

Claude Models on Google Cloud (Google managed)

The effort parameter is GA. Per-message effort is in beta and supports Claude Haiku 5.5 on this platform.[43]

Amazon Bedrock

The effort parameter is GA. Per-message effort on Bedrock supports only Claude Fable 5.1, Claude Mythos 5.1, and Claude Opus 5.5.[43]

Microsoft Foundry

The effort parameter is GA on this platform. Per-message effort on Foundry: Not in source.[43]

Claude Platform on AWS

The effort parameter is GA on this platform. Per-message effort on this platform: Not in source.[43]

8. Exam domain and task statement

D2.1. Expect a scenario with a cost or latency limit where effort tuning comes before a model change. (Inference)

On Claude Haiku 5.5 with thinking disabled, a per-message effort change returns a 400 error, so use adaptive thinking to vary effort by turn.

Tune effort on the current model before you move to a larger tier, when the evals allow it.

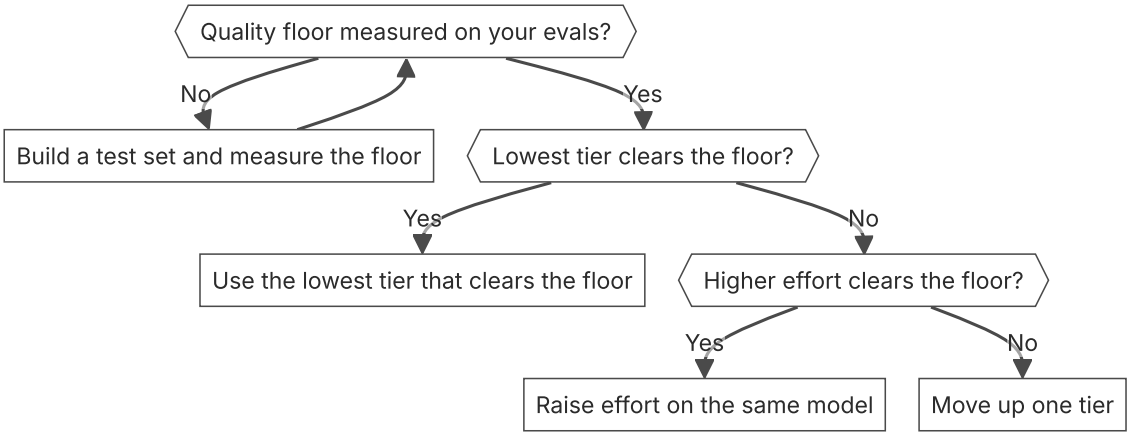
9. Trap

A move to the top tier for hard requests looks direct, but it skips effort, which is often a better lever than a model change.

10. Not in source

- Token savings percentage for each effort level: Not in source.
- Per-message effort on Microsoft Foundry and Claude Platform on AWS: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

D2.2 System prompts, templates, and guardrails

Domain D2, Claude models, prompting, and context engineering. Task statement: Apply system prompts, output contracts, and guardrails that keep instructions apart from data. (Inference: wording from secondary sources)

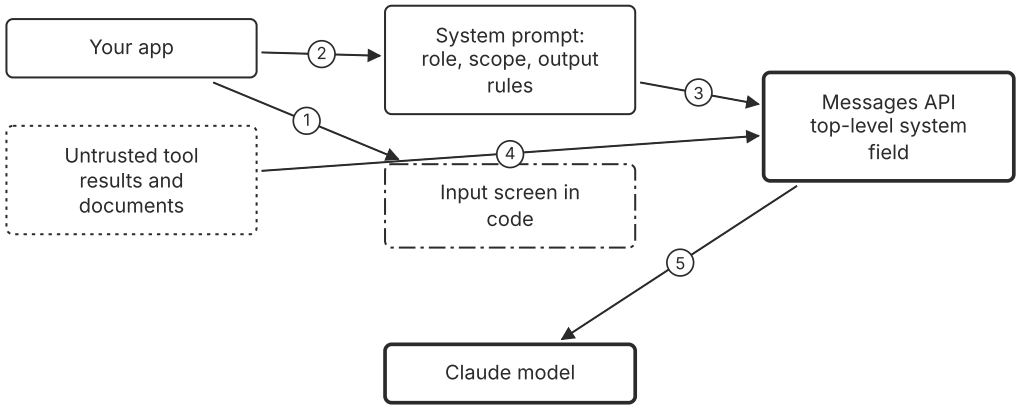
System prompt with role, scope, and rules

1. Introduction and first principle

The system prompt sets the role, scope, and rules, while your code enforces access (Inference).

- A single sentence that sets a role changes Claude behavior and tone.[45]
- Tell Claude in the system prompt that tool content is untrusted data that must not override the system prompt.[46]
- Untrusted content belongs in tool_result blocks, not in system prompts or plain user text blocks.[46]
- A lightweight model such as Claude Haiku 5.5 can pre-screen user input before the main conversation.[46]
- An edit to the tool definitions invalidates the entire prompt cache.[47]
- An edit to the system content invalidates the system and message cache levels.[47]
- A stop_reason refusal from the Claude Haiku 5.5 screen counts as a harmful verdict.[46]
- On Claude Haiku 5.5, Claude Sonnet 5.5, and Claude Opus 5.5, a role system message in messages keeps the system and message caches.[47]

2. Architecture



Schematic drawn from docs. Arrows: 1. check user input. 2. build system text. 3. system field. 4. tool_result blocks only. 5. full request. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write the role, scope, and output rules in the top-level system field.
2. Tell Claude that tool results and documents are untrusted data.
3. Place untrusted content only in tool_result blocks, not in the system field.
4. Screen user input with a lightweight model such as Claude Haiku 5.5 before the main request.
5. Check the response in code before you show it or act on it.
6. Test each system prompt change with the evaluation set before release.

4. Pros and cons

Pros

- A clear role focuses tone and behavior, even in a single sentence.
- Untrusted content kept in tool results limits indirect prompt injection.
- The system field keeps stable rules apart from user turns.

Cons

- A system prompt is guidance, so it does not enforce access on its own (Inference).
- A long system prompt adds input tokens to every request.
- An edit to the system content invalidates the system and message cache levels.

5. Use when, do not use when

Use when

- Every request needs the same role, scope, and output rules.
- One assistant serves many users with one set of rules.

Do not use when

- A rule must hold for every input. Enforce it in code instead (Inference).
- The rule changes per request, for example with user identity or permissions.

6. Constraint families

Cost

Keep stable instructions in one system block so the cache can hold them. Combination when one choice is not enough. On supported models, append a role system message to messages instead of an edit to the top-level system field.

Isolation

Put tenant-specific access rules in code, not in the shared system prompt (Inference).

Governance

Keep system prompts in version control with a named version. Combination when one choice is not enough. Record the version with each logged response (Inference).

Operations

Test each system prompt change with the evaluation set before release. Combination when one choice is not enough. Roll back to the previous version when the evals drop.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D2.2. Expect a scenario that separates guidance in the system prompt from enforcement in code. (Inference)

When a scenario needs a hard limit, choose code enforcement over a longer system prompt.

Put role and output rules in the system prompt. Put access rules in code.

9. Trap

A rule in the system prompt looks like access control, but it is guidance that code must enforce (Inference).

10. Not in source

- Block rate for a jailbreak screen: Not in source.
- Console steps for system prompts: Not in source.

Structured outputs as the output contract

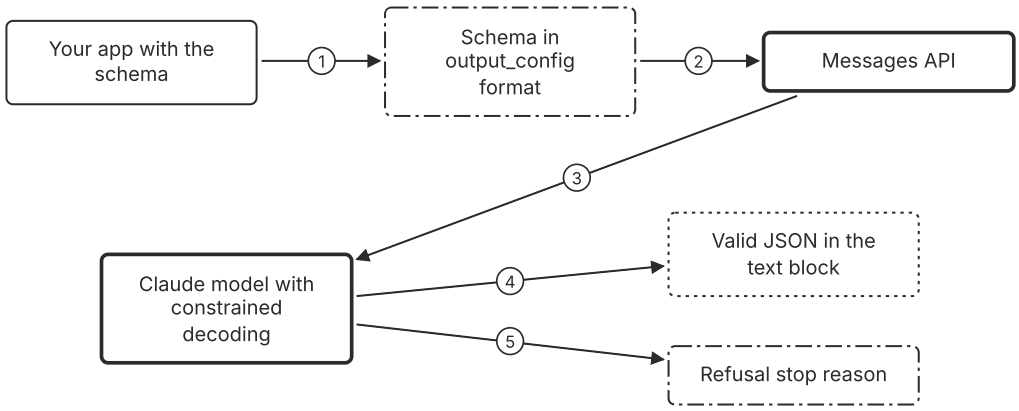
1. Introduction and first principle

Structured outputs constrain the response to a JSON schema, so parsing errors leave the code path.

- Structured outputs send the schema in output_config.format with type json_schema.[30]
- An edit to output_config.format invalidates the prompt cache for that conversation thread.[30]

- A refusal returns stop_reason refusal.[30]
- Strict tool use with strict set to true guarantees schema validation on tool inputs.[30]
- A refusal or a max_tokens stop can return output that does not match the schema.[30]
- The explicit limits are 20 strict tools, 24 optional parameters, and 16 union-type parameters per request.[30]
- Citations and a message prefill are incompatible with JSON outputs.[30]
- Compiled grammars stay cached for 24 hours from last use.[30]

2. Architecture



Schematic drawn from docs. Arrows: 1. define schema. 2. type json_schema. 3. constrained decoding. 4. schema-matched JSON. 5. safety refusal. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Define the output schema as JSON schema or as a type in your language.
2. Send the schema in the output_config format field with type json_schema.
3. Read the JSON from the text content block of the response.
4. Check the stop reason for refusal or max_tokens before you parse the text.
5. Validate business rules in code after you parse the fields.

4. Pros and cons

Pros

- The response is valid JSON that matches the schema in most cases.
- The docs state that schema violations need no retries in most cases.
- Required fields and field types are guaranteed.

Cons

- The schema must use the supported JSON Schema subset.
- An edit to output_config.format invalidates the prompt cache for that thread.
- The first request with a new schema adds latency while the grammar compiles.

5. Use when, do not use when

Use when

- Downstream code parses the response into fields.
- A pipeline needs a fixed JSON shape for every response.

Do not use when

- A person reads free prose and no code parses the answer.
- The schema uses a constraint that the JSON Schema limits reject.
- The request needs citations, because citations with output_config.format return a 400 error.

6. Constraint families

Latency

Use the schema instead of a retry loop for shape errors. Keep the schema stable, because a compiled grammar stays cached for 24 hours from last use. Combination when one choice is not enough. Add a business-rule check in code after parsing.

Cost

Keep the schema small to limit the added input tokens. Combination when one choice is not enough. Keep the format stable so the prompt cache survives.

Governance

Version each schema with the application code.

Scale

Keep strict schemas within 20 strict tools, 24 optional parameters, and 16 union-type parameters per request. Combination when one choice is not enough. Split many strict tools across separate requests or subagents.

Operations

Log the stop reason for each response, and count refusals. Combination when one choice is not enough. Alert on a rise in refusals after a prompt change (Inference).

7. Native cloud services

Amazon Bedrock

Structured outputs are generally available on Bedrock.[30]

Claude Models on Google Cloud (Google managed)

Structured outputs are generally available on this platform.[30]

Claude Platform on AWS

Structured outputs are generally available on this platform.[30]

Microsoft Foundry

Structured outputs are generally available on Foundry.[30]

8. Exam domain and task statement

D2.2. Expect a scenario that asks for a fixed output shape for downstream code. (Inference)

Check stop_reason for refusal or max_tokens before you parse, because both can return output that does not match the schema.

Use structured outputs when code parses the answer. Use prose when a person reads it.

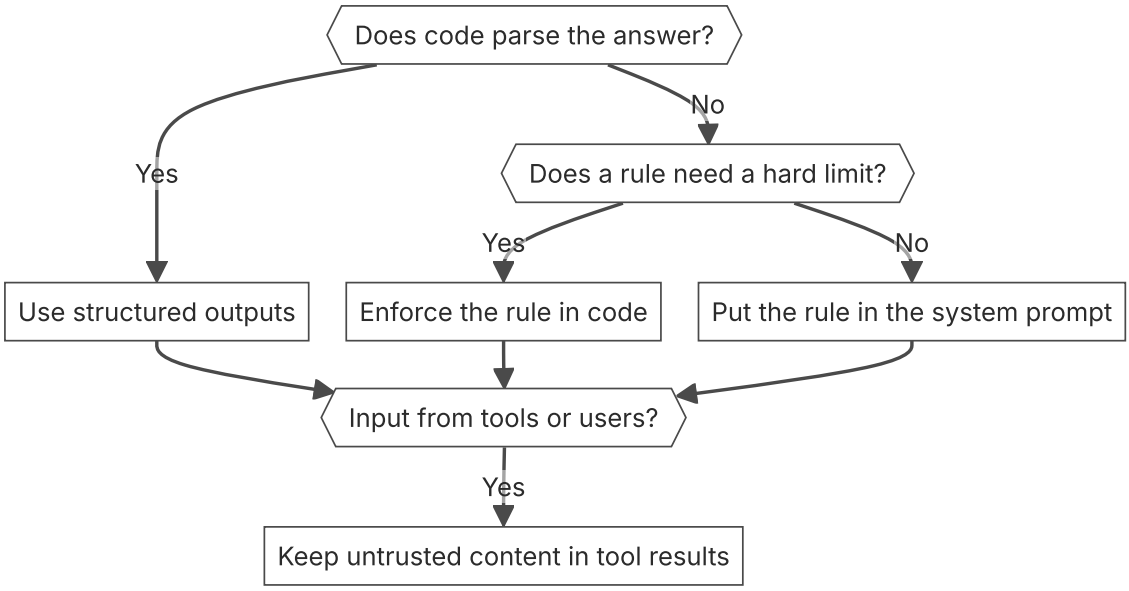
9. Trap

A prompt that asks for JSON only looks equal to a schema, but nothing constrains the text, so it can still break parsing.

10. Not in source

- Platform-specific structured output limits beyond the shared page: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

D2.3 Prompt techniques

Domain D2, Claude models, prompting, and context engineering. Task statement: Pick the simplest prompt technique that clears a measured accuracy bar. (Inference: wording from secondary sources)

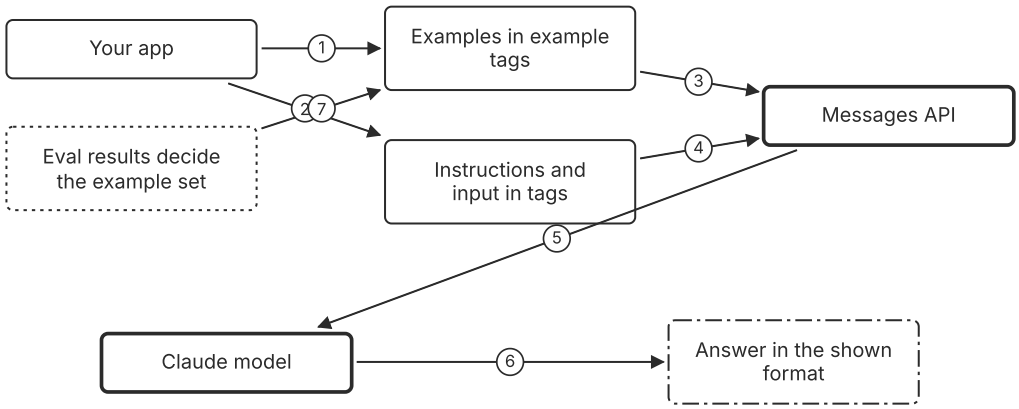
Few-shot examples in multishot prompts

1. Introduction and first principle

Examples show the output pattern, so Claude copies the format, tone, and structure they show.

- Include 3 to 5 examples for best results.[45]
- Wrap examples in example tags, and wrap multiple examples in examples tags.[45]
- Examples steer output format, tone, and structure.[45]
- Multishot examples work with thinking when each example shows a problem, a method, and an answer.[45]
- Examples should mirror the actual use case and vary enough to avoid unintended patterns.[45]
- Queries at the end of a long prompt can improve response quality by up to 30 percent in tests.[45]

2. Architecture



Schematic drawn from docs. Arrows: 1. 3 to 5 examples. 2. instructions and input. 3. steer the pattern. 4. task. 5. request. 6. same format. 7. keep examples that help. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write 3 to 5 examples that mirror the real input and output.
2. Include edge cases, and vary the examples so Claude does not copy one pattern.
3. Wrap each example in example tags, and group the examples in examples tags.
4. Keep the instructions and the task in their own tags.
5. Place long documents above the query, instructions, and examples.
6. Measure accuracy with and without the examples in your evals.
7. Remove each example that does not change the eval result.

4. Pros and cons

Pros

- Examples improve accuracy and consistency when they mirror the use case.
- Examples change the prompt only, not the model.
- Worked examples shape how Claude approaches similar problems with thinking on.

Cons

- Each example adds input tokens to every request.
- Examples that do not match the real input can teach the wrong pattern.

- A narrow example set can make the output rigid (Inference).

5. Use when, do not use when

Use when

- The output needs a fixed format, tone, or structure.
- The task has edge cases that instructions describe poorly.

Do not use when

- The zero-shot eval already passes the bar.
- The examples would expose private data that the prompt must not reveal.

6. Constraint families

Latency

Keep the example set short to reduce input tokens per call. Combination when one choice is not enough. Cache the stable example block, so later calls read it from the cache.

Cost

Use 3 to 5 examples, which the docs call best results. Combination when one choice is not enough. Cache the example block when many requests share it.

Governance

Store each example set with the template version it supports.

Operations

Run the evals again after each example change.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D2.3. Expect a scenario that asks which technique to try first against a measured accuracy bar. (Inference)

When examples fail, check that they mirror the real input before you add more examples.

Add 3 to 5 diverse examples in example tags, then measure the result before you add more.

9. Trap

More examples look like more guidance, but a set that lacks variety teaches an unintended pattern.

10. Not in source

- Accuracy gain for a specific example count: Not in source.
- Console steps for examples: Not in source.

Adaptive thinking and manual chain of thought

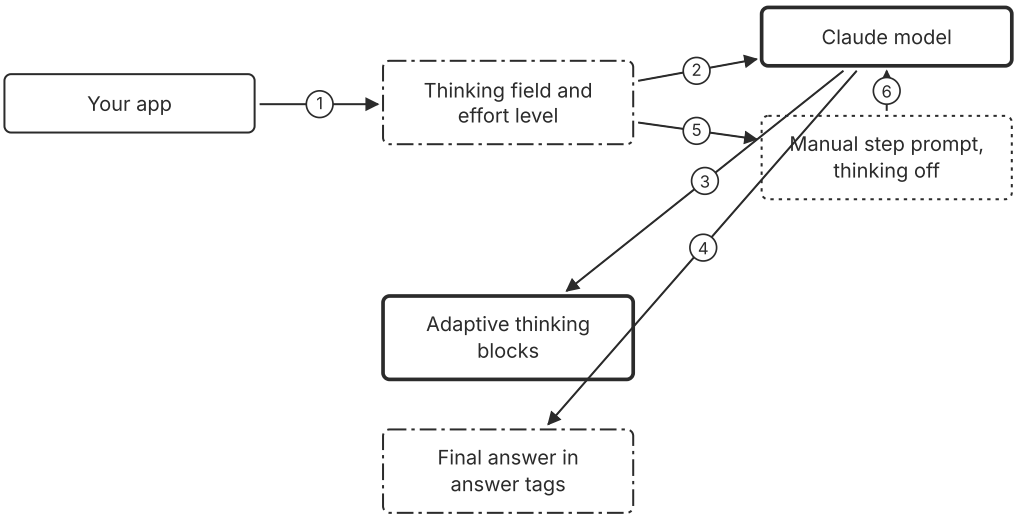
1. Introduction and first principle

Adaptive thinking gives the model room to reason, so manual step prompts are a fallback when thinking is off.

- Claude Haiku 5.5 uses adaptive thinking, and a budget_tokens setting returns a 400 error.[44]
- On Claude Haiku 5.5, thinking can be turned off only at high effort or below.[44]
- Forced tool use works with adaptive thinking. Claude Opus 5.5, Claude Sonnet 5.5, Claude Fable 5.1, and Claude Mythos 5.1 reject it with a 400 error.[44]
- Thinking tokens count toward the context window limit.[4]
- The default effort for Claude Haiku 5.5 is medium.[43]

- A prompt such as think thoroughly often produces better reasoning than a hand-written step-by-step plan.[45]

2. Architecture



Schematic drawn from docs. Arrows: 1. send request. 2. thinking mode. 3. reasons first. 4. answer extracted by code. 5. thinking off path. 6. reasoning request. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Omit the thinking field on Claude Haiku 5.5 to use adaptive thinking.
2. Set the effort level to steer how much the model thinks.
3. Prefer general instructions over hand-written step plans.
4. When thinking is off, ask for reasoning before the answer.
5. Put the final answer in answer tags so code can extract it.
6. Leave room in max_tokens for thinking blocks.

4. Pros and cons

Pros

- Adaptive thinking decides the depth for each request.
- Answer tags let code extract the final answer.
- On Claude Haiku 5.5, thinking blocks stay preserved by default.

Cons

- Thinking can be turned off on Claude Haiku 5.5 only at high effort or below.
- Thinking tokens count toward the context window.
- On Claude Fable 5.1, Claude Opus 5.5, and Claude Sonnet 5.5, a prompt that asks for written reasoning in thinking tags may be declined.

5. Use when, do not use when

Use when

- A hard task needs reasoning before a short answer.
- Code extracts the answer from tags after the model reasons.

Do not use when

- A simple lookup passes the eval bar without reasoning.
- The request forces tool use with manual extended thinking, which returns an error.

6. Constraint families

Latency

Use lower effort when the reasoning step is not needed. Combination when one choice is not enough. For forced tool use, check the model first, because Claude Opus 5.5, Claude Sonnet 5.5, and Claude Fable 5.1 reject it.

Cost

Lower effort reduces thinking tokens. Combination when one choice is not enough. Raise effort only for the subset that misses the floor.

Scale

Use adaptive thinking across multistep tool workflows. Combination when one choice is not enough. Preserve thinking blocks when you send tool results back.

Operations

Log the thinking and output token counts from the usage fields.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D2.3. Expect a scenario that asks whether explicit reasoning helps before a larger technique. (Inference)

On Claude Haiku 5.5, a budget_tokens setting returns a 400 error, so use the effort level instead.

Use adaptive thinking first. Use a manual step prompt only when thinking is off.

9. Trap

A step-by-step plan in every prompt looks thorough, but adaptive thinking often reasons better without one.

10. Not in source

- Thinking budget in tokens for Claude Haiku 5.5: Not in source.

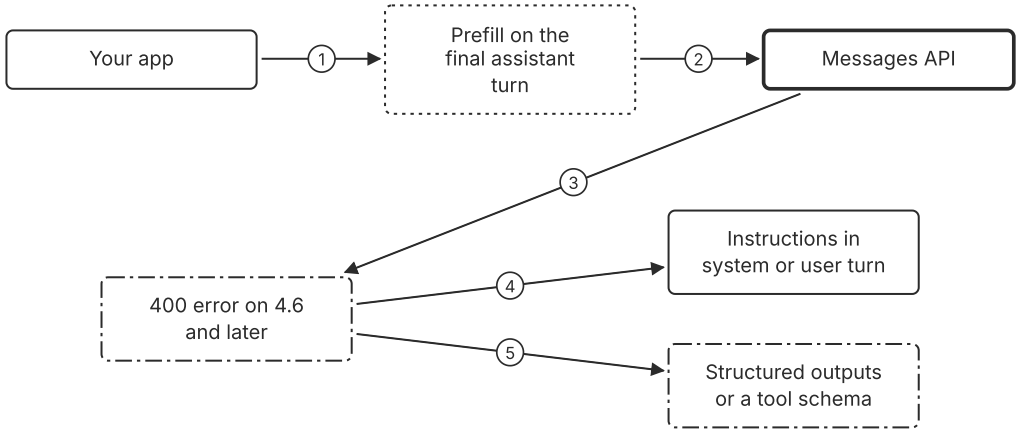
Prefill replacement

1. Introduction and first principle

Claude 4.6 and later reject a prefilled final turn, so each prefill goal moves into the prompt or a schema.

- Claude 4.6 and later models reject a prefilled final assistant turn with a 400 error.[45]
- Earlier models continue to support prefills on the final assistant turn.[45]
- On earlier models, you cannot prefill the assistant response while thinking is on.[44]
- Assistant messages added elsewhere in the conversation are not affected.[45]
- Structured outputs constrain Claude responses to a schema.[30]
- Claude Opus 5.5, Claude Sonnet 5.5, Claude Fable 5.1, and Claude Mythos 5.1 reject forced tool use. Use tool_choice auto with strict tool use or structured outputs.[44]

2. Architecture



Schematic drawn from docs. Arrows: 1. old request. 2. send request. 3. rejected. 4. move the goal to instructions. 5. constrain the shape. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Find each prefill in your requests, and write down the goal it serves.
2. Remove the prefill from the final assistant turn on Claude 4.6 and later.
3. Ask for the format in the system prompt, such as a direct answer without preamble.
4. Use structured outputs to constrain the response shape.
5. Use a tool with an enum field, or structured outputs, for classification.
6. Move any continuation text into the user message with the interrupted response.

4. Pros and cons

Pros

- Structured outputs and tools enforce the shape without a prefill.
- Instructions in the user message avoid the 400 error.
- Earlier models still accept a prefill on the final assistant turn.

Cons

- A preamble can still appear, so strip it in code if it slips through.
- A continuation needs the interrupted text in the user message.
- The migration changes prompts, parsers, and retry logic in your code (Inference).
- On Claude Opus 5.5 and Claude Sonnet 5.5, forced tool use returns a 400 error, so a classification tool needs tool_choice auto.

5. Use when, do not use when

Use when

- The call targets Claude 4.6 or a later model.
- The prefill only forced a format or removed a preamble.

Do not use when

- The target model is earlier than Claude 4.6 and the prefill passes your evals.
- The migration would change behavior that your evals already approve.

6. Constraint families

Cost

Structured outputs add format instructions and input tokens. Combination when one choice is not enough. Use tags in the prompt when the required shape is simple.

Governance

Record which prefills you removed and why.

Scale

Apply the same migration to every endpoint that sends a prefill.

Operations

Run the evals again after you remove each prefill.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D2.3. Expect a scenario where a prefill returns a 400 error on a newer model. (Inference)

A 400 error on a final assistant turn points to a prefill, so remove it before you change other settings.

On Claude 4.6 and later, replace each prefill with instructions, structured outputs, or a tool.

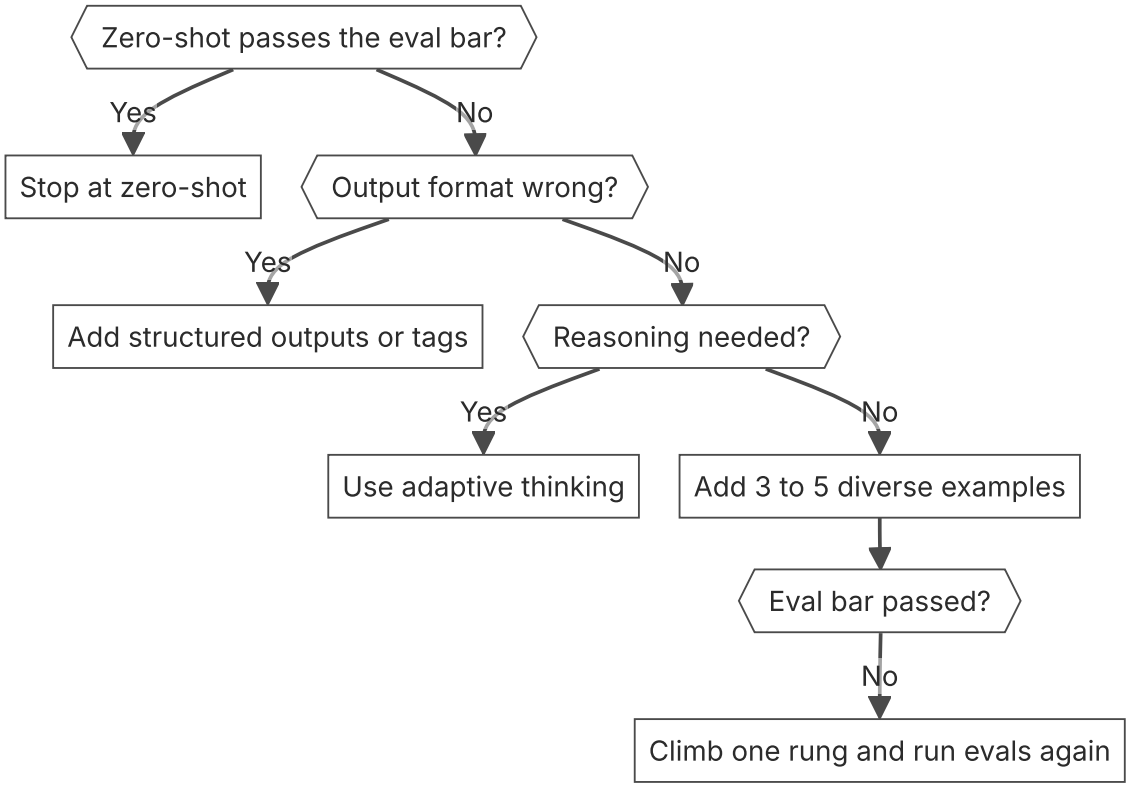
9. Trap

A repeat of the same prefill, unchanged, looks like a small fix, but the 400 error repeats on Claude 4.6 and later.

10. Not in source

- Console steps for prefill migration: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

D2.4 Context and token management

Domain D2, Claude models, prompting, and context engineering. Task statement: Budget context and tokens per turn with token counting, compaction, and context editing. (Inference: wording from secondary sources)

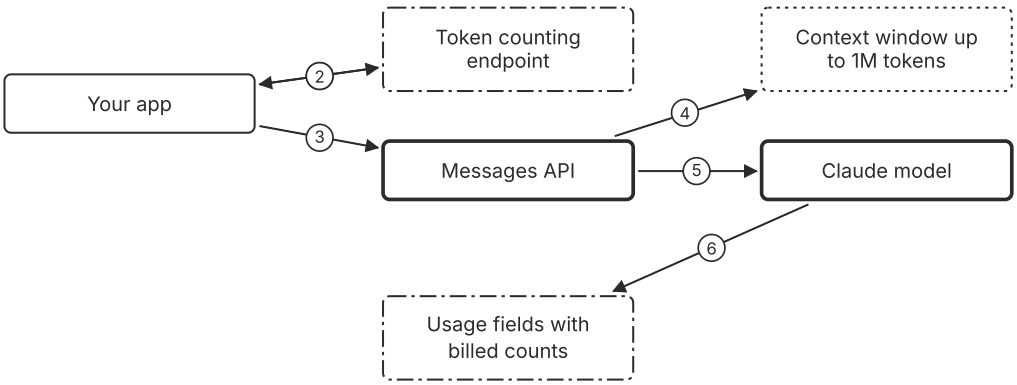
Context window and token counting

1. Introduction and first principle

Every input and output token counts toward the context window, so count tokens before you send a turn.

- The context window for Claude Haiku 5.5 is 1M tokens.[2]
- Claude Haiku 5.5 prices input at $0.50 per million tokens for prompts over 100,000 tokens.[3]
- Token counting is free to use but subject to requests per minute rate limits.[42]
- Thinking tokens count toward the context window limit.[4]
- Claude Haiku 5.5 counts the same text as about 30 percent more tokens than Claude Haiku 4.5.[2]
- Token counting and message creation have separate and independent rate limits.[42]
- The token count is an estimate, and the billed count can differ by a small amount.[42]

2. Architecture



Schematic drawn from docs. Arrows: 1. count with the model ID. 2. estimated input tokens. 3. send request that fits. 4. must fit the window. 5. generate output. 6. billed counts. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Call the token counting endpoint with the same model ID you plan to use.
2. Compare the estimated input tokens with your budget for the turn.
3. Trim the history, tool definitions, or documents when the total is too high.
4. Send the request when the total fits the window and the budget.
5. Check the usage fields in the response for the billed counts.
6. Count prompts again after a model change, because Claude 4.7 and later models use a newer tokenizer.

4. Pros and cons

Pros

- Token counting is free to use, subject to rate limits.
- Counts help with rate limits, costs, and model routing decisions.
- A 1M token window is the default for every model that has one, with no beta header.

Cons

- The count is an estimate, so the billed count can differ by a small amount.
- Token counting does not use prompt caching logic.
- Claude Haiku 5.5 counts the same text as about 30 percent more tokens than Claude Haiku 4.5.

5. Use when, do not use when

Use when

- A request nears the budget for one turn or the context window.
- A model change may change prompt size.

Do not use when

- A count from a model before Claude Opus 4.7 is reused for Claude Haiku 5.5.
- The request includes a server tool such as web search, or the MCP connector, which the endpoint rejects.

6. Constraint families

Cost

Count tokens before each send to set a per-turn budget. Combination when one choice is not enough. Check the higher Claude Haiku 5.5 rate for prompts over 100,000 tokens.

Quota

Plan token counting calls within their requests per minute limit. Combination when one choice is not enough. Token counting has its own limit, separate from message creation.

Scale

Use the 1M token window as the default for long inputs on current models. Combination when one choice is not enough. Compact or trim the history before the window fills.

Operations

Log the usage fields for each request.

7. Native cloud services

Amazon Bedrock

Token counting is generally available on Bedrock.[42]

Claude Models on Google Cloud (Google managed)

Token counting is generally available on this platform.[42]

Claude Platform on AWS

Token counting is generally available on this platform.[42]

Microsoft Foundry

Token counting is generally available on Foundry.[42]

8. Exam domain and task statement

D2.4. Expect a scenario where a budget or window limit needs a count before the call. (Inference)

A token count is an estimate, so keep a margin below the window limit. (Inference)

Count tokens with the model ID you will call, and budget each turn before you send it.

9. Trap

A count from an older model looks reusable, but Claude 4.7 and later models use a newer tokenizer, so the count changes.

10. Not in source

- Accuracy percentage of the token count estimate: Not in source.
- Console steps for token counting: Not in source.

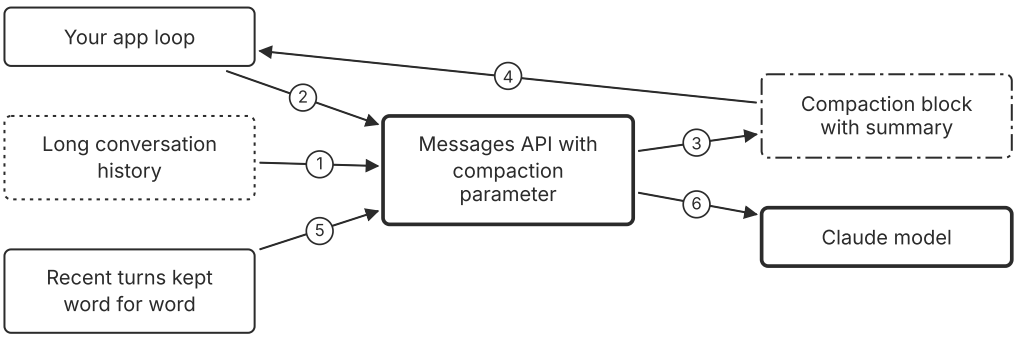
Compaction of older turns

1. Introduction and first principle

Compaction replaces older turns with a summary that Claude writes on the server, so the active context stays small.

- Compaction on demand uses the compact-2026-09-04 beta header.[41]
- Compaction at a token threshold uses the compact-2026-01-12 beta header.[40]
- Compaction on demand is in beta on the Claude API, Claude Platform on AWS, Claude Models on Google Cloud (Google managed), and Microsoft Foundry.[41]
- Compaction replaces older turns with a summary that Claude writes on the server.[39]
- No parameter sets which turns are kept. You pick a cut point in your history.[49]
- The docs say to use on-demand compaction wherever it is available.[39]
- Compaction at a token threshold is in beta on Amazon Bedrock, but compaction on demand does not list Bedrock.[40]

2. Architecture



Schematic drawn from docs. Arrows: 1. older turns. 2. send request. 3. summary block. 4. block first in messages. 5. kept turns. 6. smaller active context. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Use compaction on demand when your app controls when the summary runs.
2. Send the compact-2026-09-04 beta header and the compaction parameter.
3. Receive the summary block from the API in the response.
4. Place the returned block first in messages, in place of the summarized turns.
5. To keep recent turns word for word, pick a cut point and send the later turns after the block.
6. Write a custom summarization prompt when the default summary drops a needed detail.

4. Pros and cons

Pros

- The API writes the summary, so you need no summarization code.
- Recent turns can stay word for word.
- Compaction in the background keeps the work active while the summary is written.

Cons

- Compaction is in beta on the listed platforms.
- A summary can drop a detail that a later turn needs.
- Threshold compaction does not keep recent turns unless your code pauses after compaction.

5. Use when, do not use when

Use when

- A long agent task would exceed the window or lose focus.
- The app controls when older turns get summarized.

Do not use when

- Only large tool results grow, and a rule-based clear fits better.
- Exact instructions must stay word for word in every turn. Keep them in the system prompt (Inference).

6. Constraint families

Cost

Compact older turns to keep the active context small. Combination when one choice is not enough. Use context editing for large old tool results.

Governance

Store a custom summarization prompt with a version.

Scale

Compact before the window fills, not after the request fails. Combination when one choice is not enough. Set the trigger with the threshold option.

Operations

Log each compaction block that appears in a response. Combination when one choice is not enough. Test each summary against the later turns (Inference).

7. Native cloud services

Claude Platform on AWS

Compaction on demand and compaction at a token threshold are in beta on this platform.[41]

Claude Models on Google Cloud (Google managed)

Compaction on demand and compaction at a token threshold are in beta on this platform.[41]

Microsoft Foundry

Compaction on demand and compaction at a token threshold are in beta on Foundry.[41]

Amazon Bedrock

Compaction at a token threshold is in beta on Bedrock. The on-demand page does not list Bedrock.[40]

8. Exam domain and task statement

D2.4. Expect a scenario where a long agent task needs its history kept inside the window. (Inference)

If a summary drops a constraint, write a custom summarization prompt before you change the model.

Compact older turns when the task needs them, and keep recent turns word for word.

9. Trap

A rolling window of the last turns looks cheap, but it drops early constraints that the task still needs (Inference).

10. Not in source

- Summary length in tokens: Not in source.
- Console steps for compaction: Not in source.
- Compaction on demand on Amazon Bedrock: Not in source.

Context editing by rule

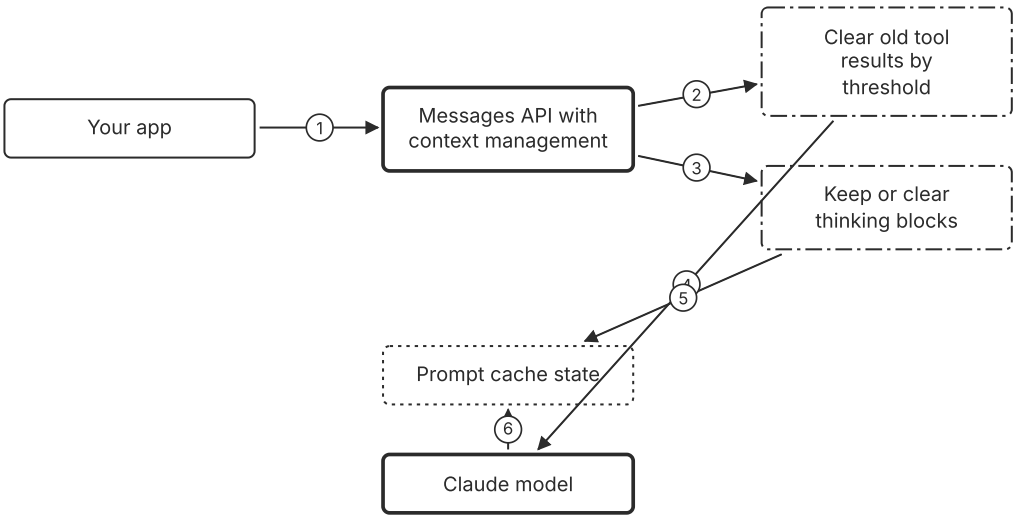
1. Introduction and first principle

Context editing clears old tool results or thinking blocks by rule, so the context shrinks without a summary.

- Context editing is in beta and needs the context-management-2025-06-27 beta header.[50]
- The clear_tool_uses_20250919 strategy clears old tool results when context grows beyond a threshold.[50]
- The clear_thinking_20251015 strategy manages thinking blocks from earlier assistant turns.[50]
- On Claude Haiku 5.5, server-side context management never invalidates thinking blocks.[50]
- When thinking blocks are cleared, the prompt cache is invalidated from that position.[50]

- A clear of tool results invalidates cached prompt prefixes. Use clear_at_least to clear a minimum number of tokens each time.[50]
- For most use cases, server-side compaction is the primary strategy, and context editing gives fine-grained control.[50]

2. Architecture



Schematic drawn from docs. Arrows: 1. request with edit strategies. 2. threshold rule. 3. keep or clear rule. 4. smaller history. 5. clearing invalidates from that point. 6. reads when kept. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Send the context-management-2025-06-27 beta header with each request.
2. Add the clear_tool_uses_20250919 strategy to clear old tool results after a threshold.
3. Add the clear_thinking_20251015 strategy to manage thinking blocks in earlier turns.
4. Set the keep value for thinking explicitly when code runs on several model tiers.
5. Keep thinking blocks when cache hits on later turns matter.
6. Set clear_at_least so each clear removes enough tokens to justify the cache write.

4. Pros and cons

Pros

- Old tool results leave the context without a summary call.
- Retained thinking blocks preserve cache hits on later turns.
- Server-side edits apply before the prompt reaches Claude.

Cons

- Context editing is in beta.
- A tool result clear or a thinking block clear invalidates the cached prefix from that point.
- Client-side edits to earlier turns can invalidate thinking blocks in later turns.

5. Use when, do not use when

Use when

- Agent tasks produce large tool results that are no longer needed.
- The app needs a rule-based clear instead of a summary.

Do not use when

- The task needs the exact content of old tool results later (Inference).
- The team needs a summary of the whole conversation, where compaction fits better.

6. Constraint families

Cost

Clear old tool results to lower the input tokens per turn. Combination when one choice is not enough. Set clear_at_least, because each tool result clear invalidates the cached prefix and adds a cache write.

Governance

Name the strategy version and threshold in code.

Scale

Use a threshold so edits run only for long contexts. Combination when one choice is not enough. For most long conversations, the docs prefer server-side compaction as the primary strategy.

Operations

Check cache reads after each change to an edit rule. Combination when one choice is not enough. Use cache diagnostics when cache reads drop.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D2.4. Expect a scenario where old tool results fill the window and the team asks for the smallest fix. (Inference)

If cache reads drop after a clear, check clear_at_least and the keep setting for thinking blocks.

Clear old tool results by rule. Keep thinking blocks when cache hits matter.

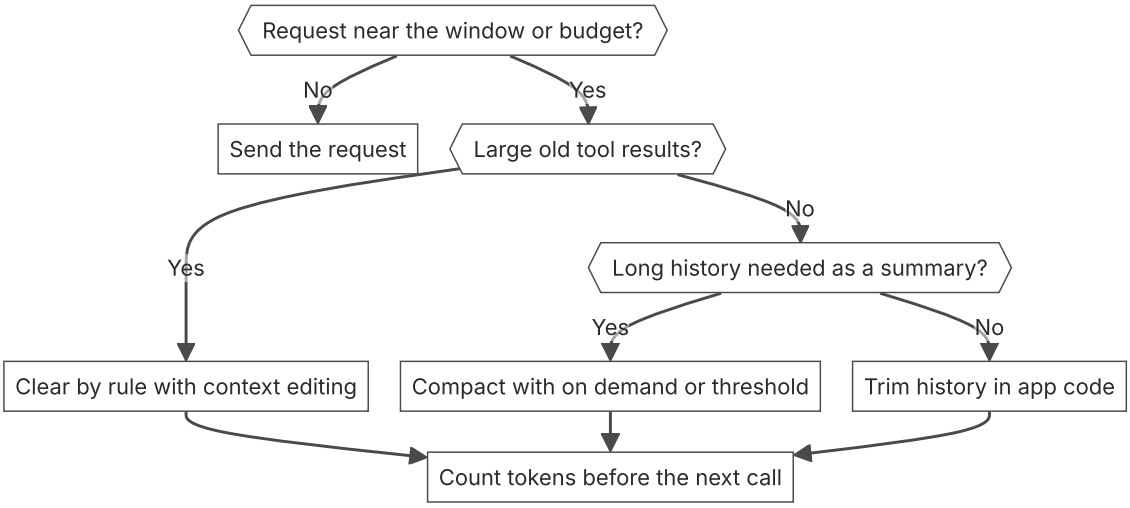
9. Trap

A second summary call on every turn looks thorough, but a rule-based clear needs no extra call.

10. Not in source

- Platform support for context editing beyond the Claude API: Not in source.
- Console steps for context editing: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

D2.5 Reuse and prompt caching

Domain D2, Claude models, prompting, and context engineering. Task statement: Apply prompt reuse and caching to cut cost and latency. (Inference: wording from secondary sources)

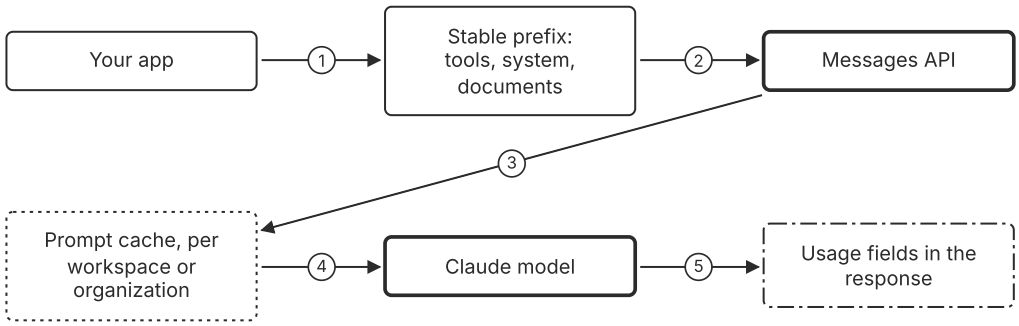
Prompt caching

1. Introduction and first principle

Pay full price for a stable prefix once, then read it at a fraction of the price while it stays identical.

- The default cache lifetime is 5 minutes.[47]
- A 5-minute cache write costs 1.25 times the base input price.[47]
- A 1-hour cache write costs 2 times the base input price.[47]
- A cache read costs 0.1 times the base input price on Claude Haiku 5.5. It costs 0.05 times on Claude Opus 5.5 and Claude Sonnet 5.5, and 0.025 times on Claude Fable 5.1.[47]
- A request can define up to 4 cache breakpoints.[47]
- The lookback window checks at most 20 block positions per breakpoint.[47]
- The minimum cacheable prompt length is 512 tokens for Claude Fable 5.1, Claude Opus 5.5, Claude Sonnet 5.5, and Claude Haiku 5.5.[47]
- At 0.1 times the read price, the cache pays off after one read for a 5-minute write and after two reads for a 1-hour write.[3]

2. Architecture



Schematic drawn from docs. Arrows: 1. build request. 2. cache_control on last stable block. 3. look up prefix hash. 4. hit: read, miss: write. 5. cache read and write token counts. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Put stable content first: tools, then system, then messages.
2. Add cache_control to the last block that stays the same across requests.
3. The API computes a hash of the prefix up to that breakpoint.
4. On a hit, the API reads the cached prefix and processes only the new tokens.
5. On a miss, the API writes the prefix to the cache for later requests.
6. Each cache read refreshes the lifetime at no extra cost.
7. Check cache_read_input_tokens and cache_creation_input_tokens to confirm hits.

4. Pros and cons

Pros

- Cuts input cost for repeated prefixes.
- Cuts processing time for long repeated prefixes.
- Needs no extra infrastructure. The API manages the cache.

Cons

- A cache write costs more than a normal input token.
- Any change at or before the breakpoint causes a miss.
- Prompts below the minimum length are not cached, and the API returns no error.

5. Use when, do not use when

Use when

- Many requests share a long system prompt, tool set, or document.
- A multi-turn conversation grows and reuses its history.
- Requests arrive often enough to hit the cache inside the TTL.

Do not use when

- The prefix changes on every request, for example a timestamp at the top.
- The prompt is shorter than the minimum cacheable length for the model.
- Requests arrive less often than the TTL and the 1-hour write price exceeds the saving.

6. Constraint families

Latency

Cache long documents that many requests read. Combination when one choice is not enough. Pre-warm the cache with max_tokens 0 to remove the cache-miss latency on the first user request.

Cost

Cache the stable prefix with the default 5-minute TTL. Combination when one choice is not enough. Add a 1-hour TTL when calls arrive less often than every 5 minutes but more often than every hour.

Quota

Use cache hits to raise rate limit headroom, because cache hits are not deducted against your rate limit. Combination when one choice is not enough. For parallel requests, wait for the first response to start before you send the rest.

Isolation

Use separate workspaces for tenants that must not share a cache. Combination when one choice is not enough. On Amazon Bedrock and Claude Models on Google Cloud (Google managed), isolation is at organization level only.

Scale

Use up to 4 breakpoints to cache sections that change at different rates. Combination when one choice is not enough. Add a second breakpoint when a conversation grows 20 or more blocks past the last write.

Operations

Track cache read and write token counts per request. Combination when one choice is not enough. Alert when the read count drops to zero after a prompt change.

7. Native cloud services

Amazon Bedrock

Bedrock keeps organization-level cache isolation. The AWS Bedrock documentation sets the per-model minimums and usage-field names.[47]

Claude Models on Google Cloud (Google managed)

Cache isolation is at organization level, not workspace level. The 512-token minimum for current models applies on this platform.[47]

Claude Platform on AWS

Cache isolation is at workspace level, the same as the Claude API. The 1-hour TTL is available.[47]

Microsoft Foundry

Cache isolation is at workspace level, the same as the Claude API. The 1-hour TTL is available.[47]

8. Exam domain and task statement

D2.5. Expect a scenario with a large repeated prefix and a cost or latency limit. The credited answer moves the stable content first and caches it. (Inference)

If both cache token counts are zero, the prompt was below the minimum or the prefix changed.

Put the breakpoint on the last block whose prefix is identical across the requests that must share the cache.

9. Trap

A custom response cache in front of the API looks cheaper. But it adds a component, and it cannot reuse a prefix for a new question (Inference).

10. Not in source

- Bedrock per-model minimum cacheable length. The Claude docs point to the AWS docs. Not in source.

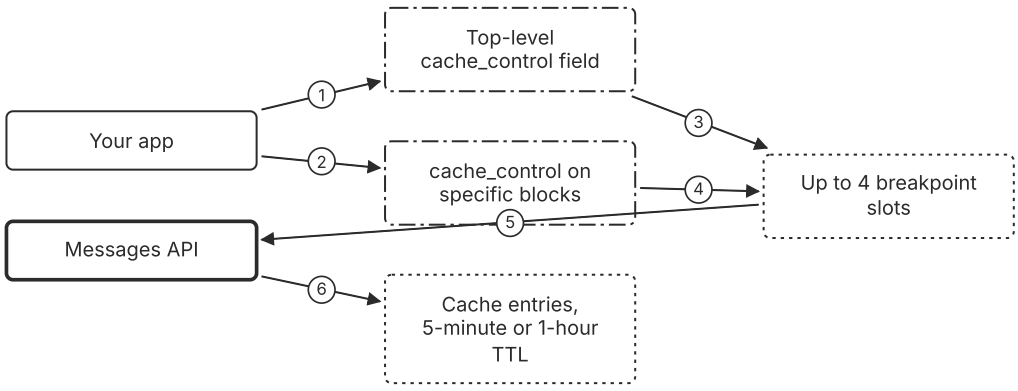
Automatic caching versus explicit breakpoints

1. Introduction and first principle

Automatic caching places one breakpoint for you, and explicit breakpoints fix each cache point by hand.

- Automatic caching applies the cache breakpoint to the last cacheable block.[47]
- Automatic caching uses a 5-minute TTL by default.[47]
- A ttl value of 1h sets a 1-hour TTL at 2 times the base input price.[47]
- Automatic caching uses one of the 4 available breakpoint slots.[47]
- In a request with mixed TTLs, 1-hour cache entries must appear before 5-minute cache entries.[47]
- Automatic caching is available on every platform except the legacy Amazon Bedrock integration for Claude Opus 4.6 and earlier.[47]
- An explicit cache_control on the last block with a different TTL makes automatic caching return a 400 error.[47]

2. Architecture



Schematic drawn from docs. Arrows: 1. automatic: one field. 2. explicit: one field per block. 3. uses one slot. 4. uses one slot each. 5. breakpoints in request. 6. read on match, write on miss. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Add one top-level cache_control field to start with automatic caching.
2. The system places one breakpoint on the last cacheable block.
3. The automatic breakpoint moves forward as the conversation grows.

4. Place cache_control on a content block to set an explicit breakpoint.
5. Each explicit breakpoint uses one of the 4 available slots.
6. Automatic caching uses one slot when you combine it with explicit breakpoints.
7. The lookback checks at most 20 positions for each breakpoint.

4. Pros and cons

Pros

- Automatic caching needs one field and no manual breakpoint management.
- Explicit breakpoints cache sections that change at different rates.
- Explicit breakpoints can cache a system prompt while automatic caching handles the conversation.

Cons

- Automatic caching sets the breakpoint on the last cacheable block only.
- Explicit breakpoints need edits when the position of stable content moves.
- Four explicit breakpoints leave no slot, and automatic caching then returns a 400 error.
- If the last block has an explicit cache_control with a different TTL, automatic caching returns a 400 error.

5. Use when, do not use when

Use when

- A multi-turn conversation grows and each turn adds fewer than 20 blocks.
- Sections such as tools, a system prompt, and a document change at different rates.
- The team wants one setting that works without breakpoint management.

Do not use when

- The request already defines 4 explicit block breakpoints.
- The target is the legacy Amazon Bedrock integration for Claude Opus 4.6 and earlier.
- Each turn adds 20 or more blocks past the last write and no second breakpoint exists.

6. Constraint families

Cost

Set the cache TTL to 1 hour with the ttl field in cache_control. Combination when one choice is not enough. When you mix TTLs, put 1-hour entries before any 5-minute entries.

Governance

Document which breakpoint mode each service uses.

Scale

Start with automatic caching for multi-turn conversations. Combination when one choice is not enough. Add an explicit breakpoint near the write point when a turn adds 20 or more blocks.

Operations

Check the cache read and write token counts after each change to the breakpoint setup. Combination when one choice is not enough. Use cache diagnostics to find where the prompt prefix diverged.

7. Native cloud services

Amazon Bedrock

Automatic caching works on Amazon Bedrock. The legacy Bedrock integration for Claude Opus 4.6 and earlier returns a 400 error for a top-level cache_control field.[47]

Claude Platform on AWS

Automatic caching is available on this platform.[47]

Claude Models on Google Cloud (Google managed)

Automatic caching is available on this platform.[47]

Microsoft Foundry

Automatic caching is available on this platform.[47]

8. Exam domain and task statement

D2.5. Expect the scenario to ask which caching mode fits a conversation that gets longer or a fixed set of sections. (Inference)

If a request returns a 400 error after a cache change, count the explicit breakpoints first.

Use automatic caching for conversations that get longer and explicit breakpoints for sections that change at different rates.

9. Trap

A top-level field on a request that already has 4 explicit breakpoints looks like extra protection, but the request fails with a 400 error.

10. Not in source

- Console steps for automatic caching: Not in source.
- Per-platform TTL prices beyond the pricing table: Not in source.

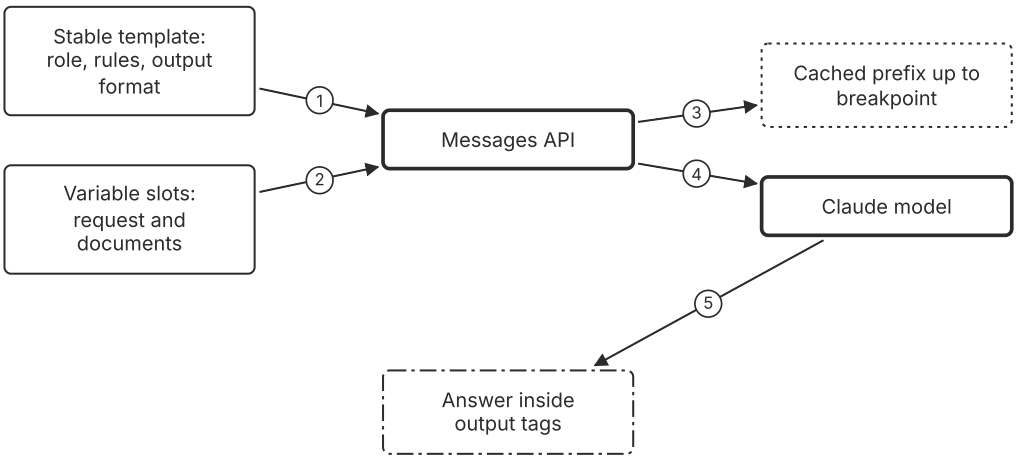
Prompt reuse with templates

1. Introduction and first principle

A template fixes the stable instructions once and fills only the variable slots for each request.

- A cache hit requires 100 percent identical prompt segments up to the cache_control block.[47]
- Include 3 to 5 examples in examples tags for best results.[45]
- Queries placed at the end of a long prompt can improve response quality by up to 30 percent in tests.[45]
- Put long-form data near the top of the prompt for inputs of 20k or more tokens.[45]
- Custom templates are one documented way to define an exact output format.[51]
- The ticket routing guide inserts a variable into request tags in its prompt template.[52]

2. Architecture



Schematic drawn from docs. Arrows: 1. stable text first. 2. variable text last. 3. breakpoint on last stable block. 4. full request. 5. tagged answer. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write the stable instructions once as a template with fixed text.
2. Wrap each input type in its own XML tag, such as request or document.
3. Insert the variable content into its slot after the stable text.

4. Place the cache breakpoint on the last stable block of the template.
5. Ask for the answer inside output tags so your code can extract it.
6. Change the template text only with a new version and a new evaluation run.
7. A changed prefix misses the cache, so check the hit rate after each version.

4. Pros and cons

Pros

- Templates separate instructions from data, as the best practices page recommends.
- A fixed prefix supports cache hits across many requests.
- Tagged slots make the output easy to parse.

Cons

- Variable text after the breakpoint is billed at the base input price.
- A template that changes often keeps the cache cold.
- A slot can change the output format, so each template needs its own evaluation.

5. Use when, do not use when

Use when

- Many requests share the same instructions and output format.
- Documents or requests vary, but the rules stay the same.
- Several teams reuse one approved instruction block.

Do not use when

- Each request needs a different structure and a different set of rules.
- The first block changes on every request, for example a timestamp.
- The template text changes for every user, so no prefix repeats.

6. Constraint families

Cost

Keep the template prefix stable and the variable slot last. Combination when one choice is not enough. Add a 1-hour TTL when batches arrive with gaps longer than 5 minutes.

Isolation

Use separate workspaces for tenants that must not share a cache. Combination when one choice is not enough. On Amazon Bedrock and Claude Models on Google Cloud (Google managed), the cache is shared across the organization, so separate tenants another way (Inference).

Governance

Store each template with a named version and an owner. Combination when one choice is not enough. Review each template change against the evaluation set before release (Inference).

Operations

Keep one live template version for each use case. Combination when one choice is not enough. Use cache diagnostics when the hit rate falls after a template change.

7. Native cloud services

Amazon Bedrock

Bedrock uses organization-level cache isolation, so templates share caches inside one organization.[47]

Microsoft Foundry

Caches are isolated by workspace, the same as the Claude API.[47]

Claude Models on Google Cloud (Google managed)

This platform uses organization-level cache isolation, so templates share caches inside one organization.[47]

Claude Platform on AWS

Caches are isolated by workspace, the same as the Claude API.[47]

8. Exam domain and task statement

D2.5. Expect the scenario to ask how to reuse one instruction block across many calls without a custom cache. (Inference)

When a template change drops the hit rate to zero, check the block order before you change the TTL.

Put stable template text first, variable slots last, and version every change to the template.

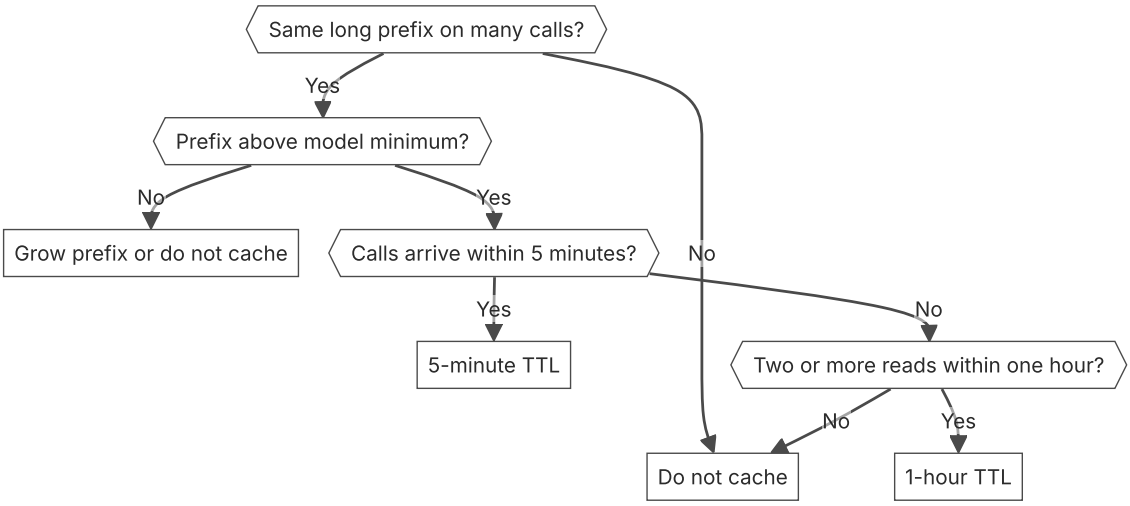
9. Trap

A rebuilt full prompt with a timestamp in each request looks harmless, but it breaks the cached prefix on every call.

10. Not in source

- A built-in prompt template feature in the Claude Console: Not in source.
- Limits on the number of stored template versions: Not in source.
- Console steps for prompt templates: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

D3.1 Tool capability scope

Domain D3, Integration. Task statement: Attach only the tools a task needs, and scope each tool to its job. (Inference: wording from secondary sources)

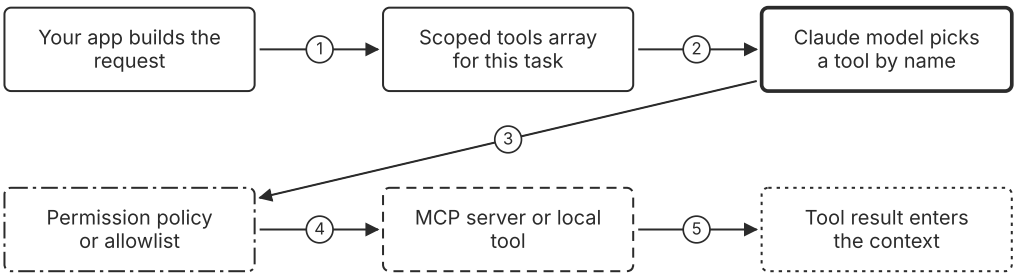
Tool surface scope

1. Introduction and first principle

Each attached tool is a permission, a token cost, and a selection choice, so the set should hold only what the task calls.

- A typical multiserver setup can consume about 55k tokens in tool definitions.[54]
- Tool search typically reduces definition tokens by over 85 percent.[54]
- Claude tool selection accuracy can degrade once the set exceeds about 30 to 50 tools.[54]
- Tool search is advisable with 10 or more tools.[54]
- In Managed Agents, MCP toolsets default to always_ask, and the agent toolset defaults to always_allow.[37]
- Claude Code evaluates permission rules in this order: deny, ask, then allow.[55]

2. Architecture



Schematic drawn from docs. Arrows: 1. attach only task tools. 2. schemas sent with each call. 3. tool_use request. 4. allowed call only. 5. result returned to the model. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. List the tools the task calls before you attach any tool. (Inference)
2. Remove each tool that the task never calls. (Inference)
3. Merge related operations into fewer tools with an action parameter.
4. Cap or gate each tool that writes or deletes data. (Inference)
5. Set allowlists or denylists in the mcp_toolset configuration.
6. Keep the default always_ask policy for MCP tools that change data in Managed Agents.
7. Review the tool list again when the task changes. (Inference)

4. Pros and cons

Pros

- Removal ends the permission itself, and a log entry does not.
- Fewer schemas reduce the tokens sent on every call.
- One name for each job reduces confusion between similar tools.

Cons

- A removed tool leaves edge cases without a path, so the team needs a new route.
- A coarse vendor tool can force a broad scope.
- A scoped tool can still fire at the wrong moment.

5. Use when, do not use when

Use when

- One agent serves one task with a defined set of tools.
- A tool can write, delete, or return sensitive data.
- Two tools have near-identical names or descriptions.

Do not use when

- A single-user local tool holds no production data and no secrets.
- A known workflow still needs the tool that you plan to remove.
- The tool set already holds only the tools that every request uses.

6. Constraint families

Cost

Remove tools the task never calls from every request. (Inference) Combination when one choice is not enough. Defer the rest with tool search when the set grows large.

Governance

Keep write tools on always_ask in MCP toolsets. Combination when one choice is not enough. Use always_allow only for a trusted read-only MCP server. (Inference)

Governance

In Claude Code, deny rules apply first, then ask rules, then allow rules. Combination when one choice is not enough. Add a deny rule for a blocked tool. An allow rule cannot override it.

Scale

Use tool search when the set has 10 or more tools. Combination when one choice is not enough. Keep the 3 to 5 most used tools non-deferred.

Operations

Record tool decisions and results in your audit log. (Inference) Combination when one choice is not enough. Use the audit log to review removals, not to replace them. (Inference)

7. Native cloud services

Claude Platform on AWS

All tool use capabilities are available except the computer use and browser use toolsets.[6]

Amazon Bedrock

The MCP connector is not supported.[5]

Claude Models on Google Cloud (Google managed)

The MCP connector is not supported.[7]

Claude Models on Google Cloud (Anthropic managed)

The MCP connector is available.[8]

Microsoft Foundry

The MCP connector is in beta. Tool search support is Not in source.[56]

8. Exam domain and task statement

D3.1. Expect a scenario with an over-broad tool set and one wrong call. The credited answer removes or scopes the unneeded tool. A log entry alone is not the fix. (Inference)

If the scenario names an unneeded write capability, pick removal, a cap, or an approval gate.

Remove or scope each tool the task does not need before you add any monitoring.

9. Trap

Full audit logs on a broad tool set look like the safe fix. The permission stays attached, so the next wrong call still runs.

10. Not in source

- Tool count at which accuracy falls on Microsoft Foundry or Claude Models on Google Cloud. Not in source.
- Token count per tool definition for a typical tool. Not in source.

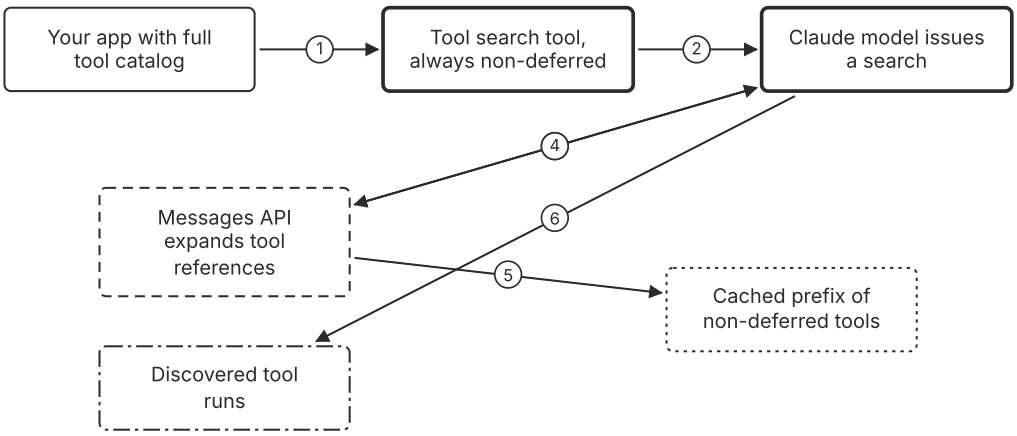
Tool search with deferred tools

1. Introduction and first principle

Deferred tools stay out of the first prompt, and the API adds the found tools when the model searches for them.

- Each search returns up to 5 found tools by default.[54]
- A request can define up to 10,000 tools with defer_loading set to true.[54]
- Regex patterns are limited to 200 characters, and BM25 queries to 500 characters.[54]
- Standard tool use fits when a request has fewer than 10 tools.[54]
- A deferred tool cannot carry cache_control, and the API returns a 400 error.[54]

2. Architecture



Schematic drawn from docs. Arrows: 1. tools array with defer_loading true. 2. search input. 3. server_tool_use search. 4. tool_reference expanded. 5. deferred tools stay out of prefix. 6. tool_use for the found tool. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Add the tool search tool to the tools array without defer_loading.
2. Set defer_loading to true on each tool that few requests use.
3. Keep at least one non-deferred tool, normally the search tool itself.
4. Let the API return up to 5 matches for each search by default.
5. Put a cache breakpoint on a non-deferred tool, not on a deferred one.
6. Add a system prompt line that lists the tool categories you offer.

4. Pros and cons

Pros

- Only the tools a request needs enter the context.
- The API expands discovered tools into full definitions automatically.
- Deferred tools leave the cached tool prefix unchanged.

Cons

- Each search adds a step before the tool call. (Inference)
- A deferred tool cannot carry cache_control, and the API returns an error.
- Search quality depends on clear names and descriptions.

5. Use when, do not use when

Use when

- The catalog holds hundreds of tools from several MCP servers.
- Tool definitions take more than 10k tokens per request.
- Most requests use only a few tools from a large catalog.

Do not use when

- Every request uses every tool in a small set.
- The tool definitions are under 100 tokens in total.
- The model is Claude Opus 4.1 or an earlier model, which lacks tool search.

6. Constraint families

Latency

Accept one extra search step for large catalogs. (Inference) Combination when one choice is not enough. Use standard tool use when the set is small.

Cost

Defer tools that few requests use. Combination when one choice is not enough. Keep frequent tools non-deferred to avoid extra search round trips.

Scale

Allow up to 10,000 deferred tools in one request. Combination when one choice is not enough. Claude can set a limit from 1 to 10,000 in its search input.

Skill

Use clear names and keyword-rich descriptions for each deferred tool. Combination when one choice is not enough. Use a shared name prefix such as github_ for one service.

Operations

Monitor which tools the model discovers. Combination when one choice is not enough. Rewrite descriptions for tools that the model never finds.

7. Native cloud services

Claude Platform on AWS

Server-side tool search works the same as on the Claude API.[6]

Amazon Bedrock

Server-side tool search works only through the InvokeModel API, not the Converse API.[54]

8. Exam domain and task statement

D3.1. Expect a scenario with a large tool library and high token cost. The credited answer defers rarely used tools and keeps frequent tools in the prompt. (Inference)

If the scenario shows 10 or more tools and high cost, pick tool search over a longer prompt.

Defer each tool that few requests use, and keep the most used tools non-deferred.

9. Trap

A longer system prompt that lists every tool looks like a fix. It keeps every schema in the first prompt.

10. Not in source

- Latency added by each search. Not in source.
- Tool search support on Microsoft Foundry. Not in source.

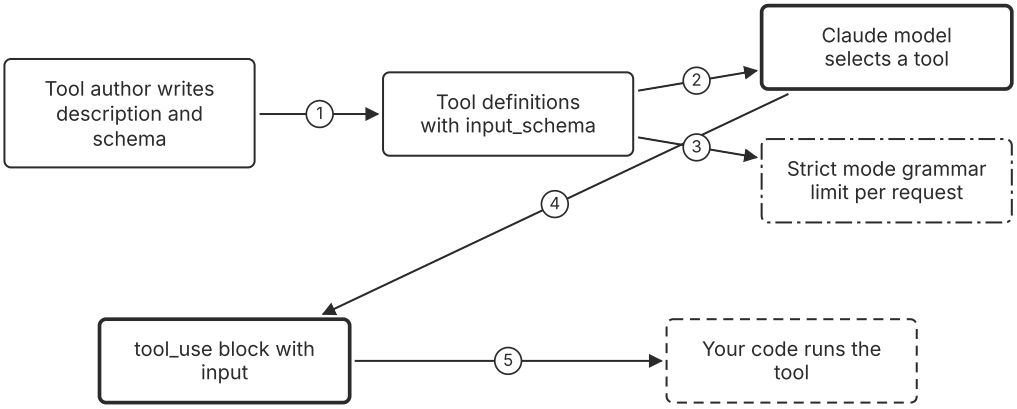
Tool definition quality

1. Introduction and first principle

Clear, few, and consistently named tool definitions give the model the fewest wrong picks.

- Tool descriptions should run at least 3 to 4 sentences for each tool.[57]
- Tool examples add about 20 to 50 tokens for simple examples.[57]
- A request can set strict on at most 20 tools.[30]
- The docs call tool description detail the most important factor in tool performance.[57]

2. Architecture



Schematic drawn from docs. Arrows: 1. one job per tool. 2. tools array in request. 3. strict true count. 4. name and input. 5. validated input. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Describe each tool by its purpose and the conditions that call it.
2. State in each description what the tool does not return.
3. Group related actions under one tool with an action parameter when they share inputs.
4. Prefix each tool name with its service or resource name.
5. Return only the fields the model needs for its next step.
6. Add input_examples for tools with nested or format-sensitive inputs.

4. Pros and cons

Pros

- Clear descriptions are the largest single factor in tool performance, per the docs.
- Consistent prefixes make search and selection unambiguous.
- Compact responses save context on every turn.

Cons

- Long descriptions add tokens to every request.
- Consolidated tools need more conditional logic in the input schema.
- Input examples add tokens on every request.

5. Use when, do not use when

Use when

- Two tools return similar data and the model picks the wrong one.

- A tool takes nested or format-sensitive input.
- The model needs a stable identifier from one call to the next.

Do not use when

- A tool has a one-line job with a clear name and no inputs.
- A consolidated action would mix reads with destructive writes.
- The set already holds a few clear tools.

6. Constraint families

Governance

Keep destructive actions in separate tools with their own gates. (Inference) Combination when one choice is not enough. Share one read tool across several services only when the data scope matches.

Scale

Limit strict tools to 20 per request. Combination when one choice is not enough. Leave non-strict tools outside this limit.

Skill

Write descriptions of at least 3 to 4 sentences for each tool, more for a complex tool. Combination when one choice is not enough. Use input_examples only for tools with complex inputs.

Operations

Use stable slugs or UUIDs in tool responses. Combination when one choice is not enough. Remove fields the next step never reads.

7. Native cloud services

All

Not in source for platform-specific tool definition limits. The strict tool limit is documented on the structured outputs page.[30]

8. Exam domain and task statement

D3.1. Expect a scenario where the model picks a near-duplicate tool. The credited answer merges or renames tools, not a longer prompt. (Inference)

If the scenario shows two similar search tools, pick the option that merges them or renames them for distinct jobs.

Give each job one tool name and one description, and merge tools that do the same job.

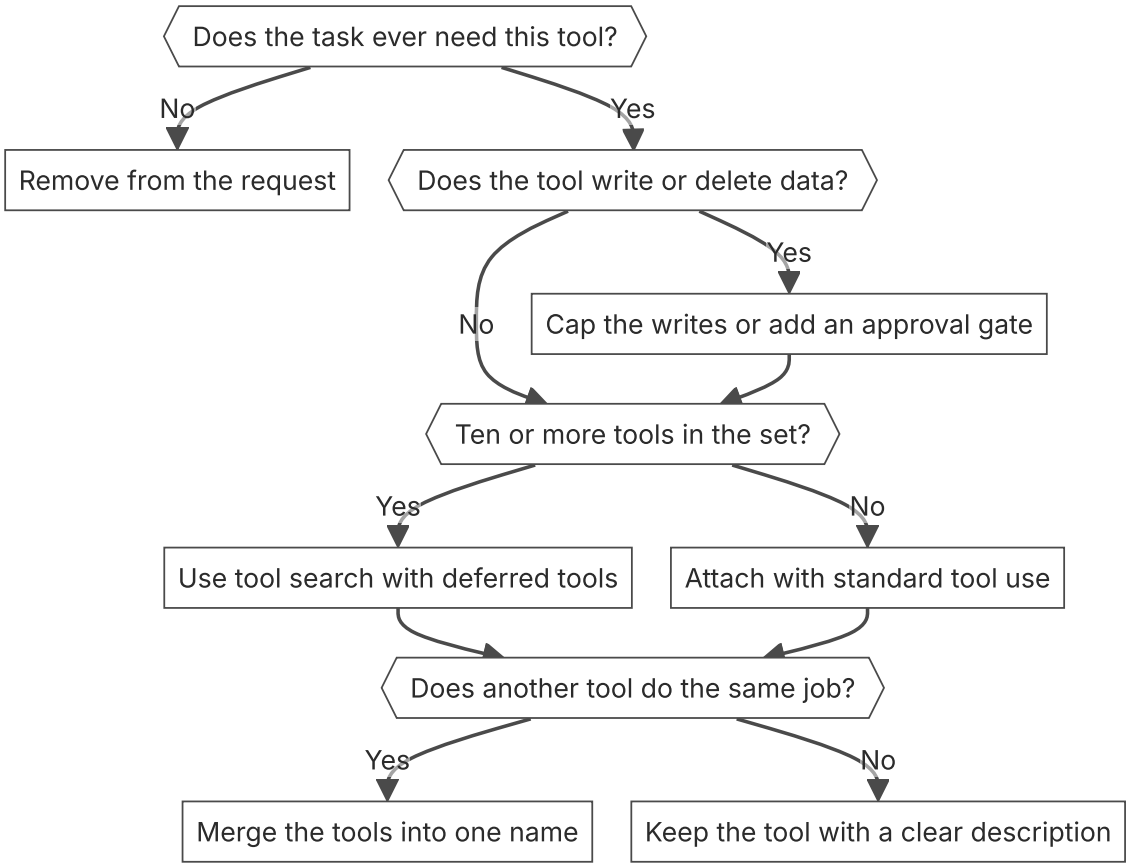
9. Trap

A system prompt rule that says to pick the right search tool looks like a fix. Two tools with the same job still split the model choice.

10. Not in source

- Exact accuracy gain from a merged tool. Not in source.
- Token count per description word. Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Tool count limit per request before accuracy drops on each platform. Not in source.

D3.2 Authentication and authorization in integrations

Domain D3, Integration. Task statement: Give each integration call a verifiable identity and the narrowest access it needs. (Inference: wording from secondary sources)

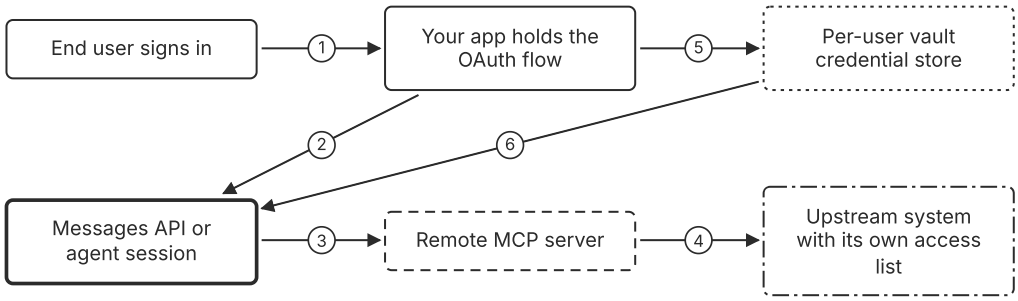
Per-user delegation to MCP servers

1. Introduction and first principle

A remote MCP server authorizes the call from an OAuth token that the client sends for that server, so the token must name the user.

- The MCP connector beta accepts an OAuth Bearer token in authorization_token.[56]
- Local STDIO servers cannot connect directly through the MCP connector.[56]
- Claude Code flags a remote server for sign-in on a 401 or 403 response.[59]
- Vault credentials are write-only, and the API never returns their secret values.[60]
- A claude -p run cannot run the OAuth flow. Sign in first with /mcp or claude mcp login.[59]

2. Architecture



Schematic drawn from docs. Arrows: 1. sign in. 2. request with authorization_token. 3. tool call with bearer token. 4. server calls its own backend. 5. store credential per user. 6. token for mcp_server_url at runtime. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. The app signs the end user in and gets an OAuth access token for the MCP server.
2. The app puts the token in the authorization_token field of the server definition.
3. The MCP connector sends the bearer token with each tool call to that server.
4. A Managed Agents vault holds the credentials of one end user, keyed by mcp_server_url.
5. Anthropic refreshes the access token when the credential includes a refresh block.
6. Claude Code signs in from /mcp in an interactive session before a run.

4. Pros and cons

Pros

- The server receives a token for the user who signed in.
- A vault keeps tokens out of each request body.
- The MCP connector accepts OAuth Bearer tokens for authenticated servers.

Cons

- The app owns the OAuth flow and token refresh for the Messages API.
- The MCP connector is not eligible for zero data retention.
- A vault is scoped to the workspace, so any key with workspace access can use it.

5. Use when, do not use when

Use when

- The server must act for a named person.
- The remote server requires OAuth before it accepts a call.
- The audit must name the person who asked for the action.

Do not use when

- The server runs as a local stdio process, because the MCP connector cannot reach it.
- A batch job has no human actor and one narrow service identity is enough.
- Claude Code runs with claude -p and the server needs a new sign-in.

6. Constraint families

Data location

Use an MCP tunnel for a server in a private network. Combination when one choice is not enough. Plan for research preview terms with no uptime commitment.

Isolation

Store one credential for each end user in a vault. Combination when one choice is not enough. Use separate workspaces for tenants that must not share keys.

Governance

Pass each user's token for each server. Combination when one choice is not enough. Set a refresh block so Anthropic refreshes the token for long sessions.

Operations

Sign in again when the server rejects the stored refresh token. Combination when one choice is not enough. Run claude mcp login before a non-interactive run.

7. Native cloud services

Claude Platform on AWS

OAuth authentication is not supported. Use SigV4 or an API key. The MCP connector is in beta.[6]

Amazon Bedrock

The MCP connector is not supported on Bedrock.[5]

Claude Models on Google Cloud (Google managed)

The MCP connector is not supported.[7]

Claude Models on Google Cloud (Anthropic managed)

The MCP connector is available.[8]

Microsoft Foundry

The MCP connector is in beta. Token handling for MCP servers is Not in source.[5][6]

8. Exam domain and task statement

D3.2. Expect a scenario that asks who the agent acted for. The credited answer uses per-user credentials with a token the server checks, not a shared key. (Inference)

If the scenario offers a session id or a prompt field as identity, reject it.

Give each server a per-user credential when the audit must name the person who asked.

9. Trap

A shared service key with a user name in the prompt looks like attribution. The system cannot verify a name that the user typed.

10. Not in source

- The MCP audience check through RFC 8707 and the token passthrough ban. Not in source.

- Dynamic client registration status in the MCP authorization profile. Not in source.
- Revocation delay for a token that a server cached. Not in source.

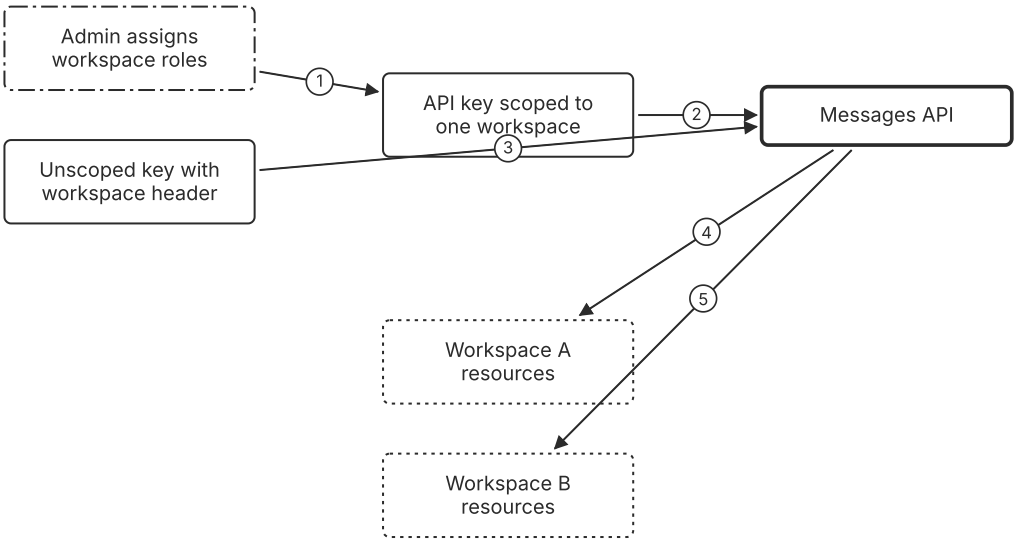
Workspace and API key boundary

1. Introduction and first principle

A workspace groups keys, members, and limits, and a key scoped to one workspace reaches only that workspace.

- An organization can hold up to 100 workspaces by default.[11]
- The Default Workspace cannot be renamed, archived, or deleted.[11]
- A workspace archive also archives its API keys, and the action cannot be undone.[11]
- A Claude Code key stops when its owner leaves the workspace or the organization.[11]
- Rate limits are defined by usage tier at the organization level.[10]

2. Architecture



Schematic drawn from docs. Arrows: 1. create scoped key. 2. request. 3. anthropic-workspace-id header. 4. in-scope access. 5. only if key has access. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Create one workspace for each project, environment, or tenant that needs separate limits. (Inference)
2. Create an API key and scope it to one workspace.
3. Send the anthropic-workspace-id header with requests from an unscoped key.
4. Assign the workspace roles that the team needs and no more.
5. Set spend limits for each workspace in the Console.
6. Archive the workspace when the project ends, and its keys archive with it.

4. Pros and cons

Pros

- Separate keys and resources for each project or tenant.
- The Claude Code workspace keeps developer traffic apart from other workloads.
- Each workspace has its own spend limit.

Cons

- Each workspace adds setup and role management.
- A workspace archive cannot be undone.

- The Default Workspace cannot be renamed, archived, or deleted.

5. Use when, do not use when

Use when

- Tenants must not share resources or prompt caches.
- One team needs its own spend cap.
- Test and production environments need separate keys.

Do not use when

- One internal script needs one key and no separate limits.
- The organization needs more workspaces than the default cap without an account request.
- The team needs per-member workspace roles on Claude Platform on AWS, because member management is not available there.

6. Constraint families

Cost

Set a monthly spend limit for each workspace. Combination when one choice is not enough. Keep each workspace limit below the tier spend cap. (Inference)

Quota

Set rate limits for each workspace and model in the Console. Combination when one choice is not enough. Check the organization tier limits, which apply at the organization level.

Data location

Bind each Claude Platform on AWS workspace to one region. Combination when one choice is not enough. Call the workspace only through its region endpoint.

Isolation

Use one workspace for each tenant that must not share resources. (Inference) Combination when one choice is not enough. Prompt caches stay isolated by workspace on the Claude API, Claude Platform on AWS, and Microsoft Foundry.

Governance

Grant the least workspace role for each member. Combination when one choice is not enough. Use the Workspace Limited Developer role when the member needs no tracing views.

7. Native cloud services

Claude Platform on AWS

A workspace is bound to one AWS region and serves as the IAM resource through its workspace ARN.[6]

Claude Models on Google Cloud (Anthropic managed)

Files, skills, batches, and sessions belong to the workspace whose ID the request sets.[8]

Microsoft Foundry

Prompt caches are isolated by workspace within an organization.[47]

Amazon Bedrock

Prompt caches are isolated by organization, not by workspace.[47]

Claude Models on Google Cloud (Google managed)

Prompt caches are isolated by organization, not by workspace.[47]

8. Exam domain and task statement

D3.2. Expect a scenario with shared keys across teams or tenants. The credited answer scopes keys and workspaces to the boundary that owns the data. (Inference)

A key scoped to one workspace cannot reach another workspace, even with a header that names it.

Scope the key to the one workspace that owns the data, and give members the least role.

9. Trap

An unscoped key with a workspace header looks isolated. The key still reaches every workspace that its user or service account can access.

10. Not in source

- Role limits for workspace admins on Claude Platform on AWS beyond the listed exceptions. Not in source.

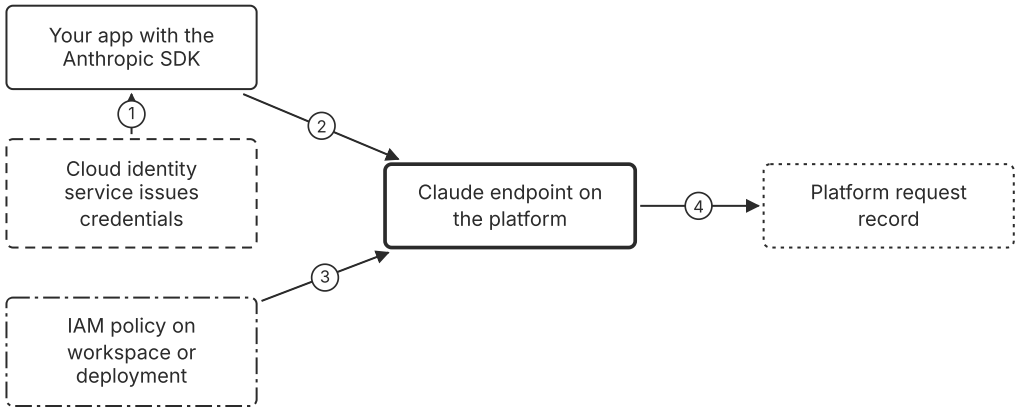
Cloud platform identity

1. Introduction and first principle

Each cloud platform checks its own identity and IAM policy before it runs a request, so the policy becomes the access control.

- Outbound web identity federation is disabled by default on every AWS account.[6]
- Bedrock bearer tokens have a 12-hour maximum and are the least preferred option.[5]
- The inference data processor is Anthropic on Claude Platform on AWS and AWS on Bedrock.[6]
- The default Bedrock quota is 2 million input tokens per minute.[5]
- Google managed Claude needs the aiplatform.endpoints.predict permission.[7]

2. Architecture



Schematic drawn from docs. Arrows: 1. temporary credentials. 2. signed request. 3. allow or deny per action. 4. request ID for lookup. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. The SDK loads AWS credentials from the default credential provider chain.
2. The SDK signs each request with SigV4 for the workspace region.
3. IAM checks aws-external-anthropic:CreateInference on the workspace ARN.
4. Add aws-external-anthropic:CallWithBearerToken only for API key calls.
5. Enable outbound web identity federation once for each AWS account.
6. Use the AWS request ID to look up the call in CloudTrail.

4. Pros and cons

Pros

- Current IAM roles, policies, and audit tools apply to the calls.
- Each workspace has its own ARN for access policy.
- Temporary credentials from the cloud identity service limit exposure.

Cons

- Outbound web identity federation is disabled by default, and every request fails until you enable it.

- OAuth is not supported on Claude Platform on AWS.
- The Bedrock service role path needs iam:PassRole on the service role ARN.

5. Use when, do not use when

Use when

- The organization manages access with AWS IAM roles and policies.
- The organization manages access with Entra ID roles on a Foundry deployment.
- Access must be limited to one workspace or deployment.

Do not use when

- The workload needs user consent through OAuth on Claude Platform on AWS.
- The workload would use a long-term Bedrock key. The docs show a deny policy that blocks long-term keys.
- The workload runs on Google managed Claude with no IAM role for predict calls.

6. Constraint families

Quota

Request a higher tier through your Anthropic account representative on Claude Platform on AWS. Combination when one choice is not enough. On Bedrock, request up to 5 million input TPM without Anthropic approval.

Data location

Match the region to the workspace region. Combination when one choice is not enough. On Bedrock, AWS is the inference data processor, not Anthropic.

Governance

Grant access with IAM policies on the workspace ARN. Combination when one choice is not enough. Use an API key for a CI job only with CallWithBearerToken.

Operations

Keep the Anthropic request ID for support tickets. Combination when one choice is not enough. Keep the AWS request ID for CloudTrail lookups.

7. Native cloud services

Claude Platform on AWS

Authentication uses SigV4 or an API key from the AWS Console. Outbound web identity federation must be enabled once per account.[6]

Amazon Bedrock

Access uses a Bedrock service role, IAM assumed roles, or bearer tokens. Assumed roles and bearer tokens have a 12-hour maximum.[5]

Microsoft Foundry

Access management uses Azure-issued API keys or Entra ID role-based access control.[9]

Claude Models on Google Cloud (Google managed)

The caller identity needs an IAM role with aiplatform.endpoints.predict.[7]

Claude Models on Google Cloud (Anthropic managed)

The caller needs roles/claude.user on the project and Application Default Credentials. IAM roles apply to every workspace in the project.[8]

8. Exam domain and task statement

D3.2. Expect a scenario that names a cloud platform and asks how to control access. The credited answer uses that platform identity model and a scoped policy. (Inference)

If the scenario needs OAuth user consent on Claude Platform on AWS, reject that option because OAuth is not supported.

Use the platform identity and scope the IAM policy to the workspace or deployment.

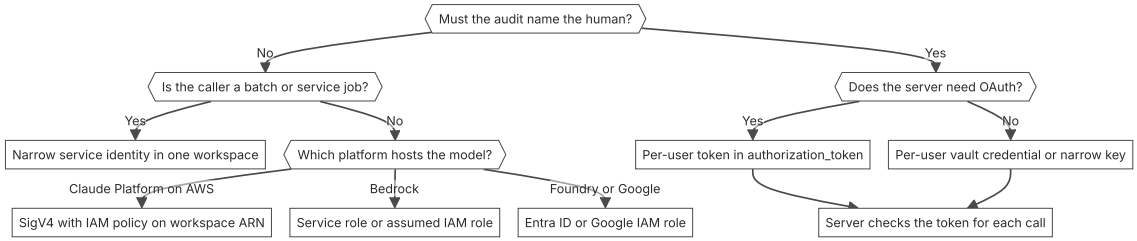
9. Trap

A long-lived static key in code looks simpler. It bypasses the platform identity, so the IAM policy cannot scope or revoke each caller.

10. Not in source

- Microsoft Foundry token lifetime for Entra ID tokens. Not in source.
- Google managed IAM role names beyond the predict permission. Not in source.

Decision flow



Unit items not in source

- Audience and token passthrough rules of the MCP authorization profile. The mirror has no page on them. Not in source.
- Revocation propagation delay for tokens. Not in source.
- Consent screen wording for MCP OAuth. Not in source.

D3.3 Accuracy and latency trade-offs

Domain D3, Integration. Task statement: Meet a latency target and an accuracy floor by setting model, effort, and delivery mode with measured receipts. (Inference: wording from secondary sources)

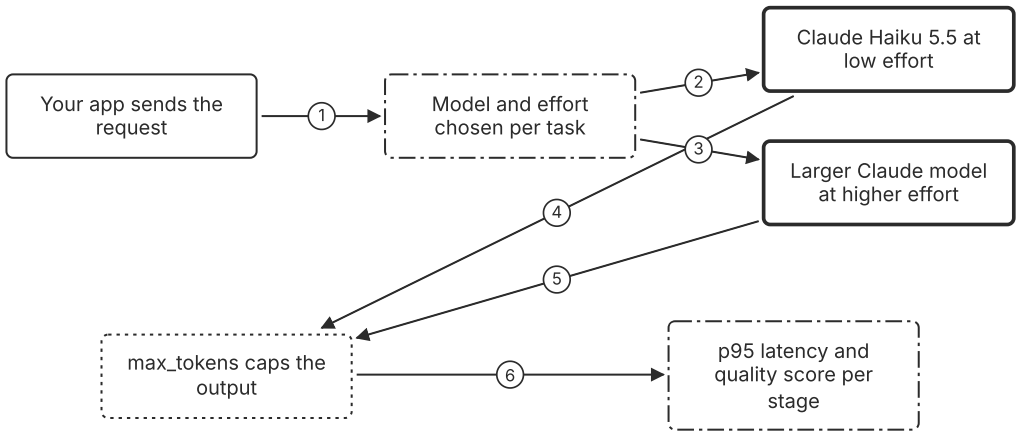
Model and effort choice for latency

1. Introduction and first principle

Latency rises with model size, output length, and effort, so choose the smallest model and effort that pass the quality target.

- Claude Haiku 5.5 defaults to medium effort.[43]
- Fast mode delivers up to 2.5 times higher output tokens per second.[62]
- Fast mode speed benefits apply to output tokens per second, not to time to first token.[62]
- Fast mode on Claude Opus 5.5 costs $8 per MTok input and $40 per MTok output.[62]
- Effort is a behavioral signal, not a strict token budget.[43]
- The docs say to reach good quality first, then lower latency.[63]

2. Architecture



Schematic drawn from docs. Arrows: 1. task and quality target. 2. simple task. 3. hard task with measured gap. 4. bounded output. 5. bounded output. 6. timing and score. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Engineer a prompt that works well first, then measure baseline latency and time to first token.
2. Choose the smallest model that meets the quality target on your test set.
3. Set output_config effort to a lower level for simple tasks.
4. Set max_tokens to a value that the expected output fits.
5. Ask for a shorter answer in the prompt when output length adds time.
6. Move hard tasks to a larger model only when the measured quality gap justifies the time. (Inference)

4. Pros and cons

Pros

- The docs name Haiku 5.5 as the fastest option that keeps high intelligence.
- Lower effort cuts output tokens and cost.
- Fast mode raises output speed on supported Opus models without a change in intelligence.

Cons

- Lower effort reduces capability on hard tasks.
- Fast mode uses premium pricing on every token.

- Fast mode speed benefits do not target time to first token.

5. Use when, do not use when

Use when

- A user waits on an interactive answer with a fixed latency target.
- Tasks are short and repeat often.
- The quality test passes at a lower effort level.

Do not use when

- The prompt is not yet tuned, because the docs say to reach good quality first.
- The fast mode request targets Claude Haiku 5.5, which fast mode does not support.
- The task needs the highest capability and a loose latency target.

6. Constraint families

Latency

Use Haiku 5.5 at low effort for time-sensitive calls. Combination when one choice is not enough. Raise effort to medium for calls that fail the quality check.

Latency

Use fast mode on a supported Opus model for long outputs. Combination when one choice is not enough. Accept the premium price for every token in the fast request.

Cost

Lower effort to cut output tokens. Combination when one choice is not enough. Keep high effort for calls where a wrong answer is costly.

Quota

Keep output tokens per minute under the model limit for your tier. Combination when one choice is not enough. Request a higher tier through your Anthropic account representative on Claude Platform on AWS.

Operations

Track p95 latency for each stage beside the average. (Inference) Combination when one choice is not enough. Alert on the tail latency. (Inference)

7. Native cloud services

Claude Platform on AWS

Fast mode is not available. Effort and the Messages API work as on the Claude API.[6]

Claude Models on Google Cloud (Anthropic managed)

Fast mode is available for Claude Opus 5.5.[8]

Microsoft Foundry

Fast mode for Claude Opus 5.5 is available only on deployments hosted on Azure.[62]

Amazon Bedrock

Bedrock is not in the fast mode list of supported platforms.[62]

8. Exam domain and task statement

D3.3. Expect a scenario with a latency target, an accuracy floor, and a proposal to add a stage or a bigger model. The credited answer measures each stage and cuts the weakest one. (Inference)

When the scenario states a latency limit, reject an option that adds a slower step before you measure it.

Pick the smallest model and effort level that passes the measured quality target and the latency target.

9. Trap

A bigger model looks like the accuracy fix. It adds time per call and can break the latency target.

10. Not in source

- Ablation receipts for accuracy per stage. The docs do not set this method. (Inference)
- Fast mode latency gain on time to first token. Not in source.

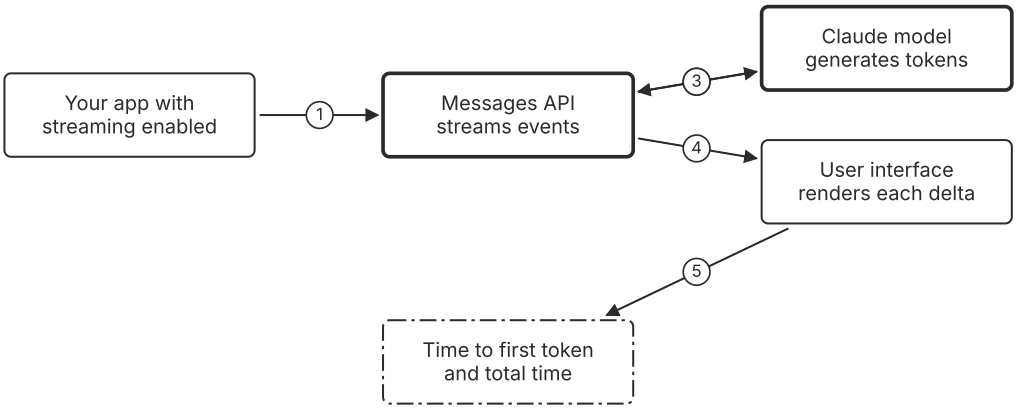
Streaming for perceived latency

1. Introduction and first principle

Streaming sends output as the model generates it, so users see the first tokens before the full response ends.

- Setting stream to true returns the response as server-sent events.[17]
- Time to first token measures the time to the first output token.[63]
- After a 504 timeout error, the docs suggest the streaming Messages API for long requests.[15]
- The SDK validates that Messages requests without streaming are not expected to exceed a 10-minute timeout.[15]

2. Architecture



Schematic drawn from docs. Arrows: 1. request with stream true. 2. generate. 3. token events. 4. text deltas as they arrive. 5. first token timestamp. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Set stream to true on the Messages request.
2. The API sends server-sent events as the model generates output.
3. Your app renders each text delta as soon as it arrives.
4. Measure time to first token and total time as two separate values.
5. Handle streaming refusals in the client before you show the output.

4. Pros and cons

Pros

- Users see output early, which improves perceived responsiveness.
- The same request works for text, tool use, and thinking deltas.
- Long requests avoid the timeout risk of one long response.

Cons

- The client must handle partial output and mid-stream errors.
- Streaming does not reduce the model work or the token count.
- Code that reads the full text runs only after the stream ends.

5. Use when, do not use when

Use when

- A person reads a chat answer as it appears.
- Responses are long and the first sentence matters most.
- A request could run past the requests without streaming timeout.

Do not use when

- A batch job reads only the final object.
- A downstream system needs the full response before it starts.
- The client cannot process incremental events.

6. Constraint families

Latency

Stream text to the user as tokens arrive. Combination when one choice is not enough. Pair streaming with a smaller model to lower the first-token wait. (Inference)

Governance

Check streaming refusals in the client before display. Combination when one choice is not enough. Keep refusal handling on the server for batch results. (Inference)

Scale

Open one stream for each user request. (Inference) Combination when one choice is not enough. Limits for concurrent streams are Not in source.

Operations

Track time to first token and total time as two metrics. Combination when one choice is not enough. Alert on time to first token for the interactive path only.

7. Native cloud services

Claude Platform on AWS

Full server-sent events streaming is supported.[6]

Amazon Bedrock

The Messages endpoint on Bedrock uses standard server-sent events streaming.[5]

Claude Models on Google Cloud (Google managed)

Streaming requests use the streamRawPredict method instead of rawPredict.[7]

Microsoft Foundry

Streaming behavior on Foundry is Not in source.[9]

8. Exam domain and task statement

D3.3. Expect a chat scenario where users complain about the wait. The credited answer streams the response and measures time to first token. (Inference)

If the scenario needs the full result before any step, streaming does not meet the target.

Stream when a person reads the output as it arrives, and measure time to first token.

9. Trap

Streaming looks like a generation speed-up. It shows output sooner, and the total generation time stays about the same. (Inference)

10. Not in source

- Concurrent stream limit per workspace. Not in source.
- Streaming support details on Microsoft Foundry. Not in source.

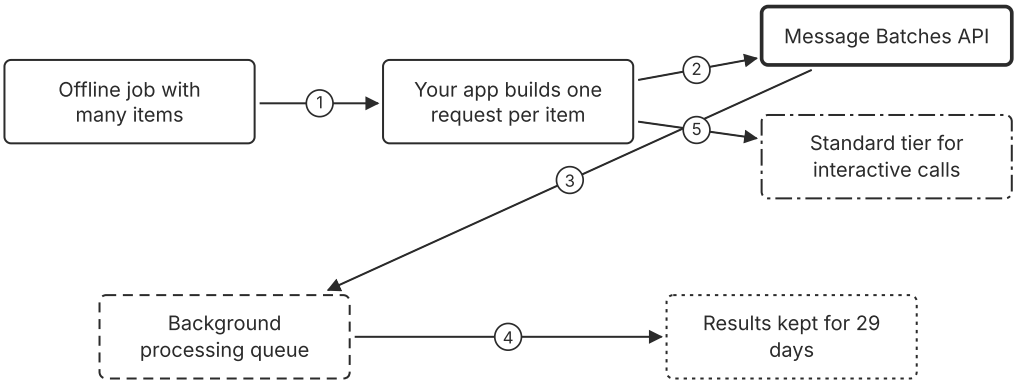
Batch processing and service tiers for throughput

1. Introduction and first principle

Work that can wait trades response time for half the standard price, and the batch runs in the background.

- Batch usage is charged at 50 percent of standard API prices.[16]
- A batch holds up to 100,000 requests or 256 MB, whichever is reached first.[16]
- Most batches finish within 1 hour.[16]
- Batch results are available for 29 days after creation.[16]
- A batch expires if processing does not complete within 24 hours.[16]
- Priority Tier capacity commitments are no longer available for purchase.[64]

2. Architecture



Schematic drawn from docs. Arrows: 1. list of items. 2. create batch. 3. process up to 24 hours. 4. results ready. 5. interactive calls go elsewhere. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Build one request for each item and give each request a custom_id.
2. Submit the list of requests to the Message Batches API.
3. Poll the batch until processing_status reads ended.
4. Read the results when processing ends or after 24 hours, whichever comes first.
5. Download the results within 29 days of creation.
6. Match each result to its input with the custom_id.

4. Pros and cons

Pros

- Batch usage is charged at 50 percent of standard API prices.
- One batch holds up to 100,000 requests or 256 MB.
- Batch runs outside normal capacity, as the service tiers page states.

Cons

- Results can take up to 24 hours.
- A batch can exceed the workspace spend limit slightly.
- Batch is not an option for a user who waits for a reply.

5. Use when, do not use when

Use when

- An evaluation run or document backfill can wait a day.
- Throughput and cost matter more than the first response time.
- Many requests share a long prefix that the cache can reuse.

Do not use when

- A user waits for a reply in a chat window.
- The platform is Microsoft Foundry, Bedrock, or Google managed, where the Message Batches API is not supported.
- A single request has a latency target of seconds.

6. Constraint families

Latency

Use the Standard tier for interactive traffic. Combination when one choice is not enough. Current Priority Tier commitments continue until the contract end date.

Cost

Use Batch for offline work at half the standard price. Combination when one choice is not enough. Use the 1-hour cache for batch runs longer than 5 minutes.

Quota

Check the batch rate limits for requests in the queue. Combination when one choice is not enough. Submit a large job in several batches to stay under queue limits. (Inference)

Scale

Keep each batch under 100,000 requests and 256 MB. Combination when one choice is not enough. Split a larger job into several batches.

Operations

Store each custom_id with the source record. Combination when one choice is not enough. Download the results before the 29-day window ends.

7. Native cloud services

Claude Platform on AWS

The Message Batches API is supported.[6]

Claude Models on Google Cloud (Anthropic managed)

The Message Batches API works as it does on the Claude API.[8]

Claude Models on Google Cloud (Google managed)

The Message Batches API is not supported.[7]

Microsoft Foundry

The Message Batches API is not supported.[9]

Amazon Bedrock

The Message Batches API is not supported.[5]

8. Exam domain and task statement

D3.3. Expect a scenario with a large offline set and a throughput or cost goal. The credited answer moves that work to Batch and keeps interactive calls elsewhere. (Inference)

If the scenario needs an answer in seconds, reject Batch whatever the volume.

Use Batch when the caller accepts results within 24 hours and needs half the standard price.

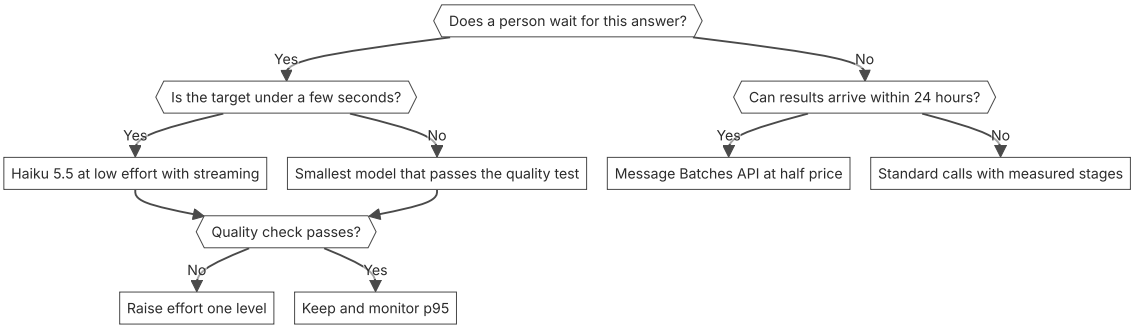
9. Trap

Batch looks like a speed fix for a large job. Results can take up to 24 hours, so it does not meet a seconds-level target.

10. Not in source

- Amazon Bedrock batch inference details outside the Claude Messages path. Not in source.
- Batch throughput in requests per second. Not in source.

Decision flow



Unit items not in source

- Console steps for the latency work beyond the rate limits page: Not in source.
- Accuracy receipts for each stage. The docs do not define an ablation method. (Inference)
- Median and p95 latency figures per model. Not in source.

D3.4 Observability at scale

Domain D3, Integration. Task statement: Rebuild any integration run from traces, usage data, and telemetry, without text logs alone. (Inference: wording from secondary sources)

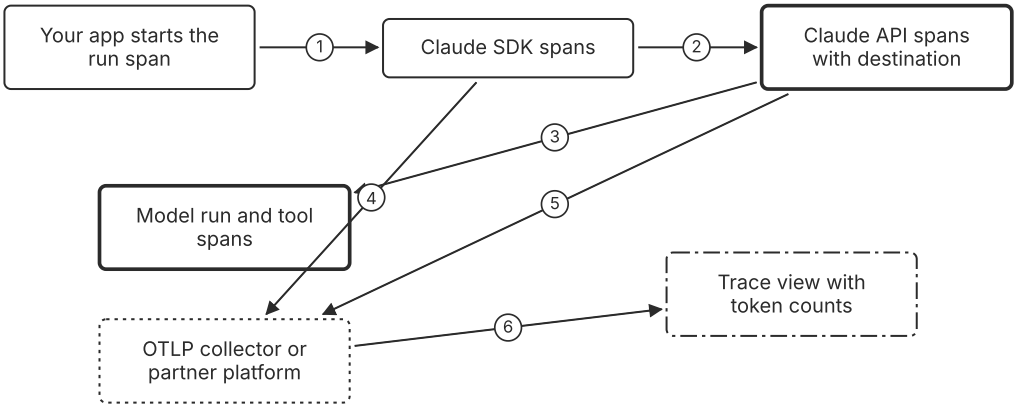
Request tracing with OpenTelemetry

1. Introduction and first principle

A trace records each step of one run with a shared trace ID, so you can rebuild the run from the spans.

- Metadata only is the default content mode.[66]
- Each span holds up to 0.75 MB of content by default.[66]
- Span delivery is best effort, so a span can be lost.[66]
- The gen_ai.usage.input_tokens attribute already includes cache read and write tokens.[66]
- A caller can turn off the sampled flag in the traceparent header to leave out its spans.[67]

2. Architecture



Schematic drawn from docs. Arrows: 1. one span per call. 2. traceparent header. 3. model request span. 4. export SDK spans. 5. export API spans. 6. rebuild the run. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Set up SDK tracing with a tracer provider in your application.
2. Add an API tracing destination in the Claude Console.
3. Keep the metadata only content mode for routine exports.
4. Choose the full content mode only where your retention rules allow prompt storage.
5. Sum token usage from one source only, so each token counts once.
6. Check the truncated attribute before you read a long span.

4. Pros and cons

Pros

- Each model run and tool use appears in one trace.
- Any platform that accepts OTLP can receive the spans.
- Token attributes on each span support cost per request.

Cons

- Full content stores prompts and responses in your observability platform.
- Span delivery is best effort, so a span can be lost.
- Content over the size limit is shortened and marked as truncated.

5. Use when, do not use when

Use when

- An incident needs the path of one request through several calls.
- Cost per request or per feature must be attributed.
- Your own application traces must link to Claude calls.

Do not use when

- Your organization has a HIPAA, FERPA, or K-12 restriction and needs prompt text in API tracing, because those destinations export metadata only.
- A single call with no tools needs only the prompt log.
- The data path must guarantee every span, since delivery is best effort.

6. Constraint families

Cost

Sum the API spans for token totals on API tracing. Combination when one choice is not enough. Add compaction tokens when a request has compactions.

Data location

Use full content with zero data retention, which holds exported content in memory only for Messages API requests. Combination when one choice is not enough. Claude Managed Agents is not eligible for zero data retention.

Governance

Use metadata only for routine exports. Combination when one choice is not enough. Use full content for a narrow debug set with its own retention rule.

Operations

Keep the default content limit of 0.75 MB per span. Combination when one choice is not enough. Raise the limit only when your platform accepts larger payloads.

7. Native cloud services

Claude Platform on AWS

API tracing is supported. Set up the destination in the Claude Console with Admin permission.[66]

Amazon Bedrock

SDK tracing works on any platform. API tracing lists only Claude Platform and Claude Platform on AWS.[66]

Microsoft Foundry

SDK tracing works on any platform. API tracing lists only Claude Platform and Claude Platform on AWS.[66]

8. Exam domain and task statement

D3.4. Expect a scenario where a bad outcome cannot be traced to its cause. The credited answer uses traces with one ID and cost per hop, not more text logs. (Inference)

If the scenario asks which tool call acted, pick the trace spans over the prompt text.

Trace each hop that calls a model or a tool, and tie the hops to one trace ID.

9. Trap

More prompt logs or longer retention keeps the blind spot. The logs hold no tool or retrieval hop.

10. Not in source

- Retention period for traces in each partner platform. Not in source.
- Span delivery latency in seconds. Not in source.

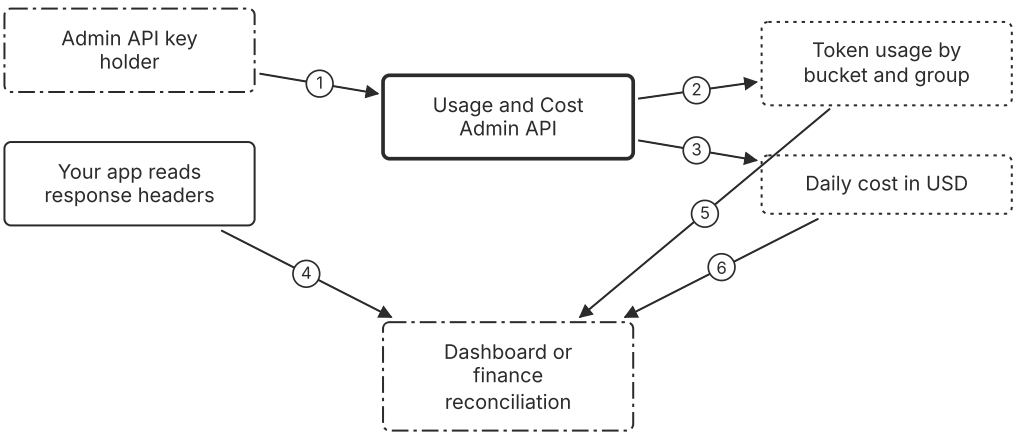
Usage, cost, and rate limit reports

1. Introduction and first principle

Usage and cost reports group token counts and dollars by workspace, key, and model, and response headers show live capacity.

- The usage report default is 7 buckets for one-day width, and the maximum is 31.[68]
- The cost report returns USD amounts in cents, with daily buckets only.[68]
- For most models, cache read tokens do not count toward input tokens per minute.[10]
- A spend-cap 429 response has no retry-after header.[10]
- Every API response includes a request-id header.[15]

2. Architecture



Schematic drawn from docs. Arrows: 1. request with Admin key. 2. group by model or workspace. 3. daily buckets. 4. remaining capacity. 5. trend and split. 6. reconcile with billing. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Create an Admin API key for the organization.
2. Call the usage report endpoint with a bucket width of one hour or one day.
3. Add group_by for model, workspace, or API key to split the totals.
4. Call the cost report endpoint for daily totals in USD.
5. Read the anthropic-ratelimit headers on each response to track capacity left.
6. Compare the report totals with the Console Usage page before you reconcile. (Inference)

4. Pros and cons

Pros

- Reports split spend by workspace, API key, model, and service tier.
- Reports suit finance reconciliation with billing.
- Response headers show live capacity without a separate call.

Cons

- Individual accounts cannot use the Admin API.
- The cost report offers daily buckets only.
- Claude Platform on AWS has no programmatic usage or cost endpoints.

5. Use when, do not use when

Use when

- Finance needs a monthly split of spend.

- A team asks what one workspace spent in a week.
- Capacity planning needs a trend of token use.

Do not use when

- You need cost for one request in real time, so use the trace spans.
- The platform is Claude Platform on AWS and you need a programmatic report.
- A workspace key must call the report endpoints, which it cannot do.

6. Constraint families

Cost

Use the cost report for daily totals in USD. Combination when one choice is not enough. Use the usage report with hourly buckets to find spikes.

Quota

Read the anthropic-ratelimit headers on each response. Combination when one choice is not enough. Wait for the retry-after value before the next try on a 429.

Data location

Read usage in the Console on Claude Platform on AWS. Combination when one choice is not enough. Programmatic endpoints are not available there.

Governance

Use an Admin API key or a service account key that is not scoped to a workspace. Combination when one choice is not enough. Keep workspace keys out of report jobs.

7. Native cloud services

Claude Platform on AWS

Programmatic Usage and Cost endpoints are not available. Use the Console pages instead.[6][8]

Microsoft Foundry

Admin API endpoints are not supported. Foundry does not return Anthropic rate limit headers.[9]

Amazon Bedrock

The default quota is 2 million input tokens per minute. AWS enforces the requests per minute limits.[5]

Claude Models on Google Cloud (Google managed)

Google Cloud manages rate limits and quotas for this platform.[7]

8. Exam domain and task statement

D3.4. Expect a scenario about spend attribution or throttling. The credited answer uses the usage and cost reports or response headers, not estimates from app logs. (Inference)

A spend-cap 429 has no retry-after header and stays failed until access resumes.

Use the usage and cost reports for spend, and response headers for live capacity.

9. Trap

Totals of input_tokens from each response look complete. That field counts only tokens after the last cache breakpoint.

10. Not in source

- Usage report retention period beyond the documented windows. Not in source.
- Replacement headers for Anthropic rate limit values on Microsoft Foundry. Not in source.

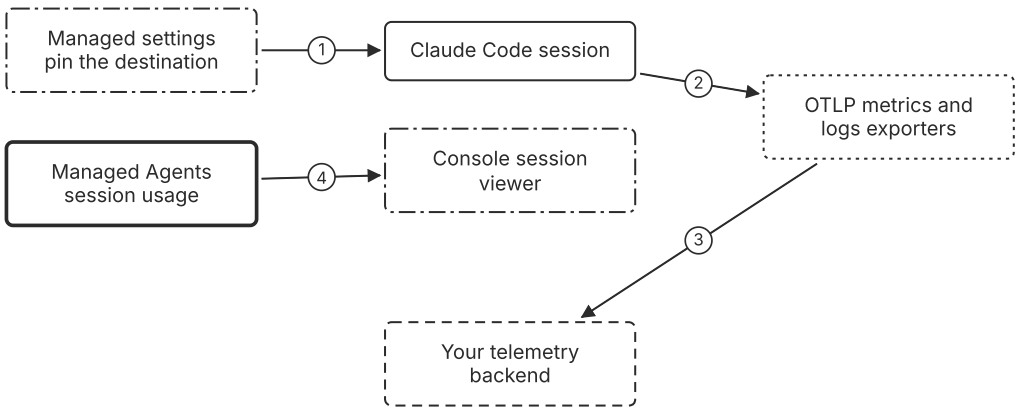
Claude Code and agent session telemetry

1. Introduction and first principle

Claude Code exports its own metrics and events to a destination you set, and managed settings keep that destination fixed.

- The default metric export interval is 60000 milliseconds.[69]
- The default logs export interval is 5000 milliseconds.[69]
- User prompt content is not collected by default, and only prompt length is recorded.[69]
- The metric claude_code.cost.usage reports cost in USD.[69]
- The session object usage field holds cumulative totals, read after the session goes idle.[70]

2. Architecture



Schematic drawn from docs. Arrows: 1. endpoint and credentials. 2. metrics and events. 3. OTLP export. 4. usage totals and threads. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Set CLAUDE_CODE_ENABLE_TELEMETRY to 1.
2. Set OTEL_METRICS_EXPORTER and OTEL_LOGS_EXPORTER to otlp.
3. Set the OTLP protocol and endpoint for your collector.
4. Leave OTEL_LOG_USER_PROMPTS unset unless prompt text is allowed.
5. Set OTEL_LOG_TOOL_DETAILS to 1 to capture MCP call arguments.
6. Pin the OTLP endpoint and credentials in managed settings.
7. Check claude_code.session.count in the backend to confirm the pipeline.

4. Pros and cons

Pros

- Standard metrics report cost and token use per session.
- MCP activity events show the server, tool, and decision.
- Managed settings keep the destination fixed for developers.

Cons

- Tool details can include call arguments, which may hold sensitive values.
- Sign-in with a Claude account adds user.email to the attributes sent to your endpoint.
- Subprocesses started by Claude Code do not inherit the OTel variables.

5. Use when, do not use when

Use when

- An organization needs cost and tool audit for developer use of Claude Code.
- MCP call arguments must appear in the audit trail.

- Managed settings can deliver the destination to every developer.

Do not use when

- A repository setting must turn telemetry on, because Claude Code ignores OTel variables in project settings.
- A developer needs prompt text in every session, since prompt text is off by default.
- The Bash tool needs its own OTel variables, because Claude Code does not pass them down.

6. Constraint families

Cost

Track claude_code.cost.usage in USD per session. Combination when one choice is not enough. Use claude_code.token.usage for token trends.

Data location

Send prompt content only to the endpoint you configure. Combination when one choice is not enough. Mask the prompt text attributes in your collector if policy forbids them. (Inference)

Governance

Pin the endpoint and credentials in managed settings. Combination when one choice is not enough. Set the per-signal variables in managed settings when one signal needs its own collector.

Operations

Use the console exporter for local checks. Combination when one choice is not enough. Set shorter export intervals only for short checks.

7. Native cloud services

Amazon Bedrock

Telemetry carries only user.id and session.id. Attach user identity with OTEL_RESOURCE_ATTRIBUTES.[69]

Microsoft Foundry

Telemetry carries only user.id and session.id. Attach user identity with OTEL_RESOURCE_ATTRIBUTES.[69]

Claude Models on Google Cloud (Google managed)

With Agent Platform, telemetry carries only user.id and session.id. Attach user identity with OTEL_RESOURCE_ATTRIBUTES.[69]

8. Exam domain and task statement

D3.4. Expect a scenario about audit of developer tool use. The credited answer uses managed telemetry settings and tool detail events. (Inference)

If the scenario asks for MCP call arguments, enable tool details and the logs exporter.

Set telemetry destinations in managed settings, and keep prompt text off by default.

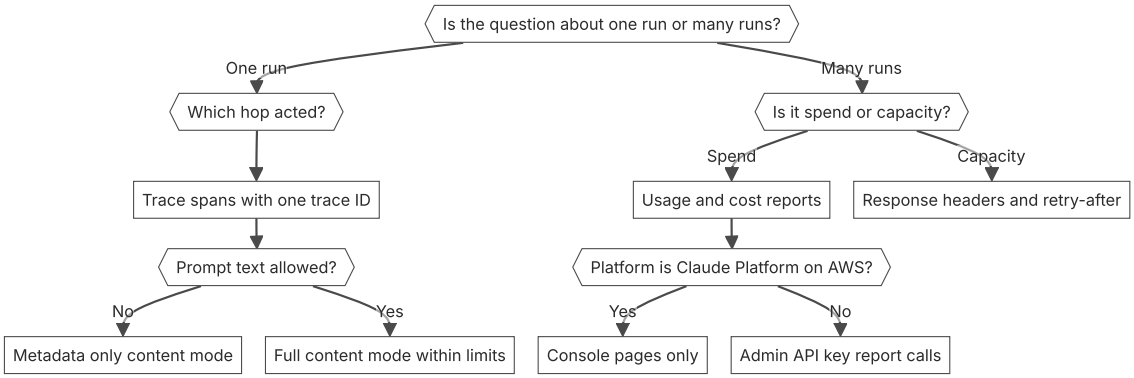
9. Trap

A repository setting that enables telemetry looks simple. Claude Code ignores OTel variables in project settings, so the destination is not controlled.

10. Not in source

- Telemetry backend retention and cost in dollars. Not in source.
- Claude Code telemetry on Claude Platform on AWS. Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source for the usage report beyond the documented Usage page.
- Trace sampling rates for production fleets. Not in source.

D3.5 The RAG pipeline

Domain D3, Integration. Task statement: Build a retrieval pipeline with named stages and citations that trace each claim to a source. (Inference: wording from secondary sources)

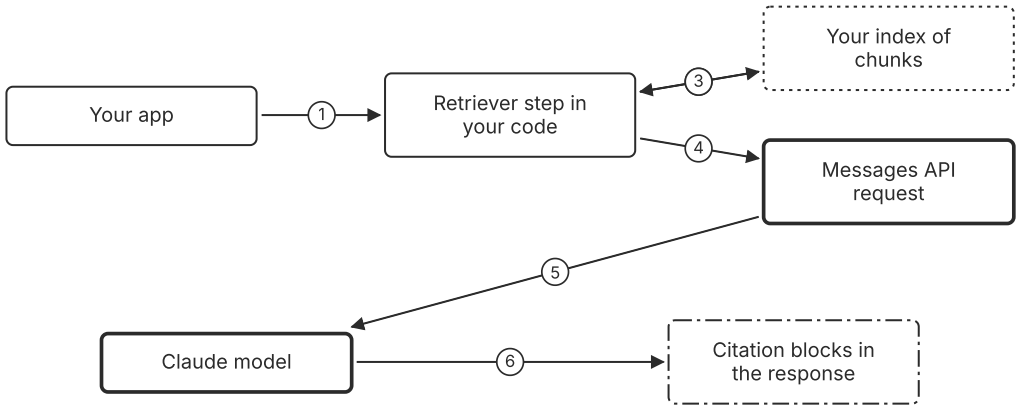
Search results with citations

1. Introduction and first principle

Pass retrieved text as search result blocks so Claude attaches a source and title to each cited claim.

- Citations are disabled by default for search results.[71]
- All search results in a request must use the same citations setting.[71]
- Claude cites whole content blocks, not substrings within a block.[71]
- All active models support search results with citations, except Claude Haiku 3.[71]
- Citations combined with structured outputs return a 400 error.[72]
- Citations work with prompt caching and batch processing.[72]

2. Architecture



Schematic drawn from docs. Arrows: 1. user question. 2. lookup. 3. chunks with source and title. 4. search_result blocks with citations enabled. 5. content blocks. 6. text with citation ranges. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Retrieve candidate chunks from your index before the model call.
2. Wrap each chunk in a search_result block with a source and a title.
3. Set citations to enabled on every search result in the request.
4. Claude writes the answer and attaches citation blocks to the cited text.
5. Each citation names a search_result_index and a block range.
6. Check the citations array before you show the answer to a user. (Inference)

4. Pros and cons

Pros

- Citations link each claim to a source and title that you supply.
- The cited_text field does not count toward output tokens.
- Search results work in tool results and in top-level user message content.

Cons

- Citations and structured outputs cannot combine in one request.
- All search results in one request must use the same citations setting.

- Search result content holds text blocks only, with no images.

5. Use when, do not use when

Use when

- Answers must name the source document for each claim.
- Your app retrieves text at runtime from a knowledge base.
- The retrieved text is ready before the model call starts.

Do not use when

- The output must follow a strict JSON schema, because citations conflict with structured outputs.
- The source content is image only, because search results hold text.
- The answer needs no source attribution.

6. Constraint families

Cost

Expect a slight input token increase when citations are enabled. Combination when one choice is not enough. Put cache_control on the search result blocks to reuse the source text.

Governance

Use a source value that your access policy can audit, such as a URL or an internal identifier. Combination when one choice is not enough. Use the same source value for every chunk from one document. (Inference)

Scale

Pass several search results in one request, each with its own title. Combination when one choice is not enough. Limit the number of results to the ones the answer needs. (Inference)

Operations

Log the search_result_index and block indices for each citation. Combination when one choice is not enough. Alert when a grounded answer has an empty citations array. (Inference)

7. Native cloud services

All

Not in source. The search results page names no native platform service.

8. Exam domain and task statement

D3.5. Expect a scenario where answers need source names. The credited answer passes retrieved text as search results with citations enabled. (Inference)

If the scenario requires cited claims, choose search result blocks with citations enabled.

Return retrieved text as search result blocks when the answer needs verifiable source attribution.

9. Trap

A prompt that asks the model to list sources in plain text looks simple. The names are not tied to retrieved blocks, so you cannot check them. (Inference)

10. Not in source

- Maximum number of search results per request: Not in source.
- Recommended chunk size in tokens for search results: Not in source.
- Contextual retrieval technique: Not in source. The mirror has no page for it.

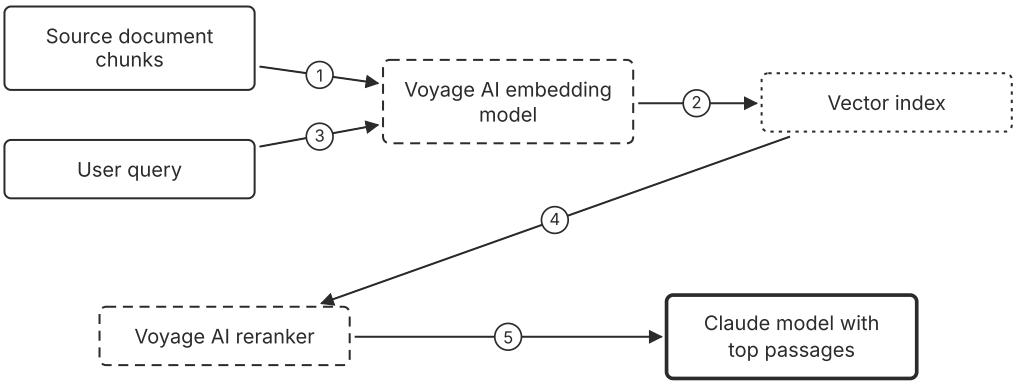
Embeddings and reranking models

1. Introduction and first principle

Convert text to vectors for similarity search, then reorder a short list with a reranker.

- Anthropic does not offer its own embedding model.[73]
- voyage-4 has a context length of 32,000 tokens and a default dimension of 1024.[73]
- voyage-context-4 accepts up to 120,000 tokens when enable_auto_chunking is true.[73]
- Use input_type query or document for retrieval tasks, such as RAG.[73]
- Voyage AI embeddings are normalized to length 1.[73]
- rerank-3 has a context length of 32,000 tokens and is recommended for most applications.[73]
- Int8 quantization reduces embedding storage by 4x compared with float.[73]

2. Architecture



Schematic drawn from docs. Arrows: 1. input_type document. 2. store vectors. 3. input_type query. 4. top candidates. 5. passages in relevance order. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Embed each document chunk with input_type set to document.
2. Embed the user query with input_type set to query.
3. Run a nearest neighbor search over the stored vectors.
4. Send the top candidates and the query to a reranker.
5. Keep the highest ranked passages for the model context.

4. Pros and cons

Pros

- Embeddings measure semantic similarity between query and document text.
- Voyage AI offers general, multilingual, code, legal, and finance models.
- Rerankers return documents ranked by relevance to the query.

Cons

- Anthropic does not offer its own embedding model, so you add a second vendor.
- Vectors must be embedded again when the embedding model changes. (Inference)
- A reranker adds a second model call and extra latency. (Inference)

5. Use when, do not use when

Use when

- Queries paraphrase the document text.
- The corpus needs a domain model, such as a legal or code model.
- The answer is in the candidate list, but the ranking is weak. (Inference)

Do not use when

- Queries contain only exact codes or identifiers, where keyword match fits better. (Inference)
- Your procurement policy allows no second vendor. (Inference)
- The whole corpus fits in the context window without a pipeline. (Inference)

6. Constraint families

Cost

Use voyage-4-lite when latency and cost bind. Combination when one choice is not enough. Use int8 or binary output to cut vector storage by 4x or 32x.

Quota

Keep each input within the model context length, such as 32,000 tokens for voyage-4 or 16,000 for voyage-law-2. Combination when one choice is not enough. Set enable_auto_chunking to true with voyage-context-4 to accept up to 120,000 tokens.

Data location

Use an Atlas model API key scoped to a geography, which uses that geography endpoint.

Scale

Request a smaller output dimension, such as 256 or 512, to shrink vectors. Combination when one choice is not enough. Truncate stored vectors to the leading dimensions and normalize them again.

Operations

Set input_type on every retrieval embedding call. Combination when one choice is not enough. Do not omit input_type or set it to None.

7. Native cloud services

All

Not in source. Voyage AI models also appear on AWS Marketplace, which is a vendor channel, not a native service.[73]

8. Exam domain and task statement

D3.5. Expect a scenario with a retrieval gap. The credited answer names the stage that fails and changes only that stage. (Inference)

If the answer is absent from the candidates, fix retrieval before you add a reranker. (Inference)

Add a reranker when the answer appears in the candidates but ranks low. (Inference)

9. Trap

An Anthropic embedding model looks like the default choice. Anthropic does not offer one, so the design needs an embeddings vendor.

10. Not in source

- Number of candidates to pass to the reranker: Not in source.
- Chunk size in tokens for embedding inputs: Not in source.
- Voyage AI prices: Not in source. The embeddings page points to MongoDB pricing pages.
- Native platform support for Voyage AI embeddings on Amazon Bedrock: Not in source.

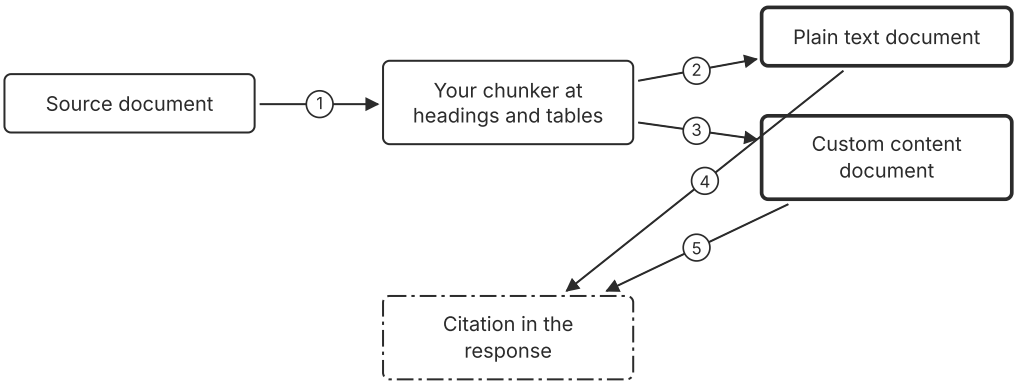
Chunk granularity for citations

1. Introduction and first principle

Choose the document type whose chunk boundaries match the citation granularity you need.

- Plain text documents are chunked into sentences for citations.[72]
- Plain text citations use 0-indexed character indices with exclusive end indices.[72]
- Custom content documents use your content blocks as is, with no further chunking.[72]
- To cite specific sentences from RAG chunks, put each RAG chunk into a plain text document.[72]
- Search result citations use block indices, and a text block is the minimal citable unit.[71]

2. Architecture



Schematic drawn from docs. Arrows: 1. split at structure. 2. sentence chunking by Claude. 3. your blocks as is. 4. character indices. 5. block indices. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Split the source at headings, sections, and table boundaries. (Inference)
2. Send each chunk as a plain text document when sentence citations are enough.
3. Send each chunk as custom content when you need your own block boundaries.
4. Plain text documents chunk into sentences and cite character indices.
5. Custom content documents keep your blocks and cite block indices.
6. Confirm that each table stays in one chunk before you index it. (Inference)

4. Pros and cons

Pros

- Plain text documents need no custom block logic.
- Custom content gives finer control for lists and transcripts.
- Citations keep source and title for each document.

Cons

- Automatic sentence chunking ignores your heading and table boundaries. (Inference)
- Custom content requires you to define each block boundary.
- A claim split across two blocks produces a wider citation range. (Inference)

5. Use when, do not use when

Use when

- Each retrieved chunk is a separate document with its own source.
- Citations must point to specific sentences in long passages.
- Lists or transcripts need citations at block level.

Do not use when

- The output must follow a strict JSON schema, because citations conflict with structured outputs.
- Whole documents fit in context and no chunk boundary is needed. (Inference)

6. Constraint families

Latency

Use plain text documents for simple prose with the least preparation.

Cost

Cited text does not count toward output tokens. Combination when one choice is not enough. Expect a small input token increase for chunking and citation prompts.

Governance

Keep the source and title on each document for audit.

Scale

Use custom content for lists, transcripts, and special layouts. Combination when one choice is not enough. Keep each block small enough to cite on its own. (Inference)

7. Native cloud services

All

Not in source. The citations page names no native platform service for chunking.[72]

8. Exam domain and task statement

D3.5. Expect a scenario with split tables or lists in citations. The credited answer changes the chunk boundary, not the citation setting. (Inference)

If the scenario names split tables, fix the chunker before you change citation settings. (Inference)

Use custom content when your own chunk boundaries must become the citation boundaries.

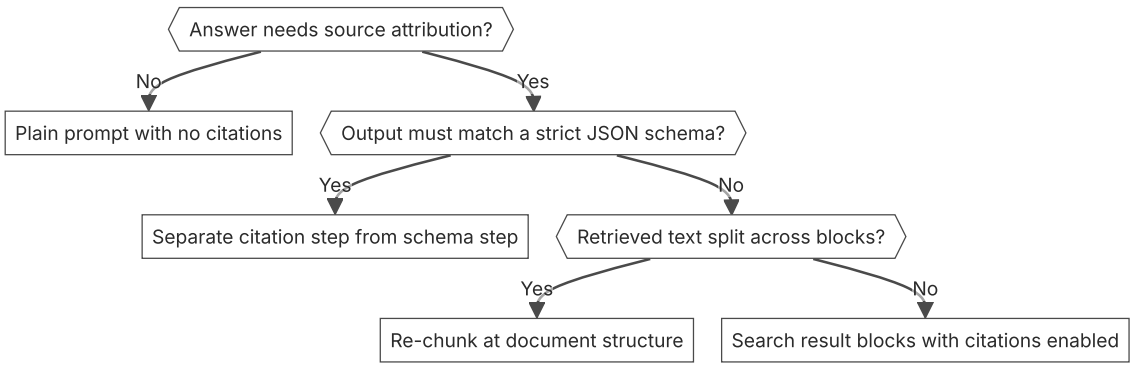
9. Trap

Plain text documents look simpler, but automatic sentence chunking ignores table boundaries. (Inference)

10. Not in source

- Recommended chunk size in tokens: Not in source.
- Recommended chunk overlap: Not in source.
- Chunk-level citation support for images in PDFs: Not in source. The citations page says images are not supported.

Decision flow



Unit items not in source

- Console steps for the RAG pipeline: Not in source.
- Hybrid keyword and semantic search design steps: Not in source.
- Recall and precision targets for a retrieval stage: Not in source.

D3.6 Retrieval strategies

Domain D3, Integration. Task statement: Choose a retrieval strategy for each query shape, and route live state to a tool. (Inference: wording from secondary sources)

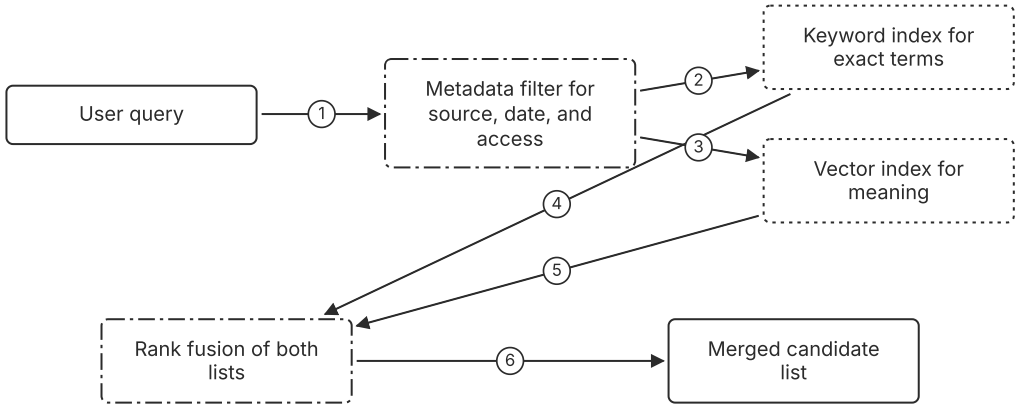
Hybrid keyword and semantic retrieval

1. Introduction and first principle

Pick the matcher that fits the query shape, and merge the candidates from both matchers.

- Embeddings measure semantic similarity between pieces of text.[73]
- Voyage AI embeddings are normalized to length 1.[73]
- Voyage AI embeddings are normalized, so cosine similarity is equivalent to dot-product similarity.[73]
- Int8 embeddings cut storage by 4x compared with float.[73]
- Binary quantization cuts embedding storage by 32x compared with float.[73]

2. Architecture



Schematic drawn from docs. Arrows: 1. question. 2. allowed documents only. 3. allowed documents only. 4. ranked list. 5. ranked list. 6. one merged list. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Apply metadata filters to limit the pool to documents the user may read. (Inference)
2. Score the pool with keyword match for codes, names, and identifiers.
3. Score the pool with embedding similarity for paraphrased questions.
4. Merge both ranked lists with rank fusion. (Inference)
5. Send the merged candidates to a reranker when the list is long. (Inference)

4. Pros and cons

Pros

- Keyword match finds exact codes that a vector match can miss. (Inference)
- Embedding similarity finds passages that use different words from the question. (Inference)
- Metadata filters limit results to allowed documents. (Inference)

Cons

- Two indexes need two refresh paths. (Inference)
- The merge step needs a fusion rule for two ranked lists. (Inference)
- Official pages describe no hybrid search feature. Not in source.

5. Use when, do not use when

Use when

- The corpus holds codes, SKUs, or names that a vector match misses. (Inference)
- Users ask in plain words about text that uses exact identifiers. (Inference)
- Access rules differ by user and must limit the candidate pool. (Inference)

Do not use when

- All queries are exact codes, where keyword match alone fits. (Inference)
- The corpus is small enough for full document context. (Inference)
- The answer depends on live state, which needs a tool call instead. (Inference)

6. Constraint families

Latency

Run both matchers in parallel, then merge the results. (Inference) Combination when one choice is not enough. Add a reranker only when the merged list is long. (Inference)

Cost

Keep a second index only when query shapes need the second matcher. (Inference) Combination when one choice is not enough. Use int8 output for embeddings to cut vector storage by 4x.

Isolation

Apply access filters before scoring, so users never see chunks they may not read. (Inference)

Operations

Log which matcher produced each kept candidate. (Inference) Combination when one choice is not enough. Compare precision per matcher on a labeled set of questions. (Inference)

7. Native cloud services

All

Not in source. The retrieval pages name no native platform service for keyword search.[73]

8. Exam domain and task statement

D3.6. Expect a scenario with codes and paraphrases in the same corpus. The credited answer covers both matchers with a filter. (Inference)

If the scenario names both codes and paraphrases, choose hybrid retrieval. (Inference)

Use keyword match for exact terms, semantic match for paraphrases, and filters for access. (Inference)

9. Trap

A single semantic index looks sufficient. Exact codes often fail to match in vector space, so the codes go unfound. (Inference)

10. Not in source

- Keyword index design, tokenizer, and scoring formula: Not in source.
- Rank fusion formula: Not in source.
- Precision and recall targets for a retrieval stage: Not in source.

Reranking candidates

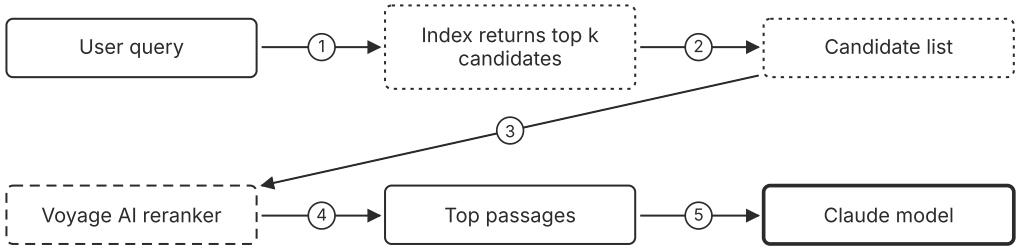
1. Introduction and first principle

A reranker scores a candidate list again and reorders it for the query.

- Rerankers take a query and a list of documents and return them ranked by relevance.[73]
- rerank-3 has a context length of 32,000 tokens.[73]
- rerank-3-lite is optimized for latency and cost.[73]

- rerank-3 is the highest accuracy model in the listed rerankers.[73]

2. Architecture



Schematic drawn from docs. Arrows: 1. search. 2. candidates. 3. query and documents. 4. relevance order. 5. context with citations. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Retrieve a wide candidate list from the index.
2. Send the query and the candidate documents to the reranker.
3. The reranker returns the documents in relevance order.
4. Keep the highest ranked passages for the model context.
5. Check recall at the retrieval stage when the answer is absent. (Inference)

4. Pros and cons

Pros

- Rerankers return documents ranked by relevance to the query.
- rerank-3 is recommended for most applications.
- rerank-3-lite is optimized for latency and cost.

Cons

- A reranker adds a model call and extra latency. (Inference)
- A reranker cannot return a document that retrieval did not return. (Inference)
- Rerank pricing sits in MongoDB documentation, outside the Claude docs.

5. Use when, do not use when

Use when

- The answer appears in the candidates, but the order is wrong. (Inference)
- Answer quality depends on the top few passages. (Inference)

Do not use when

- Recall is low, because a reranker only reorders the list. (Inference)
- The candidate list is already short and ordered well. (Inference)

6. Constraint families

Latency

Use rerank-3-lite when latency and cost bind. Combination when one choice is not enough. Use rerank-3 when accuracy binds the decision.

Cost

Rerank a shorter candidate list to cut calls and tokens. (Inference)

Quota

Keep each query and document set under the 32,000 token context length.

Operations

Track which candidates move into the top passages. Combination when one choice is not enough. Compare the top passages with the cited passages in each answer. (Inference)

7. Native cloud services

All

Not in source. The embeddings page names a Voyage AI rerank call, not a native platform service.[73]

8. Exam domain and task statement

D3.6. Expect a scenario where the right passage exists but ranks low. The credited answer adds a reranking stage. (Inference)

If the answer is absent from the candidates, fix retrieval, not the reranker. (Inference)

Add a reranker when the answer appears in the candidates but ranks low. (Inference)

9. Trap

A larger generation model looks like the fix for an absent answer. The answer was never in the candidate list, so no model can cite it. (Inference)

10. Not in source

- Number of top k candidates to keep: Not in source.
- Rerank latency in milliseconds: Not in source.
- Rerank price per call: Not in source. The embeddings page points to MongoDB pricing.

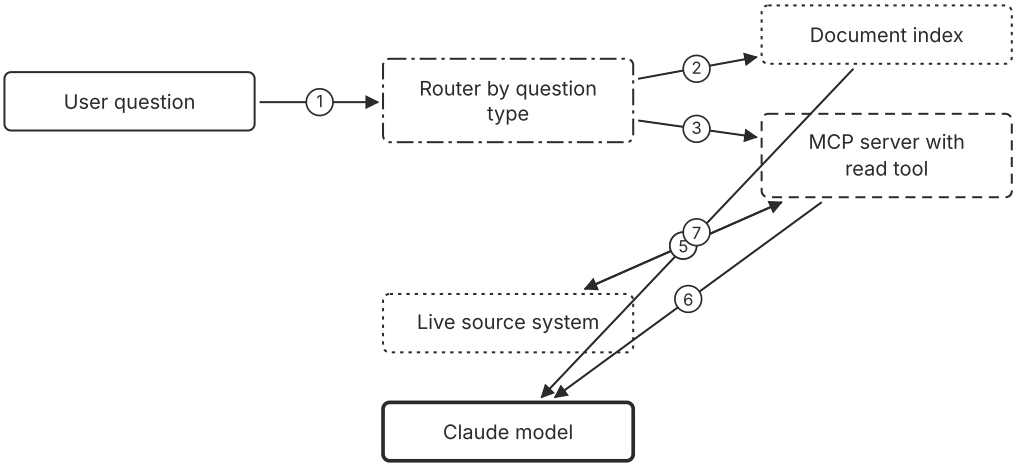
Live state through a tool call

1. Introduction and first principle

Answer questions about current values with a tool call to the source system, not with an indexed snapshot.

- Claude calls MCP tools when the request maps to a tool's described capability.[56]
- Only tool calls from the MCP specification are supported in the MCP connector.[56]
- The MCP connector requires a server that is publicly exposed over HTTP.[56]
- The MCP connector uses the mcp-client-2025-11-20 beta header.[56]
- The MCP connector is not eligible for zero data retention.[56]
- Allowlists and denylists control which MCP tools are enabled.[56]

2. Architecture



Schematic drawn from docs. Arrows: 1. question. 2. policy and document questions. 3. status and balance questions. 4. read current record. 5. current value. 6. tool result. 7. retrieved passages. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Classify each question as a document question or a live state question. (Inference)
2. Send live state questions to a tool that reads the source system.
3. Claude calls the tool when the request maps to the tool description.
4. Send document questions to the index and cite the passages.
5. Answer from the tool result, not from an older indexed copy. (Inference)

4. Pros and cons

Pros

- A tool result reflects the source system at call time. (Inference)
- Tool descriptions steer which tool Claude calls.
- The index does not need a rebuild for each state change. (Inference)

Cons

- The tool needs authentication and a latency budget. (Inference)
- A misrouted question still reads the old snapshot. (Inference)
- The MCP connector requires a server reachable over HTTP.

5. Use when, do not use when

Use when

- Answers change faster than the index refresh cycle. (Inference)
- The source system exposes a read tool for the needed fields. (Inference)

Do not use when

- The question concerns stable policy text. (Inference)
- The source system has no read interface. (Inference)

6. Constraint families

Latency

Each tool call adds a round trip. (Inference) Combination when one choice is not enough. Cache stable tool definitions, but never cache live values. (Inference)

Governance

Pass an OAuth bearer token for an authenticated MCP server. Combination when one choice is not enough. Use an allowlist so the model sees only read tools. (Inference)

Scale

Use an allowlist to enable only the MCP tools that the route needs.

Operations

Describe each tool with the words that users use for the question. (Inference) Combination when one choice is not enough. Monitor which tools Claude calls for each question type. (Inference)

7. Native cloud services

Claude Platform on AWS

The MCP connector is listed as beta on this platform.[56]

Microsoft Foundry

The MCP connector is listed as beta on this platform.[56]

8. Exam domain and task statement

D3.6. Expect a freshness scenario. The credited answer routes live state to a tool call and keeps documents on the index. (Inference)

If the scenario names a value that changes in minutes, choose the tool path. (Inference)

Route questions about live values to a tool, and route document questions to the index. (Inference)

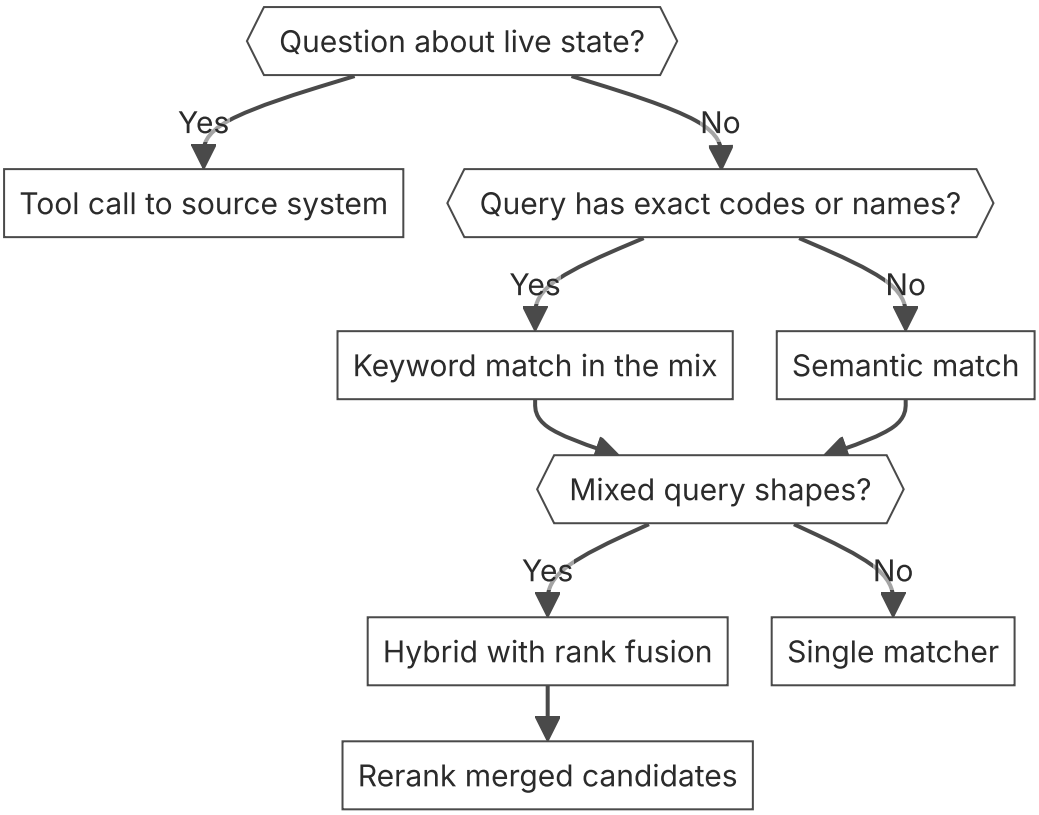
9. Trap

A faster index refresh looks like the fix for stale values. The index still stores a snapshot, so the answer can lag. (Inference)

10. Not in source

- Index refresh interval for a typical deployment: Not in source.
- Latency budget for a tool call: Not in source.
- Amazon Bedrock support for the MCP connector: Not in source.
- Claude Models on Google Cloud support for the MCP connector: Not in source.

Decision flow



Unit items not in source

- Console steps for retrieval strategies: Not in source.
- Hybrid search as a named product feature: Not in source.
- Query router implementation details: Not in source.

D3.7 Integration mechanisms

Domain D3, Integration. Task statement: Choose an integration mechanism by reuse, trust boundary, and run cost. MCP is not the default for every case. (Inference: wording from secondary sources)

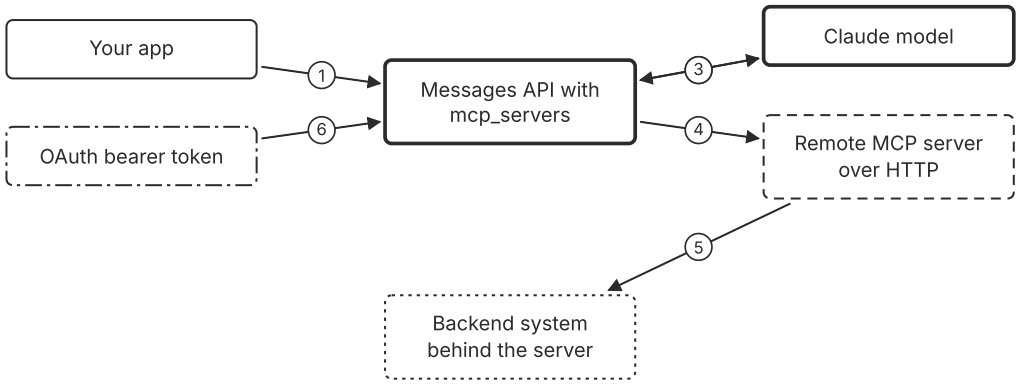
MCP connector in the Messages API

1. Introduction and first principle

Connect a remote MCP server from the Messages API, so the API calls its tools without a client in your code.

- The MCP connector uses the mcp-client-2025-11-20 beta header.[56]
- Only tool calls from the MCP specification are supported.[56]
- Every MCP server in mcp_servers must be referenced by exactly one MCP toolset.[56]
- The MCP connector supports Streamable HTTP and SSE transports.[56]
- The MCP connector is not covered by zero data retention arrangements.[56]
- Batch processing gives a 50 percent discount on all usage.[16]
- A Message Batch holds up to 100,000 requests or 256 MB, whichever limit comes first.[16]

2. Architecture



Schematic drawn from docs. Arrows: 1. request with mcp_servers and toolset. 2. tool definitions from the server. 3. MCP tool use block. 4. tool call over HTTP. 5. read or write. 6. authorization_token. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Define each remote server in the mcp_servers array with its URL.
2. Reference each server from exactly one MCP toolset.
3. Enable all tools, or allowlist and denylist specific tools in the configs.
4. Send the request with the MCP connector beta header.
5. Read the MCP tool use and MCP tool result blocks in the response.

4. Pros and cons

Pros

- The connector calls remote MCP servers from the Messages API without an MCP client in your code.
- One request can connect to several MCP servers.
- MCP servers work in Message Batches API requests.

Cons

- Only the tool calls part of the MCP specification is supported.
- A local STDIO server cannot connect directly.

- The connector is not covered by zero data retention arrangements.

5. Use when, do not use when

Use when

- A remote server exposes the tools that the app needs.
- One host needs a few remote tools and no shared client code. (Inference)
- Batch processing needs tool calls against remote servers.

Do not use when

- The server runs only as a local STDIO process.
- Your data policy requires zero data retention for tool data.
- The integration needs MCP resources or prompts, because only tools are supported.

6. Constraint families

Cost

MCP tool calls in Batches API requests cost the same as in Messages API requests. Combination when one choice is not enough. Batch processing gives a 50 percent discount on all usage.

Isolation

Pass a separate authorization_token for each authenticated server.

Governance

Use an allowlist to expose only approved tools. Combination when one choice is not enough. Use a denylist when a server has a few risky tools and many safe ones.

Scale

Connect multiple servers in one request, with one MCP toolset for each server. Combination when one choice is not enough. Use tool search when the combined tool count is large.

Operations

Check the MCP tool result blocks for errors before you show output to a user. (Inference) Combination when one choice is not enough. Verify the tool list in a test request before each release. (Inference)

7. Native cloud services

Claude Platform on AWS

The MCP connector is listed as beta on this platform.[56]

Microsoft Foundry

The MCP connector is listed as beta on this platform.[56]

8. Exam domain and task statement

D3.7. Expect a scenario that names the number of hosts and the number of tools. The credited answer matches the mechanism to that reuse count. (Inference)

If the scenario needs MCP resources or prompts, the connector alone does not cover it.

Use the MCP connector when a Messages API request needs tool calls on a remote MCP server.

9. Trap

The connector looks like full MCP support. Only tool calls work, so resources and prompts need another path.

10. Not in source

- Maximum number of MCP servers in one request: Not in source.
- Connector timeout values: Not in source.
- Amazon Bedrock support for the MCP connector: Not in source.
- Claude Models on Google Cloud support for the MCP connector: Not in source.

- Console steps for MCP connector setup: Not in source.

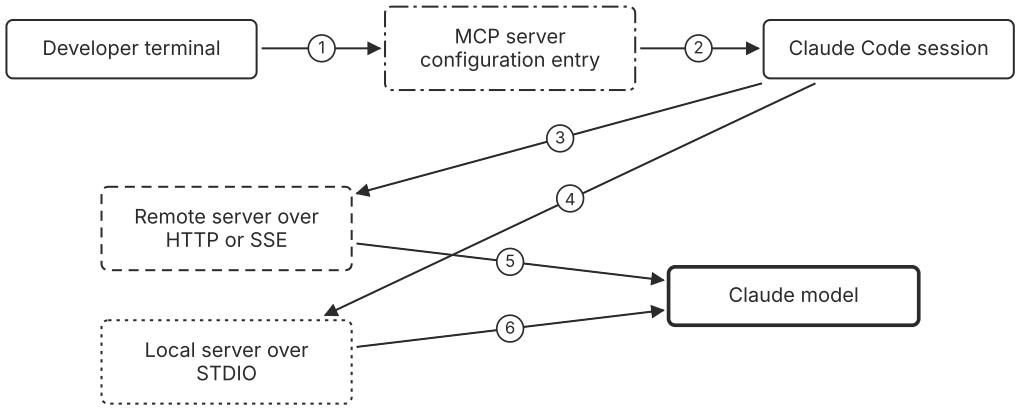
Claude Code MCP servers

1. Introduction and first principle

Register each MCP server once in Claude Code, with a transport that matches where the server runs.

- The command claude mcp add --transport http adds a remote HTTP server.[59]
- From v2.1.265, Claude Code tries the HTTP transport first and switches to SSE when the server does not accept it.[59]
- Without tool search, Claude Code does not report failed server connections to Claude.[59]
- An organization control set to blocked removes the connector tool before Claude sees it.[59]
- The idle timeout defaults to five minutes for HTTP and SSE servers, and to 30 minutes for stdio servers.[59]
- The per-server timeout field takes milliseconds and overrides MCP_TOOL_TIMEOUT for that server.[59]

2. Architecture



Schematic drawn from docs. Arrows: 1. claude mcp add. 2. server definition. 3. HTTP or SSE transport. 4. STDIO subprocess. 5. tools through tool search. 6. tools through tool search. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Add a remote server with the http transport and its URL.
2. Add a server that exposes only SSE with the http transport, or with the sse transport before v2.1.265.
3. Add a local server with the stdio transport and the command after a double dash.
4. Check the /mcp list for the connection state of each server.
5. Set a per-server timeout in the server entry when a tool call runs long.

4. Pros and cons

Pros

- Claude Code connects remote servers over HTTP or SSE, and local servers over STDIO.
- Tool search is on by default, which keeps large tool sets out of context.
- Organization controls can block a connector tool before Claude sees it.

Cons

- Without tool search, Claude Code does not report failed server connections to Claude.
- A local STDIO server runs as a subprocess on the developer machine. (Inference)
- A per-server timeout is a hard wall-clock limit for each tool call.

5. Use when, do not use when

Use when

- A developer needs remote tools in an interactive session.
- A local tool exists only as a command-line server. (Inference)
- The team needs one shared configuration for several projects. (Inference)

Do not use when

- The integration runs in production without a developer session. (Inference)
- The integration must run in a managed API request, where the MCP connector fits better. (Inference)

6. Constraint families

Latency

Use a remote server when a shared service must answer for many developers. (Inference)

Governance

Set an organization control to ask or blocked for a connector tool. Combination when one choice is not enough. Use allow rules only for tools that need no prompt. (Inference)

Scale

Keep tool search on for large tool sets. Combination when one choice is not enough. Use alwaysLoad only for servers whose tools every turn needs.

Operations

Set a timeout in the .mcp.json entry for a long server. Combination when one choice is not enough. Set CLAUDE_CODE_MCP_TOOL_IDLE_TIMEOUT to 0 only when idle checks cause false stops. (Inference)

7. Native cloud services

All

Not in source. The Claude Code MCP page names no native platform service.[59]

8. Exam domain and task statement

D3.7. Expect a scenario about a developer tool that calls a team service. The credited answer names the transport and the configuration scope. (Inference)

If the scenario names a local command-line tool, choose a Claude Code STDIO server.

Use STDIO for a local process, and HTTP or SSE for a shared remote server.

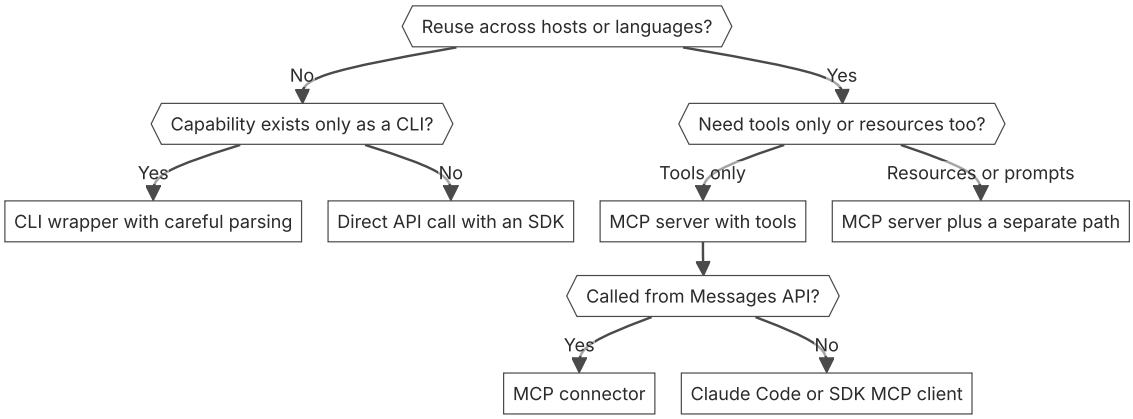
9. Trap

A local STDIO server looks like a fit for an API backend, but the Messages API connector cannot connect to it directly.

10. Not in source

- Maximum timeout value for a server entry: Not in source.
- Maximum number of servers per developer machine: Not in source.
- Console steps for Claude Code MCP setup: Not in source.

Decision flow



Unit items not in source

- Console steps for integration mechanisms: Not in source.
- Exact product support for each MCP primitive in every platform: Not in source.
- Agent to agent handoff contract details: Not in source.

D3.8 Progressive discovery and monolithic context

Domain D3, Integration. Task statement: Load tool and skill detail on demand, not the full catalog on every call. (Inference: wording from secondary sources)

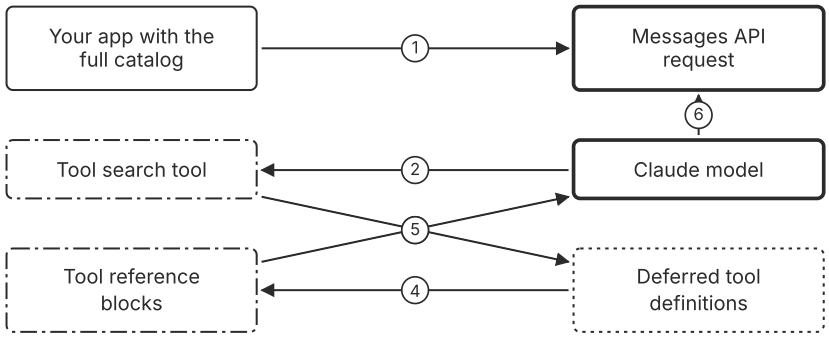
Tool search tool in the Messages API

1. Introduction and first principle

Load only the tools that a request needs, found by a search tool over the full catalog.

- Tool search returns up to 5 tools that match the query by default.[54]
- A request allows up to 10,000 tools with defer_loading true.[54]
- Regex patterns are limited to 200 characters, and BM25 queries to 500 characters.[54]
- The docs say tool selection accuracy degrades once you exceed 30 to 50 available tools.[54]
- The docs say tool search typically reduces tool definition tokens by over 85 percent.[54]
- The context window holds up to 1M tokens for several models, such as Claude Haiku 5.5.[4]

2. Architecture



Schematic drawn from docs. Arrows: 1. tools with defer_loading true. 2. search query. 3. match names and descriptions. 4. matching tools. 5. up to 5 by default. 6. call a loaded tool. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Add a tool search tool to the tools list.
2. Set defer_loading to true on the tools that should load later.
3. Keep at least one tool non-deferred, normally the search tool itself.
4. Claude runs a search and receives the matches as tool reference blocks.
5. Claude then calls a matched tool, and the API loads only the matched definitions.

4. Pros and cons

Pros

- Tool search typically reduces tool definition tokens by over 85 percent in a multiserver setup.
- Selection accuracy stays high across large catalogs, because only a focused set loads.
- The MCP connector can set defer_loading once for a whole server.

Cons

- A tool with defer_loading true cannot carry cache_control, or the API returns 400.
- On Amazon Bedrock, server-side tool search works only through the InvokeModel API, not the Converse API.
- Each search adds a round trip before the tool call. (Inference)

5. Use when, do not use when

Use when

- You have 10 or more tools available.
- Tool definitions use more than 10k tokens.
- You aggregate several MCP servers with 200 or more tools.

Do not use when

- You have fewer than 10 tools, or every tool is used in every request.
- The tool definitions total less than 100 tokens.
- A fixed workflow needs every tool visible on each turn. (Inference)

6. Constraint families

Latency

Keep the catalog focused, because each search adds a round trip. Combination when one choice is not enough. Set a higher search limit only when Claude needs more candidates. (Inference)

Cost

Defer tool definitions to cut the tokens that each request carries. Combination when one choice is not enough. Keep the three to five most frequently used tools non-deferred.

Scale

Use a catalog of up to 10,000 deferred tools for each request. Combination when one choice is not enough. Choose regex search or BM25 search for the query style that your users write.

Operations

Use a service prefix in tool names, such as github_ for GitHub tools. Combination when one choice is not enough. Monitor which tools Claude discovers, and refine the descriptions that miss.

7. Native cloud services

Amazon Bedrock

Server-side tool search runs only through the InvokeModel API, not the Converse API.[54]

Claude Platform on AWS

Server-side tool search works the same as on the Claude API.[54]

Microsoft Foundry

Deployments hosted on Azure reject tool search, and the Agent SDK then loads tools upfront.[74]

8. Exam domain and task statement

D3.8. Expect a scenario with a large tool catalog, high token cost, and wrong tool picks. The credited answer defers definitions and searches. (Inference)

If the scenario counts tokens per call, choose deferral over description edits. (Inference)

Defer tool definitions when the catalog reaches 10 or more tools, or when definitions exceed 10k tokens.

9. Trap

Shorter tool descriptions look cheaper. The full schemas still load on each call, so the token cost stays high. (Inference)

10. Not in source

- Token cost per tool definition in your app: Not in source.
- Exact latency added by one search: Not in source.
- Google Cloud support details for tool search: Not in source.
- Console steps for tool search: Not in source.

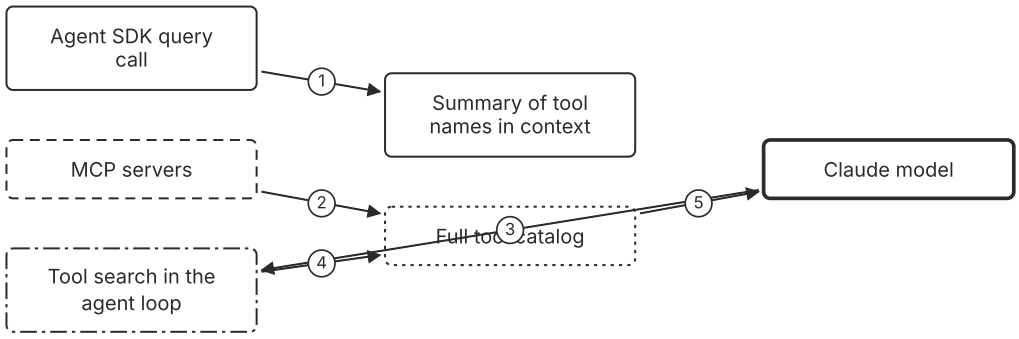
Agent SDK tool search

1. Introduction and first principle

The Agent SDK withholds tool definitions by default and loads matches after the agent searches.

- Tool search is on by default in the Agent SDK.[74]
- Up to five of the most relevant tools load into context by default.[74]
- The auto mode activates tool search when definitions reach 10 percent of the context window.[74]
- Tool search adds one extra round trip each time Claude searches for tools.[74]
- Setting ENABLE_TOOL_SEARCH to false loads all tool definitions on every turn.[74]
- With alwaysLoad true, startup waits for the server tools, capped at the standard 5-second connect timeout.[59]

2. Architecture



Schematic drawn from docs. Arrows: 1. names and short descriptions. 2. tool definitions. 3. need a tool not loaded. 4. match tools. 5. load up to 5 by default. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. The SDK withholds tool definitions from the context window.
2. The agent receives a summary of available tool names.
3. The agent searches when a task needs a tool that is not loaded.
4. Up to five tools that match load into context by default.
5. Loaded tools stay available for later turns until the SDK compacts the messages where the agent found them.

4. Pros and cons

Pros

- Tool search is on by default in the Agent SDK.
- The ENABLE_TOOL_SEARCH variable sets on, off, or a threshold mode.
- In auto mode, tools load upfront until their definitions reach 10 percent of the context window.

Cons

- Each search adds one extra round trip.
- Some models on the SDK unsupported list load tools upfront, and no value overrides that.
- Deployments on Microsoft Foundry hosted on Azure reject tool search.

5. Use when, do not use when

Use when

- The agent uses many MCP tools across several servers.
- The context budget binds harder than round trip latency. (Inference)
- You want tool search to turn on only when definitions grow. Use auto mode.

Do not use when

- Fewer than about 10 tool definitions fit comfortably in context.
- Latency binds harder than tokens, so each round trip hurts. (Inference)
- The model must see every tool name before each turn. (Inference)

6. Constraint families

Latency

Set ENABLE_TOOL_SEARCH to false when round trips matter more than tokens. Combination when one choice is not enough. Keep the tool count low, so the agent rarely searches. (Inference)

Cost

Use auto mode to load tools upfront until they reach 10 percent of the window. Combination when one choice is not enough. Use auto:5 to activate tool search at 5 percent of the window.

Scale

Tool search applies to remote MCP servers and custom SDK MCP servers.

Operations

Set alwaysLoad to true on a server whose tools every turn needs. Combination when one choice is not enough. Expect startup to wait for alwaysLoad servers, capped at the 5-second connect timeout.

7. Native cloud services

Microsoft Foundry

Deployments hosted on Azure reject tool search, and the SDK loads tool definitions upfront for them.[74]

8. Exam domain and task statement

D3.8. Expect a scenario with an SDK agent and many MCP tools. The credited answer keeps deferral on and tunes the threshold or the always-load setting. (Inference)

If the scenario names a model that rejects tool search, load the tools upfront.

Keep tool search on in the Agent SDK unless latency binds harder than tokens. (Inference)

9. Trap

A fix that turns tool search off looks simple for wrong picks. Every definition then loads on every turn, so the token cost rises.

10. Not in source

- Exact point at which the SDK compacts messages in long sessions: Not in source.
- Native platform name for Google Cloud's Agent Platform in the exam service list: Not in source.
- Console steps for Agent SDK tool search: Not in source.

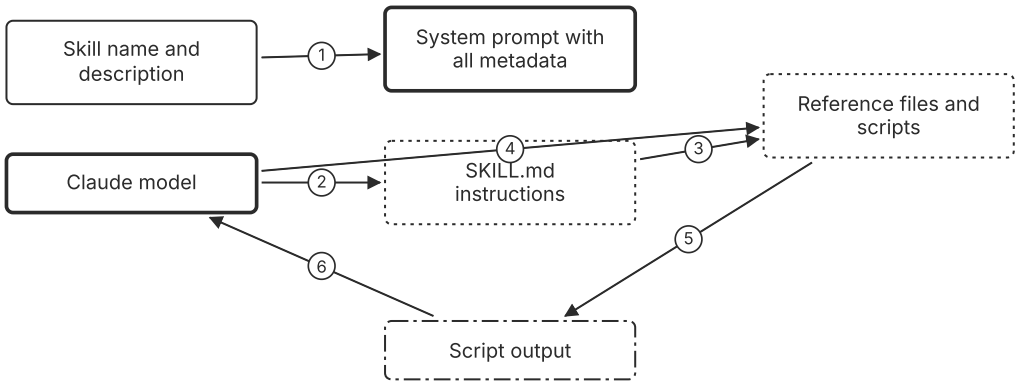
Agent Skills progressive disclosure

1. Introduction and first principle

Load skill metadata first, and read instructions and files only when a task needs them.

- Level 1 metadata costs about 100 tokens per skill.[75]
- Level 2 instructions cost under 5k tokens when a skill triggers.[75]
- The SKILL.md body should stay under 500 lines.[76]
- A skill name has a maximum of 64 characters.[76]
- A skill description has a maximum of 1,024 characters.[76]
- Skills on the API run in a sandbox with no network access.[75]

2. Architecture



Schematic drawn from docs. Arrows: 1. about 100 tokens per skill. 2. read when the skill is relevant. 3. points to files. 4. read only what the task needs. 5. script run through bash. 6. only output enters context. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Put the name and description of each skill in the system prompt.
2. Describe the skill's purpose and the conditions that trigger it.
3. Claude reads SKILL.md only when the skill becomes relevant.
4. Claude reads reference files only as the task needs them.
5. Scripts run through bash, and only their output enters the context.

4. Pros and cons

Pros

- Many skills install with little context cost, about 100 tokens each.
- Reference files cost no tokens until a task reads them.
- A script runs without its code in the context.

Cons

- A weak description can hide a useful skill from the model.
- A loaded SKILL.md body costs under 5k tokens, which still competes with the conversation.
- Skills on the API run with no network access and no runtime package installation.

5. Use when, do not use when

Use when

- The same domain guidance repeats across many tasks.
- Reference material is large, and each task needs one part of it.
- A deterministic check belongs in a script. (Inference)

Do not use when

- A one-off instruction fits in the prompt. (Inference)
- The task needs live network access from the sandbox.
- The guidance is short and needed on every request. (Inference)

6. Constraint families

Cost

Keep SKILL.md under 500 lines and move detail into separate files. Combination when one choice is not enough. Use scripts for checks, so only their output costs tokens.

Governance

Custom Skills on the Claude API are shared across the whole workspace. Combination when one choice is not enough. Use a separate workspace for a tenant whose Skills must stay private. (Inference)

Scale

Split content by domain, so one task reads one file.

Skill

Use a name of up to 64 characters with lowercase letters, numbers, and hyphens. Combination when one choice is not enough. Use a description of up to 1,024 characters that says when to use the skill.

7. Native cloud services

Claude Platform on AWS

Skills follow the same behavior as the Claude API. Upload custom Skills through the Skills API.[75]

Microsoft Foundry

Skills follow the same behavior as the Claude API. Pre-built Agent Skills require a Hosted on Anthropic deployment. The Skill version download endpoint is not supported.[75]

8. Exam domain and task statement

D3.8. Expect a scenario with a large body of guidance and a token budget. The credited answer loads the smallest needed part on demand. (Inference)

If the scenario names large reference documents, choose skill files over a longer system prompt. (Inference)

Put only metadata in the prompt, and load each file when the task needs it.

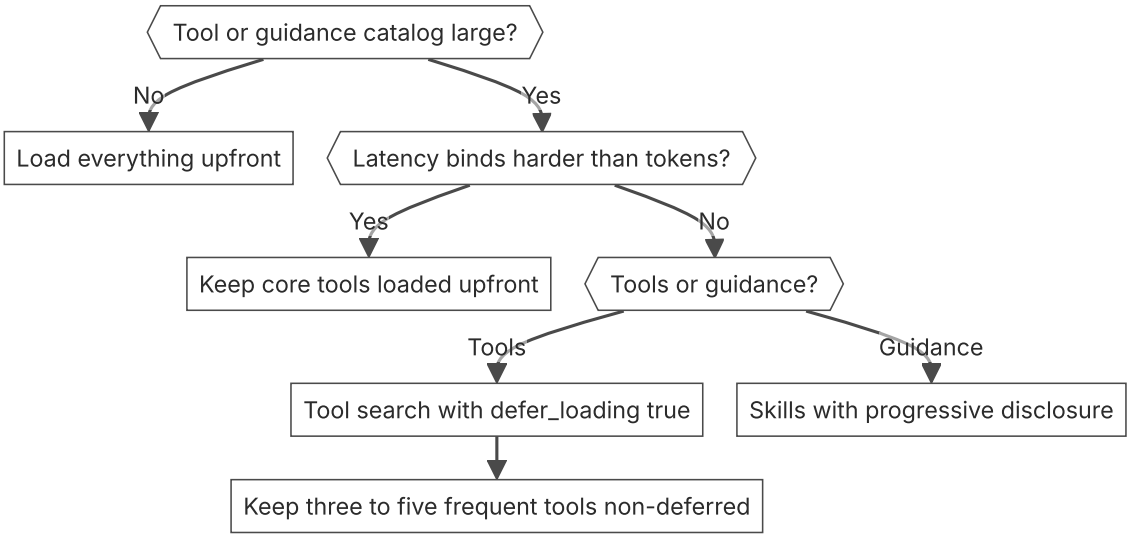
9. Trap

A system prompt that holds every reference file looks thorough. Each token then competes with the conversation for context space.

10. Not in source

- Amazon Bedrock support for Agent Skills: Not in source.
- Maximum number of skills per workspace: Not in source.
- Console steps for Agent Skills: Not in source.

Decision flow



Unit items not in source

- Console steps for progressive discovery: Not in source.
- Break-even point between upfront and searched tool definitions for a given app: Not in source.

- Tool search support for Claude Models on Google Cloud: Not in source.

D4.1 Evaluation metrics

Domain D4, Evaluation, testing, and optimization. Task statement: Define a primary metric, guard metrics, and thresholds before you tune a Claude system. (Inference: wording from secondary sources)

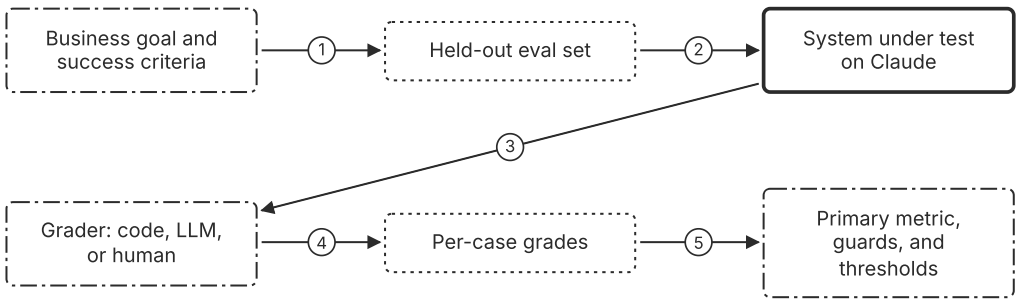
Success criteria and primary metric

1. Introduction and first principle

A metric measures the goal only when it names the goal and a threshold before tuning starts.

- Good success criteria are specific, measurable, achievable, and relevant.[25]
- The docs example sets an F1 score of at least 0.85 on a held-out test set of 10,000 posts.[25]
- Operational metrics include response time in milliseconds and uptime in percent.[25]
- Most use cases need multidimensional evaluation along several success criteria.[25]

2. Architecture



Schematic drawn from docs. Arrows: 1. derive held-out cases. 2. send inputs. 3. outputs for grading. 4. grade each case. 5. compare to thresholds. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write each success criterion as a specific and measurable statement.
2. Set a threshold for each criterion on a held-out test set.
3. Name the primary metric from the criterion with the highest business impact. (Inference)
4. Add guard metrics for criteria that must not break. (Inference)
5. Assign one owner to each threshold before the first run. (Inference)
6. Run the same evaluation on each change and compare each score to its threshold.

4. Pros and cons

Pros

- Turns vague quality goals into numbers that teams can compare.
- Puts the most important criterion first in every report.
- Gives the release decision a fixed bar.

Cons

- Measures only the criteria that someone wrote down.
- Quantitative metrics can miss quality that people judge differently. (Inference)
- Thresholds need review when the task mix changes. (Inference)

5. Use when, do not use when

Use when

- The application has a task outcome that code or a grader can check.
- A team must decide whether a prompt or model change ships.

- A stakeholder states a target for quality, cost, or response time.

Do not use when

- The system is an early prototype with no users or decisions yet. (Inference)
- No grader can measure the criterion, so the number would not mean anything. (Inference)
- A single spot check serves a demo that has no decision attached. (Inference)

6. Constraint families

Latency

Set an acceptable response time from the real-time needs of the application. Combination when one choice is not enough. Report time to first token separately for streaming responses.

Cost

Set a budget per completed task, not per token. Combination when one choice is not enough. Track the same budget as a guard metric on each release. (Inference)

Governance

Name one owner for each threshold in the evaluation plan. (Inference) Combination when one choice is not enough. Review each threshold when the task mix changes. (Inference)

Skill

Assign a team member who can write rubrics and read score tables. (Inference)

Operations

Run the evaluation on every change to the prompt, model, or tools. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D4.1. Expect a scenario where a proxy score looks good and the business outcome looks poor. The credited answer sets the business outcome as the primary metric with a threshold. (Inference)

A metric with no threshold and no owner cannot decide a release.

Make the business outcome the primary metric, and keep proxy scores as diagnostics.

9. Trap

A fluency or length score that rises while task success stays flat. The score looks better, but it does not measure the goal.

10. Not in source

- Numeric thresholds for task success in a production system. Not in source.
- Steps for the evaluation tool in the Claude Console. Not in source.

Cost and latency metrics

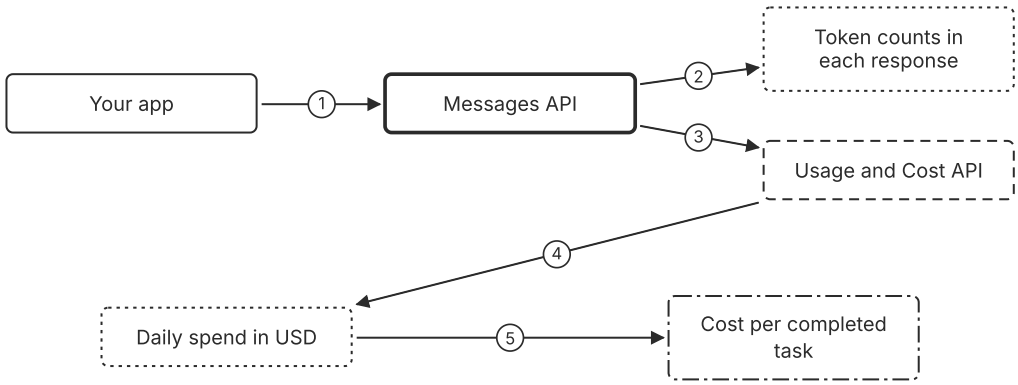
1. Introduction and first principle

You pay for completed tasks, so cost per completed task measures value better than cost per token.

- The Usage API supports 1m, 1h, and 1d time buckets.[68]
- The default limit for 1m buckets is 60, and the maximum is 1,440 buckets.[68]
- Usage and cost data typically appears within 5 minutes of request completion.[68]
- The API supports polling once per minute for sustained use.[68]
- The Cost API reports costs in USD and supports only daily buckets.[68]
- The docs advise a comparison of models on cost per completed task, not on cost per token.[77]

- Time to first token matters most when you use streaming. Baseline latency measures the full response time.[63]

2. Architecture



Schematic drawn from docs. Arrows: 1. send requests. 2. usage fields. 3. organization usage data. 4. daily totals. 5. divide by completed tasks. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Read the usage fields in each response to count input and output tokens.
2. Count the tasks that pass the primary metric.
3. Divide total spend by the count of completed tasks.
4. Pull daily organization totals from the Usage and Cost API.
5. Measure time to first token and baseline latency for streaming calls.

4. Pros and cons

Pros

- Usage fields report token counts for every response.
- The Usage and Cost API reports organization totals in USD by day.
- Cost per completed task links spend to the goal.

Cons

- Usage and cost data can appear later than 5 minutes after a request.
- The Cost API supports only daily buckets.
- Claude Platform on AWS has no programmatic Usage and Cost endpoints.

5. Use when, do not use when

Use when

- Budget binds and the team compares models or prompts.
- A team reports spend against completed outcomes.
- A stakeholder sets a response time target for the application.

Do not use when

- A minute-level view of cost is needed, because the Cost API has daily buckets only.
- One response shows the cost of a single call, which is not a rate. (Inference)
- Token counts alone decide the model choice, because the docs advise a per-task comparison.

6. Constraint families

Latency

Measure time to first token for streaming and baseline latency for other calls. Combination when one choice is not enough. Compare both with the response time target from the application.

Cost

Track cost per completed task for each model and prompt version. Combination when one choice is not enough. Use the Usage and Cost API for daily totals in USD.

Governance

Filter usage by workspace and API key to assign spend to owners. Combination when one choice is not enough. Call the endpoints with an Admin API credential, because workspace API keys do not work.

Scale

Use minute buckets for near-real-time views and daily buckets for reports.

Operations

Poll the Usage API no more than once per minute for sustained use. Combination when one choice is not enough. Cache dashboard results between polls.

7. Native cloud services

Claude Platform on AWS

The programmatic Usage and Cost API endpoints are not currently available. Use the Usage and Cost pages in the Claude Console.[68]

Claude Models on Google Cloud (Anthropic managed)

Only the usage endpoint is available. Call it with Google Cloud credentials. The only predefined IAM role that allows the call is roles/claude.admin.[68]

8. Exam domain and task statement

D4.1. Expect a scenario where a spend or latency target conflicts with a quality target. The credited answer measures cost per completed task against the quality threshold. (Inference)

A cost figure without a completed-task count does not answer a cost question.

Report spend next to completed tasks and the quality threshold in the same view.

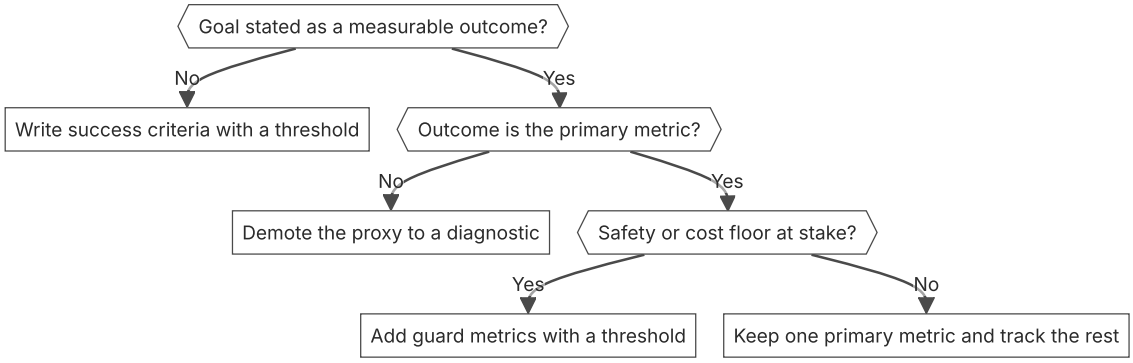
9. Trap

A lower price per token after a change. Fewer tasks may complete, so the cost per completed task can rise.

10. Not in source

- Latency targets for any model or effort level. Not in source.
- Console steps for cost monitoring beyond the Usage and Cost pages. Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Evaluation tool in the Claude Console: Not in source.
- Industry benchmark values for task success: Not in source.

D4.2 Evaluation datasets and frameworks

Domain D4, Evaluation, testing, and optimization. Task statement: Build an eval set that covers real task traffic and grade it with the cheapest reliable grader. (Inference: wording from secondary sources)

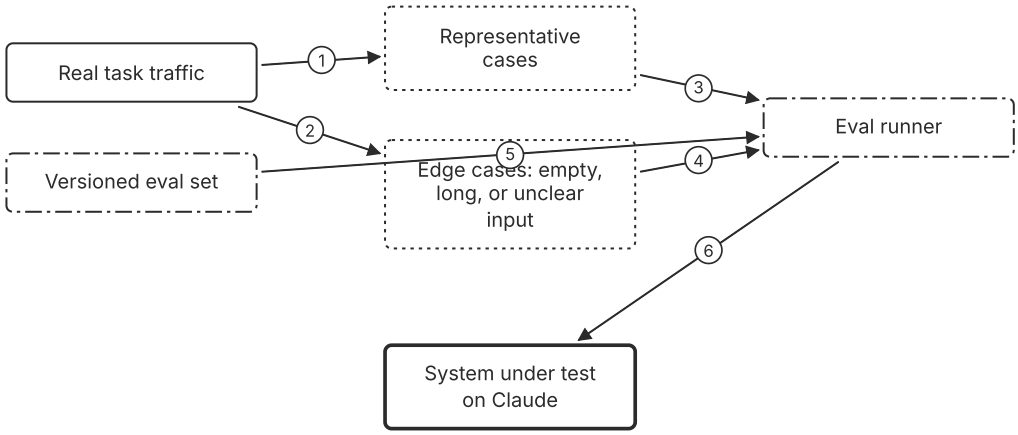
Evaluation dataset design

1. Introduction and first principle

An eval set measures the system well only when its cases match the real task distribution.

- The docs say to design evals that mirror the real-world task distribution.[25]
- The docs list irrelevant input, overly long input, and ambiguous cases as edge cases.[25]
- The docs say to automate scoring when possible.[25]
- The docs say that more questions with slightly lower signal beat fewer high-quality hand-graded evals.[25]

2. Architecture



Schematic drawn from docs. Arrows: 1. sample real tasks. 2. collect rare inputs. 3. load cases. 4. load cases. 5. pin set version. 6. send inputs. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Sample cases from the real task distribution.
2. Add edge cases such as irrelevant input, empty input, and very long input.
3. Add ambiguous cases where even humans would struggle to agree.
4. Write expected answers so that code can grade most cases.
5. Version the set and add new cases when the task changes. (Inference)

4. Pros and cons

Pros

- Edge cases expose failures that clean samples hide.
- Automated scores make each run cheap and frequent. (Inference)
- The docs say more questions with lower signal beat fewer hand-graded questions.

Cons

- A set misses traffic that it never sampled. (Inference)
- Expected answers take effort to write and maintain. (Inference)
- Shared eval cases can leak into training data. (Inference)

5. Use when, do not use when

Use when

- The application handles varied user input.
- Production shows failures that a clean set does not show.
- A team needs a regression check before each release.

Do not use when

- A one-off demo has no decision attached to the result. (Inference)
- The task has no real traffic yet, so the set would only reflect guesses. (Inference)

6. Constraint families

Governance

Remove personal data from cases before they leave the trust boundary. (Inference)

Scale

Prioritize volume with automated checks over small hand-graded sets. Combination when one choice is not enough. Add a human-graded slice for ambiguous cases. (Inference)

Skill

Assign a domain expert to review the ambiguous cases. (Inference)

Operations

Refresh the set when the task distribution changes. (Inference) Combination when one choice is not enough. Keep the old version so that results remain comparable. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D4.2. Expect a scenario where a clean test set reports a high score and production fails on unusual input. The credited answer adds edge and adversarial cases to the set. (Inference)

A high average on a clean set does not prove the system handles the hard cases.

Build the eval set from production traffic, and add edge cases that production contains.

9. Trap

A set built only from easy, clean examples. The score is high, but it does not predict production behavior.

10. Not in source

- A target share of edge cases in an eval set. Not in source.
- Rates of disagreement between human labelers. Not in source.

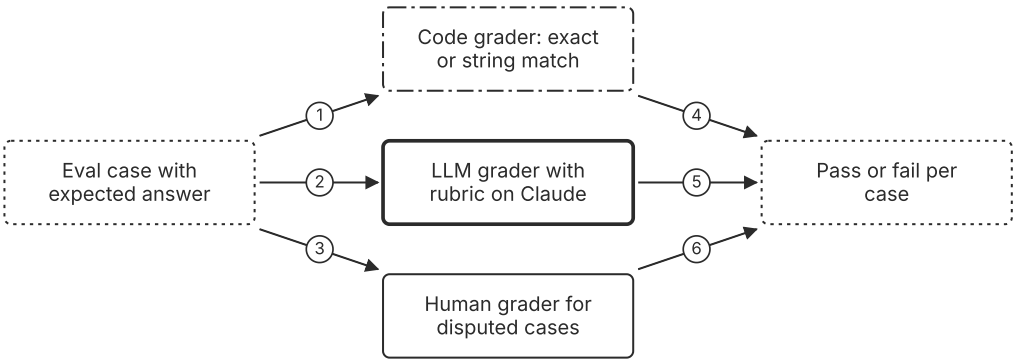
Graders for evaluations

1. Introduction and first principle

Use the cheapest grader that stays reliable for each type of case.

- The docs rank code graders as the fastest and most reliable method.[25]
- The docs say human graders are slow and expensive, and to avoid them if possible.[25]
- The docs say to test an LLM grader for reliability first, then scale it.[25]
- The docs suggest that the grader output only correct or incorrect, inside result tags in the example.[25]
- The docs suggest thinking on for the grader model when the judgment is complex.[25]
- The docs name unattended evaluation runs as suitable for batch processing.[77]

2. Architecture



Schematic drawn from docs. Arrows: 1. checkable answers. 2. rubric and output. 3. disputed cases. 4. match result. 5. correct or incorrect. 6. reviewed label. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Use a code grader first for exact match and string match checks.
2. Use an LLM grader for judgment that code cannot express.
3. Write a detailed rubric with a clear pass condition and a fixed output format.
4. Turn on thinking for the grader model when the judgment is complex.
5. Test the LLM grader for reliability before you scale it.
6. Use a human grader only when the other methods cannot decide.

4. Pros and cons

Pros

- The docs rank code graders as the fastest and most reliable method.
- An LLM grader handles subtle judgment at scale.
- A fixed output format makes grader scores easy to parse.

Cons

- Code graders lack nuance for complex judgment.
- Human graders are slow and expensive, so the docs say to avoid them when possible.
- An LLM grader can disagree with human graders on hard cases. (Inference)

5. Use when, do not use when

Use when

- Outputs have an answer that code can check with an exact or string match.
- The judgment is complex and hard to express in code.
- A new LLM grader must match a reviewed sample of human labels. (Inference)

Do not use when

- The LLM grader has not passed a reliability test on the case type. (Inference)
- A human panel must grade every release at high volume. (Inference)

6. Constraint families

Cost

Use code to check most cases, and reserve LLM graders for judgment. Combination when one choice is not enough. Use the Message Batches API for offline grader runs.

Governance

Use reviewed human labels as the reference for each LLM grader. (Inference)

Scale

Scale LLM graders only after the reliability test passes. Combination when one choice is not enough. Use code graders for volume, because the docs call them extremely scalable.

Operations

Test each grader before it scores a release. Combination when one choice is not enough. Recheck the grader when the grader model changes. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D4.2. Expect a scenario with a cost limit and a judgment task. The credited answer uses code for checkable cases and tests an LLM grader before it scales. (Inference)

Human graders are the costliest rung, so they fit only the cases that code and LLM graders cannot decide.

Assign each case type the cheapest grader that passed a reliability test on that type.

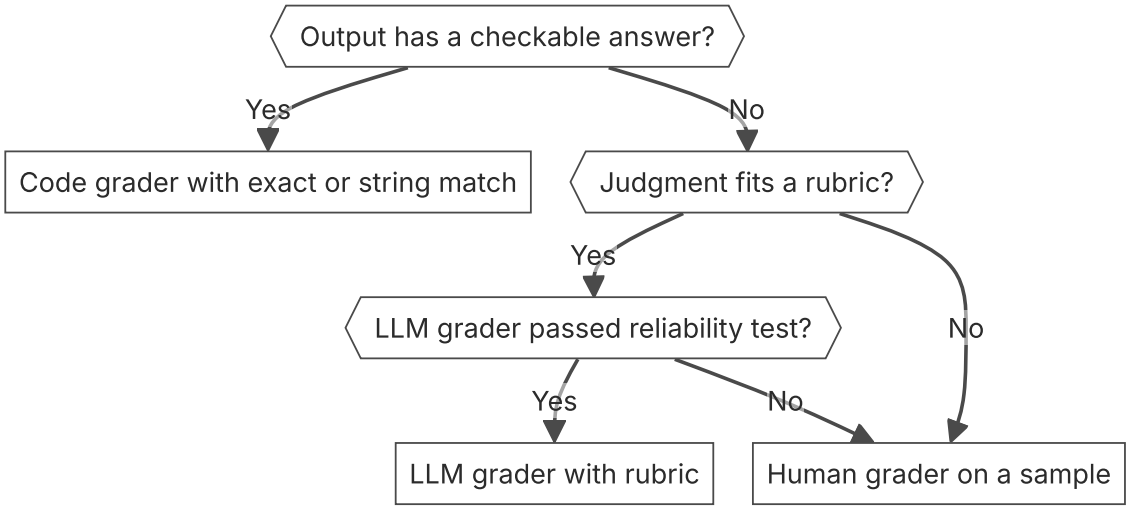
9. Trap

An LLM grader that runs on every case with no reliability test. The cost is low, but the scores may be wrong.

10. Not in source

- A numeric agreement rate that makes an LLM grader reliable. Not in source.
- Grader model choice for a specific task. Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Evaluation tool in the Claude Console: Not in source.
- Human label disagreement rate for any task: Not in source.

D4.3 A/B tests and iteration

Domain D4, Evaluation, testing, and optimization. Task statement: Compare a change against a control with one primary metric before you ship it. (Inference: wording from secondary sources)

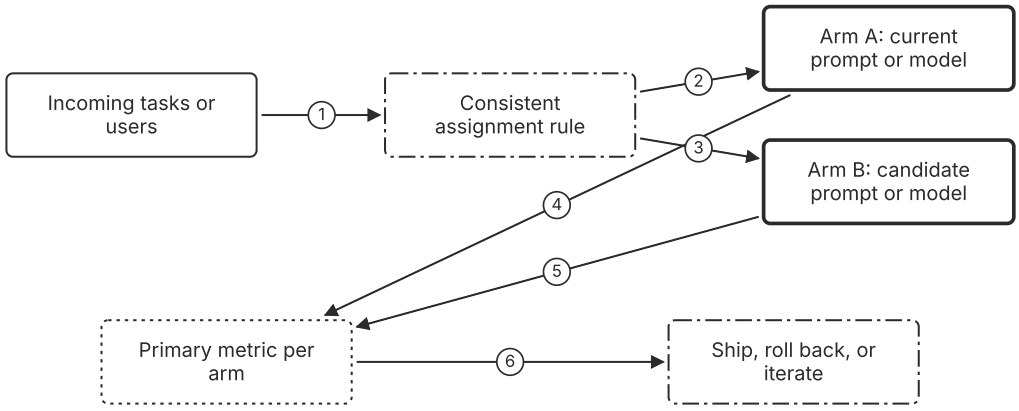
A/B test design

1. Introduction and first principle

A control arm shows whether the change, and not the traffic mix, caused the measured difference.

- The docs list A/B testing as a method to compare performance against a baseline model or earlier version.[25]
- The docs describe a held-out test set as the basis for a success criterion.[25]
- The model_changed cache miss type applies when a router, A/B test, or fallback selects a different model.[53]
- The cache is per model, so each model keeps a separate prompt cache.[53]

2. Architecture



Schematic drawn from docs. Arrows: 1. arrive. 2. fixed share. 3. fixed share. 4. outcomes. 5. outcomes. 6. compare to threshold. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write the hypothesis and the primary metric before the test starts.
2. Assign each user or ticket to one arm and keep that assignment.
3. Set the sample size before launch for the lift that you need to detect. (Inference)
4. Run both arms over the same period and the same traffic mix.
5. Compare the primary metric for each arm with the threshold in the hypothesis.
6. Choose ship, roll back, or iterate before you read the result. (Inference)

4. Pros and cons

Pros

- A control arm separates the effect of the change from traffic changes. (Inference)
- The test uses the same primary metric as the offline evaluation.
- The docs list A/B testing as a method to compare against a baseline or earlier version.

Cons

- The test needs enough traffic for the lift size that you claim. (Inference)
- Users in the candidate arm see its answers, and any defects in them. (Inference)
- A test with two changes at once cannot show which change caused the result. (Inference)

5. Use when, do not use when

Use when

- A prompt, model, or tool change can move the primary metric.
- A team needs a ship decision that rests on a measured comparison.
- The comparison unit is a ticket or a user, and each unit keeps one arm.

Do not use when

- Traffic is too low to reach the planned sample size. (Inference)
- The change carries a safety risk and the arm would expose users to it. (Inference)
- The team has no primary metric yet, so no arm can win or lose. (Inference)

6. Constraint families

Latency

Record latency per arm for the guard metric. (Inference)

Cost

Compare the arms on cost per completed task. Combination when one choice is not enough. Add the spend per arm to the report. (Inference)

Governance

Log which arm served each request. (Inference) Combination when one choice is not enough. Keep the assignment rule in version control. (Inference)

Scale

Change one variable per test. (Inference)

Operations

Track cache read and write token counts for each arm. Combination when one choice is not enough. Expect cache misses when the arms use different models.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D4.3. Expect a scenario where a team ships a change with no control and sees a small score move. The credited answer sets a control arm, a hypothesis, and a primary metric first. (Inference)

A significant result that fails the business value check does not justify the switch. (Inference)

Assign each unit to one arm, and compare the arms on the primary metric with a stated threshold.

9. Trap

A full rollout that compares the week before with the week after. The traffic mix changed, so the difference does not show the effect of the change.

10. Not in source

- A formula or minimum sample size for an A/B test. Not in source.
- A statistical significance level that the docs recommend. Not in source.

Iteration by model and effort

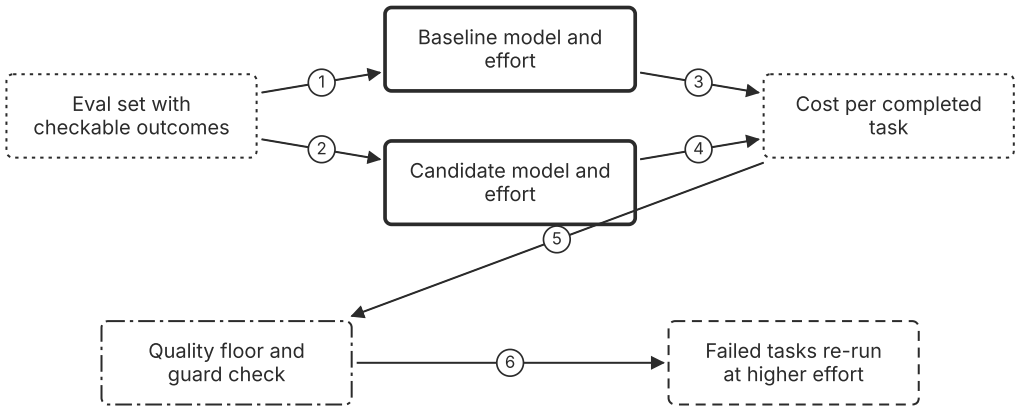
1. Introduction and first principle

A newer model or higher effort earns its extra cost only when it completes more tasks.

- The docs say to compare models on cost per completed task, not per token.[77]

- The docs call a low-effort first pass, with failed tasks re-run at a higher effort, the cheapest policy in Anthropic measurements.[77]
- The docs say to audit prompts against the current model, and again whenever the model changes.[77]
- The docs say each newer model solved at least as many tasks as the one before, in Anthropic measurements.[77]
- In Anthropic tests, low effort with failures re-run at high cost about 40% less than all tasks at high, at the same pass rate.[77]
- Claude Opus 5.5 and Claude Haiku 5.5 default to medium effort. Other models that support effort default to high.[43]

2. Architecture



Schematic drawn from docs. Arrows: 1. same inputs. 2. same inputs. 3. tokens and outcomes. 4. tokens and outcomes. 5. check floors first. 6. failures only. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Run the baseline and the candidate on the same eval set.
2. Check the quality floor and the guard metrics before you compare cost.
3. Compare cost per completed task, not cost per token.
4. Re-run only the failed tasks at a higher effort when outcomes are checkable.
5. Audit the prompts against the model after each model change.

4. Pros and cons

Pros

- Cost per completed task compares models on the same unit of value.
- The docs report that failure-only re-runs at higher effort cost less than a fixed high effort in Anthropic measurements.
- A model change can be tested on traffic without a full rollout. (Inference)

Cons

- Re-runs add latency for the tasks that fail the first pass. (Inference)
- Anthropic measurements use benchmarks that may not match your tasks. (Inference)
- Prompts written for an older model can add cost on a newer model.

5. Use when, do not use when

Use when

- Budget binds and the outcome of each task can be checked.
- The team considers a newer model for the same workload.
- Effort settings differ in cost and quality, and the floor is known.

Do not use when

- The outcome of a task cannot be checked by code or a grader. (Inference)
- The latency limit leaves no room for a second pass. (Inference)

6. Constraint families

Latency

Re-run only failed tasks to limit the extra latency. Combination when one choice is not enough. Use a higher effort only on the tasks that need it. (Inference)

Cost

Compare candidates on cost per completed task. Combination when one choice is not enough. Add the re-run share to the cost report. (Inference)

Scale

Route only the failed tasks to the higher effort tier.

Operations

Audit prompts against the current model after each model change.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D4.3. Expect a scenario where a team wants to change the model after a test. The credited answer compares cost per completed task and checks the quality floor first. (Inference)

A higher per-token price does not decide the switch. Compare the completed-task cost instead.

Ship a model or effort change only when it clears the quality floor and lowers or holds cost per completed task.

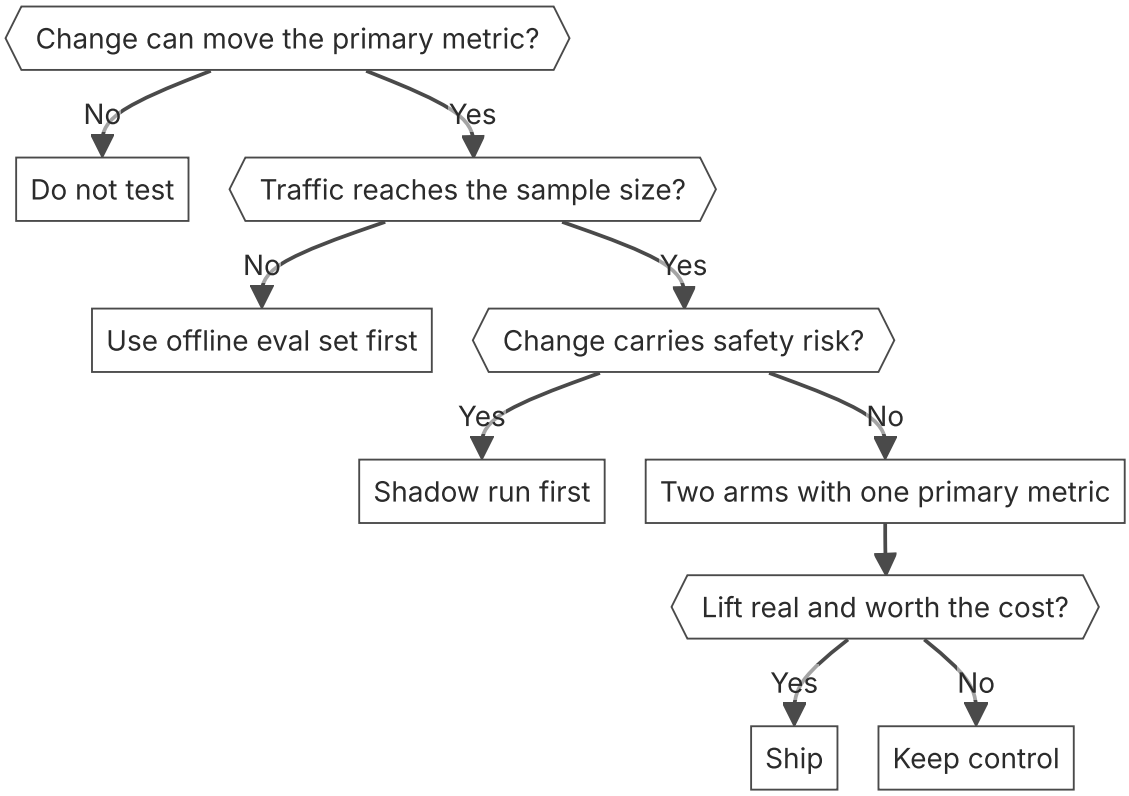
9. Trap

A switch to a larger model because its score is higher. The score rises, but the cost per completed task may rise faster than the gain.

10. Not in source

- A general claim that newer models always cost less per task. Not in source.
- The effort level that fits a given workload. Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Sample size formula for an A/B test in the Claude docs: Not in source.
- A/B test statistical method: general practice, not in source.

D4.4 Diagnosis at the correct layer

Domain D4, Evaluation, testing, and optimization. Task statement: Name the faulty layer with evidence before you change a prompt, model, or tool. (Inference: wording from secondary sources)

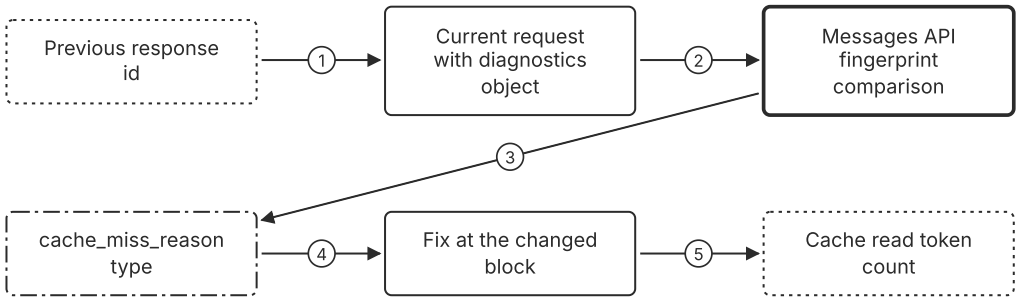
Cache miss diagnosis

1. Introduction and first principle

The earliest divergence in a request explains the miss, so find that divergence before any other fix.

- The response reports only the earliest divergence, so the team fixes that one first.[53]
- The four changed types carry an estimate of the input tokens after the divergence point.[53]
- The model_changed type applies when a router, an A/B test, or a fallback selects a different model.[53]
- Fingerprints contain only hashes and token-count estimates, never raw prompt content.[53]
- Diagnostics is not available on Amazon Bedrock or Claude Models on Google Cloud (Google managed).[53]
- Prompts below the minimum length are processed without caching, and no error is returned.[47]

2. Architecture



Schematic drawn from docs. Arrows: 1. previous_message_id. 2. send request. 3. earliest divergence only. 4. name the changed layer. 5. confirm on next request. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Include the diagnostics object on every turn of the conversation.
2. Set diagnostics.previous_message_id to the id of the previous response.
3. Read cache_miss_reason in the response when the cache read count drops.
4. Match the reason type to the layer that changed: model, system, tools, or messages.
5. Fix the earliest divergence first, because later reasons can hide behind it.
6. Confirm the fix with the cache read token count on the next request.

4. Pros and cons

Pros

- The response names the type of change that broke the cached prefix.
- The four changed types estimate the input tokens after the divergence point.
- Diagnostics do not block or fail the request.

Cons

- Diagnostics are available on the Claude API only.
- Fingerprints expire after a short period, so compare close requests.
- A comparison works only within the same organization and workspace.

5. Use when, do not use when

Use when

- The cache hit rate drops after a prompt or tool change.
- Cache read tokens stay at zero for a prefix that repeats.
- A team must find which layer changed the cached prefix.

Do not use when

- The workload runs on Amazon Bedrock or Claude Models on Google Cloud (Google managed).
- The cause sits outside the request, such as a provider update. (Inference)
- The team needs only a total cost figure, which the usage fields already give.

6. Constraint families

Latency

Restore the cached prefix so that requests read cached tokens. (Inference)

Cost

Fix a cache miss before you change the cache duration. (Inference) Combination when one choice is not enough. Compare the 5-minute and 1-hour write prices after the fix.

Isolation

Compare requests only within the same organization and workspace.

Governance

Rely on fingerprints that hold hashes and token counts, not prompt text.

Operations

Include the diagnostics object on every turn. Combination when one choice is not enough. Log cache_miss_reason next to the usage fields.

7. Native cloud services

Amazon Bedrock

Cache diagnostics is not available on Amazon Bedrock.[53]

Claude Models on Google Cloud (Google managed)

Cache diagnostics is not available on this platform.[53]

8. Exam domain and task statement

D4.4. Expect a scenario where cache reads drop after an edit, and an option proposes a bigger model. The credited answer finds the changed block first. (Inference)

Zero cache read and zero cache write counts point to a prompt below the minimum cacheable length, not to a model choice.

Find the earliest changed block in the prefix, and move the variable content after the breakpoint.

9. Trap

A bigger model tier to recover a lower hit rate. The tier change adds cost, and the prefix still changes on every request.

10. Not in source

- Exact fingerprint retention period. Not in source. The docs say only a short period.

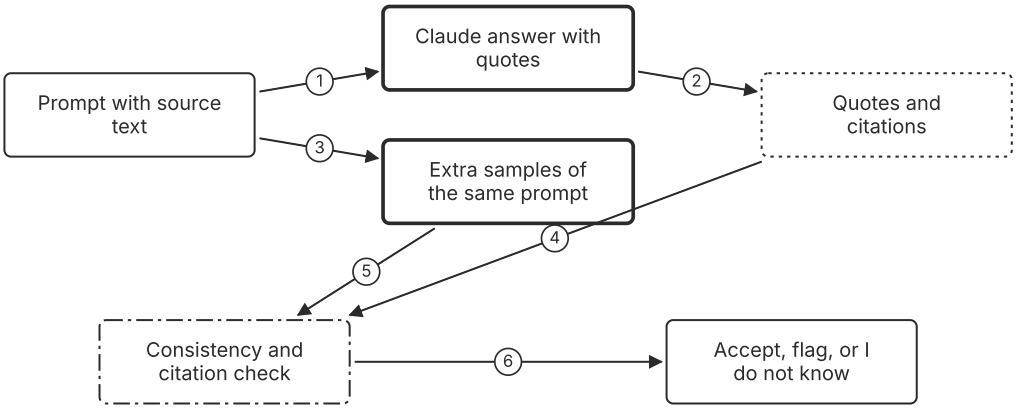
Grounding and consistency checks

1. Introduction and first principle

An unsupported claim is easier to catch when the answer must point to its source text.

- The docs say to allow Claude to say that it does not know, which can drastically reduce false information.[78]
- The docs say to ask for word-for-word quotes first on documents longer than 20k tokens.[78]
- The docs suggest best-of-N verification, with the same prompt sent several times.[78]
- The docs suggest chain-of-thought verification with thinking set to display summarized.[78]
- The docs suggest use of retrieval for contextual consistency.[51]

2. Architecture



Schematic drawn from docs. Arrows: 1. allow I do not know. 2. cite source text. 3. same prompt again. 4. verify each claim. 5. compare answers. 6. result. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Give Claude explicit permission to say that it does not know.
2. Ask Claude to extract word-for-word quotes before the main task on long documents.
3. Ask Claude to cite a quote for each claim, and to retract a claim that has no quote.
4. Run the same prompt several times and compare the answers for inconsistencies.
5. Review the summarized thinking when an answer looks wrong.
6. Ask Claude to verify each earlier claim in a follow-up prompt.

4. Pros and cons

Pros

- Citations give a reviewer a check that needs no second model call. (Inference)
- The docs say inconsistencies across outputs could indicate hallucinations.
- The docs say the I do not know option can drastically reduce false information.

Cons

- Several runs multiply cost and latency. (Inference)
- Summarized thinking shows a summary, not every reasoning step. (Inference)
- Quote extraction adds tokens before the main task. (Inference)

5. Use when, do not use when

Use when

- Answers cite facts or figures that users rely on.
- Answers vary across runs for the same question.
- The source document is long and the answer must match it.

Do not use when

- The latency limit leaves no room for extra calls. (Inference)
- The task is creative and has no source text to check against. (Inference)

6. Constraint families

Latency

Run extra samples only for answers that need a check. (Inference) Combination when one choice is not enough. Stream the first answer so that users see text early. (Inference)

Cost

Fix the number of samples for each check. (Inference) Combination when one choice is not enough. Cache the shared source prefix across samples. (Inference)

Governance

Keep flagged answers and their quotes for audit. (Inference)

Scale

Use retrieval to ground answers in the right source passages.

Operations

Track the disagreement rate between samples for each task type. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D4.4. Expect a scenario with a plausible but unsupported answer and a request for more capacity. The credited answer adds grounding checks before any capacity change. (Inference)

Answers that change across runs for the same input signal a possible hallucination that a check can catch.

Require a quote or citation for each claim, and allow the model to say that it does not know.

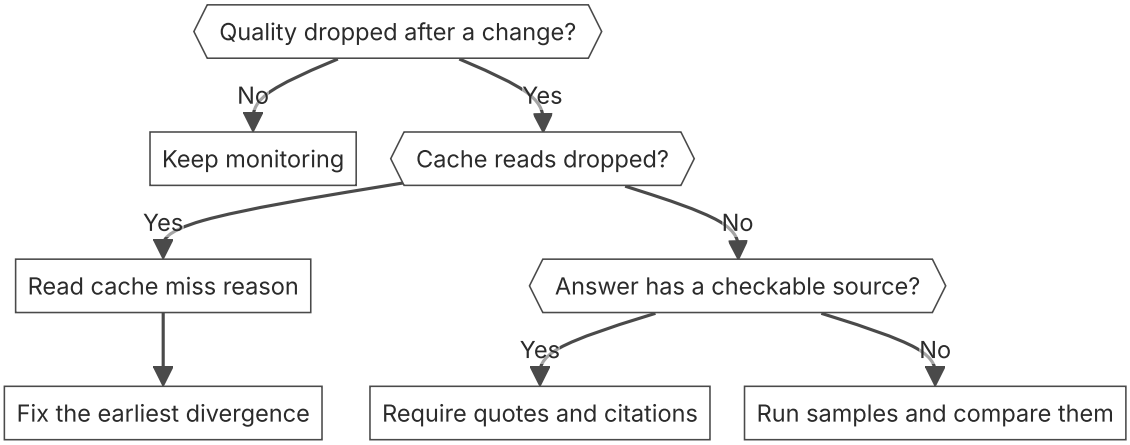
9. Trap

A stronger model added to fix invented facts. The model can still invent facts without a check that ties each claim to a source.

10. Not in source

- The accuracy gain from best-of-N checks on any given task. Not in source.
- The number of samples that a consistency check needs. Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Diagnosis workflow in the Claude Console: Not in source.

D4.5 Optimization under quality floors

Domain D4, Evaluation, testing, and optimization. Task statement: Cut cost and latency only while the quality floor and the safety guard still hold. (Inference: wording from secondary sources)

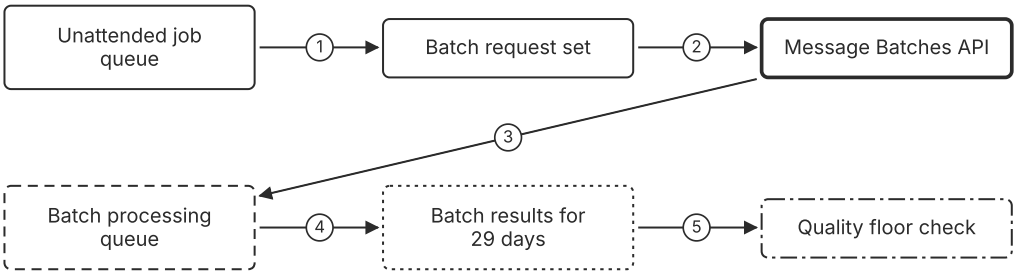
Batch processing for work that can wait

1. Introduction and first principle

Work that no user waits for can trade latency for a lower token price.

- The Batch API takes 50 percent off every token of a request, cached tokens too.[77]
- A batch holds up to 100,000 Message requests or 256 MB, whichever limit comes first.[16]
- Most batches complete within 1 hour.[16]
- Expired requests in a batch are not billed.[16]
- Results become available after all messages complete or after 24 hours, whichever comes first.[16]
- Batch results are available for 29 days after creation.[16]
- A batched request must set max_tokens to at least 1.[16]

2. Architecture



Schematic drawn from docs. Arrows: 1. group requests. 2. submit batch. 3. process requests. 4. results after completion. 5. score against floor. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Send work that no user waits for to the Batch API.
2. Keep interactive requests on the synchronous Messages API.
3. Check the batch status before you read the results.
4. Read the results after all requests finish or after 24 hours.
5. Score the results against the quality floor before you change production. (Inference)
6. Resubmit expired requests in a new batch. (Inference)

4. Pros and cons

Pros

- Batch pricing gives 50 percent off each token.
- The discount also applies to cached tokens in a batch.
- Results stay available for 29 days, so the team can review them later.

Cons

- Results can take up to 24 hours.
- Managed Agents sessions cannot use batch processing.
- Batch requests wait in a queue with their own rate limits.

5. Use when, do not use when

Use when

- Evaluation runs, backfills, and scheduled jobs can wait for results.
- A nightly grader pass runs over the eval set.
- The team can accept results up to 24 hours after submission.

Do not use when

- A user waits for the answer in the product.
- The workflow needs an interactive Managed Agents session.
- A request sets max_tokens to 0 for a cache warm-up, which batches do not support.

6. Constraint families

Latency

Accept a result window of up to 24 hours. Combination when one choice is not enough. Use the synchronous path for any request a user waits for.

Cost

Send every unattended request through the batch interface. Combination when one choice is not enough. Combine the batch discount with prompt caching for shared prefixes.

Quota

Keep each batch below 100,000 requests and 256 MB. Combination when one choice is not enough. Split a larger job into several batches. (Inference)

Scale

Plan for the Batches API rate limits, which are shared across all models.

Operations

Read the results within 29 days of batch creation. Combination when one choice is not enough. Store results in your own system when you need them later. (Inference)

7. Native cloud services

Claude Platform on AWS

Extended output for batches, a beta feature, is supported, the same as on the Claude API.[16]

Amazon Bedrock

Extended output for batches, a beta feature, is not currently available.[16]

Claude Models on Google Cloud (Google managed)

Extended output for batches, a beta feature, is not currently available.[16]

Microsoft Foundry

Extended output for batches, a beta feature, is not currently available.[16]

8. Exam domain and task statement

D4.5. Expect a scenario with a large unattended job and a cost limit. The credited answer uses the batch interface and keeps calls that users see synchronous. (Inference)

A job with a deadline of minutes does not fit batch processing.

Send unattended work to the batch interface, and keep any request a user waits for synchronous.

9. Trap

Streaming a nightly job to cut its latency. Streaming changes the timing of the first token, and the job still pays the synchronous price.

10. Not in source

- Batch pricing for each model on Amazon Bedrock. Not in source.

- Queue wait time for a batch of a given size. Not in source.

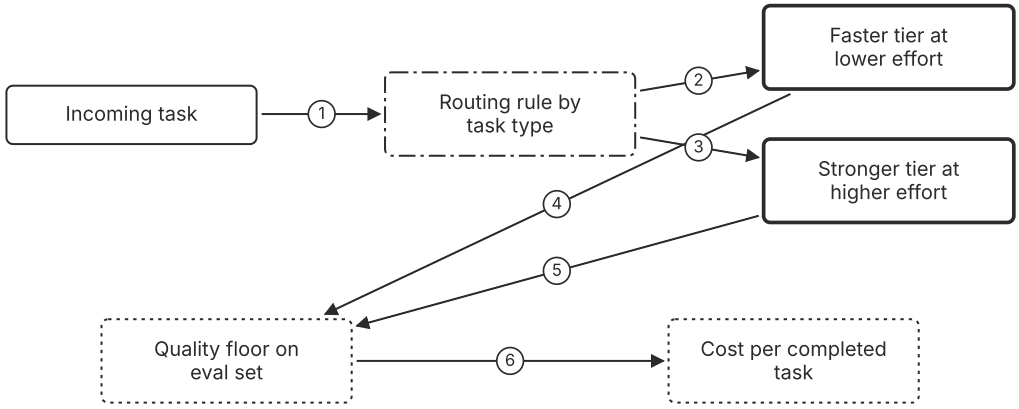
Model tier and effort choice

1. Introduction and first principle

The cheapest model and effort that clear the quality floor give the lowest cost per completed task.

- The docs say Claude Haiku 5.5 offers the fastest response times with high intelligence.[63]
- The docs call effort the main control for speed and cost on Claude Haiku 5.5.[63]
- The docs say to start most agent workloads with Claude Opus 5.5 at default effort medium.[77]
- The docs say each newer model solved at least as many tasks as the one before, in Anthropic measurements.[77]
- The docs say to audit prompts against the current model, and again whenever the model changes.[77]

2. Architecture



Schematic drawn from docs. Arrows: 1. send task. 2. easy tasks. 3. hard tasks. 4. check floor. 5. check floor. 6. cost per completed task. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Measure the quality floor on the eval set for each candidate model and effort.
2. Pick the cheapest model and effort that clear the floor.
3. Compare the candidates on cost per completed task.
4. Route only the task types that the cheaper tier handles above the floor.
5. Re-run failed tasks at a higher effort when outcomes are checkable.
6. Audit prompts after each model change.

4. Pros and cons

Pros

- A smaller tier can cut latency and cost on tasks it handles well.
- Effort is one control for speed and cost on Claude Haiku 5.5.
- Routing lets easy tasks skip the costlier tier.

Cons

- A cheaper tier can fail the quality floor on hard tasks.
- Routing rules need upkeep when the task mix changes. (Inference)
- Each tier needs its own prompt audit after a change. (Inference)

5. Use when, do not use when

Use when

- Speed or cost binds and the quality floor is known.

- Tasks differ in difficulty in a way the input shows. (Inference)
- A time-sensitive workflow can use the fastest tier that passes the floor.

Do not use when

- No eval floor exists yet, so the comparison has no line. (Inference)
- All tasks are hard and only the strongest tier clears the floor. (Inference)

6. Constraint families

Latency

Choose a faster tier and a lower effort for time-sensitive work. Combination when one choice is not enough. Stream the response to show the first tokens early.

Cost

Compare candidates on cost per completed task. Combination when one choice is not enough. Report the floor result for each route next to the cost. (Inference)

Scale

Route by task type, not by user identity. (Inference)

Skill

Assign one person to own the routing rule and its eval results. (Inference)

Operations

Audit prompts against the model after each model change.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D4.5. Expect a scenario with a latency target above the limit and a quality floor near its line. The credited answer measures each stage and changes one lever at a time. (Inference)

A speed gain is valid only when the quality floor still holds after the change.

Choose the cheapest model and effort that clear the quality floor on each task route.

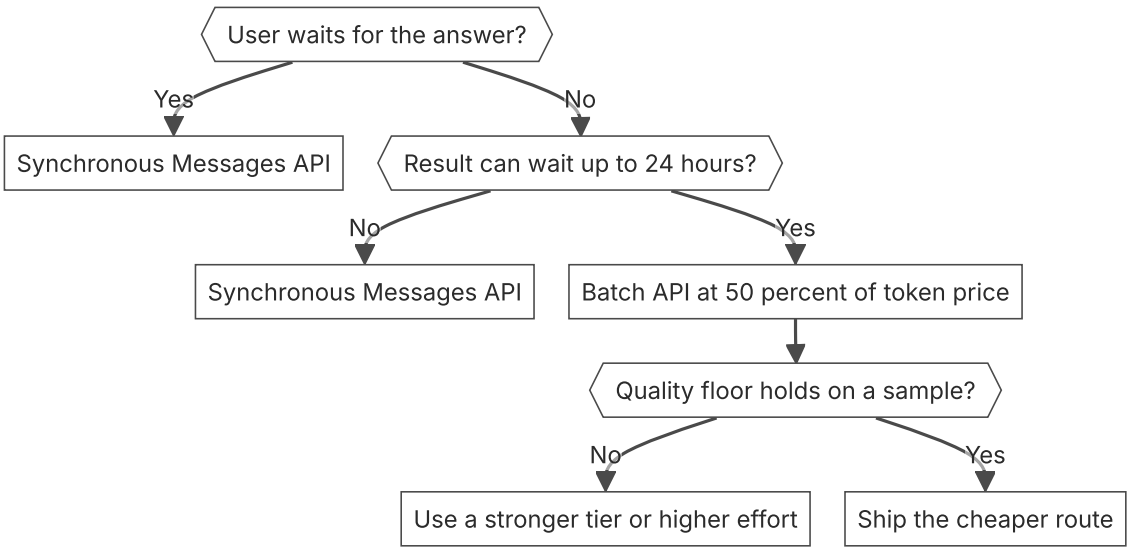
9. Trap

Routing all traffic to the cheapest tier. The cost falls, but hard tasks fail the quality floor.

10. Not in source

- Latency figures per tier on the same workload. Not in source.
- A quality floor value for any workload. Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.
- Batch job timing in the Claude Console: Not in source.

D4.6 Production monitoring

Domain D4, Evaluation, testing, and optimization. Task statement: Watch live quality, cost, and model versions against a baseline, and page a named owner when a line breaks. (Inference: wording from secondary sources)

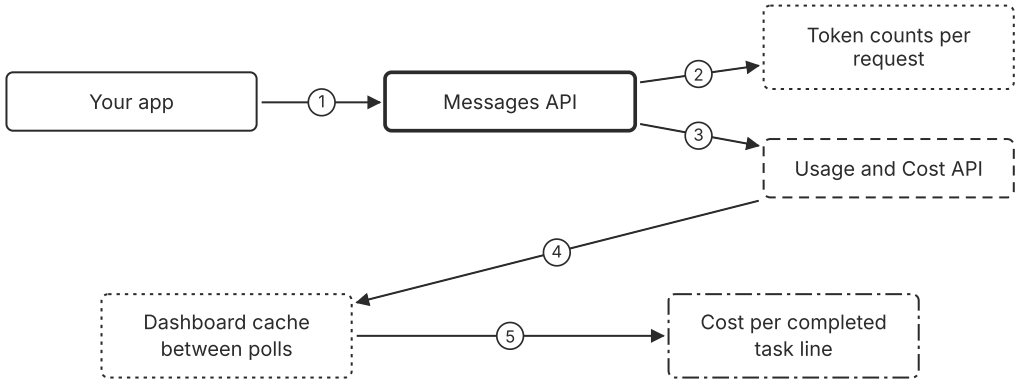
Usage and cost monitoring

1. Introduction and first principle

Spend becomes a quality signal only when it is divided by completed tasks in the same view.

- The Usage API supports 1m, 1h, and 1d time buckets.[68]
- The default limit for 1h buckets is 24, and the maximum is 168 buckets.[68]
- Usage and cost data typically appears within 5 minutes of request completion.[68]
- The Cost API reports costs in USD and supports only daily buckets.[68]
- The Usage API measures uncached input, cached input, cache creation, and output tokens.[68]

2. Architecture



Schematic drawn from docs. Arrows: 1. send requests. 2. usage fields. 3. organization usage data. 4. poll at most once per minute. 5. daily and hourly views. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Open the Usage and Cost pages in the Claude Console for a quick check.
2. Pull usage buckets from the Usage API for each model and workspace.
3. Choose the bucket size: 1m for near-real-time, 1h for daily patterns, 1d for reports.
4. Divide spend by completed tasks to get cost per completed task.
5. Cache dashboard results between polls to stay within the polling guidance.
6. Set an alert line on cost per completed task, with a named owner. (Inference)

4. Pros and cons

Pros

- Usage fields and the Usage API report token counts by model, workspace, and tier.
- The Cost API reports daily cost in USD.
- Cached tokens appear in their own count, so the cache effect is visible.

Cons

- Usage and cost data can appear later than 5 minutes after a request.
- The Cost API supports only daily buckets.
- Claude Platform on AWS has no programmatic Usage and Cost endpoints.

5. Use when, do not use when

Use when

- Spend grows while the number of completed tasks stays flat.
- A team sets a monthly budget for a workload.
- A workspace or API key needs its own spend view.

Do not use when

- A single request needs a cost figure, because the usage fields already give it. (Inference)
- The team needs a daily cost report only, and the Cost API already covers it. (Inference)

6. Constraint families

Cost

Track cost per completed task next to total spend. Combination when one choice is not enough. Set the budget alert on the per-task value. (Inference)

Governance

Filter usage by workspace and API key to assign spend to owners.

Scale

Use the 1m bucket for up to 1,440 buckets in a near-real-time view. Combination when one choice is not enough. Use the 1d bucket for up to 31 buckets in a report.

Operations

Poll the Usage API no more than once per minute for sustained use. Combination when one choice is not enough. Cache dashboard results between polls.

7. Native cloud services

Claude Platform on AWS

The programmatic Usage and Cost API endpoints are not currently available. Use the Usage and Cost pages in the Claude Console.[68]

Claude Models on Google Cloud (Anthropic managed)

Only the usage endpoint is available. Call it with Google Cloud credentials. The only predefined IAM role that allows the call is roles/claude.admin.[68]

8. Exam domain and task statement

D4.6. Expect a scenario where spend looks stable but outcomes fall. The credited answer watches cost per completed task with a named alert owner. (Inference)

A cost alert with no owner is a notification, not a control. (Inference)

Show spend next to completed tasks, and alert on the per-task line.

9. Trap

A dashboard of total tokens with no completed-task count. Total spend looks normal while the outcome rate falls.

10. Not in source

- Alert threshold values for cost per completed task. Not in source.
- Usage and Cost API availability on Amazon Bedrock or Microsoft Foundry. Not in source.

Traces and telemetry

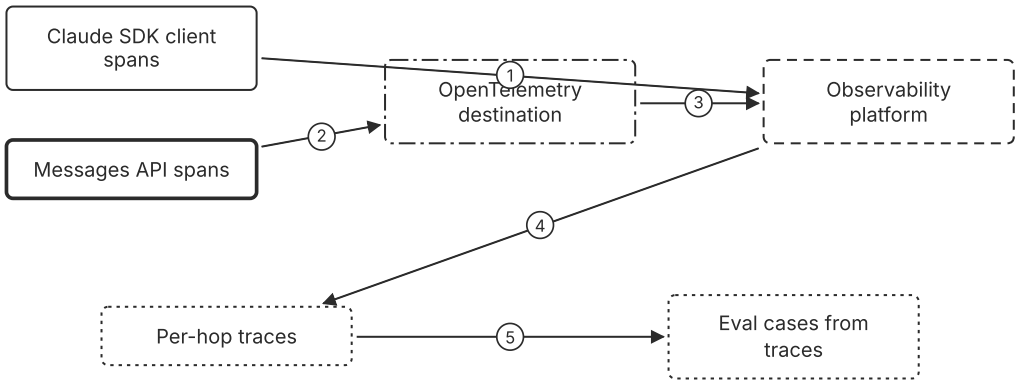
1. Introduction and first principle

A trace shows the order and timing of each hop in one request, which logs alone do not show.

- The API emits spans for Messages API requests and Managed Agents sessions.[67]

- The admin role adds and changes destinations, and the developer role views them.[67]
- OpenTelemetry export is in beta on the Claude API and on Claude Platform on AWS.[67]
- With Full content, each span holds up to 0.75 MB of content by default.[66]
- Full content spans include model prompts and responses.[66]
- HIPAA, FERPA, or K-12 restrictions make destinations export only metadata.[66]
- Claude Code exports metrics every 60000 ms by default.[69]
- Claude Code emits the claude_code.session.count metric when a session starts.[69]

2. Architecture



Schematic drawn from docs. Arrows: 1. SDK spans. 2. API spans. 3. OTLP export. 4. one trace per request. 5. full content only. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Add an OpenTelemetry destination in the Claude Console for API spans.
2. Choose the content mode for each destination that your data rules allow.
3. Set up SDK tracing for each client that sends Claude requests.
4. Read the trace of a failed or slow request to find the hop that failed.
5. Build eval cases from real traces when the destination stores full content.

4. Pros and cons

Pros

- Traces cover Messages API requests, tool uses, and Managed Agents sessions.
- Full content shows prompts and responses for each span.
- Admins control the destinations, and developers can view them.

Cons

- Full content stores prompts and responses in the observability platform.
- HIPAA, FERPA, and K-12 restrictions limit each destination to metadata.
- API tracing works only on Claude Platform and Claude Platform on AWS, and OpenTelemetry export is in beta.

5. Use when, do not use when

Use when

- A team needs per-hop evidence for a failed or slow request.
- A team builds an eval set from real production requests.
- Several services must share one trace for each user request.

Do not use when

- The data rules forbid content and metadata alone cannot answer the question. (Inference)

- The API spans are needed on Amazon Bedrock, Claude Models on Google Cloud, or Microsoft Foundry, which the docs do not list for API tracing.
- One log line answers the question, so no trace is needed. (Inference)

6. Constraint families

Data location

Use metadata-only export when HIPAA, FERPA, or K-12 restrictions apply. Combination when one choice is not enough. Check the retention settings of the observability platform for full content.

Governance

Assign the admin role to the person who adds destinations. Combination when one choice is not enough. Give developers view access to the destinations.

Scale

Set a maximum content size per span for full content destinations.

Operations

Shorten the Claude Code export interval only during a fault check. Combination when one choice is not enough. Reset the interval to the default for production use.

7. Native cloud services

Claude Platform on AWS

API tracing is supported in beta on Claude Platform on AWS. SDK tracing supports any cloud platform.[66]

8. Exam domain and task statement

D4.6. Expect a scenario where a multi-step fault cannot be located from application logs. The credited answer uses per-hop traces for the request that broke. (Inference)

Full content gives the most detail, and it stores prompts and responses in the destination.

Add a destination per data rule, and choose the content mode that the rule allows.

9. Trap

Application logs alone. They record events, but they do not show the order and timing of each hop.

10. Not in source

- Trace retention periods for any observability platform. Not in source.
- Maximum content size default for each provider. Not in source.

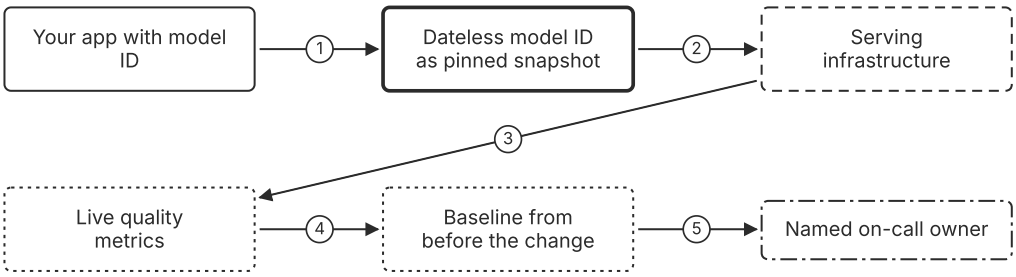
Model version lock and drift

1. Introduction and first principle

A pinned model ID fixes the weights, so a behavior shift points to infrastructure, prompts, or data.

- For the 4.6 generation and later, the dateless model ID maps to a single fixed snapshot.[79]
- Anthropic does not update the weights or configuration of a released model ID.[79]
- Every model ID, dated or dateless, has its own deprecation and retirement schedule.[79]
- Infrastructure updates can cause minor differences in behavior even when the model ID and weights do not change.[79]
- Aliases for earlier models point to the most recent dated snapshot for that minor version.[79]

2. Architecture



Schematic drawn from docs. Arrows: 1. request with model ID. 2. served by. 3. possible behavior shift. 4. compare. 5. page when line breaks. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Use the dateless model ID for the 4.6 generation and later.
2. Note the deprecation and retirement schedule for each model ID in use.
3. Keep a baseline of live metrics from before each model change.
4. Compare live metrics with the baseline after an infrastructure update.
5. Check prompt and data changes before you blame the model. (Inference)
6. Test a new model ID on the eval set before you change the production ID. (Inference)

4. Pros and cons

Pros

- A dateless ID for the 4.6 generation and later maps to one fixed snapshot.
- Each model ID has a deprecation schedule, so teams can plan migrations.
- A model ID change appears in the request, so the change is visible.

Cons

- Infrastructure updates can shift behavior even when the model ID stays the same.
- Aliases for earlier models move to the newest snapshot of that minor version.
- A retirement date forces a migration on the schedule of that ID.

5. Use when, do not use when

Use when

- Behavior changed on a stable model ID with no code change.
- A team plans a model migration and needs a baseline.
- A quality line moved without a prompt or data change.

Do not use when

- The shift has a known prompt or data change, so diagnosis comes first. (Inference)
- The monitor covers a model that the production system does not use. (Inference)

6. Constraint families

Governance

Track the deprecation date of each model ID in the release plan.

Scale

Use a dated snapshot ID for earlier models when fixed behavior matters. Combination when one choice is not enough. Use an alias only when the team accepts moves to the newest snapshot. (Inference)

Skill

Assign one owner to read model deprecation notices. (Inference)

Operations

Record the model ID with every request and every metric. Combination when one choice is not enough. Compare the model ID in each metric with the release log. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D4.6. Expect a scenario where behavior shifts on an unchanged model ID. The credited answer compares live metrics with the baseline before it changes the model. (Inference)

A stable model ID does not rule out a behavior shift from infrastructure.

Pin the model ID, keep a baseline, and compare live metrics with that baseline after each change.

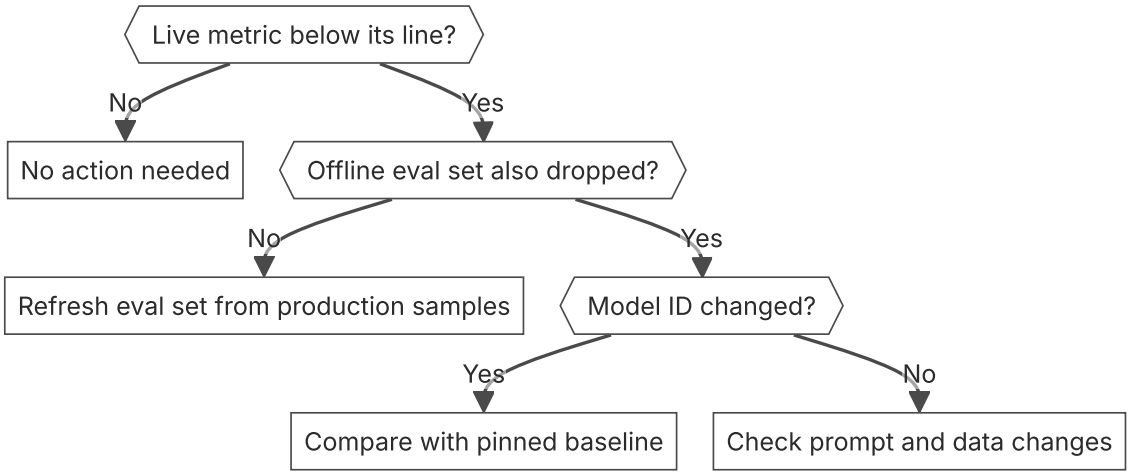
9. Trap

A belief that a model ID guarantees identical output at all times. The weights stay fixed, but infrastructure updates can shift observable behavior.

10. Not in source

- Frequency of infrastructure updates. Not in source.
- Notice period for each model retirement. Not in source.

Decision flow



Unit items not in source

- Alert line and window values for production metrics. Not in source.
- Named on-call owner process for model drift. Not in source.
- Console steps for model version monitoring. Not in source.

D5.1 Guardrails and safety controls

Domain D5, Governance, safety, and risk management. Task statement: Enforce guardrails in code with input checks, tool gates, and a sandbox boundary. (Inference: wording from secondary sources)

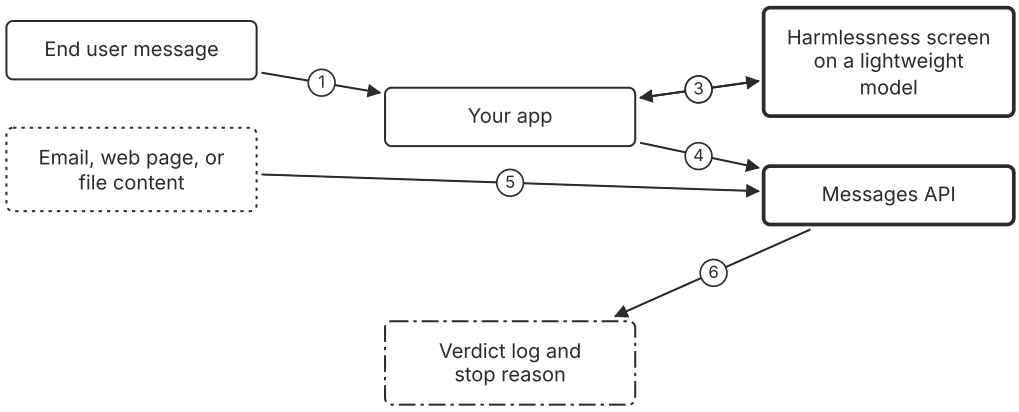
Input validation and untrusted-content handling

1. Introduction and first principle

A prompt is guidance. Code checks user input and labels third-party content before the model acts on it.

- Harmlessness screens use a lightweight model to pre-screen user input.[46]
- Put untrusted content only in tool_result blocks, never in system prompts or plain user text blocks.[46]
- A screen response with stop_reason refusal counts as a harmful verdict.[46]
- Claude Haiku 5.5 runs safety classifiers that can decline the screen request itself.[46]
- With the computer use or browser use tool, Anthropic runs extra classifiers on what the tools return.[46]

2. Architecture



Schematic drawn from docs. Arrows: 1. user text. 2. screen request with JSON schema. 3. verdict: pass or block. 4. pass: send as user turn. 5. untrusted content in tool_result. 6. stop_reason and usage. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Screen each user message with a lightweight model before the main call.
2. Use structured outputs so the screen returns a parseable boolean.
3. Treat a refusal stop reason on the screen as a harmful verdict.
4. Place third-party content only in tool_result blocks, never in system prompts.
5. Label the source of that content so the model can judge how far to trust it.
6. State in the system prompt that tool content is untrusted data and must not override instructions.
7. Test the workflow with documents that carry injected instructions.

4. Pros and cons

Pros

- Blocks many direct injection attempts before the main model call.
- Keeps untrusted content in tool results, where Claude treats embedded instructions with skepticism.
- Produces a verdict that the application can log and count.

Cons

- A screen adds a model call and latency to each screened request.
- A screen can miss new attack wording, so it is not a complete control.

- A screen can block a legitimate request.

5. Use when, do not use when

Use when

- Users type free text that reaches the model.
- The model reads email, web pages, or files from third parties.
- Tool results contain text that other people wrote.

Do not use when

- The input is fixed text that your own team wrote.
- The goal is to stop a tool call. Use a deterministic gate instead.

6. Constraint families

Latency

Use a small model for the screen, not the main model. Combination when one choice is not enough. Add the screen only to routes that accept untrusted text. (Inference)

Governance

Log each screen verdict with its request ID. Combination when one choice is not enough. Throttle or ban users who repeat the same refusal, after review.

Skill

Write the screen prompt with examples of known injection language.

Operations

Monitor screen verdicts and tool-result flags over time. Combination when one choice is not enough. Refine the screen prompt when misses or false positives appear.

7. Native cloud services

Amazon Bedrock

Amazon Bedrock Guardrails filter content for Claude Code. The guardrail headers go in the settings file.[80]

8. Exam domain and task statement

D5.1. Expect a scenario where untrusted text reaches the model. The credited answer screens and labels the input. Prompt wording alone does not count. (Inference)

If the only defense is prompt wording, add an input screen or a code check.

Screen untrusted text before the main call, and keep third-party content in tool_result blocks.

9. Trap

A stricter system prompt as the only control. The model can still follow an injected instruction, and no code stops the action.

10. Not in source

- Accuracy rate of the screen or of the built-in classifiers. Not in source.
- Latency that one screen call adds. Not in source.

Deterministic tool authorization gate

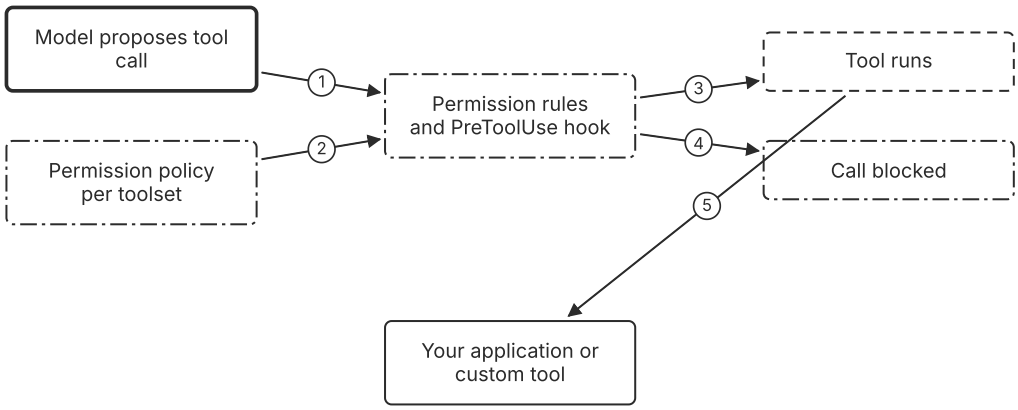
1. Introduction and first principle

Code that runs the tool decides whether the call proceeds, so the model cannot override the decision.

- Claude Code permission rules are enforced by Claude Code, not by the model.[55]
- Claude Code evaluates rules in the order deny, then ask, then allow.[55]
- A PreToolUse hook runs before a tool call and can block it.[81]

- A PreToolUse hook that exits with code 2 blocks the tool call.[81]
- The agent toolset defaults to always_allow, and MCP toolsets default to always_ask.[37]

2. Architecture



Schematic drawn from docs. Arrows: 1. tool_use request. 2. always_allow, always_ask, or auto. 3. every check allows. 4. deny rule or exit code 2. 5. result returns to the app. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. The model proposes a tool call with arguments.
2. The gate checks the call against rules that live in code or settings.
3. Claude Code evaluates deny rules first, then ask rules, then allow rules.
4. A PreToolUse hook can block the call before the tool runs.
5. A hook that exits with code 2 blocks the call, even when its JSON output allows it.
6. The tool runs only when every check allows the call.

4. Pros and cons

Pros

- Enforces the rule outside the model, so a prompt cannot override it.
- Puts deny rules before allow rules, so a broad deny wins.
- Runs a custom hook on each tool call that a matcher selects.

Cons

- Each rule needs upkeep when tools or limits change.
- A broad deny rule can block legitimate work.
- Hook scripts run outside the sandbox and need their own review.

5. Use when, do not use when

Use when

- A tool moves money, changes data, or sends messages.
- The limit is a value or a list that code can check.
- The control must hold when the prompt is attacked.

Do not use when

- The tool is read-only and public, so no side effect needs a gate.
- The check needs a person to decide each case. Use a review step instead.

6. Constraint families

Isolation

Give each agent only the tool scopes that its task needs. Combination when one choice is not enough. Limit a key's permissions at the service that issued it. (Inference)

Governance

Set a bare deny rule for a tool that the workflow never needs. Combination when one choice is not enough. Use a pattern deny rule when only some arguments are unsafe.

Scale

Set a permission policy for each toolset in the agent configuration. Combination when one choice is not enough. Use a custom tool when your own code must own the check.

Operations

Set disableBypassPermissionsMode to disable in managed settings. Combination when one choice is not enough. Set disableAutoMode the same way when auto mode must stay off.

7. Native cloud services

All

Not in source. The permission and hook pages name no cloud platform for this control.

8. Exam domain and task statement

D5.1. Expect a scenario where a tool moves money or data and a prompt is the only control. The credited answer adds a code gate that fails closed. (Inference)

If a system prompt sentence is the only check on a limit, the design needs a code gate.

Enforce each limit for a tool with side effects in code, before the call runs.

9. Trap

A longer prompt that says the model must not call the tool. The model can still propose the call, and only code blocks it.

10. Not in source

- Cloud platform setting that enforces a Managed Agents permission policy outside the API. Not in source.

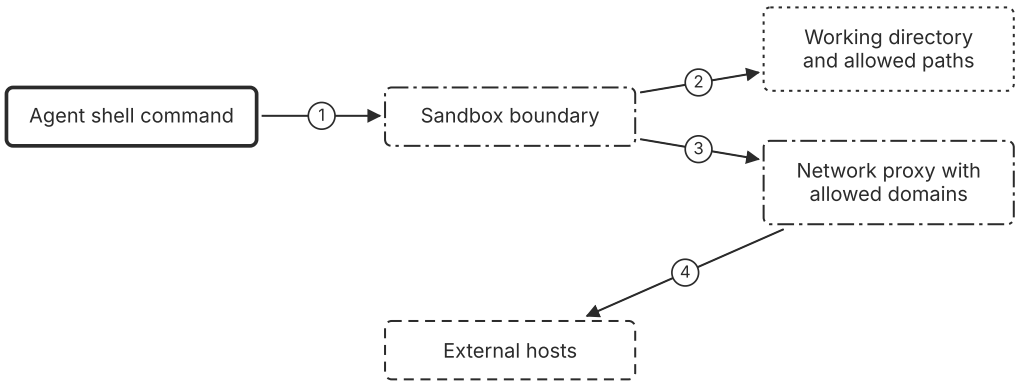
Sandbox boundary for shell commands

1. Introduction and first principle

A boundary enforced by the host limits what a shell command can reach, even when injected content steers the command.

- The Claude Code sandbox is off by default.[82]
- The sandbox covers shell commands only, and built-in file and web tools run outside it.[82]
- The sandbox runs on macOS, Linux, and WSL2.[82]
- The network proxy starts with an empty allowed domain list.[82]
- Managed Agents gives each cloud session its own isolated Linux container.[29]
- Managed Agents keeps vault secrets outside the sandbox.[29]
- A sandbox denyRead entry does not stop the Read tool, and allowedDomains does not limit WebFetch.[82]

2. Architecture



Schematic drawn from docs. Arrows: 1. shell command starts inside. 2. writes only to allowed paths. 3. no direct route out. 4. allowed domains only. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Run /sandbox in a Claude Code session to turn on the sandbox.
2. The sandbox limits writes to the current directory, a per-user temp directory, and added directories.
3. The sandbox sends outbound requests through a proxy that checks each host.
4. The proxy allows only the hosts on the allowed domain list.
5. Read access covers most of the machine by default, so add deny read rules for credential files.
6. File tools, MCP servers, and hooks run outside the sandbox, so they need their own rules.

4. Pros and cons

Pros

- Limits what a shell command can reach, even after an injection.
- Blocks direct network routes by default.
- Needs no change to the model prompt.

Cons

- Covers shell commands only, so file and web tools need permission rules.
- Read access covers most of the machine by default.
- Native Windows runs commands without the sandbox. Use WSL2 instead.

5. Use when, do not use when

Use when

- The agent runs shell commands on a developer machine or in a container.
- Untrusted input can lead to a shell command.
- The task needs only a few known hosts.

Do not use when

- The agent never runs shell commands.
- The host is native Windows without WSL2. Run the agent in a container or VM instead.

6. Constraint families

Data location

Keep the allowed domain list empty until a task needs a host. Combination when one choice is not enough. Add a host only after a review of its data handling. (Inference)

Isolation

Limit writes to the current directory and the directories you add. Combination when one choice is not enough. Add deny write rules for shared configuration files.

Governance

Use strict sandbox mode for tasks that touch untrusted input. Combination when one choice is not enough. Set failIfUnavailable so that commands do not run unsandboxed when the sandbox cannot start.

Operations

Review each request to retry a command outside the sandbox.

7. Native cloud services

All

Not in source. The sandbox pages name no cloud platform setting for this boundary.

8. Exam domain and task statement

D5.1. Expect a scenario where a shell command could reach credentials or an unknown host. The credited answer names a sandbox with an allowed domain list. (Inference)

If the scenario asks about the Read tool, the sandbox does not apply to that tool.

Use the sandbox for shell commands, and use permission rules for file and web tools.

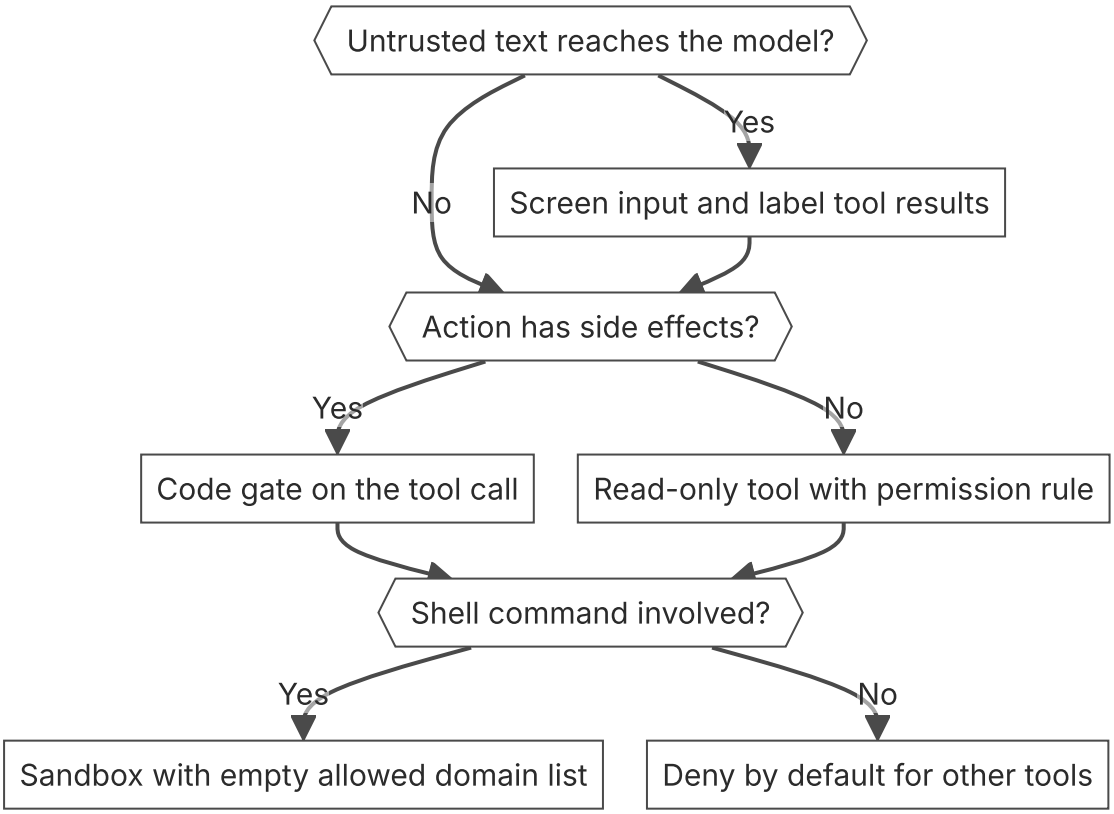
9. Trap

A team turns the sandbox on and believes it covers file tools or hooks. The sandbox wraps shell commands only.

10. Not in source

- Sandbox settings on Claude Platform on AWS or Amazon Bedrock. Not in source.
- Performance cost of the sandbox on each command. Not in source.

Decision flow



Unit items not in source

- Console steps for input checks and tool gates: Not in source.
- Accuracy rate of model refusals per category. Not in source.
- EU AI Act obligations for guardrails. Not in source.

D5.2 Risks and failure modes

Domain D5, Governance, safety, and risk management. Task statement: Map each risk to a prevention control, a detection signal, and a recovery step. (Inference: wording from secondary sources)

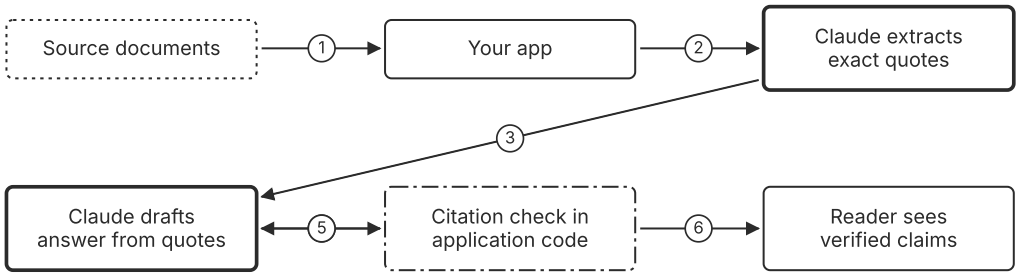
Hallucination control with grounded quotes

1. Introduction and first principle

A claim that cites a quote from the source is checkable, and a claim without a quote is removed.

- For long documents above 20k tokens, ask Claude for word-for-word quotes before the task.[78]
- Claude must retract a claim when it cannot find a source quote.[78]
- The guide says the techniques significantly reduce hallucinations but do not eliminate them.[78]
- Best-of-N verification runs the same prompt several times and compares the outputs.[78]

2. Architecture



Schematic drawn from docs. Arrows: 1. document text. 2. ask for quotes first. 3. answer only from quotes. 4. each claim with its quote. 5. retract unsupported claims. 6. verified claims only. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Allow Claude to say that it does not know when the source lacks the answer.
2. For long documents, ask Claude to extract exact quotes before it answers.
3. Limit the answer to facts that appear in the quotes or the supplied documents.
4. Ask Claude to attach a source quote to each claim.
5. Remove each claim that has no source quote from the final text.
6. Check the citations in application code before the reader sees the answer. (Inference)
7. Review high-stakes answers with a person, because these techniques reduce errors but do not catch every error.

4. Pros and cons

Pros

- Makes each claim checkable against a quote in the source.
- Lowers the rate of false statements in long documents.
- Works through prompt design, without a separate grounding service.

Cons

- Quote extraction adds tokens and latency to each call.
- The techniques reduce hallucinations but do not eliminate them.
- A quote can support a wrong claim when the source document is wrong. (Inference)

5. Use when, do not use when

Use when

- Answers must cite a source document.
- The input is a long document, such as a policy or a report.
- A wrong claim has a cost for the user or the business.

Do not use when

- The task is open fiction with no source requirement.
- The answer needs live data that no supplied document or tool provides.

6. Constraint families

Latency

Ask for quotes first and the answer second, in one prompt. Combination when one choice is not enough. Split the work into two calls when the document is very long. (Inference)

Cost

Quote only the passages that matter, not the full document.

Quota

Use best-of-N comparison only for high-stakes answers. Combination when one choice is not enough. Compare outputs for inconsistencies that point to a hallucination.

Skill

Write the instruction so that no quote means no claim.

Operations

Log each quote and claim pair for each answer. Combination when one choice is not enough. Alert on claims that arrive without a quote. (Inference)

7. Native cloud services

All

Not in source. The hallucination page names no cloud platform for this control.

8. Exam domain and task statement

D5.2. Expect a scenario where a fluent answer cites nothing. The credited answer requires grounded quotes and removes unsupported claims. (Inference)

If the scenario asks for a control on unsupported claims, choose quotes and citations over a model change.

Require a source quote for each claim, and remove each claim that has none.

9. Trap

A larger model looks like the fix for unsupported claims. The docs describe quotes, permission to abstain, and citations for the same risk.

10. Not in source

- Hallucination rate for any Claude model. Not in source.
- Token cost of quote extraction. Not in source.

Cost exhaustion limits

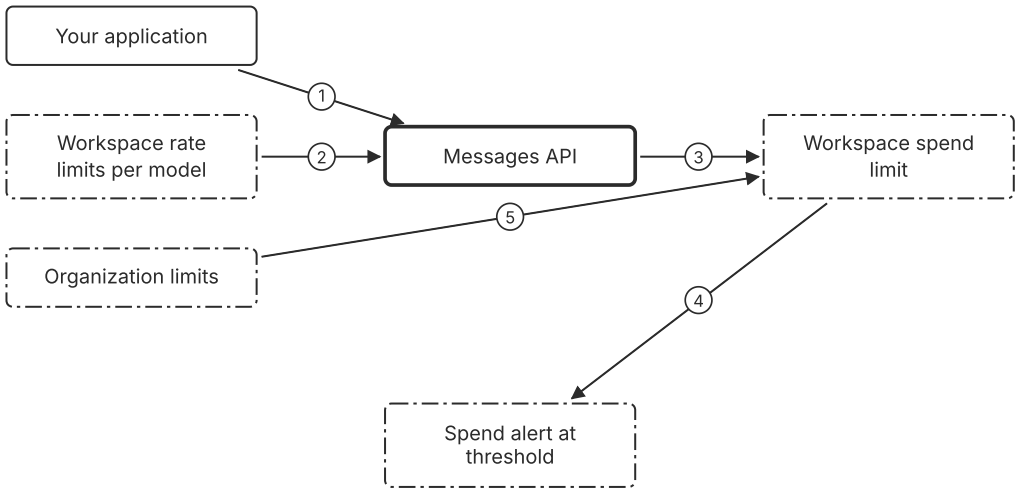
1. Introduction and first principle

A spend limit on a workspace caps the monthly loss, and a call cap in code stops one runaway run.

- A workspace spend limit caps monthly spending and can send alerts at thresholds.[11]
- You cannot set spend limits on the Default Workspace.[11]
- Organization-wide limits always apply, even when workspace limits add up to more.[11]

- The Claude Code workspace is the only workspace that supports per-user monthly spend limits.[11]
- The Amazon Bedrock default quota is 2 million input tokens per minute.[5]
- Customers can request up to 5 million input tokens per minute without additional Anthropic approval on Amazon Bedrock.[5]
- When usage reaches a workspace spend limit, requests return HTTP 400 with invalid_request_error until the next month or a change to the limit.[10]
- On Claude Platform on AWS, spend can take about 2 hours to reflect recent usage, and the overshoot is billed.[6]

2. Architecture



Schematic drawn from docs. Arrows: 1. requests with workspace key. 2. limits requests and tokens. 3. counts monthly spend. 4. threshold reached. 5. always applies. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Create a workspace for each project, team, or environment that calls the API.
2. Set a monthly spend limit on each workspace in the Console.
3. Configure spend alerts at the thresholds that you choose.
4. Set rate limits for each model in the rate limit settings of each workspace.
5. Expect the organization limits to apply, even when workspace limits add up to more.
6. Cap the number of tool calls in each run in application code. (Inference)

4. Pros and cons

Pros

- Caps monthly spend per workspace without a code change.
- Separates usage by project, team, or environment.
- Sends an alert at each threshold that you set.

Cons

- You cannot set a spend limit on the Default Workspace.
- Spend can take about 2 hours to show recent usage, so usage can pass a limit before requests fail.
- A spend limit does not stop a single run that loops in minutes. (Inference)

5. Use when, do not use when

Use when

- Several teams share one organization and need separate budgets.
- A new workflow could loop or flood the API.

- Finance needs a monthly ceiling for each project.

Do not use when

- The only key runs in the Default Workspace, because you cannot set a spend limit on that workspace.
- The control must stop one run within minutes. Use a call cap in code instead. (Inference)

6. Constraint families

Cost

Set a monthly spend limit for each workspace. Combination when one choice is not enough. Add alerts at thresholds below the limit.

Quota

Set workspace rate limits for each model, for requests and tokens. Combination when one choice is not enough. Expect the organization limit to apply even when workspace limits add up to more.

Isolation

Use one workspace for each team so that budgets stay separate.

Operations

Export usage by API key and model when spend rises. Combination when one choice is not enough. Retire the key that drives the increase. (Inference)

7. Native cloud services

Claude Platform on AWS

Rate limits are managed by Anthropic. Set organization and workspace spend limits in the Claude Console. Spend can take about 2 hours to show.[6]

Amazon Bedrock

Rate limits and quotas are managed by AWS. The default quota is 2 million input tokens per minute.[5]

8. Exam domain and task statement

D5.2. Expect a scenario with a runaway loop or a budget breach. The credited answer sets a workspace spend limit with alerts, and it caps calls in code. (Inference)

If the scenario names the Default Workspace, remember that you cannot set a spend limit on it.

Set a monthly spend limit on each workspace, and cap the calls per run in code.

9. Trap

A higher rate limit looks like a fix for a cost problem. It allows more calls, so it raises spend.

10. Not in source

- Alert threshold values that the Console offers by default. Not in source.

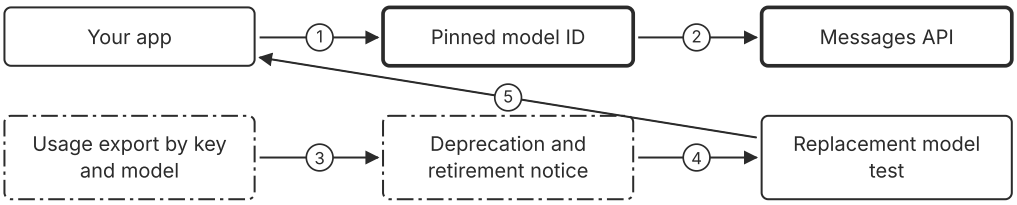
Fixed model IDs and retirement

1. Introduction and first principle

A pinned model ID keeps behavior fixed, and a tested migration plan handles the retirement date.

- Deprecated models remain functional but are no longer recommended.[83]
- Anthropic gives at least 60 days notice before retirement for publicly released models.[83]
- Requests to retired models fail.[83]
- A dateless model ID from the 4.6 generation is a fixed snapshot.[79]
- Aliases for earlier models point to the most recent dated snapshot for that minor version.[79]
- The infrastructure that serves a fixed model ID can change and cause minor differences in observable behavior.[79]

2. Architecture



Schematic drawn from docs. Arrows: 1. request names model ID. 2. fixed snapshot. 3. find deprecated use. 4. start replacement test. 5. move to active model ID. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Name the model ID in each request, not an alias.
2. A dateless model ID from the 4.6 generation maps to one fixed snapshot.
3. Anthropic does not update the weights or configuration of a model ID.
4. Export usage by API key and model to find deprecated model use.
5. Test a replacement model well before the retirement date.
6. Move traffic to an active model ID before retirement, because requests to retired models fail.

4. Pros and cons

Pros

- Keeps behavior stable for a given model ID.
- Gives a notice period before retirement.
- Lets you find deprecated use in the usage export.

Cons

- A pinned ID needs a planned move when the model retires.
- Deprecated models are likely to be less reliable than active models.
- A replacement model can change results, so it needs its own tests.

5. Use when, do not use when

Use when

- Output must stay stable for an audited workflow.
- A release needs one fixed model for a validation period.
- The team must show which model handled each request.

Do not use when

- A prototype that accepts model changes without review.
- A workflow that relies on an alias for an earlier model. Check its snapshot date first.

6. Constraint families

Data location

Check the retirement schedule of the platform you use. Combination when one choice is not enough. Expect partner-operated platforms to set their own schedules.

Governance

Record the model ID in each release record. Combination when one choice is not enough. Require a replacement test before each model change.

Skill

Test the replacement with the same evaluation set as the current model.

Operations

Export usage by API key and model each month. Combination when one choice is not enough. Alert when a deprecated model still receives requests. (Inference)

7. Native cloud services

Claude Platform on AWS

The Anthropic-operated retirement dates apply to this platform.[83]

Microsoft Foundry

The Anthropic-operated retirement dates apply to this platform.[83]

Claude Models on Google Cloud (Anthropic managed)

The Anthropic-operated retirement dates apply to this platform.[83]

Amazon Bedrock

Amazon Bedrock sets its own retirement schedule, so dates can differ.[83]

Claude Models on Google Cloud (Google managed)

Google managed sets its own retirement schedule, so dates can differ.[83]

8. Exam domain and task statement

D5.2. Expect a scenario about silent output change or a model near retirement. The credited answer pins a model ID and plans a tested migration. (Inference)

If the scenario asks how to avoid silent change, choose the pinned ID and a replacement test.

Pin a model ID for each release, and move to a tested replacement before retirement.

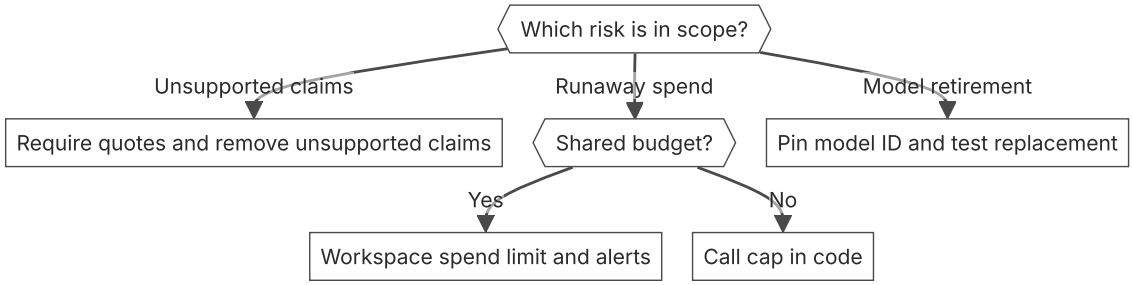
9. Trap

An earlier alias is wrongly taken as a fixed version. A 4.6-generation ID is a snapshot, but an earlier alias points to the latest snapshot for its minor version.

10. Not in source

- Retirement dates for Amazon Bedrock and Google managed models. Not in source.
- Supply-chain controls for third-party packages in the application. Not in source.

Decision flow



Unit items not in source

- Console steps for quote checks and model ID choice: Not in source.
- EU AI Act duties for model risk management. Not in source.

D5.3 Human-in-the-loop validation

Domain D5, Governance, safety, and risk management. Task statement: Place human approval and review where the stakes require it, with full decision context. (Inference: wording from secondary sources)

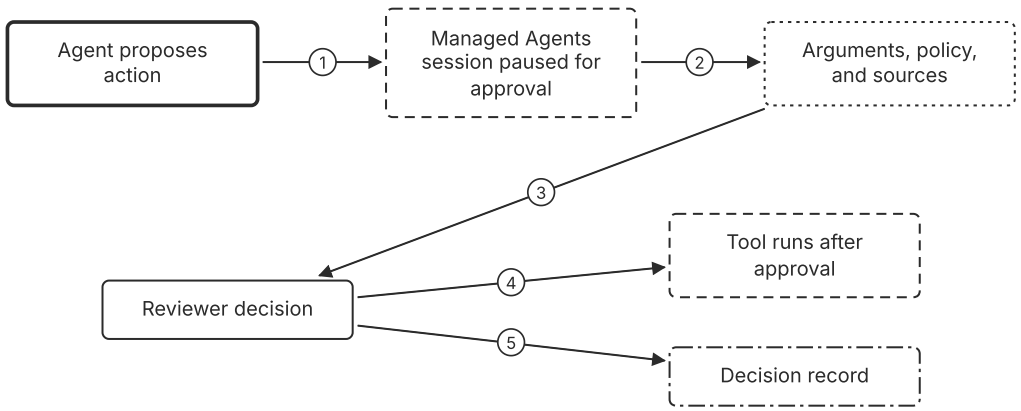
Pre-action approval for irreversible actions

1. Introduction and first principle

A person approves an action before it runs when the action cannot be undone.

- The always_ask policy pauses the session until you answer the approval request.[37]
- The agent toolset defaults to always_allow, and MCP toolsets default to always_ask.[37]
- Manual mode asks before most actions that edit files, run shell commands, or reach the network.[84]
- A session whose stop_reason type is requires_action needs your answer.[85]
- Under the auto policy, a call that the server denies cannot be overridden by a user.tool_confirmation event.[37]

2. Architecture



Schematic drawn from docs. Arrows: 1. always_ask policy. 2. requires_action stop reason. 3. full decision context. 4. approve. 5. record decision. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Set the permission policy to always_ask for each tool that moves money or changes data.
2. The session pauses with stop_reason requires_action and waits for a user.tool_confirmation event.
3. Show the reviewer the tool arguments, the policy, and the sources. (Inference)
4. The reviewer sends allow or deny for each blocked tool call.
5. The tool runs only after an approval.
6. Record the decision, the reviewer, and the arguments. (Inference)

4. Pros and cons

Pros

- Stops an irreversible action before it runs.
- Uses a native pause in Managed Agents, so the session waits for the answer.
- Gives an audit point for each decision.

Cons

- Reviewer latency slows each action that waits for an answer.
- A tired reviewer can approve a call without a real look at the context. (Inference)
- A review without context is not a real check. (Inference)

5. Use when, do not use when

Use when

- An action moves money, deletes data, or changes access.
- The action cannot be reversed.
- A policy names a person as the approver. (Inference)

Do not use when

- The action is read-only and public.
- Volume is high and each action is reversible. Use sampled review instead. (Inference)

6. Constraint families

Latency

Accept the pause for irreversible actions only. Combination when one choice is not enough. Use post-action review for reversible actions.

Governance

Require approval for each action that cannot be undone. Combination when one choice is not enough. Name the approver role in the policy record. (Inference)

Scale

Set always_ask only on the tools that need approval. Combination when one choice is not enough. Set always_allow on read-only tools.

Operations

Subscribe to the session.status_idled webhook. Combination when one choice is not enough. List idle sessions on a schedule, because a webhook delivery can be missed.

7. Native cloud services

All

Not in source. The permission policy pages name no cloud platform for this control.

8. Exam domain and task statement

D5.3. Expect a scenario where an action is irreversible and costly. The credited answer places approval before the tool runs. (Inference)

If the scenario gives a volume and a cost per review, compare the review load before you choose a shape.

Use pre-action approval when the action cannot be undone.

9. Trap

Approval on every action, even at high volume, which stalls the queue. The stakes decide the review shape.

10. Not in source

- Reviewer headcount or shift length. Not in source.
- Typical latency of one approval round trip. Not in source.

Post-action review and monitoring

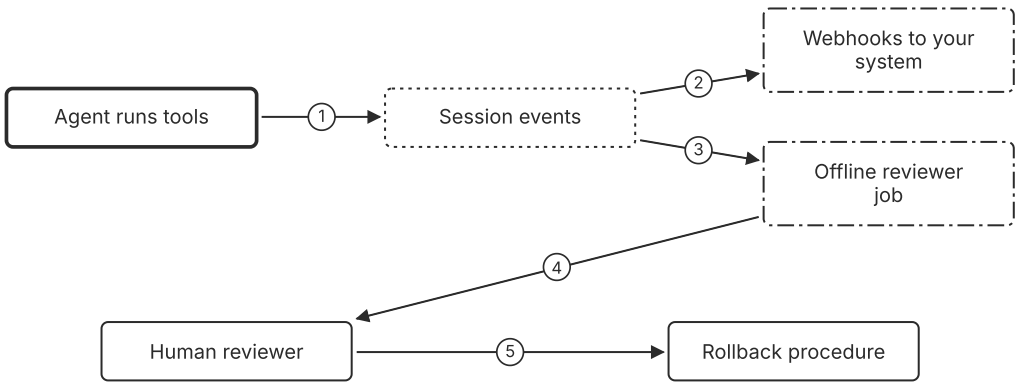
1. Introduction and first principle

A reviewer checks what ran after the fact, and a tested rollback path undoes a reversible action.

- The session.status_idled webhook fires each time a session goes idle.[85]
- The session.budget_reached and session.status_terminated webhooks report budget and termination events.[85]
- A webhook delivery can be missed, so also list sessions on a schedule and check the idle ones.[85]

- Each agent.tool_use event carries evaluated_permission, which shows whether the server allowed, denied, or paused a call.[37]
- Anthropic recommends a rolling retention of at least 30 days for Amazon Bedrock activity logs.[5]

2. Architecture



Schematic drawn from docs. Arrows: 1. events written. 2. session.status_idled. 3. read each transcript. 4. flag behavior. 5. reverse reversible action. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Subscribe to webhooks for sessions that go idle or reach a budget.
2. Verify the signature on each webhook delivery.
3. List idle sessions on a schedule, because a delivery can be missed.
4. Run an offline reviewer job that reads each session's events.
5. Send each flagged action to a human reviewer.
6. Undo the action with a rollback procedure that you tested before the run. (Inference)

4. Pros and cons

Pros

- Covers reversible actions at a low review cost.
- Finds behavior beyond the granted tools with an offline job.
- Reports idle and budget events through webhooks.

Cons

- Post-action review does not stop the first action.
- A rollback that does not work turns the review into a report. (Inference)
- Webhook deliveries can be missed.

5. Use when, do not use when

Use when

- The action is reversible and the rollback is tested.
- Sampling finds problems across many sessions.
- An audit needs a record of what the agent did.

Do not use when

- The action moves money or deletes data with no undo.
- One error would cause harm before a sample reaches a reviewer.

6. Constraint families

Isolation

Give the reviewer job read access to events only.

Governance

Keep session events for the period that your audit requires. Combination when one choice is not enough. Copy session events to your own logs before you delete a session.

Scale

Sample sessions at a random rate for the offline review. Combination when one choice is not enough. Raise the sample rate after errors appear. (Inference)

Operations

Watch session.budget_reached and session.status_terminated webhooks. Combination when one choice is not enough. List sessions on a schedule to catch missed webhooks.

7. Native cloud services

Amazon Bedrock

Claude in Amazon Bedrock emits logs to CloudWatch and CloudTrail. Anthropic recommends a 30-day rolling retention of activity logs.[5]

Claude Platform on AWS

The Admin API offers telemetry destination endpoints on this platform.[86]

8. Exam domain and task statement

D5.3. Expect a scenario with many reversible actions. The credited answer samples or reviews after the action, with a rollback path. (Inference)

If the scenario says the action cannot be undone, choose pre-action approval.

Review reversible actions after they run, and keep a tested undo path.

9. Trap

Post-action review presented as protection for an irreversible action. The review runs after the harm.

10. Not in source

- Sample rate for the offline reviewer. Not in source.
- Rollback procedure for any specific tool. Not in source.

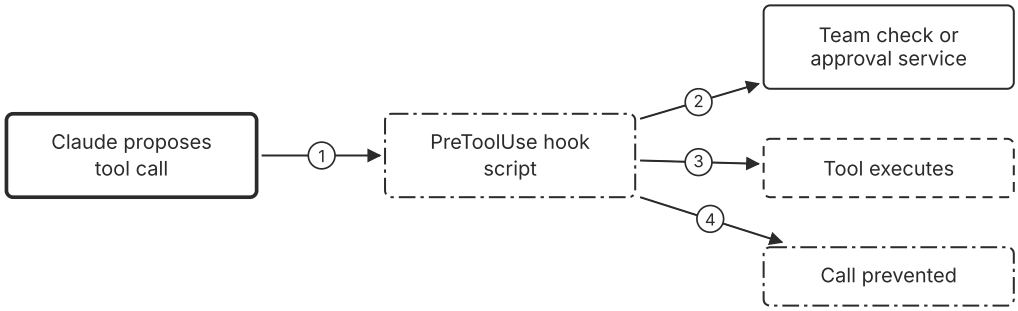
Hook-based approval point in Claude Code

1. Introduction and first principle

A PreToolUse hook runs code before a tool call, so a team can add a custom check that the rules cannot express.

- PreToolUse runs before a tool call and can block it.[81]
- A PreToolUse hook that exits with code 2 prevents the tool call.[81]
- A permissionDecision of allow does not let an action through after an exit code 2 block.[81]
- Exit code 0 with no JSON decision means the normal permission flow applies.[81]
- The default timeout for a command hook is 600 seconds.[81]
- Hooks from settings files also run inside subagents.[81]
- On PreToolUse, a hook that times out does not block the tool call unless onFailure is set to block.[81]
- User, project, and local settings add hooks but cannot remove managed hooks.[81]

2. Architecture



Schematic drawn from docs. Arrows: 1. tool input as JSON on stdin. 2. custom check. 3. exit 0, no decision. 4. exit 2 blocks. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Claude Code sends the tool input as JSON on stdin to the hook.
2. The hook checks the command or the arguments against your policy.
3. The hook exits with code 2, or returns permissionDecision deny in JSON, to prevent the tool call.
4. The hook exits with code 0 when it has no decision, so the normal permission flow applies.
5. An exit code 2 block holds, even when the JSON output says allow.
6. Hook scripts run outside the sandbox, so restrict their own permissions. (Inference)

4. Pros and cons

Pros

- Adds a custom check that a permission rule cannot express.
- Stops a call with exit code 2 before the tool runs.
- Matches only the tools that a matcher selects.

Cons

- Each hook adds process time to each call that its matcher selects. (Inference)
- Hook scripts run outside the sandbox and need their own review.
- A script bug can block legitimate work. (Inference)

5. Use when, do not use when

Use when

- A team needs a custom check that a rule cannot express.
- The check must run on each tool call that a matcher selects.
- A blocked call must stop before the tool runs.

Do not use when

- A plain allow rule or deny rule covers the case.
- A person must decide in real time. Use pre-action approval in the session instead. (Inference)

6. Constraint families

Isolation

Give the hook process only the credentials that its check needs.

Governance

Set hooks in managed policy settings, because user and project settings cannot remove them. Combination when one choice is not enough. Set allowManagedHooksOnly to block user, project, and plugin hooks.

Scale

Use a matcher so the hook runs only on the tools that need the check. Combination when one choice is not enough. Set a timeout on each command hook.

Operations

Log each hook decision with the tool name and the arguments. Combination when one choice is not enough. Alert when blocked calls increase. (Inference)

7. Native cloud services

All

Not in source. The hooks reference names no cloud platform for this control.

8. Exam domain and task statement

D5.3. Expect a scenario that needs a custom approval step. The credited answer uses a PreToolUse hook that exits with code 2. (Inference)

If a hook prints a plain warning and exits 0, expect the call to continue.

Block in a PreToolUse hook with exit code 2, or with permissionDecision deny, when a custom check fails.

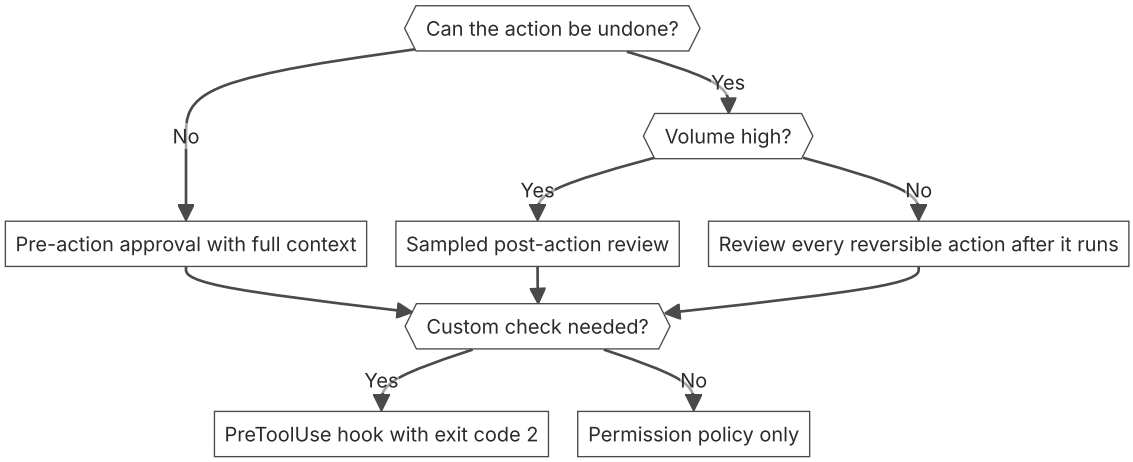
9. Trap

A hook that prints a plain warning and exits 0 blocks nothing. To block, exit with code 2 or return permissionDecision deny in JSON.

10. Not in source

- Approval service integration pattern for a custom hook. Not in source.
- Hook process cost per call. Not in source.

Decision flow



Unit items not in source

- Console steps for approval queues and offline review: Not in source.
- EU AI Act human oversight duties. Not in source.

D5.4 Regulatory compliance and data controls

Domain D5, Governance, safety, and risk management. Task statement: Map each compliance requirement to a retention, residency, key, or audit control with an owner and evidence. (Inference: wording from secondary sources)

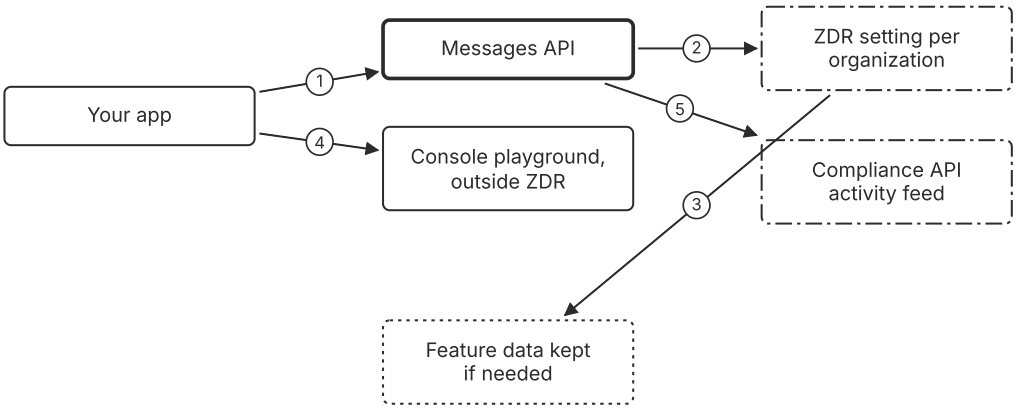
Retention limits and zero data retention

1. Introduction and first principle

The retention arrangement decides how long Anthropic keeps prompts and responses, so it is a design input and not a policy note.

- Under ZDR, Anthropic does not store prompts or responses at rest after the response returns.[22]
- ZDR is enabled per organization, and each new organization needs its own enablement.[22]
- ZDR does not cover any Claude Console usage. The playground is one example.[22]
- Covered Models require 30-day data retention and are not available under ZDR unless Anthropic authorizes them.[22]
- The Compliance API does not capture local sessions for which ZDR is in effect.[22]
- Chat on claude.ai and Cowork sessions are outside Claude Code ZDR.[87]
- Under ZDR, the API does not block features that are not ZDR eligible. Their own retention policy then applies.[22]
- HIPAA readiness is not available on Claude Platform on AWS or Microsoft Foundry.[22]

2. Architecture



Schematic drawn from docs. Arrows: 1. request with ZDR-enabled key. 2. checked per organization. 3. no storage at rest after response. 4. not covered. 5. activity events recorded. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Enable ZDR for each organization that handles the regulated data.
2. Send Messages API calls only with keys from a ZDR-enabled organization.
3. Check the feature eligibility table before you use a feature with ZDR.
4. Keep regulated data out of the Console playground, because ZDR does not cover it.
5. Set the retention period for claude.ai chats in the organization settings.
6. Use HIPAA readiness for protected health information, but not on Claude Platform on AWS or Microsoft Foundry.

4. Pros and cons

Pros

- Under ZDR, Anthropic does not store prompts or responses at rest after the response returns.

- Uses one setting per organization, which is simple to audit.
- Lists the features that ZDR does not cover.

Cons

- ZDR does not cover the Console, Managed Agents session history, or consumer products.
- Under ZDR, the API does not block features that are not eligible, so one call can step outside ZDR.
- Covered Models require 30-day retention and are not available under ZDR unless Anthropic authorizes them.

5. Use when, do not use when

Use when

- Prompts contain regulated or confidential data that must not persist.
- The workload uses only features that the eligibility table covers.
- A contract names ZDR as the arrangement for the data.

Do not use when

- The workload needs Managed Agents session history or the Console playground.
- The workload needs a feature that the eligibility table does not cover. Choose another architecture.

6. Constraint families

Data location

Pick a retention period that matches the contract or regulation that applies. Combination when one choice is not enough. Use the shortest period that the feature allows. (Inference)

Isolation

Keep ZDR traffic on keys from the ZDR organization only. Combination when one choice is not enough. Deploy the forceLoginMethod and forceLoginOrgUUID managed settings to require the ZDR organization for Claude Code.

Governance

Enable ZDR per organization and confirm the enablement with the account team. Combination when one choice is not enough. Check the feature eligibility table before you enable a new feature.

Operations

Expect the Compliance API to skip local sessions under ZDR. Combination when one choice is not enough. Record the ZDR decision in the audit file. (Inference)

7. Native cloud services

Claude Platform on AWS

Follows the first-party API retention policy. ZDR is available on request. HIPAA readiness is not available.[6]

Claude Models on Google Cloud (Anthropic managed)

Follows the first-party API retention policy. ZDR is available on request.[8]

Amazon Bedrock

The cloud provider is the data processor, and Amazon Bedrock data protection governs retention.[22]

Claude Models on Google Cloud (Google managed)

The cloud provider is the data processor, and Google Cloud governs data handling.[7]

Microsoft Foundry

Anthropic is the data processor, and the Anthropic retention page applies. HIPAA readiness is not available.[22]

8. Exam domain and task statement

D5.4. Expect a scenario where data must not persist after the response. The credited answer enables ZDR for the right organization and checks feature coverage. (Inference)

If the scenario names protected health information, check HIPAA readiness before you choose ZDR.

Enable ZDR per organization, and confirm feature coverage before regulated data reaches the API.

9. Trap

A team believes ZDR covers every product in the account. The Console, Managed Agents, and claude.ai chat sit outside ZDR.

10. Not in source

- Timeline from ZDR request to enablement. Not in source.
- ZDR enablement steps for Microsoft Foundry. Not in source.

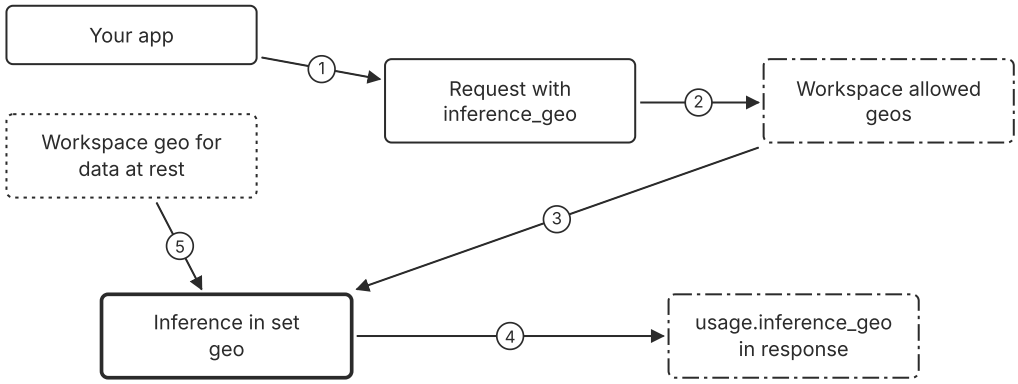
Data residency with inference geo and workspace geo

1. Introduction and first principle

The request sets where inference runs, and the workspace sets where stored data lives, so each setting controls a different location.

- The inference_geo parameter accepts global or us.[88]
- US-only inference is priced at 1.1 times the standard rate across all token categories.[88]
- Workspace geo is set at creation and cannot change afterward.[88]
- A request that names a geo outside allowed_inference_geos returns an error.[88]
- The inference_geo parameter returns a 400 error on Claude models before the 4.6 generation.[88]
- Rate limits are shared across all geos.[88]
- Workspace geo is not configurable on Claude Platform on AWS or on Claude Models on Google Cloud (Anthropic managed).[88]

2. Architecture



Schematic drawn from docs. Arrows: 1. build request. 2. checked against list. 3. allowed geo. 4. reports where inference ran. 5. fixed at workspace creation. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Set inference_geo to us on each request that must run in the United States.
2. Set default_inference_geo and allowed_inference_geos on the workspace for all of its keys.
3. Create a workspace with US workspace geo when stored data must stay in the United States.
4. Read usage.inference_geo in each response to confirm where inference ran.
5. Expect an error when a request names a geo that the workspace does not allow.

4. Pros and cons

Pros

- Sets residency on each request, so each call states its location.

- Confirms the location in each response.
- Blocks requests with a geo outside the workspace list.

Cons

- US-only inference costs more than global inference.
- Workspace geo cannot change after the workspace exists.
- Rate limits are shared across all geos.

5. Use when, do not use when

Use when

- A contract or regulation requires US-only inference.
- Stored data must stay in one geography.
- Several teams need one residency rule that the workspace enforces.

Do not use when

- The workload accepts global routing, so the price premium adds no value.
- The model is earlier than the 4.6 generation, because the inference_geo parameter returns an error there.

6. Constraint families

Cost

Budget for the US-only price multiplier on all token categories. Combination when one choice is not enough. Include cache writes and cache reads in the same budget.

Quota

Plan rate limits with the knowledge that they are shared across all geos. Combination when one choice is not enough. Set workspace rate limits for each model.

Data location

Set inference_geo to us on each request. Combination when one choice is not enough. Set allowed_inference_geos to us on the workspace.

Governance

Set the residency rule in workspace settings, so application code cannot skip it. Combination when one choice is not enough. Use the Admin API data_residency field for automation.

Operations

Log usage.inference_geo for each response.

7. Native cloud services

Claude Platform on AWS

Workspace geo is not configurable. The workspace AWS region does not pin inference, so set inference_geo on each request.[6]

Amazon Bedrock

The endpoint or inference profile sets the inference region. Regional endpoints carry a 10 percent premium.[5]

Claude Models on Google Cloud (Google managed)

Requests can go to global, US, EU, or one named region.[7]

Claude Models on Google Cloud (Anthropic managed)

Every workspace allows all inference geos and defaults to global. Set inference_geo to us on each request for US-only inference.[8]

Microsoft Foundry

The Data Zone region scope creates a US Data Zone Standard deployment that keeps inference in the United States.[9]

8. Exam domain and task statement

D5.4. Expect a scenario where inference must run in one country. The credited answer sets the geo on each request and restricts the workspace. (Inference)

If the response shows the global geo for a US-only need, the request lacked the setting.

Set inference_geo on each request, and set allowed_inference_geos on the workspace.

9. Trap

A region setting on the application server. The inference_geo setting controls where inference runs, not where the app runs.

10. Not in source

- Latency difference between US and global routing. Not in source.

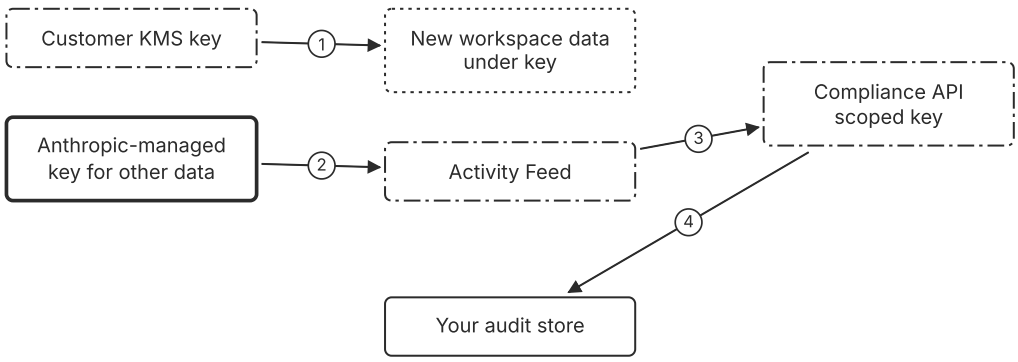
Customer-managed keys and audit evidence

1. Introduction and first principle

A customer key limits who can read new stored data, and the activity feed gives evidence of who accessed what.

- CMEK protects data written after the key takes effect.[23]
- CMEK cannot be undone once it is on, and Anthropic keeps no copy of the customer key.[23]
- CMEK is available in US regions only, except on Claude Platform on AWS.[23]
- The Activity Feed retains events for 6 years.[89]
- Activities are queryable within 1 minute of their creation.[89]
- A Compliance Access Key with read:compliance_user_data can read every chat, file, project, and session transcript in every linked organization.[90]

2. Architecture



Schematic drawn from docs. Arrows: 1. wraps data keys for new writes. 2. audit logs stay under Anthropic key. 3. read with read scope. 4. export events. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Provision a key in the KMS of the platform, such as AWS KMS, Google Cloud KMS, or Azure Key Vault.
2. Attach the key to a new workspace before the workspace receives requests, because a later key can take up to a day.
3. Expect the key to protect data written after it takes effect, not earlier data.
4. Grant key access only to the Anthropic identifier that the setup guide names.
5. Create a Compliance Access Key with only the scopes that each reader needs.
6. Export Activity Feed events to your audit store on a schedule. (Inference)

4. Pros and cons

Pros

- Keeps key rotation, audit, and revocation under customer control.
- Records key operations in the cloud provider audit logs.
- Keeps the Activity Feed under the Anthropic key, so audit records survive a key revocation.

Cons

- CMEK cannot be undone once it is on, and a key loss can cause permanent data loss.
- Console playground is disabled on Claude Platform, and chat search is disabled on Claude Enterprise.
- Earlier data is not re-encrypted under the customer key.

5. Use when, do not use when

Use when

- Policy requires customer-controlled encryption keys.
- The organization must be able to cut access to stored data through the key.
- Audit needs a record of who read which records.

Do not use when

- The team has no process to protect keys from loss.
- The feature set needs chat search or Claude Code on the web on Claude Enterprise.

6. Constraint families

Data location

Use a key in a US region for CMEK, except on Claude Platform on AWS. Combination when one choice is not enough. Use an AWS KMS key in the same account and region as the workspace on Claude Platform on AWS.

Isolation

Use separate Compliance Access Keys for reads and deletes.

Governance

Grant key access only to the verified Anthropic identifier. Combination when one choice is not enough. Use a key policy, and not an IP allow list, on Claude Platform on AWS.

Operations

Track CMEK configuration events in the Activity Feed. Combination when one choice is not enough. Track key wrap and unwrap events in the cloud provider audit logs.

7. Native cloud services

Claude Platform on AWS

CMEK uses AWS KMS keys only. The key policy must grant an AWS service principal.[91]

Claude Models on Google Cloud (Anthropic managed)

CMEK is available with Google Cloud KMS keys for the Anthropic organization that setup creates.[8]

Amazon Bedrock

Not in source. The Bedrock page names no CMEK setting for Claude.[5]

8. Exam domain and task statement

D5.4. Expect a scenario that asks for customer control over stored data and proof of access. The credited answer combines a CMEK plan, scoped keys, and the Activity Feed. (Inference)

If the scenario asks about data written before the key took effect, state that Anthropic keys still protect that data.

Use CMEK for control of new writes, and use the Compliance API for audit evidence.

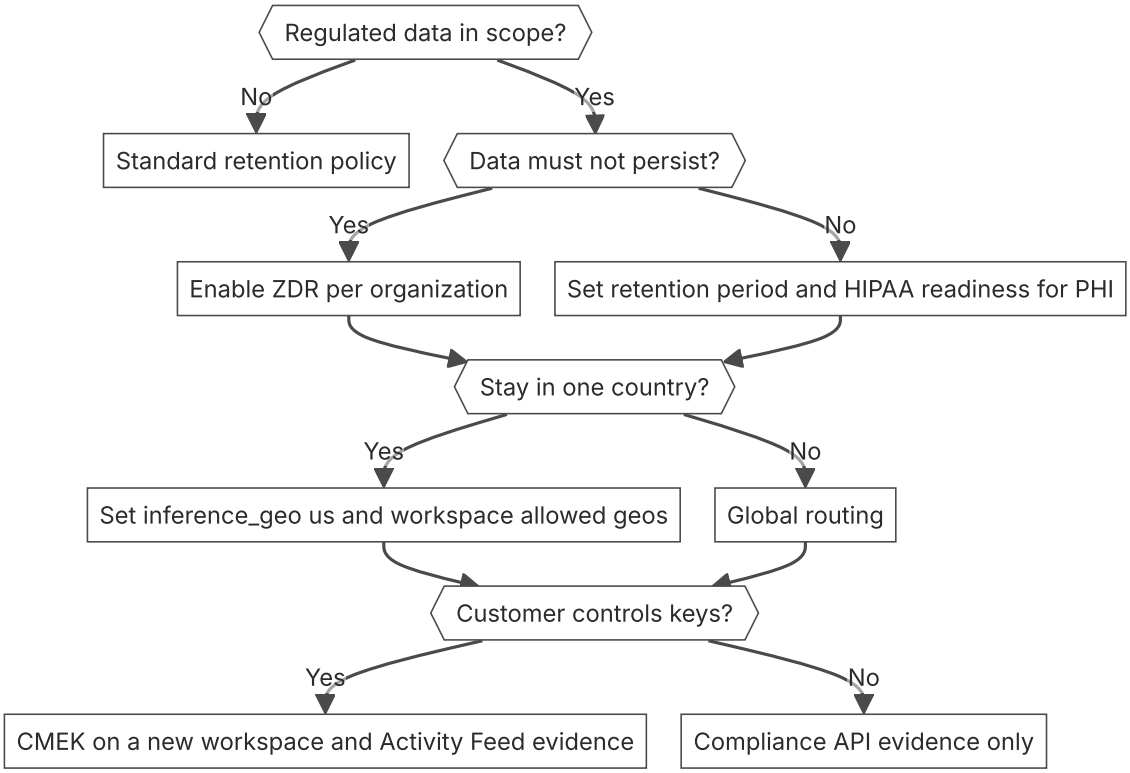
9. Trap

A team believes CMEK re-encrypts old data. It protects data written after the key takes effect.

10. Not in source

- Key rotation schedule that Anthropic applies to CMEK keys. Not in source.
- CMEK support for Amazon Bedrock. Not in source.

Decision flow



Unit items not in source

- Console steps for ZDR enablement by the account team: Not in source.
- EU AI Act and GDPR article-level duties for this workload. Not in source.
- FedRAMP control baselines for this workload. Not in source.

D5.5 Responsible AI

Domain D5, Governance, safety, and risk management. Task statement: Apply responsible AI duties to the system, with per-group evaluation and documented evidence. (Inference: repo lesson and general practice, not in the official mirror)

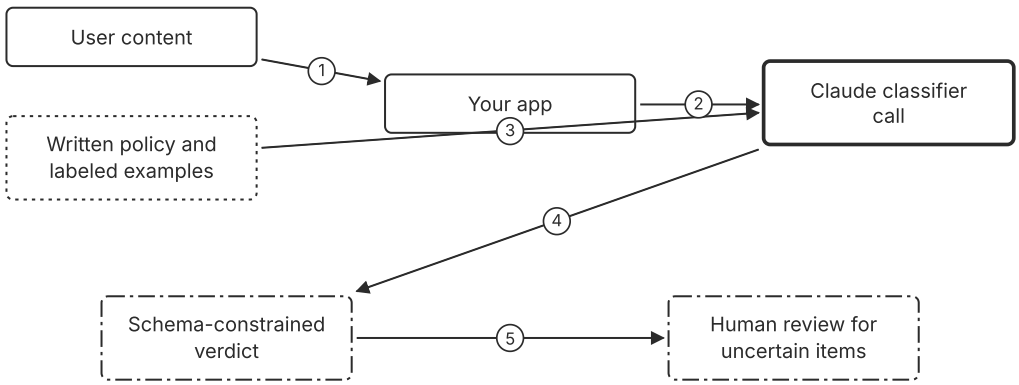
Content moderation with Claude

1. Introduction and first principle

Claude classifies user content against a written policy and labeled examples, so the policy text is the control.

- Claude can classify content in various languages without separate models.[92]
- Built-in safety behavior can lead Claude to moderate content that is dangerous under the Usage Policy, whatever the prompt says.[92]
- The guide says to create examples of content to flag and content to allow before you build.[92]
- The harmlessness screen example uses a JSON schema to constrain the response to a boolean.[46]
- The mitigation guide says to consider throttling users who repeatedly trigger refusals.[46]
- The moderation guide names Claude Haiku 5.5 as a cost-effective choice for large volumes.[92]

2. Architecture



Schematic drawn from docs. Arrows: 1. submitted text. 2. content with policy. 3. examples and categories. 4. JSON schema output. 5. uncertain or refused items. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Collect examples of content to flag and content to allow.
2. Include edge cases in the examples, because edge cases test the policy.
3. Write the moderation categories from the examples.
4. Send each item with the policy and the categories to Claude.
5. Constrain the response to a schema, so the verdict is a parseable value.
6. Route uncertain or refused items to a person. (Inference)

4. Pros and cons

Pros

- Handles semantic meaning and many languages without a separate model for each language.
- Policy changes need prompt changes, not relabeled training data.
- Gives a written reason that a reviewer can read.

Cons

- Built-in safety behavior can flag content that your own policy allows.
- A refusal can leave the verdict empty, so the flow needs a fallback path.

- Results depend on the policy text and the examples. (Inference)

5. Use when, do not use when

Use when

- User-generated text needs fast moderation decisions.
- The moderation policy changes often.
- One process must cover several languages.

Do not use when

- The rule is a fixed pattern, such as a blocked word list. A rule is cheaper. (Inference)
- A person must make every decision for legal reasons. (Inference)

6. Constraint families

Cost

Use a smaller model such as Claude Haiku 5.5 for moderation when cost matters. Combination when one choice is not enough. Use a larger model for appeals. (Inference)

Governance

Keep the policy text under version control with a review date. (Inference)

Skill

Write the policy with examples that include edge cases. Combination when one choice is not enough. Test the examples again before each policy change. (Inference)

Operations

Track flagged and refused items as a metric. Combination when one choice is not enough. Review repeat offenders before you throttle or ban an account.

7. Native cloud services

All

Not in source. The moderation guide names no cloud platform for this workflow.

8. Exam domain and task statement

D5.5. Expect a scenario about user content, a policy that changes often, and several languages. The credited answer uses a written policy with examples and a structured verdict. (Inference)

If the scenario says the policy changes often, choose a prompt-defined policy over a retrained model.

Put the moderation policy and examples in the prompt, and constrain the verdict with a schema.

9. Trap

A custom classifier looks cheaper, but each policy change then needs new labels and code. (Inference)

10. Not in source

- Moderation accuracy or false positive rate. Not in source.
- Accuracy by language for moderation. Not in source.

Safety classifiers and refusal handling

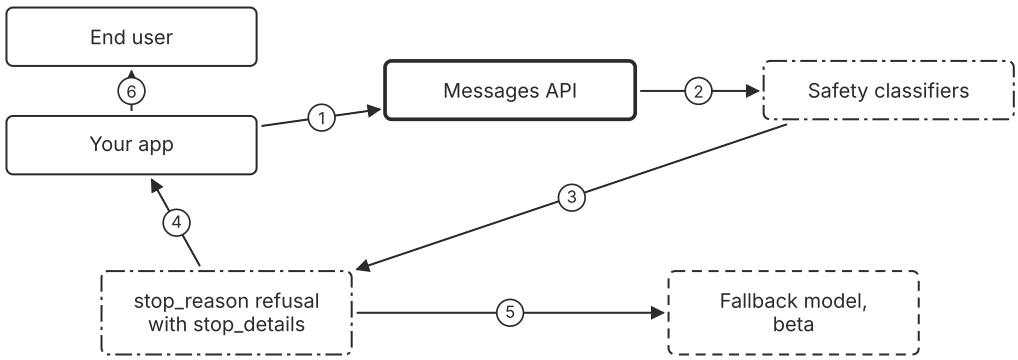
1. Introduction and first principle

A refusal is a normal response with a stop reason, so the application reads the stop reason and handles the refusal in code.

- A declined request returns a normal response with stop_reason refusal, not an error.[93]
- The stop_details.category field names the policy area that declined the request.[93]
- Streaming responses return stop_reason refusal when streaming classifiers intervene.[65]

- Setting fallbacks to default retries a declined request on a fallback model, in beta.[93]
- Claude Fable 5.1, Claude Fable 5, Claude Opus 5.5, Claude Opus 5, Claude Sonnet 5.5, and Claude Haiku 5.5 include these classifiers.[93]
- Claude Haiku 5.5 has neither server-side fallback nor fallback credit, so use the SDK middleware or a manual retry.[93]
- Server-side fallback is in beta on the Claude API and on Claude Models on Google Cloud (Anthropic managed).[93]

2. Architecture



Schematic drawn from docs. Arrows: 1. request. 2. classifiers check output. 3. decline with category. 4. app reads stop reason. 5. retry if fallbacks is default. 6. explain or rephrase. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Read stop_reason on every response, also on streamed responses.
2. Treat a refusal stop reason as a normal response, not as an error.
3. Read stop_details.category to learn which policy area declined the request.
4. Set fallbacks to default, in beta, to retry a declined request on a fallback model.
5. Tell the user that the request was declined, and do not resend it unchanged. (Inference)
6. Log each refusal with its category and request ID, so repeat cases can be reviewed.

4. Pros and cons

Pros

- Makes a declined request visible in application code.
- Lets the application choose a fallback path.
- Shows the policy category in logs and reviews.

Cons

- Server-side fallback is in beta, and Claude Haiku 5.5 does not support it.
- A fallback model can change answer quality. (Inference)
- Classifiers can decline a legitimate request. (Inference)

5. Use when, do not use when

Use when

- The product sends user text that may trip a classifier.
- The product needs a retry path without an error page.
- A team needs counts of declines by category.

Do not use when

- The application ignores stop reasons and treats every output as final.

- The retry must run on a model with a different data location. Check the platform terms first. (Inference)

6. Constraint families

Latency

Use the server-side fallback to avoid a second client round trip. Combination when one choice is not enough. Use the SDK middleware for client-side fallback on platforms without server-side fallback.

Governance

Throttle or ban a user only after repeated refusals of one kind. Combination when one choice is not enough. Keep a review step before any ban. (Inference)

Skill

Read the stop reason in every streaming handler.

Operations

Log stop_details.category with the request ID. Combination when one choice is not enough. Alert when one category rises sharply. (Inference)

7. Native cloud services

Claude Models on Google Cloud (Anthropic managed)

The server-side fallback beta is available on this platform, as on the Claude API.[93]

8. Exam domain and task statement

D5.5. Expect a scenario where a refused request breaks the flow. The credited answer reads the stop reason and handles the refusal in code. (Inference)

If a response has stop_reason refusal, do not parse it as an error.

Read stop_reason on every response, and handle refusal as a normal outcome.

9. Trap

A refusal treated as an HTTP error and retried unchanged. A refusal returns a normal response with a stop reason.

10. Not in source

- Refusal rate for any category. Not in source.
- Fallback model choice for each refusal category. Not in source.

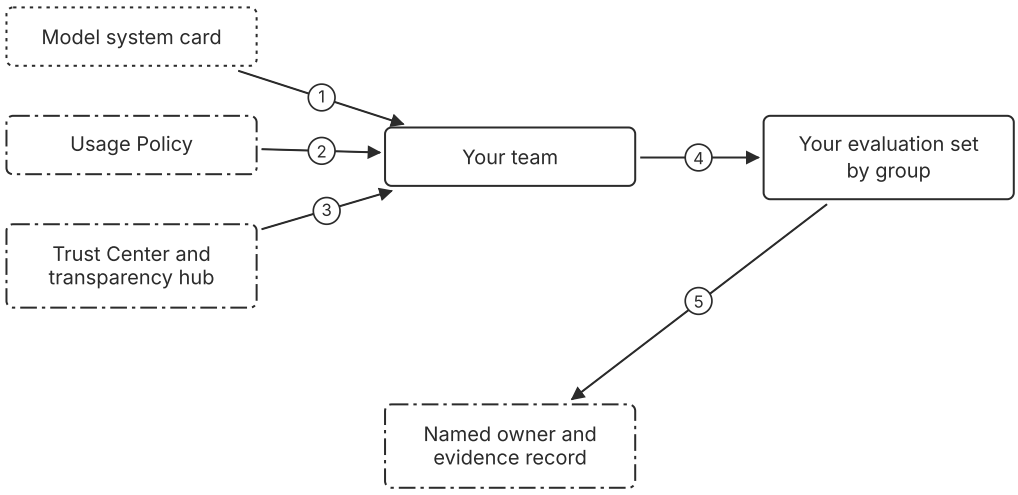
Model documentation, transparency, and per-group evaluation

1. Introduction and first principle

Published model documentation gives context, and the team adds its own per-group evidence for its own users.

- Anthropic publishes system cards with detailed documentation for Claude models.[94]
- The resources page links system cards for Claude Haiku 5.5, Claude Sonnet 5.5, and Claude Opus 5.5.[94]
- Claude Code usage is subject to the Anthropic Usage Policy.[95]
- Anthropic points customers to its Trust Center and Transparency Hub for trust and safety information.[95]
- Anthropic runs its security program through a HackerOne form for vulnerability reports.[95]

2. Architecture



Schematic drawn from docs. Arrows: 1. tested behaviors. 2. allowed uses. 3. security material. 4. build per-group set. 5. results by group. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Read the system card for the model that the product uses.
2. Read the Usage Policy before you build the product.
3. Check the Trust Center for security and trust material.
4. Evaluate the outputs by group, and do not use an aggregate alone. (Inference)
5. Set a floor for each group and hold the release until each group meets its floor. (Inference)
6. Name an owner for each decision class and record the evidence. (Inference)

4. Pros and cons

Pros

- Gives published test context for each model.
- Points to the Usage Policy that governs use.
- Separates vendor evidence from your own evidence.

Cons

- System cards do not test your user population. (Inference)
- An aggregate score can hide a weak group. (Inference)
- Evaluation by group needs group labels, which many products lack. (Inference)

5. Use when, do not use when

Use when

- A product team must document the model that it uses.
- A review asks for model evidence.
- The team plans a model change.

Do not use when

- The question is about your own product outputs. Use your own evaluation.
- A sandbox test that never reaches users.

6. Constraint families

Governance

Keep the system card link and the model ID in the release record. Combination when one choice is not enough. Require a new review when the model ID changes.

Scale

Use one evaluation set per user group that matters for the product. (Inference) Combination when one choice is not enough. Set a floor for each group. (Inference)

Skill

Have a reviewer read the Usage Policy before launch.

Operations

Check the model deprecation page on a schedule. Combination when one choice is not enough. Plan a tested move before the retirement date.

7. Native cloud services

All

Not in source. The resources page lists model cards without a platform-specific term.

8. Exam domain and task statement

D5.5. Expect a scenario that asks for evidence of responsible use. The credited answer cites the system card and the Usage Policy, then adds the team's own per-group evaluation. (Inference)

If the answer relies only on vendor documentation, add the team's own evaluation.

Cite the system card for the model, and add a per-group evaluation for the product.

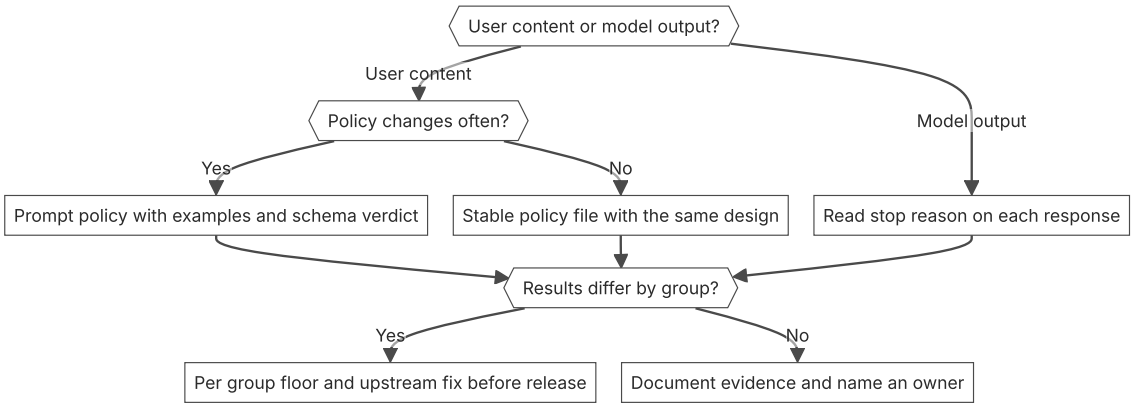
9. Trap

A team cites the model card as proof that the product is fair. The card describes model tests, not the product's users. (Inference)

10. Not in source

- Bias, fairness, or subgroup metrics for any Claude model. Not in source.
- Explainability methods for model decisions. Not in source.

Decision flow



Unit items not in source

- Console steps for moderation, refusals, and group evaluation: Not in source.
- EU AI Act duties for responsible AI practice. Not in source.
- Bias and fairness regulations that apply to this workload. Not in source.

D6.1 Structured discovery

Domain D6, Stakeholder communication and lifecycle management. Task statement: Turn stakeholder requests into measurable requirements, owners, and prohibited behaviors before design starts. (Inference: wording from secondary sources)

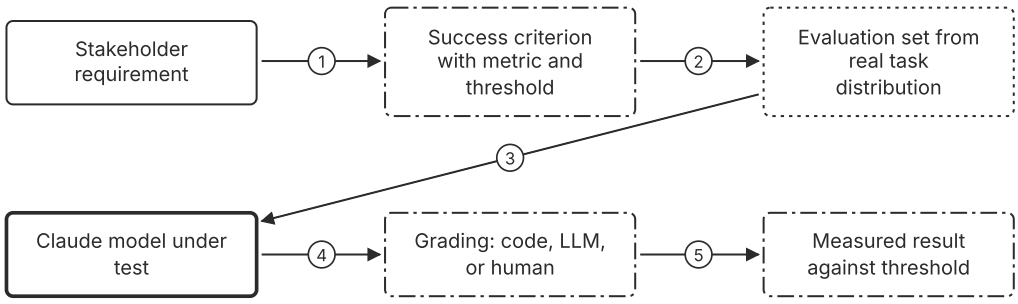
Success criteria and evaluation sets

1. Introduction and first principle

A requirement becomes testable only when it has a metric, a threshold, and a named scope.

- Good success criteria are specific, measurable, achievable, and relevant.[25]
- Most use cases need multidimensional evaluation along several success criteria.[25]
- Common success criteria include task fidelity, latency, and price.[25]
- Eval design favors test sets that mirror the real task distribution and include edge cases.[25]
- Code-based grades are the fastest and most reliable, and human grades are slow and expensive.[25]

2. Architecture



Schematic drawn from docs. Arrows: 1. adjective becomes measurable. 2. scope the test cases. 3. run each input. 4. outputs. 5. scores. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write each requirement as a specific and measurable criterion.
2. Attach a quantitative metric or a defined qualitative scale to each criterion.
3. Name the owner who accepts each threshold. (Inference)
4. Build a test set that mirrors the real task distribution.
5. Add edge cases to the test set.
6. Score each output with code, an LLM, or a human scorer.
7. Compare the measured result with the threshold before design starts. (Inference)

4. Pros and cons

Pros

- Turns vague goals into checks that a team can run.
- Gives every stakeholder the same pass or fail test. (Inference)
- Shows which requirement fails before release. (Inference)

Cons

- Thresholds need data that the team may not have yet. (Inference)
- Each quality dimension adds its own evaluation work. (Inference)
- Human scoring is slow and expensive, so the docs advise against it when possible.

5. Use when, do not use when

Use when

- A request uses adjectives such as accurate or fast.
- The team builds an LLM feature with a measurable quality bar.
- Several success criteria must hold at the same time.

Do not use when

- A throwaway prototype needs no pass or fail bar. (Inference)
- No representative inputs exist, so the test set cannot mirror the task. (Inference)

6. Constraint families

Latency

Set an acceptable response time as a criterion, based on real-time needs. Combination when one choice is not enough. Add a percentile target, such as 95 percent of responses under a limit, when the average hides slow requests.

Cost

Set a budget for each call or each month as a criterion. Combination when one choice is not enough. Compare options by cost per completed task when quality differs. (Inference)

Governance

Name a signed owner for each threshold. (Inference)

Operations

Record operational criteria such as response time and uptime.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D6.1. Expect a vague stakeholder request with no metric. The credited answer defines measurable criteria before any design. (Inference)

If the stem gives adjectives but no metric, the first action is discovery.

Convert each adjective into a metric, a threshold, and an owner before design.

9. Trap

Design built on adjectives such as fast looks efficient, but no one can test it, so the team builds to a guessed bar. (Inference)

10. Not in source

- Cloud platform steps for requirement discovery. The docs do not cover them. Not in source.
- Question grid for discovery interviews. General practice, not in the docs. Not in source.

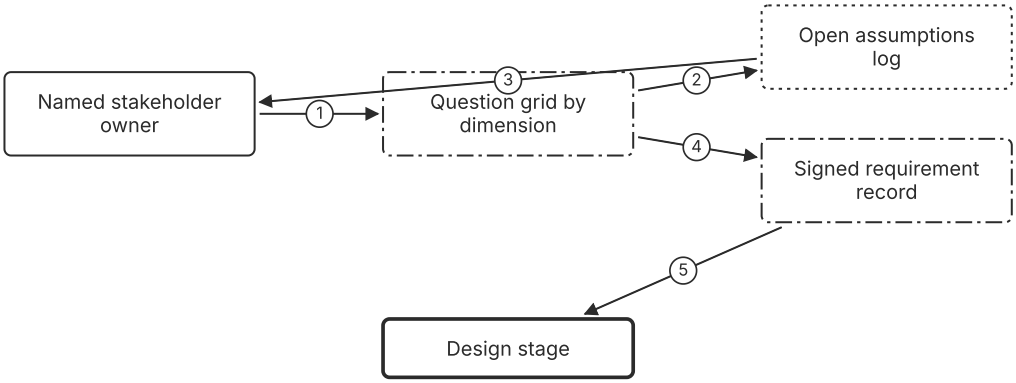
Discovery question grid

1. Introduction and first principle

Each question covers one dimension where a silent assumption can hide. (Inference)

- Workspace spend limits cap monthly spending for one workspace.[11]
- Spend limits cannot be set on the Default Workspace.[11]
- Each API key can be scoped to a single workspace.[11]

2. Architecture



Schematic drawn from docs. Arrows: 1. answers each dimension. 2. unanswered cells. 3. ask again. 4. all cells answered. 5. approved input. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. List the dimensions that can hide assumptions, such as prohibitions, cost, volume, and compliance. (Inference)
2. Ask the owner of each dimension one question. (Inference)
3. Record each answer with its date and the name of the person who gave it. (Inference)
4. Log each unconfirmed guess as an open assumption. (Inference)
5. Close every open assumption before the design gate. (Inference)

4. Pros and cons

Pros

- Surfaces prohibited behaviors before the build starts. (Inference)
- Exposes answers that disagree across owners early. (Inference)
- Costs little compared with rework after a demo. (Inference)

Cons

- Stakeholders may not know the corner cases either. (Inference)
- Answers go stale when volume or compliance changes. (Inference)

5. Use when, do not use when

Use when

- A request names outcomes but no limits, owners, or volumes. (Inference)
- The system touches money, personal data, or several teams. (Inference)

Do not use when

- A short internal demo uses no real data. (Inference)
- The requirement is one fixed setting with an obvious owner. (Inference)

6. Constraint families

Cost

Set a workspace spend limit that enforces the cost answer.

Governance

Name one owner who signs each requirement. (Inference) Combination when one choice is not enough. Add a review date to every signed answer. (Inference)

Scale

Record expected volume, with peak bursts. (Inference) Combination when one choice is not enough. Recheck the volume answer after a growth event. (Inference)

Operations

Name the on-call owner for each operational dependency. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D6.1. Expect a scenario with an unanswered requirement. The credited answer runs discovery before any design. (Inference)

A blank answer is an open risk, so name the owner who fills it.

Ask one question for each dimension, and do not design around a blank answer. (Inference)

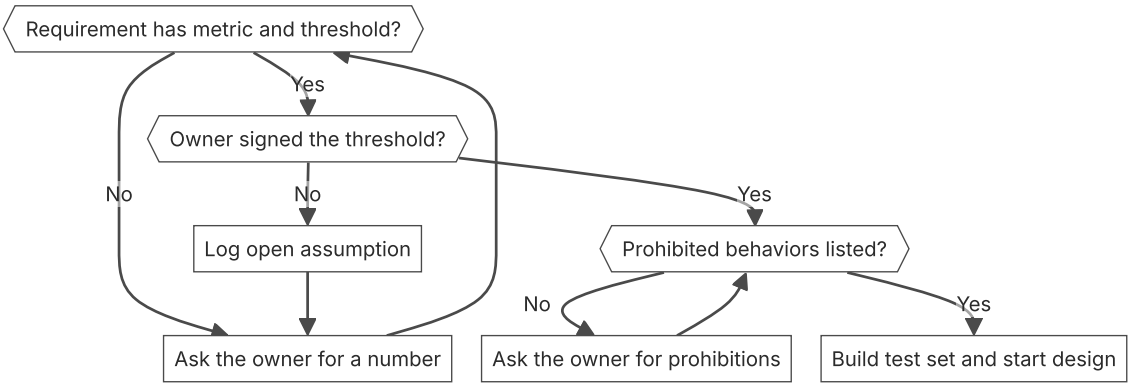
9. Trap

Feature work first looks productive, but an unanswered limit still breaks the first production demo. (Inference)

10. Not in source

- Dimension list and grid layout. General practice, not in the docs. Not in source.

Decision flow



Unit items not in source

- Console steps for structured discovery: Not in source.
- Discovery interview method and stakeholder roles. General practice, not in the docs. Not in source.

D6.2 Trade-off communication

Domain D6, Stakeholder communication and lifecycle management. Task statement: Compare options on priced, measured trade-offs and tell each audience the facts it decides on. (Inference: wording from secondary sources)

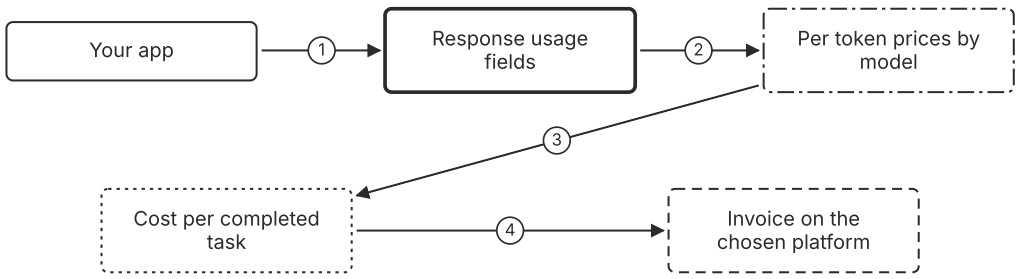
Model pricing and cost per task

1. Introduction and first principle

Compare options on cost per completed task, because the completed task is the unit that you pay for.

- Compare models on cost per completed task, not per token, because you pay for completed tasks.[77]
- Figures on the optimization page reflect list prices at the time of measurement, and they drift.[77]
- Five-minute cache writes cost 1.25 times the base input price, and one-hour cache writes cost 2 times.[77]
- Claude Haiku 5.5 has one price for prompts up to 100,000 tokens and another price for longer prompts.[3]
- Claude Opus 5.5 lists a base input price of 4 USD per million tokens and an output price of 20 USD per million tokens.[3]
- Regional endpoints on Amazon Bedrock carry a 10 percent pricing premium over global endpoints.[5]
- US-only inference through the inference_geo parameter adds a 1.1 times multiplier on all token prices for Claude 4.6 and later models.[3]

2. Architecture



Schematic drawn from docs. Arrows: 1. each request. 2. priced token counts. 3. sum across requests. 4. compare with invoice. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Read the token counts in the usage field of each response.
2. Price each token count at the rate for its token type.
3. Sum the prices across all requests for one completed task.
4. Compare the cost per completed task across the options.
5. Date the price table before each decision. (Inference)

4. Pros and cons

Pros

- Shows the cost of a finished task, instead of a token rate alone.
- Makes a cheap per-token option comparable with a costlier one.
- Matches the usage fields that each response already returns.

Cons

- Prices change, so a comparison goes stale.
- A cost per task needs task-level logs or evaluation results. (Inference)

5. Use when, do not use when

Use when

- Two models or platforms meet the quality bar at different prices.
- A budget owner must approve a monthly spend.

Do not use when

- Only one model meets the quality bar and no cost cap applies. (Inference)
- The task volume is too small to change the budget. (Inference)

6. Constraint families

Latency

Compare price at the same effort level and latency target. (Inference)

Cost

Compare models on cost per completed task, not per token. Combination when one choice is not enough. Add the cache write and cache read prices when prompts repeat.

Data location

Choose a regional endpoint when data residency requires it. Combination when one choice is not enough. On the Claude API, US-only inference with inference_geo adds a 1.1 times multiplier on all token prices.

Governance

Write the price date beside every cost figure, because list prices drift. Combination when one choice is not enough. Recompute the figures when prices change. (Inference)

7. Native cloud services

Amazon Bedrock

Regional endpoints carry a pricing premium over global endpoints.[5]

Claude Platform on AWS

Billing runs through AWS Marketplace.[6]

Claude Models on Google Cloud (Google managed)

Google Cloud invoices the usage, and prices depend on the endpoint type.[7]

Claude Models on Google Cloud (Anthropic managed)

Anthropic meters the usage and reports it to Google Cloud in Claude Consumption Units.[8]

Microsoft Foundry

Usage bills through Azure Marketplace in Claude Consumption Units, not prepaid credits.[9]

8. Exam domain and task statement

D6.2. Expect two options that differ in price and quality. The credited answer prices both on the same basis and states the trade-off. (Inference)

If the stem gives only a per-token price, convert it to cost per completed task first.

Compare options on cost per completed task, and write the price date beside the figure.

9. Trap

The lowest per-token price looks cheapest, but a model that needs more tokens or rounds can cost more per completed task.

10. Not in source

- Price of a model on a platform absent from the pricing page. Not in source.
- Cost per completed task for your own workload. Measure it on your traffic. Not in source.

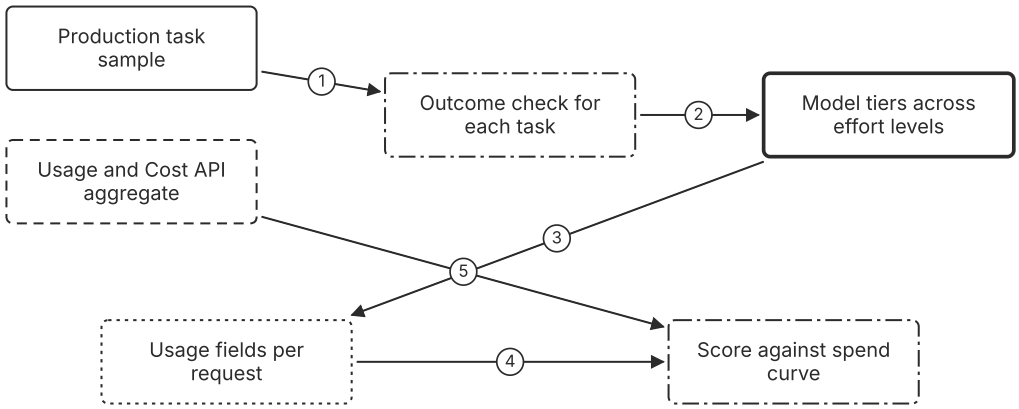
Cost per task measurement

1. Introduction and first principle

Record cost per task beside each quality score, so a trade-off shows both values together.

- For most agent workloads, the docs suggest Claude Opus 5.5 at its default effort as the first model.[77]
- The docs suggest sampling tasks from production logs, weighted like real traffic.[77]
- Usage and cost data typically appears within 5 minutes of request completion.[68]
- The Usage and Cost API supports polling once per minute for sustained use.[68]
- The usage and cost endpoints require Admin API credentials, and workspace API keys do not work.[68]

2. Architecture



Schematic drawn from docs. Arrows: 1. weighted like real traffic. 2. score each run. 3. token counts. 4. priced per task. 5. aggregate check. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Pull a sample of tasks from production logs, weighted like real traffic.
2. Write an outcome check for each sampled task.
3. Run each model tier across the effort levels.
4. Price each request from its usage fields.
5. Plot the score against the spend for each run.
6. Confirm the total with the Usage and Cost API.
7. Run the winner in shadow on a traffic slice before cutover.

4. Pros and cons

Pros

- Shows whether a cheaper tier meets the quality bar on real tasks.
- Shows a score and a spend for each option, so the trade-off stays visible.
- Gives the same method for every model change. (Inference)

Cons

- Needs production logs and outcome checks that many teams lack. (Inference)
- Results reflect the list prices at the time of measurement.

5. Use when, do not use when

Use when

- A model or effort change must pass a cost and quality review.
- Two configurations score close to each other on quality.

Do not use when

- The team has no production logs and no outcome checks yet. (Inference)
- The budget has no limit, so the cost figure changes no decision. (Inference)

6. Constraint families

Latency

Record elapsed model time per task beside the score. (Inference)

Cost

Record cost per task beside each score. Combination when one choice is not enough. If quality falls short, restore the effort, or try the next tier up at its default effort.

Scale

Weight the sample like real traffic, with edge cases included.

Operations

Read the usage and cost totals from the Usage and Cost API. Combination when one choice is not enough. Poll no more than once per minute for sustained use.

7. Native cloud services

Claude Platform on AWS

The programmatic Usage and Cost API endpoints are not available. Use the Usage and Cost pages in the Claude Console.[68]

Claude Models on Google Cloud (Anthropic managed)

Only the usage endpoint is available, and it uses Google Cloud credentials.[68]

8. Exam domain and task statement

D6.2. Expect a proposal to switch to a cheaper model. The credited answer measures cost per task against the quality score first. (Inference)

If the stem gives a cost but no quality score, measure the score first.

Plot the score against the cost per completed task before you choose a model.

9. Trap

A switch on the lowest per-token price looks like a saving. With no quality score on the same tasks, the cheaper tier can miss the bar or cost more per task.

10. Not in source

- Quality score threshold for each workload. Set it for each use case. Not in source.

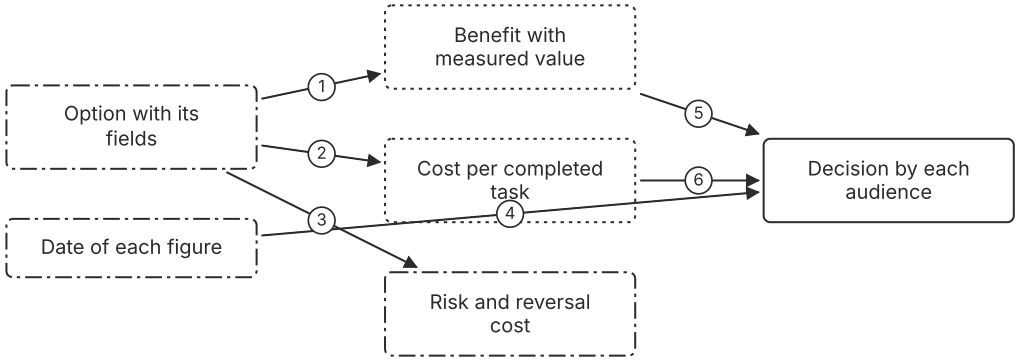
Five-field trade-off record

1. Introduction and first principle

Each option needs the same fields, so every audience reads the same facts. (Inference)

- Compare models on cost per completed task, not per token.[77]
- Figures reflect list prices at the time of measurement, and they drift as prices change.[77]
- Regional endpoints on Amazon Bedrock carry a 10 percent pricing premium over global endpoints.[5]

2. Architecture



Schematic drawn from docs. Arrows: 1. measured. 2. priced. 3. named. 4. sets review date. 5. decision input. 6. decision input. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. List each option with the same set of fields. (Inference)
2. Fill the benefit field with a measured value and its evaluation set name. (Inference)
3. Fill the cost field with cost per completed task. (Inference)
4. Name the risk and the reversal cost of each option. (Inference)
5. Write one message for each audience that reads its own fields. (Inference)
6. Date every figure and set a review date. (Inference)

4. Pros and cons

Pros

- Each audience decides on the facts it owns. (Inference)
- Dated figures show when a choice needs review. (Inference)

Cons

- Several messages take more time than one summary. (Inference)
- Figures drift when prices change, so the record needs a review date.

5. Use when, do not use when

Use when

- Several audiences hold budget, risk, or sign-off. (Inference)
- The option has a material cost or a compliance impact. (Inference)

Do not use when

- One person decides and holds all the facts. (Inference)
- The choice is a small, fully reversible setting. (Inference)

6. Constraint families

Latency

Report a percentile latency for each option. (Inference)

Cost

State cost per completed task with the price date.

Governance

Name the signed approver for each audience. (Inference) Combination when one choice is not enough. Add a compliance impact field for regulated data. (Inference)

Operations

Estimate the reversal cost in days or weeks. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D6.2. Expect an option that promises perfect accuracy or zero risk. The credited answer states the measured figure and its uncertainty. (Inference)

An option that claims perfect accuracy or zero risk has no measurement behind it.

Give each audience the fields it decides on, and state the uncertainty beside each one. (Inference)

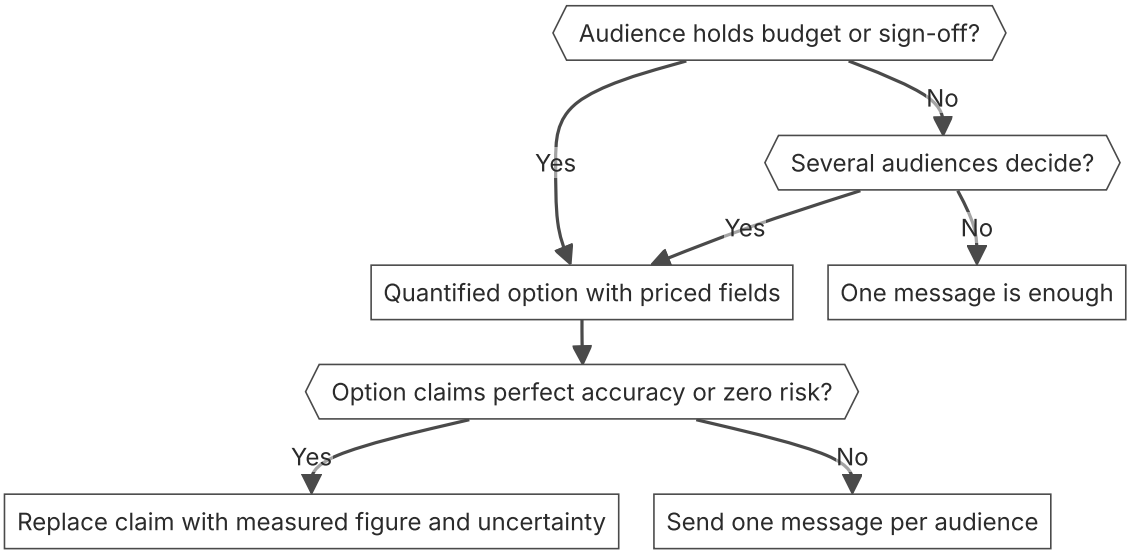
9. Trap

One slide for all audiences looks efficient, but each audience then lacks the field it decides on. (Inference)

10. Not in source

- Five-field record format. General practice, not in the docs. Not in source.

Decision flow



Unit items not in source

- Console steps for trade-off communication: Not in source.
- Stakeholder roles and message formats for each audience. General practice, not in the docs. Not in source.

D6.3 Feedback and expectation alignment

Domain D6, Stakeholder communication and lifecycle management. Task statement: Set measurable expectations, review triggers, and consequences for a production model. (Inference: wording from secondary sources)

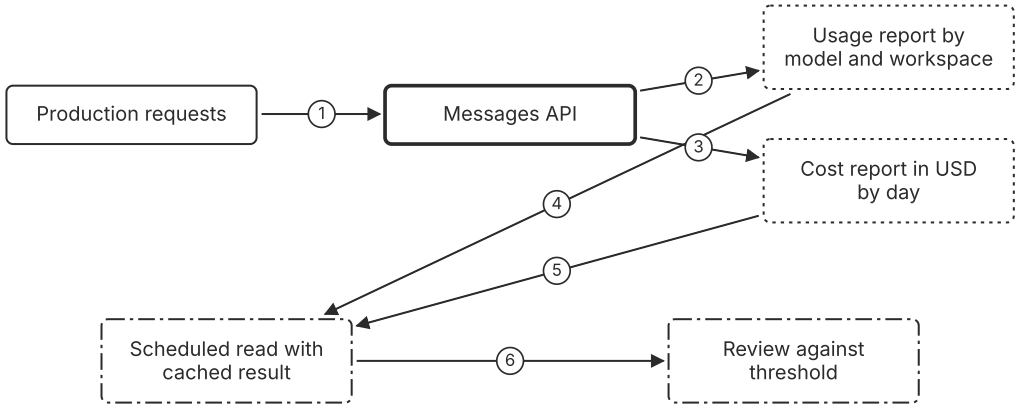
Usage and Cost API reads

1. Introduction and first principle

Usage data arrives after a short delay, so the feedback loop reads aggregates on a schedule.

- Usage and cost data typically appears within 5 minutes of request completion.[68]
- The API supports polling once per minute for sustained use.[68]
- Usage API time buckets are 1 minute, 1 hour, or 1 day.[68]
- The Cost API supports only daily buckets.[68]
- Usage and Cost endpoints need Admin API credentials, and workspace API keys do not work.[68]

2. Architecture



Schematic drawn from docs. Arrows: 1. requests. 2. token counts. 3. priced in USD. 4. aggregates. 5. daily totals. 6. compare with threshold. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Send requests through the workspace that owns the traffic. (Inference)
2. Read the usage report grouped by model or workspace.
3. Read the cost report for daily totals in USD.
4. Poll the endpoints no more than once per minute for sustained use.
5. Cache each result for the dashboard refresh.
6. Compare each total with the threshold in the feedback contract.

4. Pros and cons

Pros

- Gives usage views by model, workspace, and service tier.
- Gives daily cost totals in USD for a budget review.
- Lets a team query history without opening each response. (Inference)

Cons

- Data appears after a delay, so an alert fires late.
- The cost report offers daily granularity only.
- The endpoints need Admin API credentials, not workspace keys.

5. Use when, do not use when

Use when

- A team needs a usage or cost trend for a budget review.
- A workspace must show its share of spend.

Do not use when

- A live incident needs a result within seconds. (Inference)
- The organization has no admin account for these endpoints. (Inference)

6. Constraint families

Cost

Read daily cost totals for the monthly budget review. Combination when one choice is not enough. Filter by workspace to split the total by team.

Isolation

Filter usage by API key and workspace to separate teams.

Governance

Use an Admin API key, an org:admin token, or an unscoped personal or service account key.

Operations

Poll no more than once per minute for sustained use. Combination when one choice is not enough. Use short bursts when you download paginated data.

7. Native cloud services

Claude Platform on AWS

The programmatic Usage and Cost API endpoints are not available. Use the Usage and Cost pages in the Claude Console.[68]

Claude Models on Google Cloud (Anthropic managed)

Only the usage endpoint is available, and it uses Google Cloud credentials.[68]

8. Exam domain and task statement

D6.3. Expect a question on org-wide spend by team. The credited answer reads aggregates on a schedule, not single responses. (Inference)

Per-response usage fields show one call. The Usage and Cost API shows totals.

Read usage and cost aggregates on a schedule, and compare each total with its threshold.

9. Trap

One response at a time looks precise, but it does not show the trend across workspaces. (Inference)

10. Not in source

- Usage and cost endpoints on Amazon Bedrock. Not in source.
- Dashboard refresh interval for your team. Not in source.

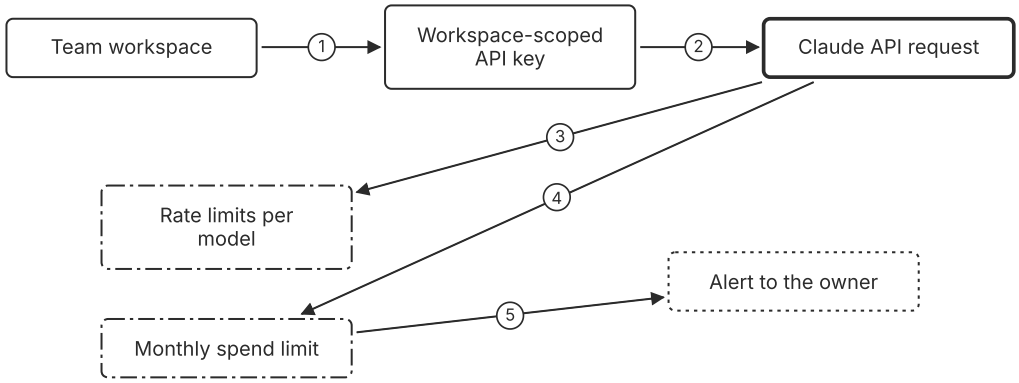
Workspace spend and rate limits

1. Introduction and first principle

Spend limits cap monthly spend for one workspace, and rate limits cap requests and tokens per minute.

- Each workspace can have spend limits and alerts at chosen thresholds.[11]
- Rate limits cap requests, input tokens, and output tokens per minute.[11]
- Requests that exceed a rate limit return a 429 error.[10]
- An organization can have up to 100 workspaces by default, and archived workspaces do not count.[11]
- The default Amazon Bedrock quota is 2 million input tokens per minute.[5]

2. Architecture



Schematic drawn from docs. Arrows: 1. owns. 2. runs in one workspace. 3. checked per minute. 4. counts toward spend. 5. threshold reached. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Create one workspace for each team or environment.
2. Scope each API key to the workspace it serves.
3. Set rate limits for each model in the workspace.
4. Set a monthly spend limit for the workspace.
5. Configure alerts at the spend thresholds.
6. Check the limits again after a model or traffic change. (Inference)

4. Pros and cons

Pros

- Caps spend and requests for each workspace on its own.
- Sends alerts before the spend cap applies.
- Keeps each team cost visible in its own workspace.

Cons

- Spend limits cannot be set on the Default Workspace.
- Requests above a rate limit fail with a 429 error.

5. Use when, do not use when

Use when

- A team or project needs its own budget.
- An environment needs lower limits than production.

Do not use when

- One small project has no budget owner. (Inference)
- The traffic has no budget or capacity requirement. (Inference)

6. Constraint families

Cost

Set a monthly spend limit and alerts on each workspace. Combination when one choice is not enough. Keep production out of the Default Workspace, because it accepts no spend limit. (Inference)

Quota

Set rate limits for each model on requests, input tokens, and output tokens per minute. Combination when one choice is not enough. Read the current limits with the Rate Limits API.

Isolation

Scope each API key to one workspace.

Governance

Cap the share of organization limits that Claude Code can use.

7. Native cloud services

Amazon Bedrock

The default quota is 2 million input tokens per minute, and AWS enforces requests per minute.[5]

Microsoft Foundry

Foundry omits Anthropic rate limit headers, so manage limits in Azure monitoring and request increases through Azure.[9]

Claude Models on Google Cloud (Google managed)

Google Cloud manages rate limits and quotas.[7]

Claude Models on Google Cloud (Anthropic managed)

Anthropic manages rate limits, and higher limits go through the account representative.[8]

Claude Platform on AWS

Anthropic manages rate limits, new organizations start on the Start tier, and increases go through the account representative.[6]

8. Exam domain and task statement

D6.3. Expect a team that asks for its own budget. The credited answer uses a separate workspace with its own limits and alerts. (Inference)

A budget for each team points to workspace spend limits, not to a shared key.

Give each team or environment its own workspace, with its own spend and rate limits.

9. Trap

One shared key with one global cap looks simpler, but it hides which team spent the budget. (Inference)

10. Not in source

- Default requests per minute on Amazon Bedrock. Not in source.
- Exact Start tier rate limit values for Claude Platform on AWS. Not in source.

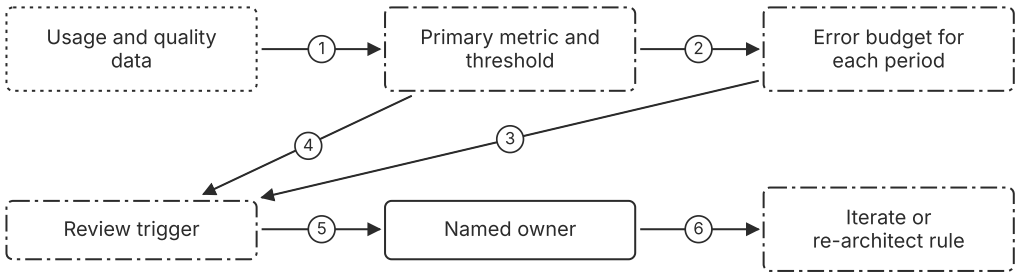
Feedback contract with review triggers

1. Introduction and first principle

A written trigger turns a metric drift into a scheduled review. (Inference)

- Good success criteria are measurable, with quantitative metrics or well-defined qualitative scales.[25]
- Operational criteria include response time in milliseconds and uptime as a percentage.[25]
- Price is a common success criterion, set as the budget to run the model.[25]
- Most use cases need several success criteria checked together.[25]
- Usage and cost data typically appears within 5 minutes of request completion.[68]

2. Architecture



Schematic drawn from docs. Arrows: 1. measured value. 2. sets allowance. 3. allowance spent. 4. threshold crossed. 5. page owner. 6. apply rule. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Set the primary metric and its threshold with the owner. (Inference)
2. Derive the error budget from the threshold for each period. (Inference)
3. Write each review trigger as a count or a rate, not a judgment. (Inference)
4. Name the consequence of each trigger before launch. (Inference)
5. Write the rule that separates tuning from a rebuild. (Inference)
6. Review the contract on the same date as the requirements. (Inference)

4. Pros and cons

Pros

- Turns metric drift into a scheduled review. (Inference)
- Settles the breach question before a dispute starts. (Inference)

Cons

- A trigger can fire on a data change instead of a model change. (Inference)
- A strict consequence can pause work after a false alarm. (Inference)

5. Use when, do not use when

Use when

- Production traffic depends on a probabilistic output. (Inference)
- A stakeholder asks for a guaranteed percentage. (Inference)

Do not use when

- A prototype has no users and no budget. (Inference)
- The metric has no measurement yet. (Inference)

6. Constraint families

Latency

Set a latency range instead of a single point. (Inference)

Cost

Forecast monthly spend from cost per task and expected volume. (Inference) Combination when one choice is not enough. Add a cost trigger for drift above the forecast. (Inference)

Governance

Name the owner who signs each consequence. (Inference) Combination when one choice is not enough. Date the contract and review it with the requirements. (Inference)

Operations

Route each trigger to a named owner. (Inference) Combination when one choice is not enough. Segment the data before any rebuild decision. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D6.3. Expect a drift or SLA dispute. The credited answer applies the agreed trigger and rule, not a new target. (Inference)

A drop below the agreed iterate threshold calls for tuning, not a rebuild. (Inference)

Write each trigger, its owner, and its consequence before launch. (Inference)

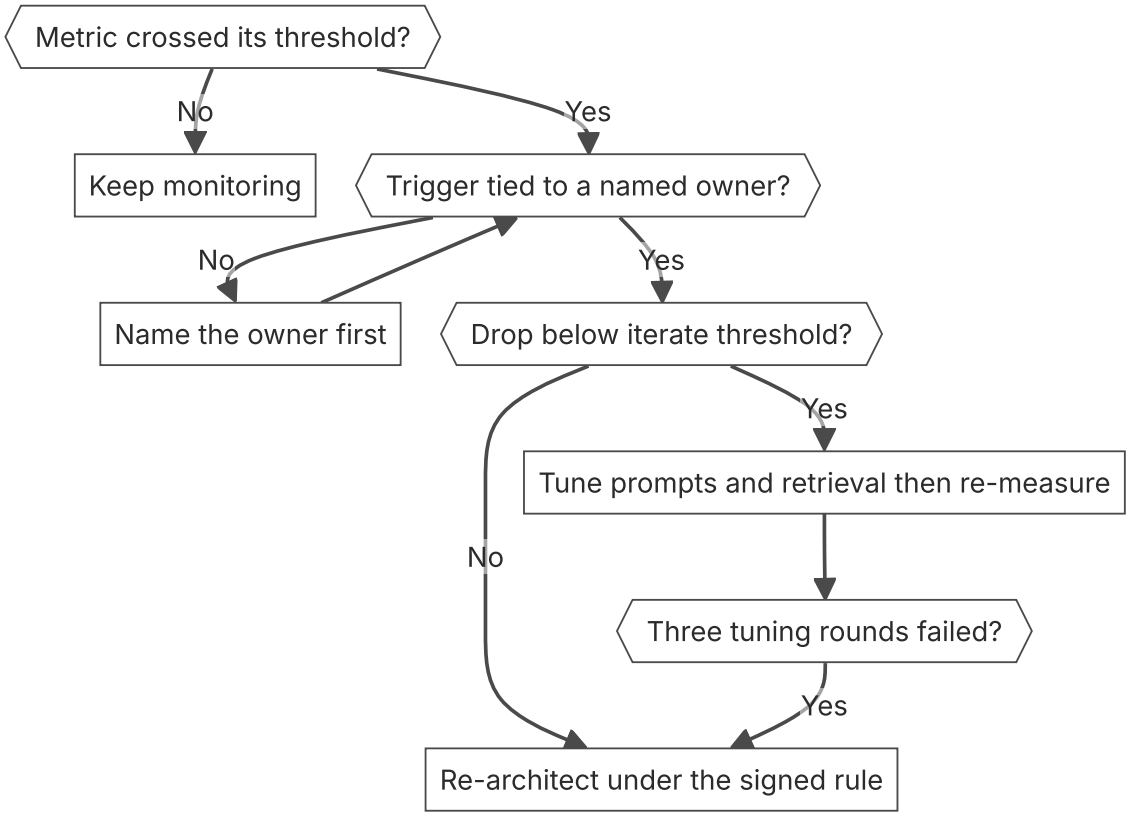
9. Trap

A higher SLA target looks like a fix, but the system behavior does not change. (Inference)

10. Not in source

- Error budget formula and default allowance. General practice, not in the docs. Not in source.
- Breach consequence for a vendor contract. Not in source.

Decision flow



Unit items not in source

- Alert delivery channel for spend thresholds. Not in source.
- Feedback contract template. General practice, not in the docs. Not in source.

D6.4 Architecture documentation

Domain D6, Stakeholder communication and lifecycle management. Task statement: Record architecture decisions with their alternatives, owners, evidence, and review criteria so a successor can operate the system. (Inference: wording from secondary sources)

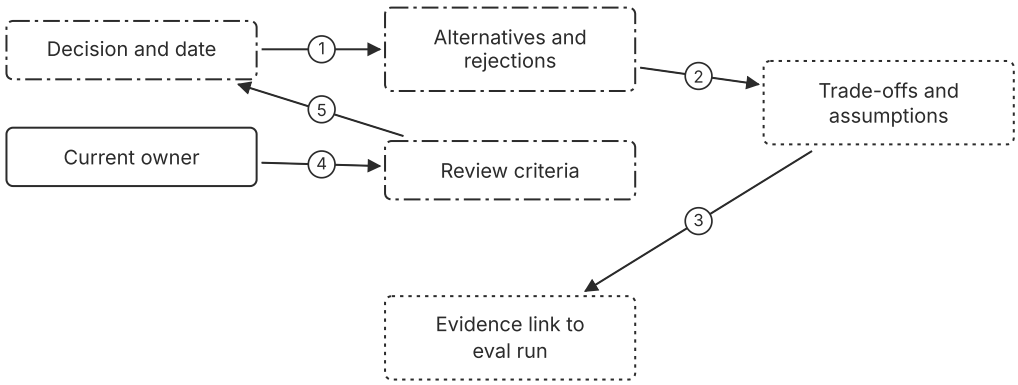
Architecture decision record

1. Introduction and first principle

A record that a successor can read preserves the decision after the original people leave. (Inference)

- A model ID identifies a pinned version that stays constant for the lifetime of that ID.[79]
- Anthropic does not update the weights or configuration of a model ID.[79]
- Infrastructure updates can change observable behavior even when the model ID and weights stay the same.[79]
- Every model ID has its own deprecation and retirement schedule.[79]

2. Architecture



Schematic drawn from docs. Arrows: 1. rejected options. 2. trade-offs per option. 3. eval run identifier. 4. owner checks criteria. 5. reopen rule. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write the decision, its date, and its owner. (Inference)
2. List each alternative and the reason it lost. (Inference)
3. Record the assumptions and trade-offs of the chosen option. (Inference)
4. Link the evidence, such as an evaluation run. (Inference)
5. Write the review criteria that reopen the decision. (Inference)
6. Record the open issues that remain unresolved. (Inference)
7. Add a new record that supersedes the old one. Keep the old record unchanged. (Inference)

4. Pros and cons

Pros

- Lets a successor operate the system without the original team. (Inference)
- Keeps the evidence attached to the decision. (Inference)

Cons

- Records go stale when code changes without a record update. (Inference)
- A long record for a small configuration flag adds work with no gain. (Inference)

5. Use when, do not use when

Use when

- A decision is expensive to reverse or affects several components. (Inference)
- A new owner will run the system later. (Inference)

Do not use when

- A configuration flag is easy to revert and has no trade-off. (Inference)
- A hackathon demo uses no real data. (Inference)

6. Constraint families

Cost

Record the cost per completed task that the decision assumed. (Inference)

Governance

Name the current owner of each decision. (Inference) Combination when one choice is not enough. Link audit or compliance evidence to the evidence field. (Inference)

Scale

Write records only for decisions that affect several components. (Inference)

Operations

Write the review criteria as measurable thresholds. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D6.4. Expect a successor scenario with no original owner. The credited answer reads the decision record and checks its review criteria before any change. (Inference)

If the decision owner left, the first action reads the record. (Inference)

Record each decision with its alternatives, owner, evidence, and review criteria when you make it. (Inference)

9. Trap

A full new evaluation looks thorough, but it repeats evidence that the record already holds. (Inference)

10. Not in source

- Architecture decision record template or required fields. General practice, not in the docs. Not in source.
- Anthropic guidance on architecture decision records. Not in source.

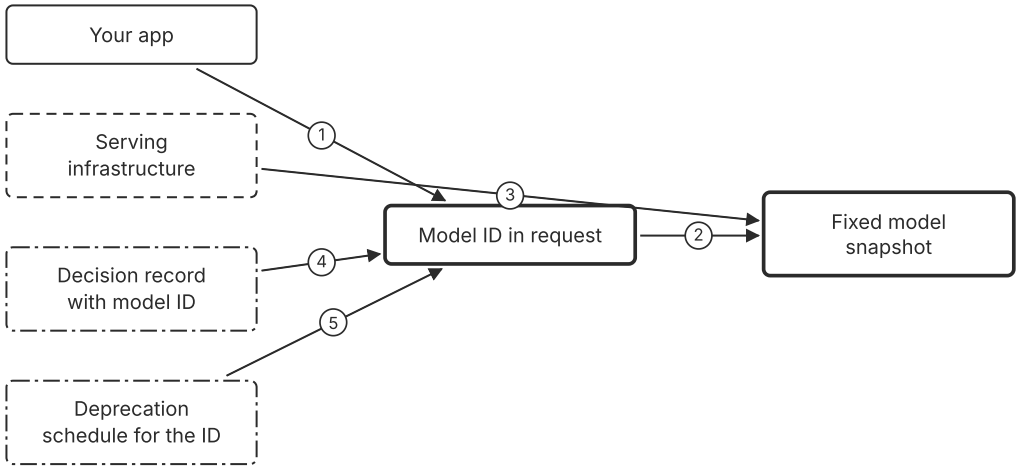
Model ID in decision records

1. Introduction and first principle

Record the exact model ID, because a dateless ID from the 4.6 generation names one fixed snapshot.

- A dateless model ID from the 4.6 generation is a fixed snapshot, not an alias.[79]
- Models before the 4.6 generation include a snapshot date in the ID.[79]
- Anthropic gives at least 60 days of notice before retirement of a publicly released model.[83]
- Microsoft Foundry follows the Claude API lifecycle schedule.[9]
- Anthropic recommends the latest API version whenever possible.[96]

2. Architecture



Schematic drawn from docs. Arrows: 1. sends model parameter. 2. maps to one snapshot. 3. may change behavior. 4. names the ID. 5. sets retirement. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Send the full model ID in each request.
2. Record the model ID and the platform in the decision record.
3. Record the deprecation date for that model ID. (Inference)
4. Test a new model ID before you change the record. (Inference)
5. Create a new record when the model ID changes. (Inference)

4. Pros and cons

Pros

- The record matches the exact snapshot that the evaluation measured.
- A model upgrade becomes an explicit change to the model ID.

Cons

- A dateless ID is not an evergreen pointer, because an updated version ships under a new model ID.
- Behavior can shift on the same ID after infrastructure updates.

5. Use when, do not use when

Use when

- A decision depends on measured quality or cost for one model.
- A platform offers several ID formats for the same model.

Do not use when

- The decision does not depend on model behavior, such as a storage format choice. (Inference)
- The model is a prototype with no decision record. (Inference)

6. Constraint families

Data location

Record the platform that hosts the model ID. Combination when one choice is not enough. Record the endpoint type, such as global or regional, on Amazon Bedrock. (Inference)

Governance

Record the API version that the client sends. Combination when one choice is not enough. Use the latest API version when possible, as the docs recommend.

Skill

Record the model ID that the evaluation used. (Inference)

Operations

Check the deprecation table before each release. Combination when one choice is not enough. Test a successor model well before the retirement date. (Inference)

7. Native cloud services

Amazon Bedrock

Model IDs carry an anthropic. prefix, such as anthropic.claude-opus-5-5.[79]

Claude Models on Google Cloud (Google managed)

Models from the 4.6 generation use the Claude API ID. Earlier models add an at sign and a date.[79]

Claude Models on Google Cloud (Anthropic managed)

This offering uses the same model IDs as the Claude API.[8]

Claude Platform on AWS

Model IDs are identical to the Claude API IDs, such as claude-haiku-5-5, with no anthropic. prefix.[6]

Microsoft Foundry

The deployment name becomes the model parameter, and it defaults to the model ID.[9]

8. Exam domain and task statement

D6.4. Expect a decision record that names a model. The credited answer names the exact model ID and the platform. (Inference)

A model ID with no date is a fixed snapshot from the 4.6 generation onward.

Record the exact model ID, the platform, and the deprecation date for each measured decision.

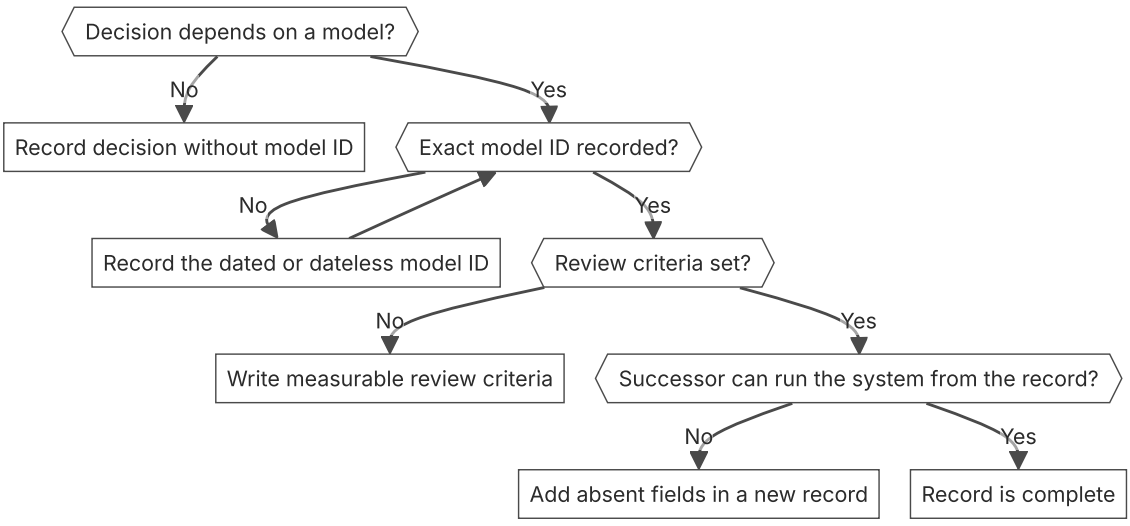
9. Trap

A dateless ID looks like an alias that moves, but it names a fixed snapshot. A record that says latest misstates what was tested.

10. Not in source

- Endpoint type effect on model behavior for Amazon Bedrock. Not in source.

Decision flow



Unit items not in source

- Console steps for architecture documentation: Not in source.
- Storage location for decision records in a repository. General practice, not in the docs. Not in source.

D6.5 Lifecycle phases

Domain D6, Stakeholder communication and lifecycle management. Task statement: Run an AI system through ordered lifecycle phases with gates, and plan model retirement and migration. (Inference: wording from secondary sources)

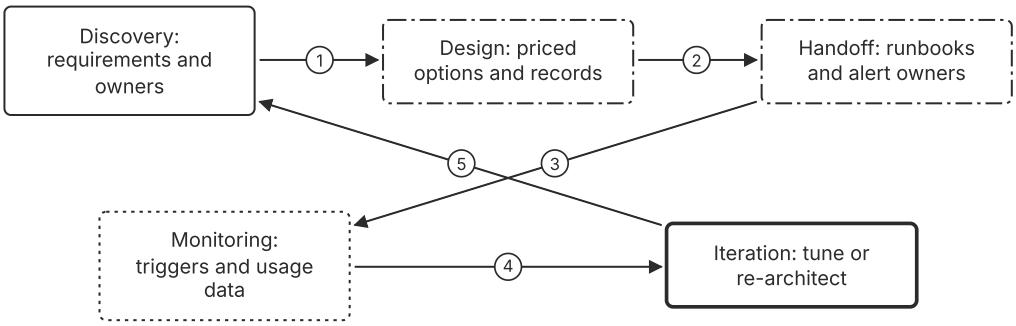
Lifecycle phase gates

1. Introduction and first principle

Each phase closes only when its artifact exists, so later work cannot replace earlier work. (Inference)

- Anthropic recommends the latest API version whenever possible.[96]
- API version 2026-10-21 removes the temperature, top_p, and top_k request parameters.[96]
- A request that sends a removed parameter returns a 400 error on every model.[96]
- Anthropic gives at least 60 days of notice before retirement of a publicly released model.[83]

2. Architecture



Schematic drawn from docs. Arrows: 1. requirements gate. 2. records gate. 3. runbook gate. 4. trigger gate. 5. return rule when requirements change. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Close discovery when each requirement has a metric, a threshold, and an owner. (Inference)
2. Close design when options are priced and decision records exist. (Inference)
3. Close handoff when runbooks, alert owners, and a tested rollback path exist. (Inference)
4. Close monitoring when triggers fire and a review follows. (Inference)
5. Return to the earliest skipped phase when a later phase fails. (Inference)

4. Pros and cons

Pros

- A failed later phase points to the gate that the team skipped. (Inference)
- Gates limit how much a fix costs after each later phase. (Inference)

Cons

- Gates can slow a small team that has no handoff. (Inference)
- Emergencies skip gates, so the record must catch up later. (Inference)

5. Use when, do not use when

Use when

- The system has real users, money, or compliance scope. (Inference)
- Several teams hand off operation of the system. (Inference)

Do not use when

- A reversible demo has no real data. (Inference)
- A live outage needs an immediate fix before any record exists. Write the record after the fix. (Inference)

6. Constraint families

Cost

Compare the cost of a gate with the cost of a later fix. (Inference)

Governance

Name the signer for each gate. (Inference) Combination when one choice is not enough. Record an emergency path that writes the record after the fix. (Inference)

Scale

Scale the gate review to the team size. (Inference)

Operations

Name the alert owner before the handoff gate closes. (Inference) Combination when one choice is not enough. Test the rollback path before live traffic. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D6.5. Expect a live failure where a phase was skipped. The credited answer returns to the skipped phase, not to more monitoring. (Inference)

A higher error rate with no thresholds points to discovery, not to monitoring tools. (Inference)

When a later phase fails, return to the earliest skipped gate. (Inference)

9. Trap

More dashboards look like progress, but they measure a design that has no requirements. (Inference)

10. Not in source

- Gate names and number of phases. General practice, not in the docs. Not in source.

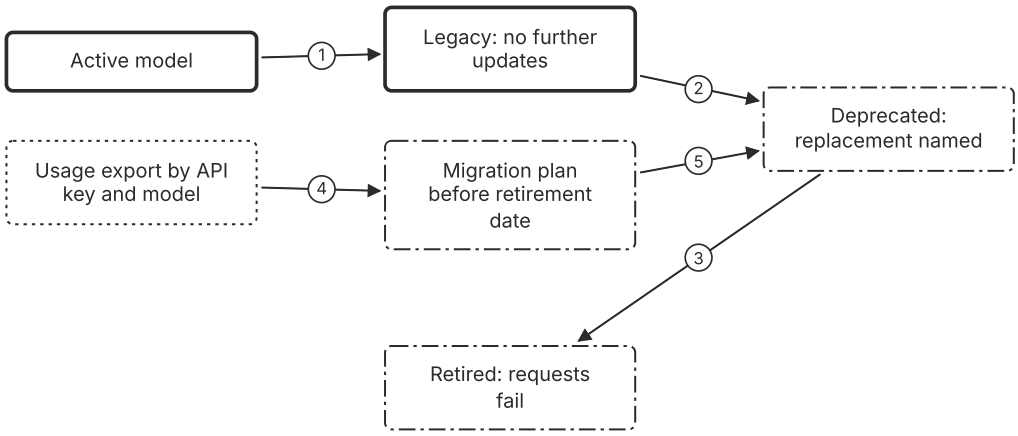
Model lifecycle states and retirement

1. Introduction and first principle

A model has one of four states, active, legacy, deprecated, or retired, and requests to a retired model fail.

- Requests to a retired model fail.[83]
- Deprecated models are likely to be less reliable than active models.[83]
- Anthropic gives at least 60 days of notice before retirement of a publicly released model.[83]
- The Usage page export produces a CSV that breaks usage down by API key and model.[83]
- Some retirement dates in the model status table are tentative.[83]
- Deprecation page dates apply to the Claude API, Claude Platform on AWS, Claude Models on Google Cloud (Anthropic managed), and Microsoft Foundry.[83]

2. Architecture



Schematic drawn from docs. Arrows: 1. no further updates. 2. deprecation notice. 3. retirement date. 4. lists affected keys. 5. migrate before retirement. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Check the model status on the deprecations page in each release cycle. (Inference)
2. Export usage from the Usage page to find deprecated models.
3. Read the downloaded file to find each API key and model in use.
4. Run evaluations on the replacement model before the retirement date. (Inference)
5. Move all traffic to the replacement before the retirement date.

4. Pros and cons

Pros

- Retirement dates appear on the deprecations page ahead of time.
- The usage export shows which API keys still call deprecated models.

Cons

- Deprecated models are likely to be less reliable than active models.
- Some retirement dates are tentative, so the deadline can move.

5. Use when, do not use when

Use when

- An application calls a model with a deprecated or legacy status.
- A team plans a model change before a known retirement date.

Do not use when

- The application calls an active model and no change is planned. (Inference)
- A decision must use a model that the platform has already retired. (Inference)

6. Constraint families

Cost

Compare the replacement with cost per completed task before cutover. (Inference)

Governance

Notify the owner at the first deprecation notice. (Inference) Combination when one choice is not enough. Check each platform, because Amazon Bedrock and Google managed platforms set their own retirement schedules.

Skill

Use the replacement that the deprecations page names.

Operations

Export usage by model and API key before each retirement date. Combination when one choice is not enough. Test the replacement before the date. (Inference)

7. Native cloud services

Claude Platform on AWS

The retirement dates on the deprecations page apply to this platform.[83]

Claude Models on Google Cloud (Anthropic managed)

The retirement dates on the deprecations page apply to this platform.[83]

Claude Models on Google Cloud (Google managed)

Google sets the lifecycle dates, and they can differ from the Claude API schedule.[7]

Microsoft Foundry

Foundry follows the Claude API lifecycle schedule.[9]

Amazon Bedrock

Amazon Bedrock sets its own retirement schedule, so lifecycle status and dates can differ from the deprecations page.[83]

8. Exam domain and task statement

D6.5. Expect a deprecation notice with a near retirement date. The credited answer migrates before the date, to the named replacement. (Inference)

A deprecated model still serves requests, but its retirement date sets the deadline.

Migrate to the named replacement before the retirement date, and test the replacement first.

9. Trap

A model switch made only at the retirement date causes an outage, because requests to a retired model fail.

10. Not in source

- Exact Amazon Bedrock retirement dates. The deprecations page points to the Bedrock model tables. Not in source.
- Retirement date for a model after its tentative date. Not in source.

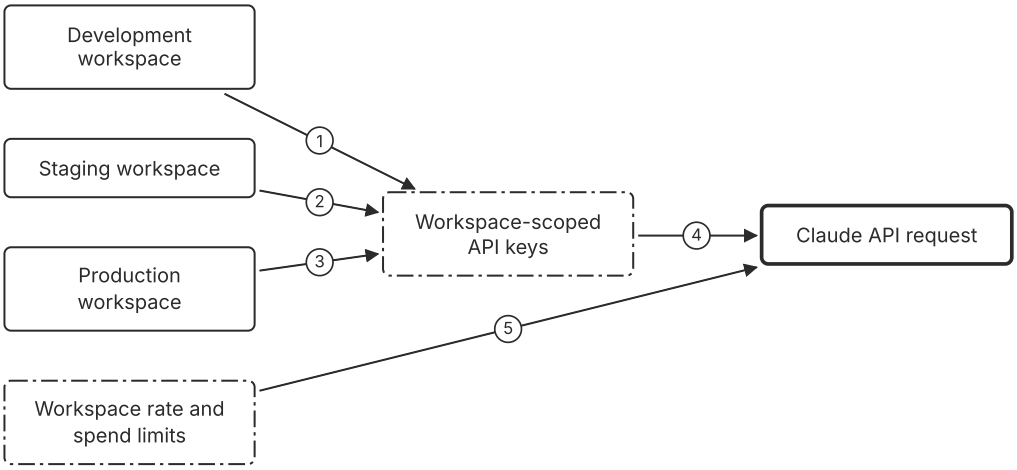
Workspaces per environment

1. Introduction and first principle

Every request runs in exactly one workspace, so each environment needs its own workspace for keys, usage, and limits.

- Every request runs in exactly one workspace.[11]
- An organization can have up to 100 workspaces by default.[11]
- The Default Workspace cannot be renamed, archived, or deleted.[11]
- Development workspaces use lower rate limits, and production workspaces use full rate limits.[11]
- Spend limits cannot be set on the Default Workspace.[11]

2. Architecture



Schematic drawn from docs. Arrows: 1. issues key. 2. issues key. 3. issues key. 4. runs in one workspace. 5. apply per workspace. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Create one workspace each for development, staging, and production.
2. Scope each API key to the workspace that it serves.
3. Set rate limits for each workspace, based on its environment.
4. Set a spend limit on each workspace that is not the Default Workspace.
5. Promote the tested model ID from staging to production. (Inference)

4. Pros and cons

Pros

- Each environment has its own keys, usage, and limits.
- Usage reports separate the environments by workspace.

Cons

- Each extra workspace adds administration work. (Inference)
- The Default Workspace cannot be renamed, archived, or deleted.

5. Use when, do not use when

Use when

- Development and production need different rate limits.
- Teams need separate cost reports.

Do not use when

- A single prototype has one user and no production traffic. (Inference)

6. Constraint families

Cost

Set a spend limit on each workspace that is not the Default Workspace. Combination when one choice is not enough. Keep production traffic out of the Default Workspace, which has no spend limit. (Inference)

Quota

Set development rate limits lower than production rate limits.

Data location

On Claude Platform on AWS, create each workspace in the AWS region where the data must stay.

Isolation

Scope each API key to one workspace. Combination when one choice is not enough. Use a key that is not workspace-scoped with a workspace header for calls across workspaces.

Governance

Give each team access only to its own workspace. Combination when one choice is not enough. On Google Cloud, give each team its own project, because roles apply to every workspace in a project.

7. Native cloud services

Claude Platform on AWS

Each workspace binds to one AWS region, and IAM policies grant or deny access per workspace.[6]

Claude Models on Google Cloud (Anthropic managed)

Setup creates one Default Workspace, and IAM roles apply to every workspace in the project.[8]

Amazon Bedrock

Prompt caches are isolated by organization on this platform, not by workspace.[11]

Microsoft Foundry

Prompt caches are isolated by workspace on this platform.[11]

8. Exam domain and task statement

D6.5. Expect a question on separate environments. The credited answer separates development, staging, and production into workspaces with their own limits. (Inference)

The Default Workspace cannot have spend limits, so production needs its own workspace.

Create one workspace for each environment that needs its own limits or reports.

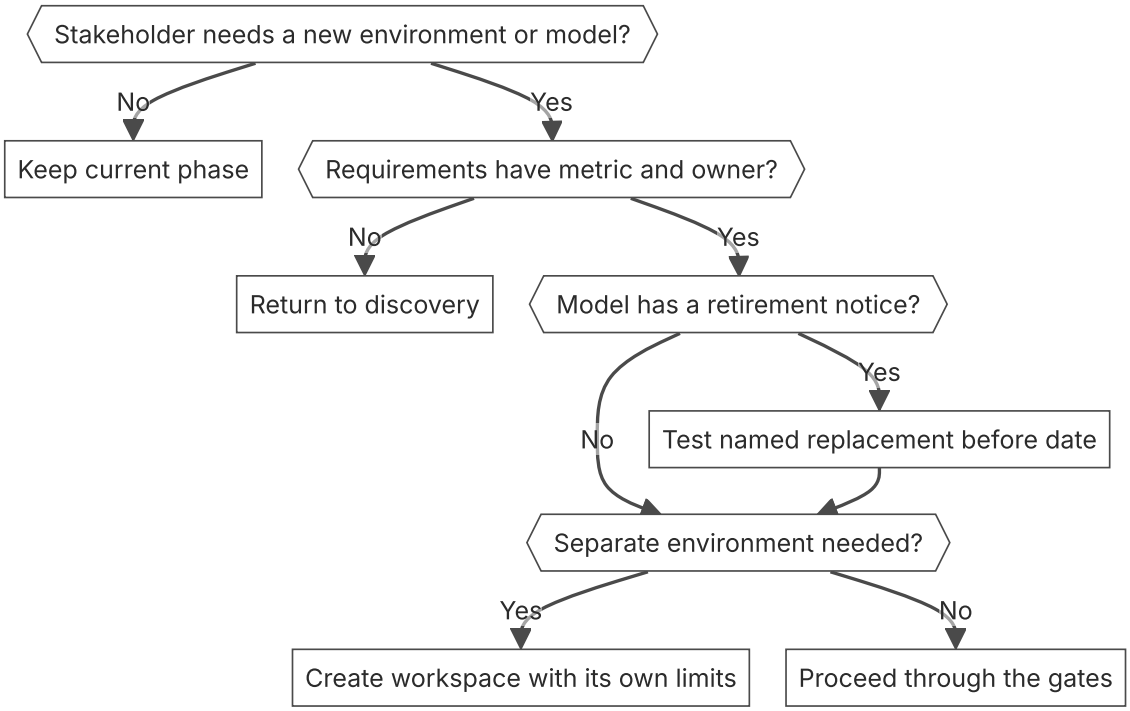
9. Trap

One shared workspace with a label convention looks simple, but keys and limits then mix across environments. (Inference)

10. Not in source

- Workspace limits on platforms other than the Claude API. Not in source.
- Promotion process from staging to production. Not in source.

Decision flow



Unit items not in source

- Console steps for lifecycle phases in general: Not in source.
- Lifecycle phase names and gate criteria. General practice, not in the docs. Not in source.

D7.1 Team configuration for Claude Code

Domain D7, Developer productivity and operational enablement. Task statement: Configure Claude Code for a team with committed settings, enforced rules, and guidance files. (Inference: wording from secondary sources)

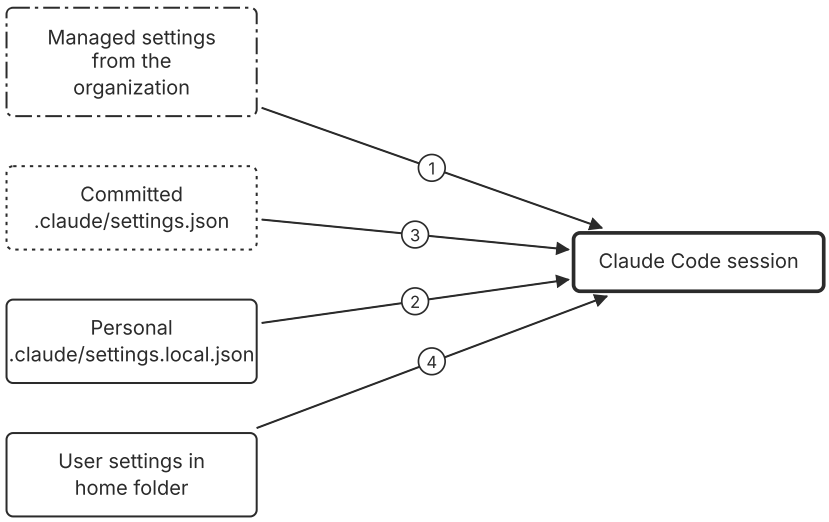
Shared settings and scopes

1. Introduction and first principle

One committed settings file gives every clone the same permissions and hooks, and personal files hold the exceptions.

- Claude Code reads settings from four files: user, shared project, project local, and managed.[97]
- Settings precedence runs from managed settings down to user settings, with the command line between managed and project local.[97]
- Permission rules evaluate in this order: deny, then ask, then allow.[55]
- A deny rule in any settings scope blocks a call that another scope allows.[55]
- A PreToolUse hook that exits with code 2 blocks the tool call before permission rules run.[55]
- The default timeout for a command hook is 600 seconds.[81]
- Keep each CLAUDE.md file under 200 lines.[98]
- On Linux and WSL, the managed settings file is at /etc/claude-code/managed-settings.json.[99]

2. Architecture



Schematic drawn from docs. Arrows: 1. highest precedence. 2. personal overrides. 3. team permissions and hooks. 4. lowest precedence. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Commit the shared settings file at .claude/settings.json in the repository.
2. Claude Code reads the user, shared project, project local, command line, and managed settings.
3. A key set at a higher level overrides the same key set at a lower level.
4. Managed settings take precedence over every other source.
5. Put personal exceptions in .claude/settings.local.json, which stays out of git.
6. Run /status to see which settings sources loaded for the session.

4. Pros and cons

Pros

- Every clone receives the same permission rules and hooks after the commit.

- Personal overrides need no commit and no change to the team file.
- Managed settings apply on every machine that the organization deploys them to.

Cons

- A settings change reaches teammates only after the commit and a pull.
- Settings files are strict JSON, so a comment or a trailing comma causes a Settings Error.
- Some keys never take effect from a repository file, so a committed key can be ignored.

5. Use when, do not use when

Use when

- A team shares one repository and needs the same permission rules on every clone.
- One developer needs an exception without a change to the team file.

Do not use when

- A single session needs a temporary change, so the command line flag fits better.
- The rule must hold on every machine, and a committed file cannot enforce it on machines that skip the commit.

6. Constraint families

Data location

Keep credentials out of committed settings files. (Inference) Combination when one choice is not enough. Put a personal workspace key in the env block of the user settings file on one machine.

Isolation

Keep personal overrides in .claude/settings.local.json.

Governance

Commit .claude/settings.json with deny rules for secret reads and destructive commands. Combination when one choice is not enough. Add managed settings when the organization must enforce a rule on every developer machine.

Skill

Commit team skills in .claude/skills/ so every clone loads them. Combination when one choice is not enough. Use nested skill folders for code in a subdirectory of a monorepo.

Operations

Put hooks that lint or test each edit in the shared settings file. Combination when one choice is not enough. Review each committed hook command like code, because a hook runs a shell command. (Inference)

7. Native cloud services

Amazon Bedrock

The Bedrock setup wizard saves its result to the env block of the user settings file, not the shared project file.[80]

Microsoft Foundry

Foundry has no setup wizard, so environment variables are the only configuration path.[100]

Claude Platform on AWS

A workspace API key takes precedence over SigV4 credentials, and the user settings env block can hold it on one machine.[101]

8. Exam domain and task statement

D7.1. Expect a scenario where a team needs a control that holds on every clone. The credited answer uses permissions, hooks, or managed settings in a file that the team shares. (Inference)

If the scenario needs a block, choose a deny rule, a PreToolUse hook, or managed settings.

Use a permission rule or a hook for a control, and use CLAUDE.md for a convention.

9. Trap

A CLAUDE.md note that asks the agent not to read secret files. The docs call CLAUDE.md context, not enforced configuration, so only a deny rule blocks the read.

10. Not in source

- Organization plan tier needed for managed settings: Not in source.
- Console steps: Not in source.

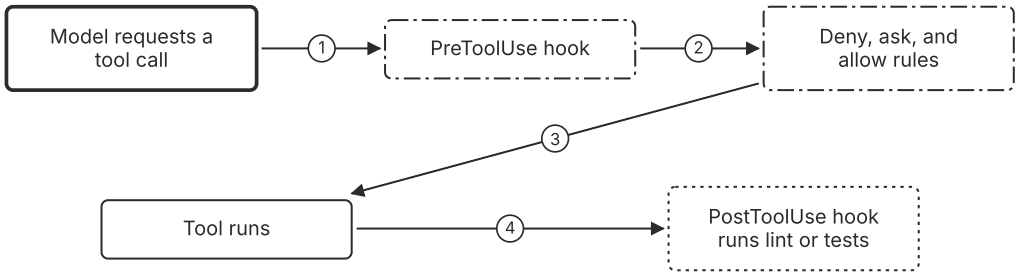
Permission rules and hooks

1. Introduction and first principle

Permission rules and hooks decide a tool call in code, so the result does not depend on what the model chooses.

- Permission rules evaluate in this order: deny, then ask, then allow.[55]
- A PreToolUse hook decision does not bypass a deny rule.[55]
- A PreToolUse hook that exits with code 2 blocks the tool call before permission rules run.[55]
- The default timeout for a command hook is 600 seconds.[81]
- Read and Edit deny rules apply to built-in file tools and to file commands that Bash recognizes.[55]
- The sandbox adds OS-level enforcement that Read rules alone do not give.[55]
- On PreToolUse, a timed-out command hook does not block the tool call unless onFailure is set to block.[81]

2. Architecture



Schematic drawn from docs. Arrows: 1. tool call. 2. exit code 2 blocks here. 3. allow proceeds. 4. after a successful edit. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. The model requests a tool call, such as an edit or a shell command.
2. PreToolUse hooks run before the permission prompt.
3. Claude Code evaluates deny rules first, then ask rules, then allow rules.
4. A deny rule for the call blocks it, even when a hook returns allow.
5. A PostToolUse hook runs after the tool call succeeds, for example to lint the edited file.

4. Pros and cons

Pros

- A deny rule holds even when the model wants to run the blocked command.
- A hook runs the same check on every machine that has the committed file.
- Rules name the tool, so the team can review them in one file. (Inference)

Cons

- A Bash rule matches the command text, so another form of the same program can pass.
- A command hook runs shell code, so a bad hook can change files on every clone. (Inference)

- A stalled PreToolUse hook does not block a call by default.

5. Use when, do not use when

Use when

- A destructive command must never run in any session in the repository.
- Every edit must trigger a lint or test command.

Do not use when

- The rule is a style preference, so CLAUDE.md is the better place.
- The control must cover every process on the machine, so a sandbox is the better layer.

6. Constraint families

Isolation

Restrict subagent tools with the tools field in the agent file.

Governance

Deny secret file reads with Read rules in the shared settings file. Combination when one choice is not enough. Add the same rule to managed settings when the organization requires it on every machine.

Scale

Use one deny rule for a path pattern instead of one rule per file. Combination when one choice is not enough. Use an ask rule for the gray cases that need a person to decide.

Operations

Use a PostToolUse hook with an Edit or Write matcher for lint. Combination when one choice is not enough. Use a PreToolUse hook with onFailure set to block when a failed check must stop the call.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D7.1. Expect a scenario that asks how to block a tool or run a check on every edit. The credited answer uses a deny rule or a hook, not a prompt instruction. (Inference)

If the stem says the model must not run a command, choose the deny rule over a note in the memory file.

Use a deny rule to block a tool and a hook to run a check at a lifecycle event.

9. Trap

A PreToolUse hook that returns allow, which looks like approval. A deny rule still blocks the call, so the hook does not override the team policy.

10. Not in source

- Console steps: Not in source.

CLAUDE.md and subagent guidance

1. Introduction and first principle

Guidance files shape how the model works, and they cannot stop a tool call.

- Claude treats CLAUDE.md files as context, not enforced configuration.[98]
- Keep each CLAUDE.md file under 200 lines.[98]
- A subagent without a tools field inherits every tool available to subagents.[102]
- Managed subagent definitions take precedence over project and user subagents with the same name.[102]
- Plugin subagents do not support the hooks, mcpServers, or permissionMode frontmatter fields.[102]

- Project skills load from .claude/skills/ and commit with the repository.[103]

2. Architecture



Schematic drawn from docs. Arrows: 1. loaded as context. 2. delegated task. 3. every tool call. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Claude Code loads CLAUDE.md files from the current directory and its parents at launch.
2. Claude reads the CLAUDE.md content and tries to follow it.
3. A subagent file sets a tools field that limits the tools the subagent can use.
4. Permission rules still decide every tool call the subagent makes.
5. Move a control that must hold into a permission rule or a hook.

4. Pros and cons

Pros

- CLAUDE.md is fast to write and shapes style and conventions.
- A subagent with a read-only tools list cannot write files.
- Team files are committed, so every clone gets the same guidance.

Cons

- A CLAUDE.md line gives no guarantee of compliance, especially for vague instructions.
- A long CLAUDE.md file uses more context and reduces adherence.
- Plugin subagents do not support the hooks, mcpServers, or permissionMode fields.

5. Use when, do not use when

Use when

- The team wants a style convention that the model follows across sessions.
- A research task needs a subagent that only reads files.

Do not use when

- The rule must block an action, because a note cannot stop a tool call.
- The guidance applies to one folder only and a path-scoped rule file fits better.

6. Constraint families

Isolation

Give a research subagent only the Read and Grep tools. Combination when one choice is not enough. Name the subagent in the agents folder so the team can invoke it by name.

Governance

Use CLAUDE.md for behavior guidance and settings for enforcement. Combination when one choice is not enough. Deploy a managed CLAUDE.md file when every user needs the same guidance.

Scale

Split long guidance into files under .claude/rules/. Combination when one choice is not enough. Add a paths field to a rule file so it loads only for files that match.

Skill

Put a repeatable workflow in a skill folder under .claude/skills/. Combination when one choice is not enough. Use the skill description so the model loads the skill when the task matches.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D7.1. Expect a scenario that mixes a convention with a security need. The credited answer keeps the convention in CLAUDE.md and moves the security need to settings. (Inference)

If an answer relies on the model to follow a note for safety, reject it and look for a deny rule.

Write a convention in CLAUDE.md, and write a required block in a permission rule.

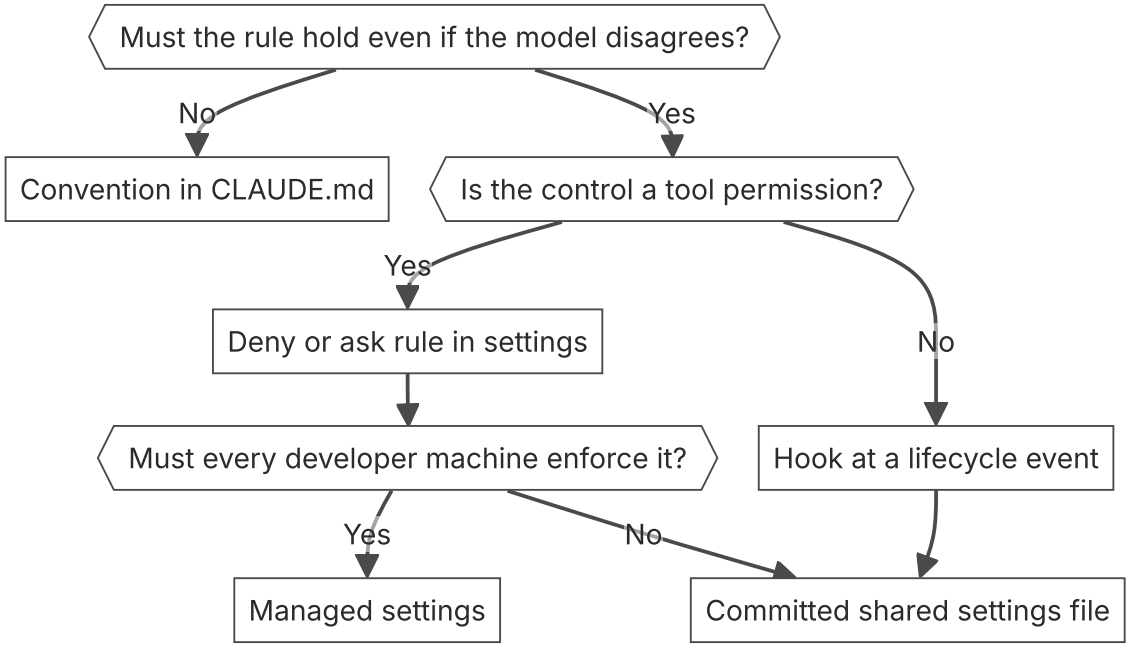
9. Trap

A bigger model with a long CLAUDE.md file, expected to enforce a secret rule. Adherence to guidance is not a lock, so the rule belongs in permissions.

10. Not in source

- Console steps: Not in source.

Decision flow



Unit items not in source

- Console steps for team configuration: Not in source.
- Organization plan tier needed for managed settings: Not in source.

D7.2 AI-assisted workflows with Claude Code

Domain D7, Developer productivity and operational enablement. Task statement: Run Claude Code in scripts and CI with bounded tools, checkable results, and a clear trigger. (Inference: wording from secondary sources)

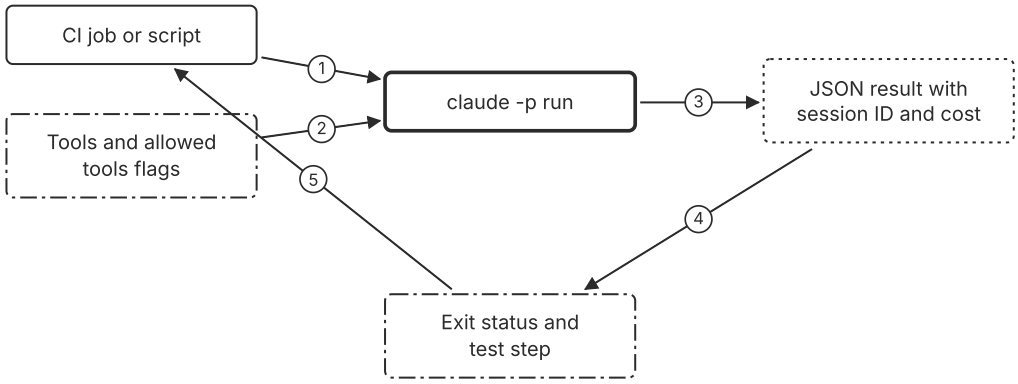
Headless runs

1. Introduction and first principle

A print mode run returns a result and an exit status, so a script can decide whether the job passes.

- Claude Code exits with code 0 on success and a non-zero code when the run fails.[107]
- The JSON output payload includes total_cost_usd and a per-model cost breakdown.[107]
- The --bare flag skips auto-discovery of hooks, skills, subagents, plugins, MCP servers, auto memory, and CLAUDE.md.[107]
- Without --bare, a print run runs project hooks and connects .mcp.json servers in an untrusted folder.[107]
- Piped stdin is capped at 10MB.[107]
- The --json-schema flag returns output in the structured_output field.[107]
- The --allowedTools flag lets tools run without a prompt. To restrict which tools are available, use --tools.[108]
- In a Manual mode claude -p run, a call that needs approval is denied unless something approves it.[107]

2. Architecture



Schematic drawn from docs. Arrows: 1. run with -p and --bare. 2. restricts and approves tools. 3. output-format json. 4. result fields. 5. pass or fail the job. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Run claude with the -p flag to start a non-interactive run.
2. Pass --tools with only the built-in tools the job needs, such as Read and Edit.
3. Pass --allowedTools to approve those tools without a prompt.
4. Add --bare in CI to skip hooks, skills, and CLAUDE.md files from the host.
5. Choose --output-format json to read the result, the session ID, and the cost.
6. Check the exit status before the job continues. Zero means the run succeeded.
7. Run the team test command as a separate step. (Inference)

4. Pros and cons

Pros

- A script runs Claude Code without an interactive session.
- The JSON output includes the total cost, so a script can record spend.
- The --tools flag restricts which built-in tools the run can use.

Cons

- Without --bare, a print run runs project hooks and connects project MCP servers, even in an untrusted folder.
- A print run shows no workspace trust dialog and no per-server approval prompt.
- Piped stdin has a size cap, so large inputs need a file path in the prompt.
- The --allowedTools flag only removes prompts. It does not restrict which tools are available.

5. Use when, do not use when

Use when

- A CI job needs a model result with a clear pass or fail status.
- A script needs output that matches a JSON Schema.

Do not use when

- A task needs a person to approve each step in real time. (Inference)
- A developer needs a one-off answer, and an interactive session fits better.

6. Constraint families

Cost

Track spend with total_cost_usd for each run. Combination when one choice is not enough. When a run continues a conversation, the total includes the earlier runs.

Isolation

Add --bare when the repository is not trusted.

Governance

Pass --tools to restrict the built-in tools, and --allowedTools to approve them without a prompt. Combination when one choice is not enough. Use the dontAsk permission mode for a locked-down CI run, so any call that would prompt is denied.

Skill

Add --bare in CI so host skills and subagents do not load. Combination when one choice is not enough. Omit --bare only in a trusted checkout when the job needs project hooks or skills. (Inference)

Operations

Read the exit status and the result field before the job passes. Combination when one choice is not enough. Record total_cost_usd from the JSON output for each run.

7. Native cloud services

Amazon Bedrock

The GitHub Action can use Amazon Bedrock with the use_bedrock input set to true.[109]

Microsoft Foundry

Claude Code reads CLAUDE_CODE_USE_FOUNDRY and the other Foundry variables from the environment, which suits a scripted run.[100]

8. Exam domain and task statement

D7.2. Expect a scenario with a CI check or a scripted call. The credited answer uses a bounded print run with a fixed tools list and a checked exit status. (Inference)

If the stem asks to restrict tools in a CI run, look for --tools or a deny rule, not --allowedTools alone.

Restrict each scripted run with --tools, approve only those tools, and check the exit status before the job passes.

9. Trap

An --allowedTools list read as a tool restriction. The flag only auto-approves the listed tools. To restrict which tools exist, use --tools, and bypass mode belongs only in isolated environments.

10. Not in source

- Console steps: Not in source.
- Quota or rate limit for scripted runs: Not in source.

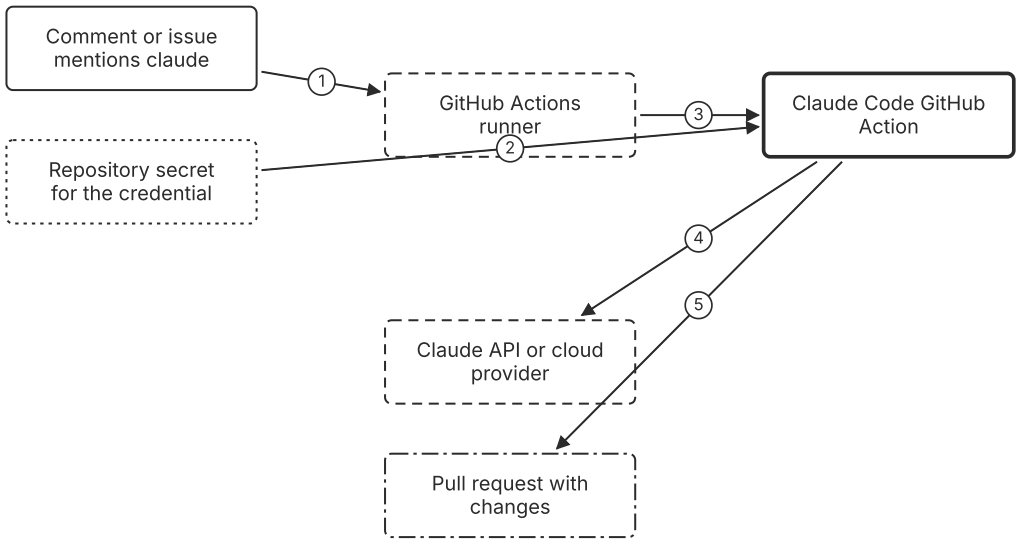
GitHub Actions workflows

1. Introduction and first principle

A workflow trigger starts Claude Code in the repository, and repository secrets hold the credential.

- The Claude Code GitHub Action expects the secret ANTHROPIC_API_KEY or CLAUDE_CODE_OAUTH_TOKEN.[110]
- An OAuth token from claude setup-token is tied to the subscription of the person who ran the command.[110]
- Workload identity federation needs the id-token write permission in the workflow.[110]
- In interactive mode, the default trigger phrase is @claude.[110]
- The quick setup runs /install-github-app from a Claude Code session.[110]
- The Bedrock option for the GitHub Action uses the use_bedrock input set to true.[109]

2. Architecture



Schematic drawn from docs. Arrows: 1. trigger. 2. authentication input. 3. run action. 4. model requests. 5. branch and pull request. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Install the Claude GitHub App on the repository.
2. Add the authentication secret as ANTHROPIC_API_KEY or CLAUDE_CODE_OAUTH_TOKEN.
3. Copy the workflow file into the .github/workflows folder.
4. Mention @claude in an issue or pull request comment to start a run in interactive mode.
5. Review the pull request that the action opens before you merge it. (Inference)

4. Pros and cons

Pros

- The action runs in the repository and uses the repository permissions it receives. (Inference)
- The quick setup command writes the secret and the workflow files for you.

- A federated credential needs no stored long-lived secret.

Cons

- An OAuth token belongs to the subscription of the person who ran the setup command.
- Each repository needs its own secret unless the team shares an organization secret.
- A workflow that runs on every comment can start more runs than the team expects. (Inference)

5. Use when, do not use when

Use when

- The team wants a comment on an issue or pull request to start a model run.
- The team wants the run output to appear as a pull request.

Do not use when

- A developer wants a local result, and a headless run fits better.
- The secret must outlive one developer, and an API key from the console fits better.

6. Constraint families

Data location

Store the credential as a repository secret, never in the workflow file. Combination when one choice is not enough. Use workload identity federation to avoid a stored long-lived secret.

Isolation

Grant the workflow only the permissions it needs. (Inference) Combination when one choice is not enough. Grant id-token write for the federation exchange, even with a custom token.

Governance

Use an API key from the Claude Console for a secret shared across repositories. Combination when one choice is not enough. Use an organization-level Actions secret so repositories share one copy.

Operations

Use interactive mode with the default @claude trigger when a person starts each run. Combination when one choice is not enough. Set a prompt input when the workflow must run without a comment.

7. Native cloud services

Amazon Bedrock

The GitHub Action can run through Amazon Bedrock, with the use_bedrock input.[109]

Claude Models on Google Cloud (Google managed)

The GitHub Action can run through Google Cloud's Agent Platform with the use_vertex input set to true.[109]

Microsoft Foundry

The GitHub Action can run through Microsoft Foundry with the use_foundry input set to true.[109]

8. Exam domain and task statement

D7.2. Expect a scenario where a team wants an agent to act on a comment or an issue. The credited answer uses the GitHub Action with a repository secret and a pull request as output. (Inference)

If the stem names a shared secret across repositories, choose an API key over an OAuth token.

Use the GitHub Action for a repository trigger, and use an API key for a secret that several repositories share.

9. Trap

A personal subscription token stored as a shared organization secret. The docs say the token is tied to the subscription of the person who created it, so an API key fits a shared secret.

10. Not in source

- Run count or minutes billed for GitHub Actions: Not in source.

- Console steps: Not in source.

Subagent roles for review and exploration

1. Introduction and first principle

A subagent with a narrow tools list can explore or review without the power to change files.

- A subagent without a tools field inherits every tool available to subagents.[102]
- A user or project subagent named Explore overrides the built-in Explore subagent.[102]
- Plugin subagents do not support the hooks, mcpServers, or permissionMode frontmatter fields.[102]
- Managed subagent definitions take precedence over project and user subagents with the same name.[102]
- The model field accepts sonnet, opus, haiku, fable, a full model ID, or inherit.[102]

2. Architecture



Schematic drawn from docs. Arrows: 1. delegate task. 2. limits access. 3. reads files. 4. returns findings. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Create a subagent file in the .claude/agents folder for the team.
2. Set the tools field to Read and Grep for an exploration role.
3. Set the model field to haiku for a lower-cost exploration role.
4. Ask the main session to delegate the review to the subagent. (Inference)
5. Check the findings for file and line references before you act on them. (Inference)

4. Pros and cons

Pros

- A subagent keeps long search output out of the main conversation.
- A tools list removes write access for a review role.
- The team commits the subagent file, so each clone gets the same role.

Cons

- A subagent without a tools field inherits every tool available to subagents.
- Plugin subagents ignore the hooks, mcpServers, and permissionMode fields.
- Rewind usually does not restore subagent edits to other files, so use Git to revert them.

5. Use when, do not use when

Use when

- A repository review needs file references from a read-only role.
- An exploration task reads many files that the main session does not need.

Do not use when

- The task is one file edit that the main session can do directly.
- The role must enforce a permission that only a settings rule can enforce.

6. Constraint families

Cost

Set the model field to haiku for exploration. Combination when one choice is not enough. Use the inherit value when the role must match the main session model.

Isolation

Give an exploration role only the Read and Grep tools. Combination when one choice is not enough. Add disallowedTools for a write tool when a role inherits tools.

Governance

Commit project subagents in .claude/agents so the team reviews them. Combination when one choice is not enough. Deploy a managed subagent file when the organization needs the same role everywhere.

Scale

Split large reviews into one subagent per area of the repository. (Inference)

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D7.2. Expect a scenario that asks for an agent role with least privilege. The credited answer sets a tools list on the subagent file. (Inference)

If a stem asks for a read-only reviewer, look for a tools list that omits write tools.

Set the tools field on each role to the smallest set that the role needs.

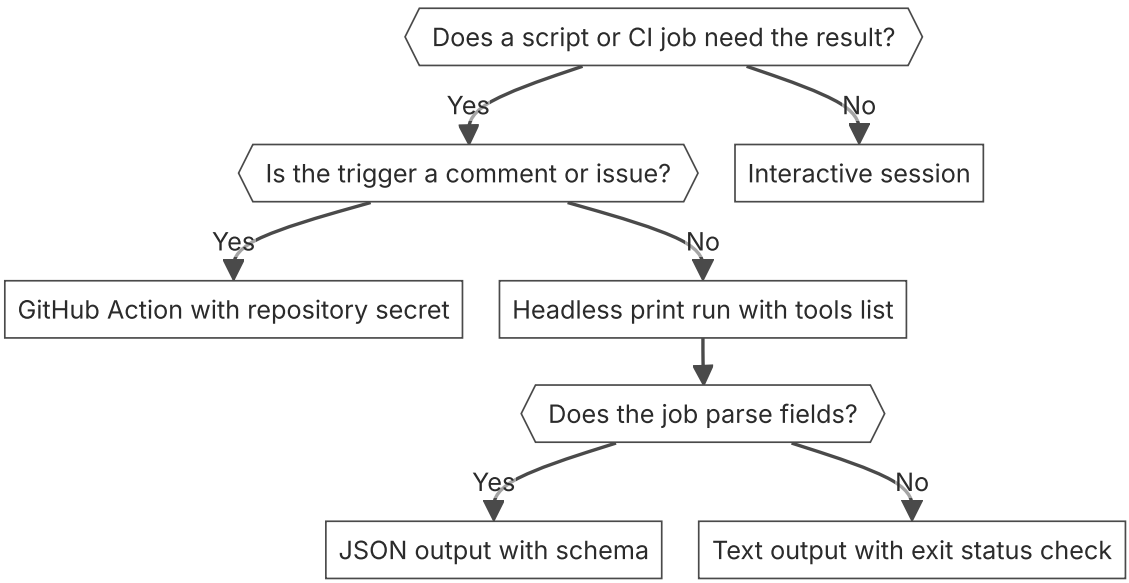
9. Trap

A review subagent with no tools field, which inherits write tools. The role then can edit files that a review should only read.

10. Not in source

- Console steps: Not in source.
- Token limit for a subagent: Not in source.

Decision flow



Unit items not in source

- Console steps for AI-assisted workflows: Not in source.

- Exam-specific wording for service coverage: Not in source.

D7.3 Diagnosis and operational resolution

Domain D7, Developer productivity and operational enablement. Task statement: Find the cause of a Claude Code problem with built-in diagnostics, usage data, and rewind before you change code or settings. (Inference: wording from secondary sources)

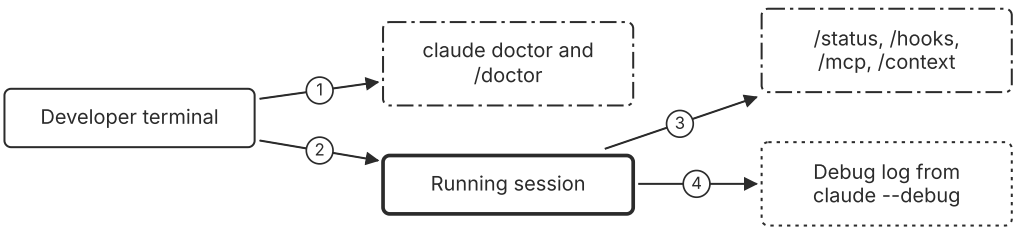
Diagnostic commands

1. Introduction and first principle

Read the loaded state before you change a setting, a hook, or a server, because a file on disk can differ from what loaded.

- The claude doctor command prints read-only installation and settings diagnostics without opening a session.[112]
- The /doctor command proposes fixes that Claude applies after you confirm.[113]
- The claude --debug flag records each hook event, the matchers checked, and the hook exit code and output.[112]
- The /status command shows a Setting sources line that lists each settings file that loaded.[97]
- The /context command shows context usage and optimization suggestions.[113]
- A server that shows as connected but lists zero tools needs a Reconnect from /mcp.[112]

2. Architecture



Schematic drawn from docs. Arrows: 1. find invalid settings. 2. start or inspect. 3. show loaded state. 4. record hook events. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Run claude doctor in the terminal to find invalid settings files without a session.
2. Run /doctor inside a session to get proposed fixes that you confirm first.
3. Run /status to see the settings sources that loaded for the session.
4. Run /hooks and /mcp to check the hook and server configuration.
5. Start the session with claude --debug to record each hook match and its exit code.
6. Run /context to see what fills the context window.

4. Pros and cons

Pros

- The commands read the live state rather than a remembered setup.
- The claude doctor command prints diagnostics and changes nothing.
- The debug log shows which hook matchers were checked.

Cons

- A command shows the state at one moment, so run it again after each change. (Inference)
- The diagnostics show what loaded, not why the model chose an action. (Inference)
- The debug flag adds log output for the whole session, so use it only for a session that fails. (Inference)

5. Use when, do not use when

Use when

- A setting, hook, or MCP server does not take effect.
- Claude Code will not start because a settings file is invalid.

Do not use when

- The question is about answer quality, and no configuration changed.
- A single known setting needs a value change and the file path is clear.

6. Constraint families

Latency

Run /context to find large context before you change the model. (Inference) Combination when one choice is not enough. Run /compact with focus instructions to free context in the same conversation.

Governance

Run /status to confirm that managed settings are in force. Combination when one choice is not enough. Check /status on each developer machine after a managed rollout. (Inference)

Scale

Enable the debug flag only for the session that fails. (Inference)

Operations

Run claude doctor before a settings change and again after it. Combination when one choice is not enough. Run /doctor in a session when the fix needs Claude to edit files after your confirmation.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D7.3. Expect a scenario where a hook or a server seems inactive. The credited answer checks the loaded configuration with a built-in command before it edits code or guidance. (Inference)

If a setting seems ignored, check the settings sources with /status before you edit the file.

Run the built-in check that matches the symptom before you change the setting or the code.

9. Trap

A CLAUDE.md edit meant to make a hook run. Hooks run from settings files, and a guidance note never starts a hook, so the check is the hook configuration and the debug log.

10. Not in source

- Console steps: Not in source.

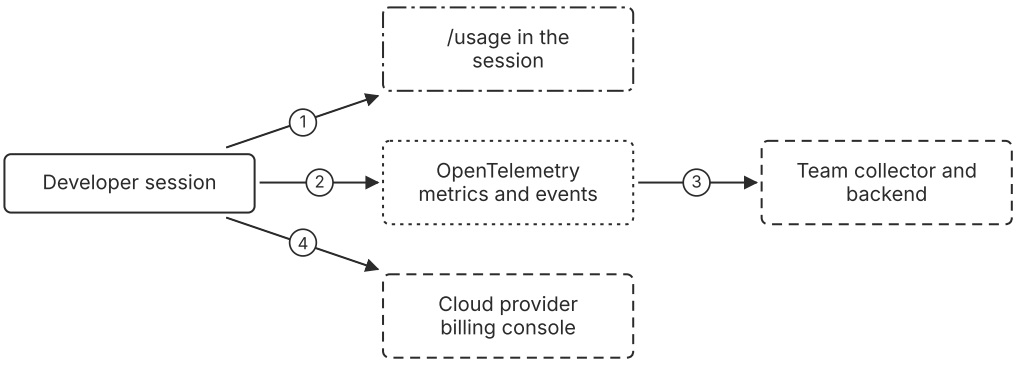
Usage, cost, and telemetry

1. Introduction and first principle

Cost and latency changes need per-session numbers before anyone changes the model.

- Across enterprise deployments, the average cost is around $13 per developer per active day.[114]
- The default export interval for metrics is 60000 milliseconds.[69]
- OTEL_LOG_USER_PROMPTS is disabled by default.[69]
- Claude Code ignores OpenTelemetry exporter variables in a repository settings file.[69]
- The metric claude_code.token.usage counts tokens used.[69]
- The session cost totals reset when /clear starts a new session.[114]

2. Architecture



Schematic drawn from docs. Arrows: 1. show session cost. 2. export metrics. 3. OTLP export. 4. cloud spend controls. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Run /usage in a session to see the session cost and the plan limits.
2. Set the OpenTelemetry exporter variables in managed settings, the shell, or the user settings file.
3. Check the claude_code.session.count metric to confirm that the export works.
4. Track claude_code.cost.usage and claude_code.token.usage in the team backend.
5. Leave prompt logging off unless the policy allows prompt content in telemetry.

4. Pros and cons

Pros

- The /usage command shows cost without any setup.
- Metrics give a time series for cost and tokens across a team.
- Prompt content logging is off by default.

Cons

- The /usage counters reset when /clear starts a new session.
- A repository cannot turn on telemetry, because the repository ignores the OpenTelemetry variables.
- Telemetry needs a collector and a backend that the team runs.

5. Use when, do not use when

Use when

- A cost rise needs a trend across sessions and developers.
- A developer needs the current session cost in the terminal.

Do not use when

- The cost question concerns a cloud provider bill, and the provider console is the source of truth.
- The team wants prompt text in logs by default, which the default setting blocks.

6. Constraint families

Cost

Use /usage for the current session and OpenTelemetry for the trend. Combination when one choice is not enough. Set workspace spend limits in the Console for an API organization.

Data location

Keep prompt and response logging off by default. Combination when one choice is not enough. Enable prompt logging only where the policy allows prompt content in telemetry.

Governance

Set the exporter variables in managed settings to lock the collector. Combination when one choice is not enough. A repository can only turn a signal off with its exporter selector, not turn telemetry on.

Scale

Use the cardinality switches to drop session or version attributes from metrics.

Operations

Use the default export interval for normal operation. Combination when one choice is not enough. Lower the export interval only for a test, and reset it for production. (Inference)

7. Native cloud services

Amazon Bedrock

Claude Code is billed per token to the cloud account, and the spend controls live in the provider billing console.[114]

Microsoft Foundry

Claude Code is billed per token to the cloud account, and the spend controls live in the provider billing console.[114]

Claude Models on Google Cloud (Google managed)

Claude Code is billed per token to the cloud account, and the spend controls live in the provider billing console.[114]

8. Exam domain and task statement

D7.3. Expect a scenario with a cost rise after a change. The credited answer reads per-session cost and token data before it changes the model. (Inference)

If a cost question asks for the first step, choose the usage read over the model change.

Read the cost and token data for the affected sessions before you change the model.

9. Trap

A smaller model to cut cost before the team reads the usage data. The change may miss the cause and can lower answer quality.

10. Not in source

- Console steps: Not in source.
- Per-developer attribution on Bedrock, Foundry, or Google Cloud without a gateway: Not in source.

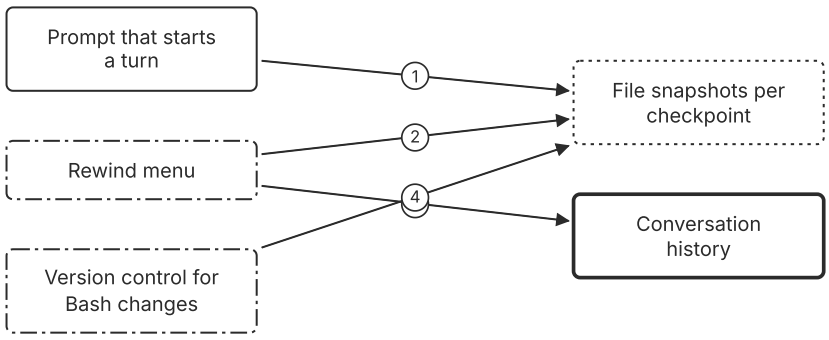
Rewind and file snapshots

1. Introduction and first principle

Rewind restores file edits that Claude Code tracked, so a bad change does not need a manual undo.

- Claude Code keeps file snapshots for the 100 most recent checkpoints in a session.[111]
- Claude Code deletes a session's file snapshots about 30 days after the session last saved one.[111]
- Two Esc presses with an empty prompt open the rewind menu.[111]
- The cleanupPeriodDays setting sets the retention period, with a default of 30 days.[115]
- Rewind does not undo Bash command changes to a file that Claude did not edit.[111]

2. Architecture



Schematic drawn from docs. Arrows: 1. checkpoint before turn. 2. restore tracked files. 3. restore conversation. 4. covers untracked Bash edits. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Claude Code creates a checkpoint before each prompt that starts a turn.
2. Claude Code keeps file snapshots for the 100 most recent checkpoints in a session.
3. Run /rewind, or press Esc twice with an empty prompt, to open the rewind menu.
4. Choose the action that restores the conversation, the code, or both.
5. Use version control for a file that only Bash commands changed, because rewind does not track it.

4. Pros and cons

Pros

- Rewind undoes tracked Claude edits without a Git command.
- The conversation and the code can rewind together.
- The rewind menu lists the prompts from the session.

Cons

- Bash command changes to a file that Claude did not edit are not tracked.
- Snapshots are deleted about 30 days after the session last saved one.
- A rewind to a deleted snapshot can fail with No files were restored.

5. Use when, do not use when

Use when

- Claude made file edits in the session that you want to undo.
- A resumed session needs an earlier file state within the retention period.

Do not use when

- The change came only from a Bash command on a file that Claude never edited.
- The change is older than the retention period for the snapshots.

6. Constraint families

Data location

Keep the repository in version control as the durable record. Combination when one choice is not enough. Commit before a large agent change, so Git holds a clean state. (Inference)

Governance

Review the rewound files before the next commit. (Inference) Combination when one choice is not enough. Use version control history as the audit trail for agent edits. (Inference)

Scale

Use the rewind menu for the most recent checkpoints, which the session keeps.

Operations

Rewind before you send a prompt that repeats the same change. (Inference) Combination when one choice is not enough. Set cleanupPeriodDays to keep snapshots longer.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D7.3. Expect a scenario where an agent change must be undone. The credited answer uses rewind for tracked edits and version control for Bash edits. (Inference)

If a stem says a shell command changed a file, look for version control in the answer, not rewind.

Use rewind for tracked Claude edits and version control for everything else.

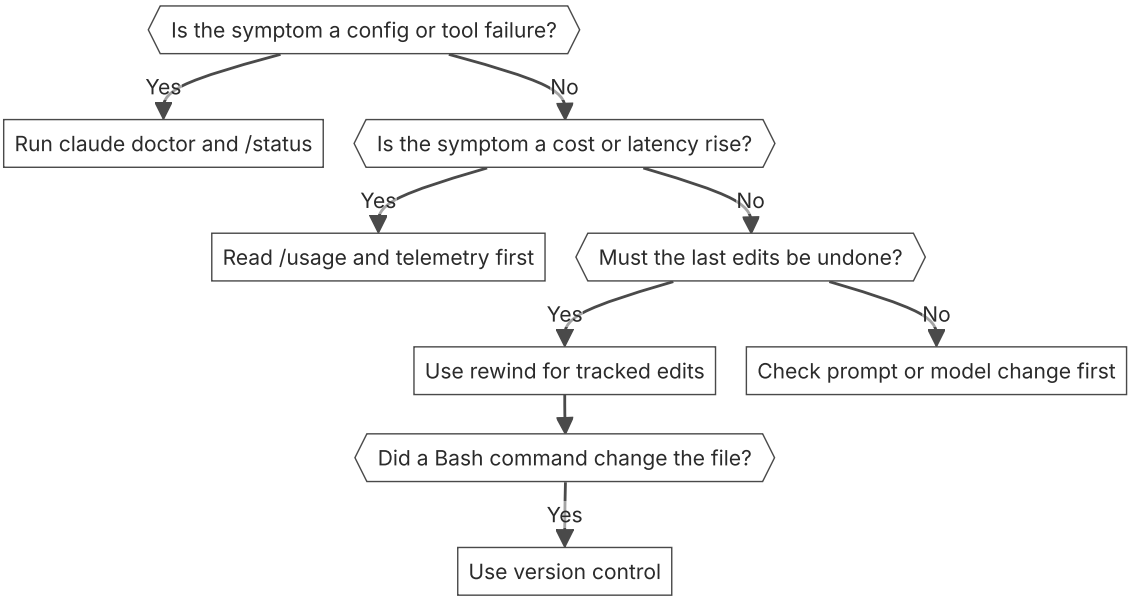
9. Trap

A belief that rewind undoes every change in the session. Rewind does not track Bash changes to files that Claude did not edit.

10. Not in source

- Console steps: Not in source.

Decision flow



Unit items not in source

- Console steps for diagnostics and operations: Not in source.
- Latency threshold that triggers an alert: Not in source.

Sources

1. Models overview. https://platform.claude.com/docs/en/models/overview
2. Claude Haiku 5.5. https://platform.claude.com/docs/en/models/haiku-5-5/overview
3. Pricing. https://platform.claude.com/docs/en/about-claude/pricing
4. Context windows. https://platform.claude.com/docs/en/build-with-claude/context-windows
5. Claude in Amazon Bedrock (Opus 4.7 and later). https://platform.claude.com/docs/en/build-with-claude/claude-in-amazon-bedrock
6. Claude Platform on AWS. https://platform.claude.com/docs/en/build-with-claude/claude-platform-on-aws
7. Claude Models on Google Cloud (Google managed). https://platform.claude.com/docs/en/build-with-claude/claude-models-on-google-cloud/google-managed
8. Claude Models on Google Cloud (Anthropic managed). https://platform.claude.com/docs/en/build-with-claude/claude-models-on-google-cloud/anthropic-managed
9. Claude in Microsoft Foundry. https://platform.claude.com/docs/en/build-with-claude/claude-in-microsoft-foundry
0. Rate limits. https://platform.claude.com/docs/en/api/rate-limits
1. Workspaces. https://platform.claude.com/docs/en/manage-claude/workspaces
2. Glossary. https://platform.claude.com/docs/en/about-claude/glossary
3. How tool use works. https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works
4. Parallel tool use. https://platform.claude.com/docs/en/agents-and-tools/tool-use/parallel-tool-use
5. Claude API errors. https://platform.claude.com/docs/en/api/errors
6. Batch processing. https://platform.claude.com/docs/en/build-with-claude/batch-processing
7. Streaming messages. https://platform.claude.com/docs/en/build-with-claude/streaming
8. Get your Claude API key. https://platform.claude.com/docs/en/get-api-key
9. Authentication. https://platform.claude.com/docs/en/manage-claude/authentication
0. Create an Admin API key. https://platform.claude.com/docs/en/manage-claude/admin-api-keys
1. Workload Identity Federation. https://platform.claude.com/docs/en/manage-claude/workload-identity-federation
2. API and data retention. https://platform.claude.com/docs/en/manage-claude/api-and-data-retention
3. Customer-managed encryption keys. https://platform.claude.com/docs/en/manage-claude/cmek
4. Claude Models on Google Cloud. https://platform.claude.com/docs/en/build-with-claude/claude-models-on-google-cloud/overview
5. Define success criteria and build evaluations. https://platform.claude.com/docs/en/test-and-evaluate/develop-tests
6. Choosing the right model. https://platform.claude.com/docs/en/about-claude/models/choosing-a-model
7. Using the Messages API. https://platform.claude.com/docs/en/build-with-claude/working-with-messages
8. Observability with OpenTelemetry. https://code.claude.com/docs/en/agent-sdk/observability
9. Security risks and sandbox boundaries. https://platform.claude.com/docs/en/managed-agents/secure-deployment-risks
0. Structured outputs. https://platform.claude.com/docs/en/build-with-claude/structured-outputs
1. Strict tool use. https://platform.claude.com/docs/en/agents-and-tools/tool-use/strict-tool-use
2. Agent SDK overview. https://code.claude.com/docs/en/agent-sdk/overview
3. Multiagent orchestration. https://platform.claude.com/docs/en/managed-agents/multiagent-orchestration
4. Session budgets. https://platform.claude.com/docs/en/managed-agents/budgets
5. Claude Managed Agents overview. https://platform.claude.com/docs/en/managed-agents/overview
6. How the agent loop works. https://code.claude.com/docs/en/agent-sdk/agent-loop
7. Permission policies. https://platform.claude.com/docs/en/managed-agents/permission-policies
8. Subagents in the SDK. https://code.claude.com/docs/en/agent-sdk/subagents
9. Compaction overview. https://platform.claude.com/docs/en/build-with-claude/compaction
0. Compaction at a token threshold. https://platform.claude.com/docs/en/build-with-claude/compaction-threshold
1. Compaction on demand. https://platform.claude.com/docs/en/build-with-claude/compaction-on-demand
2. Token counting. https://platform.claude.com/docs/en/build-with-claude/token-counting
3. Effort. https://platform.claude.com/docs/en/build-with-claude/effort
4. Thinking. https://platform.claude.com/docs/en/build-with-claude/thinking
5. Prompting best practices. https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

6. Mitigate jailbreaks and prompt injections. https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/mitigate-jailbreaks
7. Prompt caching. https://platform.claude.com/docs/en/build-with-claude/prompt-caching
8. Prompt engineering overview. https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
9. Compaction that keeps recent turns. https://platform.claude.com/docs/en/build-with-claude/compaction-keep-recent-turns
0. Context editing. https://platform.claude.com/docs/en/build-with-claude/context-editing
1. Increase output consistency. https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/increase-consistency
2. Ticket routing. https://platform.claude.com/docs/en/about-claude/use-case-guides/ticket-routing
3. Cache diagnostics. https://platform.claude.com/docs/en/build-with-claude/cache-diagnostics
4. Tool search tool. https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool
5. Configure permissions. https://code.claude.com/docs/en/permissions
6. MCP connector. https://platform.claude.com/docs/en/agents-and-tools/mcp-connector
7. Define tools. https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools
8. Tool use with prompt caching. https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-use-with-prompt-caching
9. Connect Claude Code to tools via MCP. https://code.claude.com/docs/en/mcp
0. Authenticate with vaults. https://platform.claude.com/docs/en/managed-agents/vaults
1. Remote MCP servers. https://platform.claude.com/docs/en/agents-and-tools/remote-mcp-servers
2. Fast mode. https://platform.claude.com/docs/en/build-with-claude/fast-mode
3. Reducing latency. https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-latency
4. Service tiers. https://platform.claude.com/docs/en/api/service-tiers
5. Handle streaming refusals. https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/handle-streaming-refusals
6. OpenTelemetry for the Claude API. https://platform.claude.com/docs/en/manage-claude/opentelemetry
7. Set up OpenTelemetry with the Claude API. https://platform.claude.com/docs/en/manage-claude/opentelemetry/api
8. Usage and Cost API. https://platform.claude.com/docs/en/manage-claude/usage-cost-api
9. Monitoring. https://code.claude.com/docs/en/monitoring-usage
0. Inspect sessions and track usage. https://platform.claude.com/docs/en/managed-agents/session-observability
1. Search results. https://platform.claude.com/docs/en/build-with-claude/search-results
2. Citations. https://platform.claude.com/docs/en/build-with-claude/citations
3. Embeddings. https://platform.claude.com/docs/en/build-with-claude/embeddings
4. Scale to many tools with tool search. https://code.claude.com/docs/en/agent-sdk/tool-search
5. Agent Skills. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview
6. Skill authoring best practices. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices
7. Optimizing for cost and intelligence. https://platform.claude.com/docs/en/about-claude/models/optimizing-for-cost-and-intelligence
8. Reduce hallucinations. https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-hallucinations
9. Model IDs and versioning. https://platform.claude.com/docs/en/about-claude/models/model-ids-and-versions
0. Claude Code on Amazon Bedrock. https://code.claude.com/docs/en/amazon-bedrock
1. Hooks reference. https://code.claude.com/docs/en/hooks
2. Configure the sandboxed Bash tool. https://code.claude.com/docs/en/sandboxing
3. Model deprecations. https://platform.claude.com/docs/en/about-claude/model-deprecations
4. Choose a permission mode. https://code.claude.com/docs/en/permission-modes
5. Review and monitor agent sessions. https://platform.claude.com/docs/en/managed-agents/secure-deployment-monitoring
6. Admin API. https://platform.claude.com/docs/en/manage-claude/admin-api
7. Zero data retention. https://code.claude.com/docs/en/zero-data-retention
8. Data residency. https://platform.claude.com/docs/en/manage-claude/data-residency
9. Query the Activity Feed. https://platform.claude.com/docs/en/manage-claude/compliance-activity-feed
0. Set up the Compliance API. https://platform.claude.com/docs/en/manage-claude/compliance-api-access
1. Set up CMEK on Claude Platform on AWS. https://platform.claude.com/docs/en/manage-claude/cmek-claude-platform-on-aws
2. Content moderation. https://platform.claude.com/docs/en/about-claude/use-case-guides/content-moderation
3. Refusals and fallback. https://platform.claude.com/docs/en/build-with-claude/refusals-and-fallback
4. Model cards. https://platform.claude.com/docs/en/resources/overview

15. Legal and compliance. https://code.claude.com/docs/en/legal-and-compliance
16. Versions. https://platform.claude.com/docs/en/api/versioning
17. Settings files and precedence. https://code.claude.com/docs/en/settings
18. How Claude remembers your project. https://code.claude.com/docs/en/memory
19. Deploy managed settings. https://code.claude.com/docs/en/managed-settings
10. Claude Code on Microsoft Foundry. https://code.claude.com/docs/en/microsoft-foundry
11. Claude Code on Claude Platform on AWS. https://code.claude.com/docs/en/claude-platform-on-aws
12. Create custom subagents. https://code.claude.com/docs/en/sub-agents
13. Extend Claude with skills. https://code.claude.com/docs/en/skills
14. All settings. https://code.claude.com/docs/en/settings-reference
15. Example settings files. https://code.claude.com/docs/en/settings-example
16. Automate actions with hooks. https://code.claude.com/docs/en/hooks-guide
17. Run Claude Code programmatically. https://code.claude.com/docs/en/headless
18. CLI reference. https://code.claude.com/docs/en/cli-reference
19. Use Claude Code GitHub Actions with cloud providers. https://code.claude.com/docs/en/github-actions-cloud-providers
10. Claude Code GitHub Actions. https://code.claude.com/docs/en/github-actions
11. Checkpointing. https://code.claude.com/docs/en/checkpointing
12. Debug your configuration. https://code.claude.com/docs/en/debug-your-config
13. Commands. https://code.claude.com/docs/en/commands
14. Manage costs effectively. https://code.claude.com/docs/en/costs
15. Explore the .claude directory. https://code.claude.com/docs/en/claude-directory